



智能体互联网基础设施开源项目

OpenAgenet（OAN）技术手册 v1.0

作者：许金良

jlxfly@gmail.com

2026 年 9 月 14 日

<https://www.openagenet.xyz>

<https://github.com/OpenAgenet>

目录

第一章 文档范围与状态	1
1.1 修订记录	1
1.2 目的与规范范围	2
1.3 与 OAN 白皮书和黄皮书的关系	2
1.4 与 did:oan 方法规范的关系	3
1.5 与实现仓库和部署文档的关系	3
1.6 规范性语言与符合性术语	4
1.7 参考来源	4
第二章 系统目标与设计原则	6
2.1 OAN 与智能体互联网	6
2.2 OAN 所解决的问题	7
2.3 资源优先的互联模型	7
2.4 可信注册、发布、发现与调用	8
2.4.1 资源注册与控制证明	8
2.4.2 根平台验证与可信发布	9
2.4.3 内容分发与发现服务节点索引	9
2.4.4 调用前验证与接入	9
2.5 资源控制、节点授权与治理的分离	9
2.6 联邦化、互操作与渐进式采用	10
2.7 控制面、数据面与语义面	10
2.8 公开可见性、可发现性与调用授权	11
2.9 参考来源	11
第三章 术语、角色与信任边界	13
3.1 核心术语与缩写	13
3.1.1 统一术语表	13
3.2 资源控制方与资源提供方	16
3.3 主体身份与本地身份	17
3.3.1 主体身份与资源控制身份	17
3.3.2 本地密钥生成、导入与备份	17
3.3.3 私钥保管与节点可见范围	18
3.4 根平台	18
3.4.1 信任授权枢纽	18

3.4.2	数据分发枢纽	19
3.4.3	语义治理枢纽	19
3.4.4	根平台职责边界	19
3.5	注册服务节点	19
3.6	发现服务节点	20
3.7	内容分发平台	20
3.8	链下信任索引器	21
3.9	调用方、用户、应用与智能体	21
3.10	治理权威与授权域	21
3.11	信任边界与责任矩阵	22
3.11.1	资源控制方与资源提供方	22
3.11.2	注册服务节点、根平台与发现服务节点	22
3.11.3	调用方与验证方	22
3.11.4	关键动作的责任归属	22
3.12	参考来源	23
第四章	总体架构与网络拓扑	25
4.1	逻辑架构	25
4.2	根平台作为信任授权枢纽	27
4.3	根平台作为数据分发枢纽	27
4.4	根平台作为语义治理枢纽	28
4.5	注册节点到根平台的发布路径	28
4.5.1	注册节点接入校验	29
4.5.2	上游提交与身份认证	29
4.5.3	根平台验证与发布确认	29
4.5.4	失败、拒绝与重试	29
4.6	根平台到发现节点的分发路径	29
4.6.1	分发内容与发布检查点	30
4.6.2	发现节点同步与索引	30
4.6.3	序列号、新鲜度与收敛	30
4.6.4	分发失败与恢复	31
4.7	多个注册节点与发现节点	31
4.7.1	多注册节点接入	31
4.7.2	多发现节点同步	31
4.7.3	节点间数据一致性	31
4.7.4	节点故障与隔离	32
4.8	第三方节点接入模型	32
4.8.1	接入前授权	32
4.8.2	节点能力与授权范围	32
4.8.3	协议符合性检查	32
4.8.4	运行期状态管理	33
4.9	公开网络、私有网络与联盟网络部署	33
4.10	可用性、隔离性与故障边界	34

4.11 控制面、数据面与语义面的交互	34
4.12 事实来源与派生读模型	36
4.13 网络拓扑与序列一致性	37
4.14 参考来源	38
第五章 OAN 资源模型	39
5.1 资源模型概览	39
5.2 智能体服务	40
5.3 技能包	41
5.4 MCP 服务	41
5.5 工具 API	42
5.6 资源关系与组合	43
5.6.1 资源之间的依赖关系	44
5.6.2 智能体服务与技能包组合	44
5.6.3 MCP 服务、工具 API 与调用关系	44
5.6.4 组合完整性与版本约束	44
5.7 作为独立可标识对象的资源	44
5.8 资源元数据与能力描述	45
5.8.1 基本身份与描述字段	45
5.8.2 能力标签、能力描述与用例	46
5.8.3 协议、端点与认证信息	46
5.8.4 状态、版本与治理信息	46
5.9 用例、协议绑定、端点与认证声明	46
5.9.1 用例描述	46
5.9.2 协议绑定	46
5.9.3 服务端点	46
5.9.4 认证与调用要求	47
5.10 资源包与发布产物	47
5.10.1 ResourcePackage 内容	49
5.10.2 元数据与资源内容	49
5.10.3 哈希与完整性证明	49
5.10.4 发布、下载与验证产物	49
5.11 资源生命周期与可见状态	49
5.11.1 注册状态	50
5.11.2 发布与索引状态	50
5.11.3 暂停、撤销与恢复状态	50
5.11.4 公开可见与可信可发现状态	50
5.12 资源所有权、控制权与委托关系	50
5.13 资源依赖与组合完整性	51
5.14 参考来源	51

第六章	did:oon 方法与资源身份	53
6.1	方法目的与标识符范围	53
6.2	DID 语法与方法特定标识符	54
6.3	资源 DID 与主体/控制者关系	54
6.3.1	主体 DID	54
6.3.2	资源 DID	54
6.3.3	控制关系与签名关系	55
6.3.4	主体、资源与提供方分离	55
6.4	DID Document 结构	55
6.4.1	DID 标识与方法元数据	56
6.4.2	verificationMethod	56
6.4.3	authentication 与 assertionMethod	56
6.4.4	service 端点	57
6.4.5	资源元数据与扩展字段	57
6.5	验证方法与公钥	57
6.6	服务端点与发现引用	58
6.7	资源元数据绑定	58
6.8	控制证明与 DID Document 证明	59
6.9	DID 生成与本地密钥处理	60
6.9.1	DID 生成	60
6.9.2	密钥生成与存储	60
6.9.3	本地身份备份	60
6.9.4	导入、恢复与密钥使用边界	61
6.10	DID 解析要求	61
6.11	DID 更新、版本管理、暂停、恢复与停用	61
6.11.1	同 DID 更新	62
6.11.2	版本选择	62
6.11.3	暂停与恢复	62
6.11.4	停用与历史状态	62
6.12	DID 方法安全与隐私考量	62
6.13	DID 方法符合性要求与测试向量	63
6.14	DID 方法互操作与解析器行为	63
6.15	参考来源	64
第七章	资源元数据与资源包类型	66
7.1	资源元数据模式	66
7.2	必填字段与可选字段	67
7.3	资源类型与主体类型约束	68
7.4	名称、描述、标签与用例	68
7.5	语义标签与协议绑定	68
7.6	服务端点与调用要求	69
7.7	资源版本与兼容性字段	69
7.8	包哈希与元数据哈希	70

7.9	规范化序列化与确定性哈希	70
7.10	ResourcePackage 构造规则	71
7.10.1	输入材料	71
7.10.2	字段规范化	71
7.10.3	哈希计算	72
7.10.4	签名与证明生成	72
7.10.5	构造后的校验	72
7.11	根平台验证的 ResourcePackage 结构	72
7.11.1	资源身份部分	73
7.11.2	资源元数据部分	73
7.11.3	根平台验证证明	73
7.11.4	分发和索引所需字段	73
7.12	ResourcePackage 完整性与来源验证	73
7.13	ResourcePackage 来源链与来源归属	74
7.13.1	资源控制方来源	74
7.13.2	注册节点来源	74
7.13.3	根平台发布来源	74
7.13.4	来源链验证	74
7.14	包大小、内容限制与校验策略	75
7.15	模式演进与向后兼容	75
7.16	参考来源	75
第八章	身份、密钥、签名、VC 与 VP	77
8.1	主体身份与资源身份	77
8.2	密钥生成、保管、备份与恢复	78
8.3	资源控制签名	79
8.4	节点身份密钥与节点签名密钥	79
8.5	基础设施节点的根平台授权 VC	80
8.6	向资源控制方签发的注册 VC	81
8.7	治理与授权证据	82
8.8	VC 与 VP 结构要求	82
8.8.1	VC 基本结构	82
8.8.2	VP 基本结构	82
8.8.3	签发者、持有者与主体	83
8.8.4	凭证状态与验证	83
8.9	签发者、持有者、控制者与凭证主体关系	84
8.9.1	签发者	84
8.9.2	持有者	84
8.9.3	资源控制者	84
8.9.4	凭证主体	84
8.10	签名验证与状态验证	84
8.11	凭证隐私与披露边界	85
8.12	密钥轮换与失陷响应	86

8.13 注册 VC 与资源发布证据	86
8.14 节点授权 VC 与基础设施权限	86
8.15 凭证吊销、暂停与状态传播	87
8.15.1 凭证状态变更	87
8.15.2 节点授权 VC 状态	87
8.15.3 资源注册 VC 状态	87
8.15.4 状态传播与缓存更新	87
8.16 签名格式、规范化与验证测试向量	87
8.17 参考来源	88
第九章 注册协议	89
9.1 注册目标与状态	90
9.2 注册请求结构	91
9.3 本地身份初始化与导入	93
9.3.1 新建本地身份	94
9.3.2 导入本地身份	94
9.3.3 私钥与主体配置	94
9.3.4 身份备份与恢复	94
9.4 DID Document 草稿生成	94
9.5 提交前本地校验	94
9.6 资源控制证明验证	95
9.7 注册节点授权与范围验证	95
9.8 资源类型、元数据、端点与资源包校验	96
9.9 重复注册与幂等性	96
9.9.1 同 DID 重复注册	96
9.9.2 幂等判断依据	96
9.9.3 VC 重发	96
9.9.4 根平台重复发布控制	96
9.10 资源更新与同 DID 版本更新	97
9.10.1 更新授权验证	97
9.10.2 DID Document 更新	97
9.10.3 资源元数据和端点更新	97
9.10.4 新版本发布与旧版本处理	97
9.11 注册 VC 签发与获取	97
9.11.1 VC 签发条件	97
9.11.2 VC 返回方式	97
9.11.3 VC 缺失或为空的处理	98
9.11.4 VC 本地保存	98
9.12 注册响应与错误处理	98
9.13 注册审计记录	99
9.14 资源更新授权与同 DID 替换	99
9.15 提交重试、幂等键与重复处理	100
9.16 注册速率限制与滥用控制	100

9.17 参考来源	100
第十章 根平台验证与可信发布	102
10.1 根平台发布职责	102
10.2 注册节点的上游提交	103
10.3 资源 DID 与控制证明验证	103
10.4 注册节点授权验证	104
10.5 ResourcePackage 与哈希验证	104
10.6 元数据与端点策略检查	104
10.7 根平台可信发布证明签发	104
10.7.1 签发前提	104
10.7.2 证明内容	105
10.7.3 证明签名	105
10.7.4 证明获取与验证	105
10.8 发布序列与版本选择	105
10.8.1 发布序列	105
10.8.2 同 DID 版本选择	105
10.8.3 重复提交处理	105
10.8.4 冲突版本处理	105
10.9 发布状态与拒绝原因	106
10.10 内容分发准备	106
10.11 根平台发布审计轨迹	107
10.12 重新发布、回滚与恢复	107
10.13 同 DID 更新授权与版本接受	107
10.14 发布拒绝、隔离与重新处理	108
10.15 根平台发布状态机	108
10.15.1 待验证	108
10.15.2 验证通过	108
10.15.3 已发布	108
10.15.4 已拒绝、隔离与恢复	108
10.16 参考来源	108
第十一章 内容分发与网络传播	110
11.1 分发模型与内容类别	110
11.2 内容分发服务职责	110
11.3 分发清单与发布检查点	110
11.4 资源包、元数据、证明与索引数据分发	111
11.4.1 资源包分发	111
11.4.2 DID Document 与元数据分发	111
11.4.3 VC 与根平台证明分发	111
11.4.4 索引数据分发	111
11.5 分发边界完整性验证	111
11.6 版本新鲜度与同步游标	112

11.6.1 同步游标	112
11.6.2 版本新鲜度	112
11.6.3 过期数据判断	112
11.6.4 缺口补偿	112
11.7 初始同步与增量同步	112
11.7.1 初始全量同步	112
11.7.2 增量同步	112
11.7.3 同步确认	113
11.7.4 中断后恢复	113
11.8 重试、退避与中断恢复	113
11.9 缓存行为与过期数据处理	113
11.9.1 浏览器或客户端缓存	113
11.9.2 节点侧缓存	113
11.9.3 旧快照优先展示	113
11.9.4 后台刷新与失败降级	114
11.10 传播可观测性与审计记录	114
11.11 发布顺序与冲突解决	114
11.12 分发一致性与新鲜度保证	114
11.13 部分失败与恢复语义	114
11.14 参考来源	115
第十二章 发现协议与查询语义	116
12.1 发现目标与查询类型	116
12.1.1 DID 精确发现	116
12.1.2 结构化条件发现	117
12.1.3 能力标签发现	117
12.1.4 文本与语义发现	117
12.1.5 组合条件发现	117
12.2 DID 精确查询	117
12.3 结构化资源查询	118
12.4 能力标签查询	118
12.5 文本与语义查询	119
12.6 语义与结构化条件组合查询	119
12.7 授权域过滤	120
12.8 最新有效版本选择	120
12.9 搜索排序与确定性结果顺序	120
12.9.1 匹配条件	120
12.9.2 信任和治理状态	120
12.9.3 版本和新鲜度	121
12.9.4 排序稳定性	121
12.9.5 重复结果处理	121
12.10 搜索解释与证据展示	121
12.10.1 匹配原因	121

12.10.2 资源来源	121
12.10.3 DID、VC 与根平台证明	121
12.10.4 治理状态与版本信息	122
12.11 发现响应模式	122
12.12 发现结果新鲜度与状态	123
12.13 发现错误与降级响应	124
12.14 公开可见性与可信可发现性	125
12.14.1 DID 级去重	126
12.14.2 版本级去重	126
12.14.3 注册节点重复记录	126
12.14.4 资源内容相同但 DID 不同的处理	126
12.15 结果去重与资源身份匹配	126
12.16 查询限制、分页与结果稳定性	127
12.17 发现隐私与授权域可见性	129
12.18 参考来源	129
第十三章 语义治理与语义发现	131
13.1 语义模型与能力词汇	131
13.2 根平台管理的能力标签树	131
13.2.1 标签树的组织方式	132
13.2.2 标签标识与层级关系	132
13.2.3 标签推荐与发现使用	132
13.2.4 标签树版本管理	132
13.3 标签层级、标识符、别名与本地化	132
13.4 注册过程中的标签选择与推荐	133
13.4.1 资源提供方选择	133
13.4.2 根平台推荐	133
13.4.3 自定义标签补充	133
13.4.4 标签校验与保存	133
13.5 自定义标签与提供方描述	133
13.6 语义治理变更流程	134
13.7 发现节点的语义索引构建	134
13.7.1 索引输入	134
13.7.2 标签索引	134
13.7.3 文本和语义索引	134
13.7.4 索引更新与删除	134
13.8 标签、文本、协议与资源类型匹配	134
13.9 查询解释与匹配说明	135
13.9.1 查询意图解析	135
13.9.2 标签和文本匹配	135
13.9.3 结构化条件匹配	135
13.9.4 结果解释	135
13.10 多语言语义发现	136

13.10.1 中英文资源描述	136
13.10.2 多语言标签	136
13.10.3 查询语言识别	136
13.10.4 跨语言匹配	136
13.11 面向向量检索与混合检索的扩展性	136
13.12 语义治理兼容性与迁移	137
13.13 语义词汇版本管理与变更影响	137
13.14 语义治理可审计性	137
13.15 自然语言查询与结构化条件映射	137
13.16 参考来源	138
第十四章 节点授权与链上治理	139
14.1 治理目标与链上、链下边界	139
14.1.1 链上治理事实	139
14.1.2 根平台运行状态	140
14.1.3 注册和发布状态	140
14.1.4 发现节点读模型	140
14.2 基础设施节点授权模型	140
14.3 授权域与能力范围	141
14.4 治理事件模型	141
14.4.1 节点授权事件	143
14.4.2 节点暂停与恢复事件	143
14.4.3 节点撤销事件	143
14.4.4 事件时间、序列与来源	144
14.5 节点授权、暂停、恢复与撤销	144
14.5.1 授权	144
14.5.2 暂停	144
14.5.3 恢复	144
14.5.4 撤销	144
14.5.5 状态对注册、分发和发现的影响	144
14.6 治理通知与授权状态	145
14.7 信任索引器作为治理读侧投影	145
14.7.1 链上事件读取	145
14.7.2 状态投影	145
14.7.3 同步游标与延迟	146
14.7.4 查询接口	146
14.7.5 同步失败与恢复	146
14.8 根平台运行时治理状态强制执行	146
14.9 发现侧治理过滤	147
14.10 治理事件历史与公开可观测性	147
14.11 链端点、包、对象与部署配置	147
14.12 治理升级与迁移规则	148
14.13 治理事件顺序与最终性	148

14.14 链上事实、链下发布状态与读模型一致性	149
14.14.1 链上最终事实	149
14.14.2 根平台发布状态	149
14.14.3 发现节点索引状态	149
14.14.4 状态不一致与收敛	149
14.15 治理权威密钥管理与应急流程	150
14.16 参考来源	150
第十五章 网络节点协议与互操作	151
15.1 通用协议信封与请求头	151
15.2 节点间认证	152
15.3 签名上游请求	152
15.4 节点授权 VC 交换	152
15.5 注册节点到根平台协议	153
15.6 根平台到内容分发协议	153
15.7 内容分发到发现节点协议	154
15.8 发现响应签名与来源证明	154
15.9 序列号、请求标识符与重放防护	155
15.10 分页、过滤与兼容性规则	155
15.11 超时、重试与幂等操作	155
15.12 协议版本协商	156
15.13 消息大小、压缩与传输限制	156
15.14 关联、可追踪性与审计标识符	156
15.15 协议兼容性与弃用规则	157
15.16 参考来源	157
第十六章 公开 HTTP API	159
16.1 API 约定与基础 URL	159
16.1.1 服务基础地址	159
16.1.2 路径和版本约定	159
16.1.3 请求认证	160
16.1.4 响应格式	160
16.2 健康、就绪与服务状态端点	160
16.3 注册节点注册端点	160
16.4 根平台发布与状态端点	161
16.5 内容分发端点	161
16.6 发现查询与资源详情端点	162
16.7 DID 解析端点	162
16.8 VC 与证明获取端点	162
16.9 治理状态与事件端点	163
16.10 指标与可观测性端点	163
16.11 请求与响应模式	163
16.11.1 注册请求与响应	164

16.11.2 发布请求与响应	165
16.11.3 发现请求与响应	165
16.11.4 状态和错误响应	165
16.12 HTTP 状态码与常见错误	165
16.13 速率限制、缓存与安全请求头	166
16.14 CORS、浏览器客户端与跨源策略	166
16.15 公开端点与仅限运维端点的边界	166
16.15.1 公开查询接口	167
16.15.2 节点间接口	167
16.15.3 运维接口	167
16.15.4 访问控制和暴露边界	167
16.16 API 示例与机器可读接口描述	167
16.17 参考来源	168
第十七章 客户端 SDK、社区 Skill 与开发者体验	169
17.1 TypeScript SDK 范围	169
17.2 SDK 中的 DID 身份与密钥操作	170
17.3 注册流程示例	170
17.4 DID 精确发现示例	171
17.5 语义发现示例	171
17.6 验证与信任证据处理	172
17.7 社区 Skill 包职责	172
17.8 资源编写与注册辅助	173
17.9 节点运营者与部署辅助	173
17.10 包分发与兼容性管理	173
17.11 示例资源与集成模板	174
17.12 客户端安全与秘密处理	174
17.13 浏览器注册与本地身份备份行为	174
17.13.1 浏览器端密钥生成	175
17.13.2 本地身份导入	175
17.13.3 本地备份下载	175
17.13.4 注册结果打包下载	175
17.14 DID、VC 与资源包下载格式	175
17.15 参考来源	177
第十八章 安全、隐私与滥用防护	178
18.1 威胁模型与信任假设	178
18.1.1 资源控制权威胁	179
18.1.2 节点信任威胁	179
18.1.3 数据污染和滥用威胁	179
18.1.4 可用性威胁	179
18.1.5 调用端点安全威胁	179
18.2 资源冒用与 DID 控制权保护	179

18.3 伪造注册与未授权更新防护	180
18.4 恶意或受损注册节点	180
18.5 恶意或受损发现节点	180
18.6 根平台受损与密钥保护	180
18.7 垃圾注册与资源目录污染	181
18.7.1 虚假资源注册	181
18.7.2 重复注册和版本污染	181
18.7.3 语义标签污染	181
18.7.4 限流、审核和清理	181
18.8 拒绝服务与资源耗尽	181
18.8.1 注册接口攻击	182
18.8.2 发现查询攻击	183
18.8.3 节点间同步压力	183
18.8.4 数据库和索引资源耗尽	183
18.9 重放、重复提交与过期发布	183
18.10 端点滥用与不安全资源调用	184
18.11 敏感元数据与凭证暴露	185
18.12 日志、审计与事件响应	185
18.13 第三方节点安全要求	185
18.14 端点验证、回调安全与 SSRF 考量	185
18.14.1 端点格式和协议验证	186
18.14.2 内网地址与回环地址限制	186
18.14.3 回调请求安全	186
18.14.4 重定向和 DNS 解析风险	186
18.15 包与依赖的供应链完整性	186
18.16 可用性目标与优雅降级	187
18.17 参考来源	187
第十九章 资源生命周期、治理与版本管理	188
19.1 资源生命周期状态机	188
19.2 初始注册	188
19.3 同 DID 资源更新	189
19.4 DID Document 版本管理	189
19.5 资源包版本管理	190
19.6 元数据与端点变更	190
19.7 注册 VC 生命周期	190
19.8 资源暂停与恢复	191
19.9 资源撤销与停用	191
19.10 节点暂停与资源可见性	191
19.11 发现节点的版本选择	191
19.12 历史版本与可审计性	192
19.13 资源 VC 与发布证据生命周期	192
19.14 更新传播与过期索引收敛	192

19.15 资源所有权转移策略	193
19.16 参考来源	193
第二十章 可观测性、指标与运维	194
20.1 服务健康与就绪模型	194
20.2 注册指标	195
20.3 发布与传播指标	195
20.4 发现查询指标	196
20.5 索引新鲜度与同步指标	196
20.6 治理与授权指标	196
20.7 网络级与节点级统计	197
20.7.1 网络级指标	197
20.7.2 节点级指标	197
20.7.3 注册、发布和发现指标	197
20.7.4 指标时间窗口	197
20.7.5 数据缺失和降级展示	197
20.8 链上治理事件可见性	198
20.9 日志、追踪与关联标识符	198
20.10 快照、缓存与降级模式行为	198
20.10.1 旧快照加载	199
20.10.2 后台数据刷新	199
20.10.3 新数据覆盖	199
20.10.4 长期失败与告警	199
20.10.5 无缓存时的首次加载	199
20.11 备份、恢复与灾难恢复	199
20.12 部署烟测与功能验证	200
20.12.1 注册功能烟测	200
20.12.2 发现功能烟测	200
20.12.3 节点健康检查	200
20.12.4 数据库和索引器检查	201
20.12.5 失败后的部署处理	201
20.13 数据保留、删除与隐私操作	201
20.14 运维告警与长期失败检测	201
20.15 公共服务状态与维护通信	201
20.16 参考来源	202
第二十一章 部署与符合性要求	203
21.1 参考部署拓扑	203
21.2 配置与秘密管理	203
21.3 构建与发布产物	204
21.4 数据库与存储要求	204
21.5 TLS、域名、反向代理与网络暴露	205
21.6 部署顺序与依赖检查	205

21.6.1 代码同步与构建	205
21.6.2 数据库准备	205
21.6.3 节点启动顺序	205
21.6.4 服务健康检查	205
21.6.5 部署后功能验证	206
21.7 注册节点功能烟测	206
21.8 发现节点功能烟测	206
21.9 根平台、分发与索引器验证	206
21.10 治理同步验证	207
21.11 升级、回滚与兼容性检查	207
21.11.1 升级前检查	207
21.11.2 数据和配置兼容	207
21.11.3 回滚条件	207
21.11.4 回滚后验证	207
21.12 第三方节点接入与符合性测试	207
21.13 构建可复现性与发布来源证明	208
21.14 环境特定配置文件	208
21.15 部署后功能与安全验证	208
21.16 参考来源	208
第二十二章 外部协议与生态映射	209
22.1 W3C DID Core 与 did:oon	209
22.1.1 DID Core 数据结构对应	210
22.1.2 did:oon 方法扩展	210
22.1.3 解析行为	210
22.1.4 互操作边界	210
22.2 A2A Agent Card 映射	211
22.2.1 能力描述映射	211
22.2.2 用例和技能映射	211
22.2.3 服务端点映射	212
22.2.4 身份、签名和可信证据映射	212
22.3 MCP 资源与服务器元数据映射	212
22.3.1 MCP Server 映射	212
22.3.2 工具定义映射	213
22.3.3 输入输出约束映射	213
22.3.4 调用和认证信息映射	213
22.4 工具与 API 描述映射	213
22.5 VC 与 VP 生态互操作	213
22.6 外部组织与平台身份映射	214
22.7 外部凭证与信任证据映射	214
22.8 字段来源、版本与哈希要求	214
22.9 兼容性配置文件与迁移路径	214
22.10 外部协议能力协商	214

22.11 外部签名与证据保留	215
22.12 参考来源	215
第二十三章 端到端参考流程	216
23.1 创建本地身份	217
23.2 注册智能体服务	217
23.3 注册技能包	217
23.4 注册 MCP 服务	218
23.5 注册工具 API	218
23.6 发布与分发资源	218
23.6.1 注册结果	219
23.6.2 根平台发布	219
23.6.3 内容分发	219
23.6.4 发现节点索引	219
23.7 按 DID 发现	219
23.8 按结构化条件发现	220
23.9 按自然语言任务描述发现	220
23.10 调用前验证已发现资源	220
23.10.1 DID 解析	221
23.10.2 控制权验证	221
23.10.3 注册 VC 验证	222
23.10.4 根平台可信发布证明验证	222
23.10.5 治理状态检查	222
23.11 在保留 DID 的情况下更新资源	223
23.12 暂停、撤销与恢复资源	223
23.13 接入第三方注册节点	224
23.14 接入第三方发现节点	225
23.15 通过网络观察治理事件	225
23.16 解析并验证资源 DID	225
23.17 获取资源历史版本	225
23.18 处理不可用或数据过期的发现节点	225
23.18.1 发现节点不可用	226
23.18.2 返回旧快照	226
23.18.3 数据新鲜度判断	226
23.18.4 切换其它发现节点	226
23.18.5 恢复后的重新同步	227
23.19 参考来源	227
第二十四章 符合性配置文件与验收标准	229
24.1 资源提供方符合性配置文件	229
24.2 注册节点符合性配置文件	230
24.3 根平台符合性配置文件	230
24.4 内容分发服务符合性配置文件	231

24.5 发现节点符合性配置文件	231
24.6 信任索引器符合性配置文件	232
24.7 解析器与客户端符合性配置文件	232
24.8 SDK 符合性配置文件	233
24.9 安全与密钥保管验收标准	233
24.10 互操作与兼容性验收标准	234
24.11 性能与可用性验收标准	234
24.12 符合性实现所需证据	235
24.12.1 协议和数据结构证据	235
24.12.2 密码学和身份证据	236
24.12.3 节点行为证据	236
24.12.4 部署和烟测证据	236
24.12.5 版本和来源证据	236
24.13 测试向量与互操作夹具	236
24.14 负向测试与拒绝行为	237
24.15 符合性证据留存与审查	237
24.16 参考来源	237
第二十五章 版本、兼容性与变更管理	239
25.1 规范版本管理	239
25.2 协议版本管理	240
25.3 资源模式版本管理	240
25.4 DID 方法兼容性	241
25.5 向后兼容变更与破坏性变更	241
25.5.1 兼容性变更	241
25.5.2 破坏性变更	242
25.5.3 版本标识	242
25.5.4 迁移和过渡期	242
25.6 治理控制的规范变更	243
25.6.1 变更提案	243
25.6.2 治理审批	243
25.6.3 发布和生效	243
25.6.4 回滚和紧急变更	244
25.7 弃用与迁移策略	244
25.8 变更记录与发布说明	244
25.8.1 规范变更记录	245
25.8.2 协议变更记录	245
25.8.3 实现变更记录	245
25.8.4 迁移提示和已知影响	245
25.9 参考来源	245

附录 A 规范性数据模式	247
A.1 DID Document 模式	247
A.2 资源元数据模式	248
A.3 ResourcePackage 模式	249
A.4 根平台可信发布证明模式	250
A.5 注册 VC 模式	251
A.6 节点授权 VC 模式	252
A.7 治理事件模式	252
A.8 注册请求与响应模式	253
A.9 发现请求与响应模式	254
A.10 错误响应模式	254
A.11 节点授权与治理状态模式	255
A.12 分发清单与同步游标模式	255
A.13 搜索解释与证据模式	257
A.14 参考来源	258
附录 B 协议与 API 参考	259
B.1 端点目录	259
B.2 请求头字段与签名	260
B.3 响应头字段与缓存指令	261
B.4 状态码与错误码	261
B.5 分页与游标规则	262
B.6 版本与兼容性头字段	262
B.7 健康、就绪与存活响应格式	263
B.8 公开、认证与仅限运维 API 矩阵	263
B.9 参考来源	264
附录 C 密码学与编码	265
C.1 支持的算法	265
C.2 规范化 JSON 与哈希	266
C.3 JWS 与分离式签名	266
C.4 DID 与密钥编码	267
C.5 VC 与 VP 编码	268
C.6 密钥备份与恢复格式	269
C.7 DID 与资源标识符测试向量	269
C.8 哈希与签名验证夹具	270
C.9 参考来源	270
附录 D 符合性测试矩阵	272
D.1 资源注册测试	273
D.2 根平台发布测试	274
D.3 分发与同步测试	274
D.4 发现与语义搜索测试	275
D.5 治理授权测试	276

D.6 版本更新与生命周期测试	276
D.7 安全与滥用防护测试	277
D.8 第三方节点接入测试	277
D.9 DID 解析与方法符合性测试	278
D.10 API 兼容性与错误契约测试	278
D.11 新鲜度、收敛与恢复测试	279
D.12 参考来源	279
附录 E 参考部署与示例	281
E.1 官方网络部署	281
E.2 私有网络与联盟网络部署	282
E.3 最小注册节点部署	283
E.4 最小发现节点部署	283
E.5 SDK 注册示例	284
E.6 DID 与语义发现示例	286
E.7 资源验证示例	286
E.8 运维烟测示例	287
E.9 同 DID 更新示例	287
E.10 资源暂停与恢复示例	288
E.11 跨协议 A2A 与 MCP 映射示例	288
E.12 参考来源	289
附录 F 仓库与实现映射	290
F.1 oan-protocol-common	290
F.2 oan-root-services	292
F.3 oan-registrar-node	292
F.4 oan-discovery-node	293
F.5 oan-trust-indexer	294
F.6 oan-sdk-ts	295
F.7 oan-community-skill	296
F.8 官网与公开开发者入口	296
F.9 规范章节与实现模块映射	297
F.10 参考来源	298

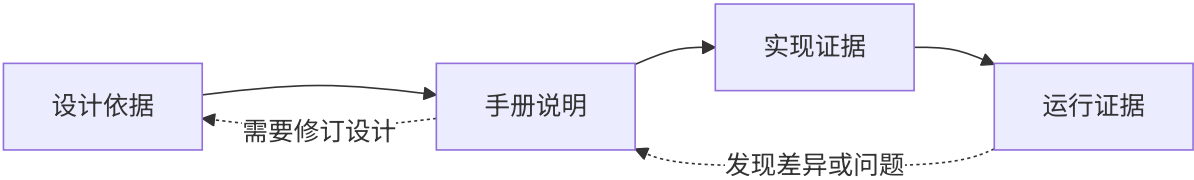
第一章 文档范围与状态

本章是本技术手册的阅读入口，面向资源提供方、节点运营者、客户端和 SDK 开发者、协议实现者、集成开发者以及需要评估 OpenAget (OAN) 的技术人员。手册覆盖智能体服务、技能包、MCP 服务和工具 API 等资源，也覆盖主体身份、资源 DID、DID 文档、注册 VC、根平台可信发布证明、内容分发、发现索引和调用前验证等支撑机制。读者可以把它作为一份从概念到实现的导航：先通过第二至五章理解目标、角色、架构和资源模型，再按需要进入身份、注册、发布、发现、运维和符合性章节。

手册中的内容需要区分四种状态：**设计依据**说明为什么这样设计；**当前系统事实**说明现有代码、接口或部署已经表现出的行为；**参考实现**说明 OAN 代码仓采用的一种实现方式；**运行证据**说明某个环境在特定时间的构建、部署和测试结果。规范性要求只在需要保证互操作、安全或数据完整性时使用，不能把官网展示、示例配置或一次部署结果扩大解释为所有节点的强制能力。后续章节引用接口、代码或运行数据时，应明确其证据类型和适用范围。

```
{
  "manual": "technical manual",
  "scope": ["resource identity", "registration", "trusted publication", "discovery", "governance"],
  "evidence_types": ["design", "current-fact", "reference-implementation", "runtime-evidence"],
  "status_rule": "每项结论都应说明来源、适用范围和核验时间"
}
```

本手册把材料分为设计依据、规范规则、实现证据和运行证据。设计依据解释目标和取舍；规范规则说明互操作底线；实现证据说明代码如何落实；运行证据说明特定环境在特定时间是否正常。四类材料可以互相引用，但不能跨层级替代。发布或审查时，应记录手册版本、引用材料版本、实现提交、配置版本、测试时间和适用环境。



1.1 修订记录

时间	变更说明	修改人
2026-09-14	发布 v1.0	许金良

1.2 目的与规范范围

手册覆盖一条资源从本地准备到被发现和验证的完整路径：资源提供方准备资源和本地身份，客户端向注册服务节点提交材料，注册服务节点完成接入校验，根平台验证并发布可信资源包，内容分发平台承载可获取材料，发现服务节点同步并建立索引，调用方在连接前检查 DID、凭证、版本和治理状态。对应章节分别解释资源模型、did: oan、DID 文档、资源包（ResourcePackage）、VC/VP、签名消息、节点授权、语义标签、公共 HTTP API、SDK、社区 Skill、生命周期、运维和符合性。

读者要完成的任务	建议阅读路径	主要结果
理解 OAN 的定位和组件	第 1、2、3、4 章	了解目标、角色、网络拓扑和资源类型
注册或发布资源	第 5、6、7、8、9、10 章	理解身份、资源包、注册、可信发布和分发
接入发现和语义查询	第 11、12、14、15、16 章	理解查询、语义治理、节点互操作和 SDK 使用
部署、排障和评估	第 17、19、20、23、24 章及附录	获得安全、运维、部署、测试和版本依据

手册不规定智能体内部采用何种模型或推理算法，不规定业务系统的全部业务逻辑，也不替代 A2A、MCP 等外部协议本身。每个章节应说明适用角色、输入输出、责任主体、验证主体、状态变化、异常处理和可留存证据；如果某个结论只适用于参考实现或当前部署，应明确标注版本、环境和边界。

1.3 与 OAN 白皮书和黄皮书的关系

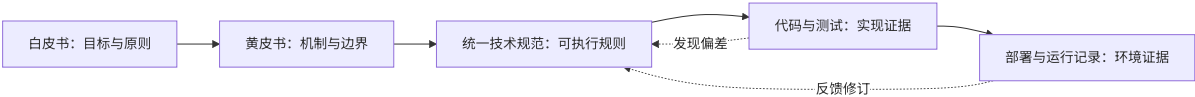
白皮书是项目的概念入口，主要回答为什么建设面向智能体互联网（Internet of Agents，IoA）的资源身份、可信发布和语义发现基础设施；黄皮书更关注机制如何工作，解释角色、数据对象、信任边界、发布链路、授权域、状态机、安全不变量和参考验证方法；本手册则把这些材料组织成读者能够查阅和使用的技术说明，补充实现映射、示例、操作边界和验证路径。

材料	读者主要从中获得什么	使用时的边界
白皮书	背景、愿景、总体定位和设计原则	不直接等同于接口字段或部署承诺
黄皮书	角色、机制、信任边界和工程推导	参考算法和推导不自动等同于所有实现的固定格式
本技术手册	概念解释、流程、实现映射、示例、运维和验证入口	仍需区分规范要求、参考实现和当前运行证据

阅读时可以从白皮书了解“为什么”，从黄皮书了解“如何组织机制”，再回到本手册查找“如何理解、接入、实现和验证”。如果不同材料只是抽象层级不同，应在手册中说明层级关系；如果确实存在字段、流程或安全边界冲突，应记录冲突位置、影响范围、适用版本和修订状态，不能用模糊表述掩盖。

材料或证据	主要回答的问题	可直接证明的内容	不能单独证明的内容
白皮书	为什么建设 OAN，以及总体愿景是什么	背景、目标、价值和原则	具体接口字段和实现已部署

材料或证据	主要回答的问题	可直接证明的内容	不能单独证明的内容
黄皮书	机制如何组织，以及安全边界是什么	角色、流程、状态和机制推导	所有实现都采用相同代码
统一技术规范	不同实现如何互操作	字段、行为、错误和符合性要求	某个部署环境长期可用
代码、测试和配置	参考实现如何落实规则	模块行为、测试结果和配置约束	设计愿景天然已经实现
部署与运行记录	某个环境当前是否可用	构建、启动、烟测和运行状态	协议本身的必需条件



发布或审查时，应至少记录规范版本、引用材料版本、实现提交、配置版本、测试时间和适用环境。若材料之间出现差异，先区分是抽象层级不同还是实际冲突；只有后者才进入冲突记录和修订流程。

1.4 与 did:oan 方法规范的关系

did:oan 方法材料负责方法特定 DID 的语法、方法特定标识符、DID 文档结构、验证方法、公钥表达、服务端点、解析行为和控制权证明边界。本手册的相关章节不重复发明这些方法规则，而是解释 OAN 如何使用它们：客户端如何生成和保存本地密钥，资源控制方如何证明控制关系，注册服务节点如何校验 DID 文档，根平台如何把资源版本与可信发布记录关联，发现服务节点如何建立可验证索引，调用方如何在连接前解析和验签。

一个最小的状态判断可以表示为：

DID 文档有效

- > 资源控制证明有效
- > 注册服务节点受理
- > 根平台发布
- > 发现服务节点索引
- > 调用方完成版本、治理和本地权限检查

上述步骤不是同一个“可用”状态的不同名称。DID 文档在语法、结构和签名上有效，不代表资源已经被注册服务节点受理、被根平台发布、被发现服务节点索引或获得调用许可；根平台的发布事实也不能替代调用方对 DID 文档、VC、端点、版本、治理状态和本地业务权限的检查。后续章节应通过字段、流程和状态说明它们如何衔接。

1.5 与实现仓库和部署文档的关系

手册中的实现映射用于帮助读者回答“这项能力在哪里实现、如何验证、当前有什么边界”，而不是把仓库名称当作符合性证明。共用数据结构、序列化、哈希和密码学行为优先核对 oan-protocol-common；根平台发布、治理状态和发布证明核对 oan-root-services；注册接入和注册凭证核对 oan-registrar-node；同步、索引、语义检索和发现响应核对 oan-discovery-node；链上治理事件到运行时授权读模型的投影核对 oan-trust-indexer；客户端集成核对 oan-sdk-ts 与社区 Skill。README、schema、路由处理器、测试、配置样例和部署脚本承担的证据作用不同，应在引用时说明用途。

```
mapping:
  manual_section: "第 8 章 注册协议"
  repository: "oan-registrar-node"
  evidence: "路由处理器、请求校验、集成测试"
  status: "已实现并有测试证据"
  checked_at: "YYYY-MM-DD"
  boundary: "具体提交和部署配置以目标环境为准"
```

每项映射至少应记录章节、仓库和模块或文件、对应版本、实现状态、验证方法以及已知偏差。实现状态应区分已实现并有测试证据、已实现但证据不足、部分实现和仅有设计；部署材料还应记录环境变量、外部服务、数据库、密钥边界、构建产物、启动顺序和部署后烟测。官方环境的地址、节点数量和运行指标属于运行证据，不应写成所有环境的必需条件。

1.6 规范性语言与符合性术语

本手册沿用 RFC 风格关键词来标识强制程度：MUST 对应“必须”，MUST NOT 对应“不得”，SHOULD 对应“应”，SHOULD NOT 对应“通常不应”，MAY 对应“可以”。这些词只在已经确认的互操作、安全、数据完整性或符合性要求中使用；描述当前代码行为、部署现状或操作建议时，应改用“当前实现”“通常”“建议”“可以”等更准确的说法。

符合性判断应针对具体角色和配置文件，而不是对整个 OAN 做笼统评价。资源提供方、注册服务节点、根平台、内容分发平台、发现服务节点、链下信任索引器、解析器和 SDK 的能力边界并不相同。每项测试或运行检查应说明固定输入、前置配置、执行步骤、预期输出、失败影响和证据保存方式；参考实现通过测试不等于所有第三方实现天然符合，官方部署运行正常也不等于协议覆盖所有环境。

- ☐ 已说明本条内容属于规范要求、参考实现、当前部署还是待实现能力。
- ☐ 已给出能够复核的来源、版本或提交信息。
- ☐ 已区分资源登记、根平台发布、发现索引和调用前验证的证据。
- ☐ 已说明偏离规范或实现失败时的影响范围和处理方式。

1.7 参考来源

来源	类型	链接
OAN White Paper	arXiv 白皮书：项目愿景与建设定位	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv 黄皮书：技术架构与信任模型	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	标准/方法规范：did:oan 标识与方法规则	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

来源	类型	链接
OAN Resource Identity and Discovery	IETF 草案：资源身份与发现	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Agentic Overlay Network Architecture	IETF 草案：智能体覆盖网络架构	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF 草案：高效智能体发现配置文件	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第二章 系统目标与设计原则

本章说明 OpenAgenet (OAN) 为什么需要面向智能体互联网的可信资源基础设施，以及系统目标如何落实为身份、注册、可信发布、内容分发、发现、治理和调用前验证等设计原则。章节重点不在列举单一产品功能，而在说明这些机制如何共同解决资源不可识别、来源难验证、版本难追踪、能力难发现和调用前缺少判断依据的问题，并明确开放互操作、最小信任、证据可追溯和职责分离之间的关系。

2.1 OAN 与智能体互联网

OpenAgenet (OAN) 面向智能体互联网提供一套资源互联基础设施。它关注的不是某个具体智能体的对话能力，而是让智能体服务、技能包、MCP 服务和工具 API 具备稳定的身份、可验证的控制关系、可描述的能力、可追踪的发布状态和可复用的发现入口。OAN 将这些对象视为网络资源，使资源能够脱离单一平台的人工目录，通过统一的 `did:oan`、资源元数据、注册协议和发现协议参与跨节点互联。

OAN 的核心结果是把“有什么能力”和“能否信任这个能力”连接起来。资源提供方在本地准备 DID 文档和资源描述，经注册服务节点接入后，由根平台验证注册服务节点授权、资源控制证明、元数据和版本信息，再通过内容分发平台提供给获得授权的发现服务节点。调用方获得搜索结果后，还需要根据 DID、VC、根平台证明、治理状态和端点信息完成调用前验证。这样的组织方式既保留不同资源类型和业务协议的差异，也为资源的发现、组合和后续治理提供共同基础。

从使用者角度看，OAN 的目标可以归纳为四个连续结果：资源能够被识别，资源来源能够被验证，资源能够在授权范围内被发现，调用方能够在连接前获得足够的判断依据。这四个结果分别由身份材料、注册与发布证据、发现索引和调用方验证共同支撑，不能由单一页面、目录或搜索分数替代。

设计目标	主要承载机制	读者可以观察到的结果
资源可识别	<code>did:oan</code> 、DID 文档、资源元数据	能够区分资源主体、类型、版本和服务引用
来源可验证	控制证明、注册 VC、根平台可信发布证明	能够核对控制关系、发布来源和材料哈希
资源可发现	授权域、能力标签、结构化查询和语义检索	能够按 DID 或任务需求找到候选资源
调用前可判断	版本、生命周期、治理状态、端点和本地策略	能够决定是否进入后续业务调用

2.2 OAN 所解决的问题

当智能体、技能包、MCP 服务和工具 API 分散在不同团队、不同代码仓或不同运行环境中时，资源提供方通常只能通过网页、文档、配置文件或人工沟通说明资源位置和能力。这种方式难以稳定表达资源的控制者、版本、端点、依赖、适用协议和使用条件，也难以判断一个搜索结果是当前版本、历史版本还是已经被暂停的资源。资源数量增加后，目录维护、重复登记、版本更新和跨组织协作都会产生持续的人工成本。

对调用方而言，名称或文本匹配只能解决“可能相关”，不能单独解决“是否为真实资源、是否由授权节点发布、是否仍处于有效状态、是否可以在当前授权域内使用”。如果注册、发布、发现和调用没有清晰的责任边界，任何一个中间目录都可能把未经验证的资源混入结果，或者继续展示已撤销、已替换和来源不明的版本。OAN 以 DID 控制证明、节点授权、根平台发布、资源包哈希、发现服务节点授权域过滤和调用前验证分别承接这些问题，而不是用一个平台标签替代全部信任判断。

可以把问题与 OAN 的处理方式对应起来：资源端点分散，首先需要统一的资源身份和服务描述；来源难核验，需要控制证明、注册 VC 和根平台可信发布证明；版本状态不透明，需要把版本、哈希和生命周期状态绑定到资源记录；跨组织发现困难，需要发现节点、授权域、能力标签和语义查询共同工作。这样，OAN 解决的不是“再做一个资源列表”，而是把资源登记、来源核验、网络传播和调用前判断组织成可追踪的过程。

```
{
  "problem": "同一任务需要在多个团队和运行环境中寻找可复用能力",
  "resource": {
    "type": "skill",
    "did": "did:oan:example",
    "version": "1.0.0",
    "capabilities": ["repository-search", "project-structure-summary"]
  },
  "required_evidence": ["DID document", "control proof", "publication status", "endpoint policy"]
}
```

上例只表示问题分析和资源描述的最小形状，不代表固定的生产字段或可直接提交的注册请求。真实字段、证明和校验规则以资源模型、DID 方法、注册协议和附录中的数据模式为准。

2.3 资源优先的互联模型

资源优先意味着 OAN 的注册、发布、分发和发现对象首先是一个具有资源类型、资源 DID、描述、能力标签、用例、端点、版本和状态的资源版本，而不是某个必须运行在统一平台中的智能体实例。智能体服务可以提供面向会话或任务的服务，技能包可以作为可复用能力单元，MCP 服务可以暴露工具集合，工具 API 可以提供具体接口；它们在业务形态上不同，但都通过统一的资源身份和元数据进入网络。

统一模型只统一共性，不抹平类型差异。资源类型决定必填字段、端点形态、协议绑定、调用约束和版本处理方式；资源 DID 标识具体可控制对象，资源包绑定该版本的表示和证明，能力标签与用例支持结构化和语义发现。资源之间可以通过 DID、版本或包哈希表达依赖与组合，但“可发现”只说明资源满足目录和信任条件，不自动说明组合关系中的每个业务调用都已获授权。

资源优先模型可以用一个抽象对象理解：身份字段回答“这是哪个资源”，描述字段回答“它能做什么”，版本和哈希回答“当前看到的材料是哪一版”，端点和协议字段回答“如何接入”，证明和

状态字段回答“为什么可以把它作为候选资源”。不同资源类型可以共享这些公共信息，但不能因此把技能包当成智能体服务、把 MCP 服务当成工具 API，或把资源的发现结果当成业务调用授权。

```
{
  "resourceType": "skill",
  "resourceDid": "did:oon:example",
  "version": "1.0.0",
  "description": "检索代码仓库并总结项目结构",
  "capabilities": ["code-search", "structure-summary"],
  "protocol": "skill",
  "endpoint": "https://example.invalid/skill",
  "status": "published"
}
```

这是用于解释资源优先模型的示例对象；资源类型的正式字段、端点格式、哈希和签名材料仍由对应章节定义。

2.4 可信注册、发布、发现与调用

OAN 的可信链路不是一次请求完成的单一动作，而是由资源控制方、注册服务节点、根平台、内容分发平台、发现服务节点和调用方共同完成的连续过程。资源控制方先在本地生成或导入身份材料，准备资源 DID 文档、资源元数据、能力标签、端点和控制证明；注册服务节点负责接入校验和向根平台提交；根平台负责验证授权、签名、哈希、版本和治理条件；后续节点只传播和索引已形成的可信事实；调用方则在业务连接前重新验证相关证据。

对于一个资源版本，读者可以按下面的顺序判断“走到哪一步”：

1. 本地材料是否已经生成，且控制方能够使用对应私钥完成证明；
2. 注册服务节点是否受理并返回登记结果；
3. 根平台是否形成该版本的接受和可信发布事实；
4. 内容分发和发现服务节点是否完成校验、同步和授权域过滤；
5. 调用方是否完成 DID、VC、版本、状态、端点和业务策略检查。

任何一步失败，都应保留可定位的状态或错误信息，使读者能够判断是材料问题、节点授权问题、传播延迟、索引问题还是调用方策略拒绝。

2.4.1 资源注册与控制证明

资源注册从资源提供方本地生成资源版本开始。客户端应根据资源类型收集名称、描述、能力标签、用例、协议绑定、端点、版本和依赖等信息，并将资源 DID 与控制密钥、公钥和 DID 文档建立可验证关系。提交给注册服务节点的内容应能够证明请求方对资源 DID 的控制权；注册服务节点可以执行格式、字段、标签、端点和策略校验，但其受理不等于根平台已经接受资源。

控制证明解决的是“谁能够代表该资源提交登记”这一问题，不能替代资源质量、端点可用性或调用权限判断。失败时应返回可定位的字段或证明错误，并保持本地草稿和密钥材料不被服务端接管。相同 DID 的再次提交还应区分相同版本的重复请求与有授权的新版本更新，分别交由幂等处理和版本更新规则处理。

2.4.2 根平台验证与可信发布

根平台接收注册服务节点的签名上游请求后，应先确认请求新鲜度、请求体哈希、随机数和注册服务节点的有效授权，再验证资源 DID 文档、资源控制证明、资源类型、元数据哈希、资源包和版本转换条件。只有这些事实一致且满足治理与发布策略，根平台才形成该资源版本的接受记录和根平台可信发布证明。根平台的接受记录应绑定 DID、版本、表示哈希、授权域、状态、时间或序列信息，避免后续节点仅凭可读文本判断来源。

根平台验证成功后，发布通常通过持久化任务、消息或同步游标异步推进。资源提供方可以拿到注册服务节点返回的登记结果，但在内容分发、根平台发布完成和发现服务节点索引完成前，资源不应被表述为已经在全网可发现。验证拒绝、隔离、重试和重新发布应具有可观察状态，不能用一次 HTTP 成功响应覆盖整个传播链路。

2.4.3 内容分发与发现服务节点索引

内容分发平台承载根平台已经验证的资源包、DID 文档、元数据、证明和索引所需材料，发现服务节点从分发层或根平台通知中获取候选版本。发现服务节点必须重新计算哈希、验证根平台证明和治理状态，并依据自身授权域过滤资源，然后才把合格资源写入结构化索引和语义索引。分发层提高可访问性和同步效率，但不取得根平台的信任权威。

分发和索引具有异步性，因此存在根平台已发布而发现服务节点暂未可见、结构化索引已完成而语义索引尚在重试、或节点暂时只能提供旧快照等正常中间状态。实现应保存同步游标、版本新鲜度、拒绝原因和待处理任务，使运营者能够区分未发布、未同步、校验失败、授权域不匹配和搜索索引失败。

2.4.4 调用前验证与接入

调用方从发现服务节点获得候选资源后，不应把搜索结果直接当作调用许可。至少应验证响应来源和新鲜度、资源 DID 文档、资源控制关系、注册 VC 或相应登记证据、根平台可信发布证明、资源包哈希、当前版本、生命周期状态、发现服务节点授权域以及目标端点的协议和认证声明。对于高风险操作，还应在实际请求中使用时间戳、随机数、目标 DID、请求体哈希和签名，防止结果被重放或替换。

验证通过只说明候选资源满足 OAN 信任路径和调用方本地策略的前置条件，最终能否执行仍取决于资源服务端点自身的认证、授权、业务状态和安全策略。验证失败时，客户端应明确区分 DID 或签名错误、凭证状态错误、根平台证明不匹配、版本过期、治理状态无效、授权域不符和端点调用失败，避免把所有问题归结为“搜索不到”。

2.5 资源控制、节点授权与治理的分离

资源控制权由资源 DID 对应的控制密钥和控制证明表达，回答“谁有权代表这个资源提交或更新文档”；节点授权由根平台授权材料及其治理状态表达，回答“哪个注册服务节点、发现服务节点或其它基础设施节点有资格执行某类网络动作”；链上治理记录表达节点授权变化等治理事实，回答“授权何时生效、暂停、恢复或撤销”。根平台发布证明则回答“某个资源版本已经通过根平台验证并进入发布链路”，它不是资源所有权凭证。

这三类事实在验证时应分别检查。拥有资源私钥不能让一个未授权节点获得注册资格，持有旧节点授权 VC 也不能绕过当前治理状态，资源已经被根平台发布也不能替代调用方对端点和业务权

限的判断。分离设计让 OAN 能够在不接触资源控制私钥的前提下管理基础设施授权，同时允许治理状态、资源身份和发布状态分别更新。

判断问题	主要证据	不能由它替代的判断
谁能代表资源提交或更新	资源 DID、控制密钥和控制证明	节点是否有基础设施授权
哪个节点可以执行网络动作	节点授权材料和治理状态	某个资源是否由该节点控制
哪个版本已进入可信发布链路	根平台可信发布证明、版本和哈希	资源端点是否满足调用方业务要求
是否可以实际调用	调用方综合验证和本地业务策略	不能仅由搜索结果或页面状态决定

2.6 联邦化、互操作与渐进式采用

OAN 的联邦化体现在多个注册服务节点和发现服务节点可以在同一根平台信任与发布规则下独立运行。注册服务节点面向不同组织、团队或接入场景提供资源受理，发现服务节点可以按授权域、网络位置或服务能力提供索引；根平台负责统一验证和发布事实，而不是要求所有资源都集中存储在一个业务平台中。第三方节点只有在完成身份、授权、接口和符合性检查后，才能进入相应的运行状态。

渐进式采用可以从单个组织或试验网络开始：先采用 `did: oan` 资源身份和注册发现流程，再接入现有 Agent、MCP 或工具 API，随后增加多个节点、授权域、语义标签和治理读模型。互操作的重点是保持 OAN 的身份、证明、版本和状态边界，同时通过协议适配层接入既有业务系统，不要求业务系统立即改造成统一运行时。

一个组织可以按“先可见、再可信、后联邦”的顺序推进：第一阶段整理已有资源并建立资源身份和描述；第二阶段接入注册服务节点，形成控制证明、注册 VC 和根平台发布证据；第三阶段部署或接入发现服务节点，启用结构化和语义查询；第四阶段增加多个组织节点、授权域、治理同步和协议映射。每个阶段都可以保留原有业务端点，OAN 主要负责资源身份、登记、发布、发现和调用前判断，不要求一次性替换现有业务系统。

资源整理与身份建立

- > 注册和可信发布
- > 发现节点索引
- > 多节点接入与授权域治理
- > 与既有 Agent、MCP、Tool/API 协议映射

2.7 控制面、数据面与语义面

控制面维护身份、授权、版本、生命周期和发布状态等决定“能否进入网络、能否传播、能否被视为有效”的事实；数据面承载 DID 文档、资源包、元数据、证明和面向客户端的分发内容；语义面维护能力标签树、标签层级、别名、描述和查询解释，用于注册时的选择推荐以及发现阶段的语义匹配。根平台同时承担三个有边界的枢纽职责，但各平面维护的事实不能互相替代。

三平面存在明确依赖顺序。语义匹配可以帮助发现服务节点找到相关资源，数据分发可以让节点取得资源包，但只有控制面确认资源版本已被接受、发布且在发现服务节点授权域内，候选结果才具有可信可发现资格。语义相关性、端点可访问性或界面展示都不能反向授予 DID 控制权、节点授权或治理资格。

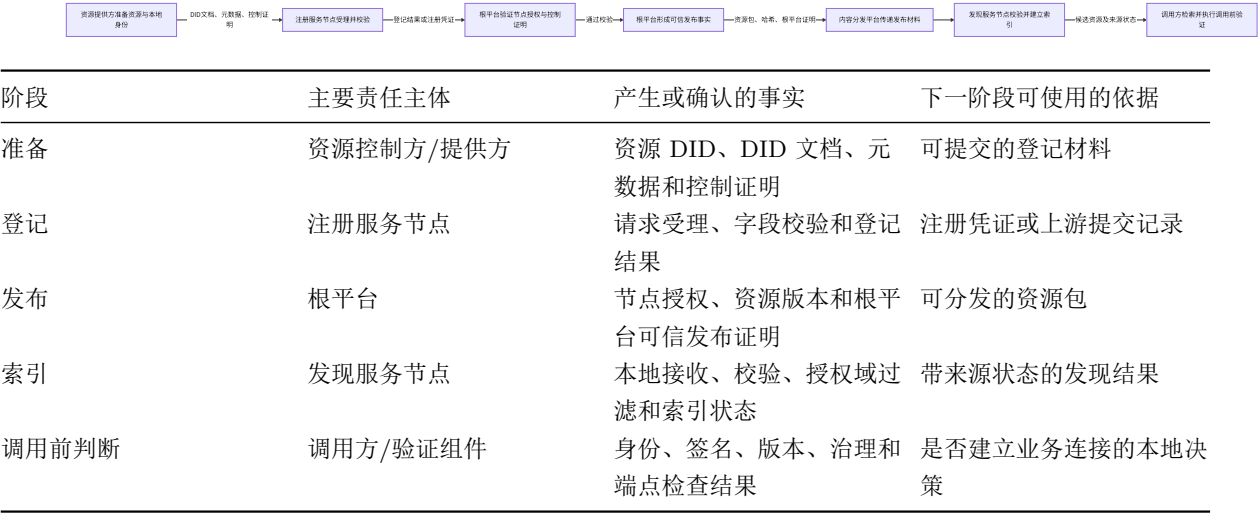
平面	维护的主要事实	典型参与组件	对其他平面的限制
控制面	身份、授权、版本、生命周期和发布状态	根平台、治理组件、注册服务节点、链下信任索引器	决定哪些材料可以进入传播和可信发现
数据面	DID 文档、资源包、元数据、证明和分发内容	内容分发平台、注册服务节点、发现服务节点	必须保留哈希、版本和来源，不能改写控制面事实
语义面	能力标签、层级、别名、描述和查询解释	根平台语义治理、发现服务节点	只帮助理解和排序，不能替代身份和授权判断

2.8 公开可见性、可发现性与调用授权

注册完成表示注册服务节点已对请求进行受理或登记处理，关联证据通常是注册响应、注册 VC 或登记记录；根平台已发布表示根平台已经接受某个资源版本并形成相应发布证明；发现服务节点已索引表示某个获得授权的发现服务节点已经完成资源包校验并写入本地索引；授权域内可见表示该节点依据自身授权范围允许向特定查询方返回资源；调用许可则是调用方在完成 OAN 证据验证后，结合端点认证和自身业务策略作出的本地判断。

这些状态可以按顺序推进，也可能因异步任务或故障暂时不一致。旧快照可以在刷新失败时继续提供有限的已验证历史结果，但必须带有新鲜度或状态信息；资源暂停、撤销、版本替换和节点授权变化应在相应状态传播后影响发现和调用前验证。任何公开门户、统计页面或缓存都只是观察界面，不能成为替代根平台接受记录、治理状态和签名验证的信任来源。

下图把本章的目标压缩为一个可检查的资源版本流程。每个箭头代表事实或材料向下一阶段传递，不表示前一阶段的结果自动授予下一阶段权限；异步分发和索引可能产生时间差，调用方仍需在最终连接前重新验证。



可发现不等于可调用。发现服务节点返回候选资源，调用方仍须根据自身权限、端点认证、资源状态和业务风险完成最终判断。

本章的设计原则可以用三条验收问题复核：资源是否有可验证身份，发布是否有可追踪证据，发现结果是否能够携带足以支持调用前判断的来源和状态信息。若其中一项只能依赖页面文字或名称匹配，则不能视为完整的可信互联流程。

2.9 参考来源

来源	类型	链接
OAN White Paper	arXiv 白皮书：项目愿景、生态价值和建设必要性	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv 黄皮书：技术原则、信任边界和系统机制	https://arxiv.org/abs/2606.03163
GRAIL Semantic Discovery Paper	arXiv 论文：语义发现和资源匹配方向	https://arxiv.org/abs/2605.02489
Agentic Overlay Network Architecture	IETF 草案：智能体网络架构	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

第三章 术语、角色与信任边界

本章定义 OpenAgenet (OAN) 中参与资源互联的主体、资源、凭证、节点和信任边界。角色定义服务于后续协议，而不是对组织架构或部署进程作抽象描述：主体负责控制身份和资源，注册服务节点负责受理和提交，根平台负责授权、验证、发布和语义治理，内容分发平台负责提供已经验证的材料，发现服务节点负责同步、过滤、索引和返回结果，调用方负责验证候选资源并决定是否连接。每个角色的权限都应与其能够产生或验证的事实相匹配。

OAN 中需要特别区分三类关系：资源控制方是否有权代表资源 DID，基础设施节点是否得到根平台和治理状态认可，资源版本是否已经通过根平台发布并被发现服务节点索引。DID、签名、VC、根平台可信发布证明和链上治理事件分别承载不同事实。任何角色都不能仅凭自己保存的一份 JSON、一次 HTTP 响应或一个搜索排名，替代其它角色负责的验证。

阅读本章时，可以把一个术语放回四个问题中理解：它描述的是谁、它证明了什么、它由谁产生或保存、它能否替代其他证据。例如，资源 DID 描述被控制的资源对象，控制证明表达资源控制方的操作授权，节点授权表达基础设施节点的运行资格，根平台可信发布证明表达特定资源版本已经通过根平台验证。四者可以同时出现在一次注册或发现流程中，但不能合并成一个笼统的“可信”字段。

判断维度	典型问题	主要证据
对象身份	当前材料描述的是哪个主体或资源？	主体 DID、资源 DID、DID 文档
控制关系	谁能够代表该对象提交或更新？	控制公钥、控制证明、签名
基础设施资格	哪个节点有资格执行网络动作？	节点授权、治理状态、授权域
发布与可见性	哪个版本已被接受、分发和索引？	根平台可信发布证明、同步记录、发现响应
业务调用	调用方是否应当连接和调用？	本地验证结果、端点认证和业务策略

3.1 核心术语与缩写

3.1.1 统一术语表

英文全称	缩略语	中文名称
OpenAgenet	OAN	OpenAgenet/OAN
Internet of Agents	IoA	智能体互联网
Decentralized Identifier	DID	分布式标识符
DID Document		DID 文档
did: oan Method		did: oan 方法
Verifiable Credential	VC	可验证凭证
Verifiable Presentation	VP	可验证呈现

英文全称	缩略语	中文名称
ResourcePackage		资源包
Resource Metadata		资源元数据
oanMetadata		OAN 资源元数据对象
Resource Controller		资源控制方
Resource Provider		资源提供方
Agent Service		智能体服务
Skill		技能包
MCP Server		MCP 服务
Tool/API		工具 API
Registrar Service Node		注册服务节点
Discovery Service Node		发现服务节点
Root Platform		根平台
Content Delivery Network	CDN	内容分发平台
Trust Indexer		链下信任索引器
User Agent		用户助手智能体
Model Context Protocol	MCP	模型上下文协议
Agent2Agent Protocol	A2A	Agent2Agent 协议
A2A Agent Card		A2A 智能体卡片
Software Development Kit	SDK	软件开发工具包
Application Programming Interface	API	应用程序编程接口
Hypertext Transfer Protocol	HTTP	超文本传输协议
JavaScript Object Notation	JSON	JavaScript 对象表示法
Request for Comments	RFC	RFC 技术规范文档
Root-verified ResourcePackage		根平台验证资源包
Root Proof		根平台可信发布证明
Registration Verifiable Credential		注册凭证
Resource DID		资源 DID
Local Identity		本地身份
Node Authorization		节点授权
Governance Status		治理状态
Resource Version		资源版本
Service Endpoint		服务端点
Capability Tag		能力标签
Capability Tag Tree		能力标签树
Semantic Discovery		语义发现
Structured Discovery		结构化发现
Authorization Domain		授权域
Control Plane		控制面
Data Plane		数据面
Semantic Plane		语义面
Governance Authority		治理权威
Verifiable Control Proof		可验证控制证明
Public HTTP API		公共 HTTP API
Message Authentication Code		消息认证码
Resource Type		资源类型
Subject Type		主体类型
implementationLinks		实现链接集合
Protocol Binding		协议绑定

英文全称	缩略语	中文名称
Publication Cursor		发布游标
Publication Sequence		发布序列
Governance Event Sequence		治理事件序列
Synchronization Cursor		同步游标
Freshness		新鲜度
Governance Event		治理事件
Publication Fact		发布事实
Discovery Index		发现索引
Data Integrity Proof		数据完整性证明
Public Key Multibase		多重编码公钥
Crypto Suite		密码套件
Nonce		随机数
Hash Algorithm		哈希算法
Package Claims		资源包声明
Protocol Version		协议版本
Version Constraint		版本约束
Previous Version		前序版本
Challenge		控制挑战
Proof		证明
Holder		凭证持有者
Controller		控制者
Credential Subject		凭证主体
Request ID		请求标识
Replay Protection		重放防护
Idempotency		幂等性
Rate Limiting		速率限制
Signature		签名
Discovery Query		发现查询
Query Projection		查询投影
Exact Lookup		精确查找
Lexical Search		词法检索
Hybrid Search		混合检索
Search Score		检索分数
Query Explanation		查询解释
Pagination Cursor		分页游标
Publication Manifest		发布清单
Notification Batch		通知批次
Publication Job		发布任务
Watermark		水位
Backoff		退避
Signed Request Envelope		签名请求封装
DID Control Challenge		DID 控制挑战
Subject Control Proof Bundle		主体控制证明包
Registration Submission		注册提交
Discovery Response		发现响应
Index Snapshot		索引快照
Query Fallback		查询降级
Protocol Envelope		协议信封

英文全称	缩略语	中文名称
Capability Vocabulary		能力词汇
Governance Projection		治理投影
Sync Batch		同步批次
Correlation ID		关联标识
Checkpoint		检查点
Event Cursor		事件游标
Capability Set		能力集合
Content Digest		内容摘要
Audience		受众
Cross-Origin Resource Sharing	CORS	跨源资源共享
Entity Tag	ETag	实体标签
Server-Side Request Forgery	SSRF	服务端请求伪造
Web Cryptography API		Web 密码学 API
OpenAPI Specification		OpenAPI 接口描述规范
Node Package Manager	npm	npm 包管理器
Base64url		Base64 URL 安全编码
Base58		Base58 编码
JSON Web Key	JWK	JSON Web 密钥
Multibase		多重基编码
Ed25519		Ed25519 签名算法
SM2		SM2 签名算法
SHA-256		SHA-256 哈希算法
SM3		SM3 哈希算法

本规范中的 DID 是分布式标识符，did:oan 是 OAN 使用的方法标识。DID 文档是与 DID 对应的可解析文档，包含验证方法、服务端点及资源相关元数据；DID 控制关系由可验证的密钥和签名证明。VC 是可验证凭证，用于表达注册、节点授权或其它由签发者承担责任的声明；VP 是为验证目的组织的凭证呈现。资源包是绑定资源版本、元数据、哈希和根平台发布证据的可分发对象。

注册服务节点负责资源接入、请求校验、控制证明验证和向根平台提交；发现服务节点负责同步根平台发布内容、建立索引和返回签名发现响应；根平台承担信任授权、数据分发和语义治理三个枢纽职责；内容分发平台是承载验证后材料的分发层，不是信任权威。MCP Server、A2A 和 Tool/API 分别表示资源类型或外部协议语境中的服务与接口，术语相近不代表它们拥有相同的生命周期和权限。

统一术语表中的名称应直接用于正文、接口说明和示例。例如，资源登记材料可以同时出现资源类型、资源 DID、控制证明和服务端点，但“资源类型”不是资源 DID，“服务端点”也不是资源控制身份；MCP 是外部协议名称或资源类型语境中的协议标识，不应被泛化为所有工具资源的同义词。若一个外部协议使用了自己的对象名称，应在首次映射时说明对应关系，后续仍按第 2.1.1 节的中文名称书写。

3.2 资源控制方与资源提供方

资源控制方掌握资源 DID 对应的控制密钥，负责证明自己有权创建、登记或更新该资源的 DID 文档和资源版本。资源提供方负责提供实际能力、资源内容、服务端点、协议绑定和使用说明，并对这些声明承担业务和运行责任。在个人或小型部署中，两者通常是同一主体；在组织场景中，控

制方可以是组织或平台，提供方可以是受其委托运营服务的团队或系统。

注册服务节点和根平台只验证控制方对 DID 的控制证据以及提交材料的一致性，不因资源提供方能够访问端点就推定其拥有 DID 控制权，也不因控制方持有私钥就保证业务端点真实可用。资源元数据应明确两者关系；关系发生变化时，应通过授权更新、所有权转移或新的资源版本规则表达，而不能只修改展示名称。

```
{
  "resourceDid": "did:oon:example-resource",
  "controller": "did:oon:example-controller",
  "provider": {
    "name": "示例团队",
    "endpoint": "https://example.invalid/resource"
  },
  "controlProof": {
    "createdBy": "controller",
    "verification": "signature over resourceDid and version"
  }
}
```

该示例表达控制方和提供方可以分离：签名证明由控制方产生，端点和能力声明由提供方负责。它不表示提供方自动取得资源控制权，也不表示控制证明能够保证端点的业务质量。

3.3 主体身份与本地身份

主体身份是资源控制关系所依附的身份语境，本地身份是客户端侧保存主体配置、控制密钥、资源密钥和必要凭证的材料集合。创建本地身份时，密钥应在用户可控制的客户端环境生成或导入；注册请求只发送完成验证所需的 DID 文档、公钥、签名和公开凭证，不发送资源控制私钥或主体私钥。资源 DID 可以由同一本地身份控制多个资源，但每个资源仍应有独立的 DID、版本和控制证明边界。主体身份是资源控制关系所依附的身份语境，本地身份是客户端侧保存主体配置、控制密钥、资源密钥和必要凭证的材料集合。创建本地身份时，密钥应在用户可控制的客户端环境生成或导入；注册请求只发送完成验证所需的 DID 文档、公钥、签名和公开凭证，不发送资源控制私钥或主体私钥。资源 DID 可以由同一本地身份控制多个资源，但每个资源仍应有独立的 DID、版本和控制证明边界。本地身份更适合被理解为客户端侧的密钥和配置容器，而不是注册服务节点上的账户；它可以关联主体 DID、多个资源 DID、导入节点配置和必要的公开凭证，但不能改变各个 DID 在协议中的独立控制边界。

3.3.1 主体身份与资源控制身份

主体 DID 用于表达控制关系的主体，资源 DID 用于表达可独立注册、发布、发现和更新的资源对象。主体身份与资源身份可以由同一密钥体系建立关联，但二者在协议中不能混为一个标识：主体私钥证明主体能够管理相关资源，资源控制签名证明具体资源 DID 的当前操作获得授权。资源提供方的运行账号、网页登录账号或注册服务节点账号也不能自动替代这两类 DID 控制证据。

3.3.2 本地密钥生成、导入与备份

本地身份流程应支持在客户端新建主体身份、导入既有身份备份、保存资源控制材料并在需要时恢复使用。备份文件可以包含主体 DID、公钥、控制密钥、资源配置和必要凭证，但必须明确文

件的敏感性和保管责任；导入时应校验结构和密钥对应关系，不能把文件名或展示文本当作身份真实性证明。浏览器注册流程中的下载、导入和恢复属于客户端行为，服务端只接收协议验证所需的公开材料。

```
{
  "subjectDid": "did:oon:example-subject",
  "resources": [
    {
      "resourceDid": "did:oon:example-resource",
      "controlKey": "stored locally"
    }
  ],
  "publicMaterial": ["subject public key", "resource public key", "credential references"],
  "privateMaterial": "client-side only"
}
```

这是本地身份备份的概念示例。真实备份格式、加密方式和字段以客户端实现为准；示例中的私密材料不应被上传到注册服务节点、根平台或发现服务节点。

3.3.3 私钥保管与节点可见范围

资源控制私钥、主体私钥和节点运行私钥应由各自控制者保管，注册服务节点、根平台、发现服务节点、内容分发平台和链下信任索引器不应从普通注册或发现请求中取得这些私钥。节点可以看到 DID 文档中的公钥、签名和公开凭证，也可以保存提交记录、发布证明和索引数据，但这些公开证据不能反推出私钥。节点运营者的服务器权限、日志权限和备份策略仍属于部署安全边界，应单独管理。

3.4 根平台

根平台是 OAN 的网络级协调和验证中心，但不是所有业务能力的执行中心。它接收授权的基础设施节点提交，验证注册请求中的身份、控制证明、资源类型、版本、哈希和发布条件，维护可供节点同步的可信发布事实，并协调内容分发和发现服务节点通知。根平台保存和签发的是网络验证所需的证明与状态，不应代替资源提供方持有业务端点的控制密钥。

3.4.1 信任授权枢纽

作为信任授权枢纽，根平台负责依据治理状态确认哪些注册服务节点、发现服务节点和其它基础设施参与者有资格执行相应网络动作，并向符合条件的节点提供可验证的授权材料。该枢纽的判断对象是节点资格、角色和授权范围，不是某个资源的业务质量。节点在运行期还必须检查授权状态是否仍然有效，不能仅凭历史 VC 或曾经成功的请求继续执行受限操作。

根平台在这一枢纽上回答的是“节点能否执行某类网络动作”，而不是“节点返回的每项业务内容都值得信任”。节点授权材料通常需要与节点身份、角色、授权域、有效期和治理状态关联；执行节点在处理受限请求时，应同时核对授权材料和当前状态，不能只检查 VC 的签名是否有效。

3.4.2 数据分发枢纽

作为数据分发枢纽，根平台组织已经验证的资源版本、DID 文档、资源包、哈希、根平台可信发布证明和同步游标，并通过内容分发平台向授权发现服务节点提供可校验材料。内容分发平台可以缓存和批量传输这些内容，但不能修改根平台接受的事实；发现服务节点收到后仍应重新校验哈希、证明、版本和授权域。

```
{
  "publication": {
    "resourceDid": "did:oon:example-resource",
    "version": "1.0.0",
    "packageDigest": "sha256:example-digest",
    "rootProof": "available for verification",
    "publicationSequence": 42
  },
  "distribution": {
    "consumer": "authorized discovery service node",
    "verification": ["packageDigest", "rootProof", "governanceStatus"]
  }
}
```

该示例用于区分根平台形成的发布事实和-content分发平台承担的传输任务。分发成功只说明材料到达，不说明接收方已经完成验证，也不授予发现服务节点超出授权域的可见范围。

3.4.3 语义治理枢纽

作为语义治理枢纽，根平台维护能力标签树及其层级、标识、别名和版本，服务于注册时的标签选择与推荐，以及发现服务节点的语义索引和查询解释。语义治理决定标签如何组织和演进，但不替代资源控制证明或节点授权。发现服务节点可以在根平台词汇基础上使用本地索引和排序方法，但不得通过本地标签扩展自身的授权范围。

能力标签树解决的是“如何用共同词汇描述和查找能力”，不解决“谁控制资源”或“哪个节点有权发布资源”。因此，发现服务节点可以把自然语言任务映射到一个或多个能力标签，也可以返回标签命中和相关性解释，但最终结果仍需关联资源 DID、版本、来源、治理状态和发布证明。

3.4.4 根平台职责边界

根平台负责网络级验证、授权、发布和协调，不负责替资源提供方执行具体业务调用，不保证资源端点的业务质量，也不因资源发布而自动授予调用方业务权限。公开网站、指标、地图和治理事件展示是观察入口，不是根平台事实的替代品。根平台失效时，节点可以依据明确的新鲜度和降级规则提供有限的既有数据，但不得伪造新的接受、授权或发布事实。

3.5 注册服务节点

注册服务节点是资源提供方进入 OAN 的受理和提交边界。它可以提供表单、SDK 或 Skill 支持，检查 DID 语法、资源类型、必填字段、能力标签、端点、哈希和控制证明，并将合格请求包装为向根平台提交的签名请求。注册服务节点签发的注册 VC 或登记结果只证明其处理过相应登记事实，不能代替根平台发布证明，也不能让注册服务节点自行扩大资源的网络可见范围。

注册服务节点必须以自身有效的节点授权和允许的授权域执行受限动作，并保留足以审计的请求标识、验证结果、错误原因和提交状态。它可以拒绝格式不合格或控制权不明的请求；根平台拒绝或异步发布失败时，注册服务节点应如实返回状态，不得把本地受理显示成全网发布完成。

从证据链看，注册服务节点至少产生三类可区分结果：请求是否到达并通过本地校验，是否已经向根平台提交，以及根平台是否接受并发布。客户端或运营人员排查注册问题时，应根据请求标识、资源 DID、版本和提交状态逐级查询，而不能只依据页面上的“提交成功”判断资源已经可发现。

3.6 发现服务节点

发现服务节点是根平台发布内容的授权读侧。它从内容分发平台或根平台通知获取候选资源包，验证根平台证明、哈希、版本和治理状态，按自身授权域过滤资源，并建立结构化和语义索引。它向用户、应用和智能体返回的是经过本地处理的发现响应，因此必须保留资源来源、版本、新鲜度和验证状态，且在支持签名响应时对响应内容负责。

发现服务节点可以优化查询性能、排序和语义匹配，也可以保存旧快照用于有限降级展示，但不得修改根平台的发布事实、把未经授权的资源加入可信索引，或用本地排名覆盖撤销、暂停和版本状态。搜索结果是调用方后续验证的输入，不是自动调用许可。

```
{
  "resourceDid": "did:oon:example-resource",
  "version": "1.0.0",
  "match": {
    "type": "semantic",
    "capabilities": ["code-search", "structure-summary"],
    "score": 0.91
  },
  "source": {
    "discoveryNode": "authorized-node",
    "publicationSequence": 42,
    "freshness": "known"
  },
  "verificationRequired": true
}
```

发现响应中的分数、标签命中和解释属于索引或查询层结果；resourceDid、版本、来源和新鲜度用于后续验证。即使相关性分数较高，也不能绕过 DID 文档、根平台可信发布证明、治理状态和端点检查。

3.7 内容分发平台

内容分发平台承载根平台已验证并允许传播的资源包、DID 文档、元数据、证明、哈希和同步材料，目标是提高节点同步的可达性、批量效率和缓存能力。它是数据面组件，不是治理和身份信任的授权者；任何分发内容都必须由接收方依据根平台证明、哈希和状态重新验证。分发层可以由根平台同一组织运营，也可以采用独立基础设施，但其部署关系不应改变协议中的责任边界。

3.8 链下信任索引器

链下信任索引器是治理链上事实的读侧投影组件。它读取节点授权、授权域变更、暂停、恢复和撤销等治理事件，按序列或游标写入可供根平台、注册服务节点和发现服务节点查询的授权状态。索引结果具有同步延迟和故障可能，必须保留最后处理位置、事件来源、投影时间和错误状态；运行时不能把索引器的“已看到”简单等同于链上最终事实。

根平台或其它执行节点使用索引状态时，应明确失效、过期和回退策略。对于需要阻止未授权节点继续工作的动作，应在状态无法确认或已超过允许新鲜度时采取 fail-closed 行为；对于公开统计或历史展示，可以展示延迟和旧数据，但不得改变实际授权判断。

```
{
  "governanceProjection": {
    "nodeId": "did:oon:example-node",
    "role": "discovery",
    "authorizationDomain": "example-domain",
    "status": "active",
    "eventSequence": 128,
    "source": "governance event stream",
    "projectedAt": "2026-09-09T00:00:00Z"
  },
  "decision": "allow indexed discovery within the authorization domain"
}
```

该投影是链上治理事件的链下读模型，必须保留事件序列和投影时间。若序列中断、状态过期或来源不可验证，执行节点应按敏感动作的失败策略处理，而不是把最后一次成功投影当作永久授权。

3.9 调用方、用户、应用与智能体

用户是发起注册、发现或业务使用意图的人，浏览器或其它客户端是承载本地身份和请求的工具，应用是调用 OAN 接口并执行本地策略的软件，用户助手智能体是可以代表用户查询和发起调用的智能体。它们可以共同出现在一次流程中，但权限来源不同：用户控制本地密钥，客户端保护私密材料，应用验证网络证据，用户助手智能体依据验证结果选择资源。

调用方在发现阶段是依赖方，在实际业务调用阶段还要承担请求签名、凭证呈现、重放防护和本地授权策略。它不得只信任搜索排序、网站显示名称或第三方转发的资源描述；在连接前应验证 DID、VC、根平台证明、治理状态、版本和端点声明，并根据具体业务风险决定是否继续。

3.10 治理权威与授权域

治理权威负责决定基础设施节点是否进入、保持或退出 OAN 的受信任运行范围。授权域是节点获准处理的能力或资源范围，可以约束注册服务节点接收哪些资源、发现服务节点同步和返回哪些资源，以及节点能够执行哪些网络动作。授权域不是资源控制关系，也不是能力标签；标签用于描述和匹配，授权域用于限制权限和可见范围。

治理状态可以由链上事件、根平台运行状态和节点授权材料共同构成，但执行节点必须明确事实来源和优先级。历史授权 VC、缓存状态或人工展示不能覆盖当前撤销或暂停状态；授权域变化传播存在延迟时，节点应按规定的新鲜度和 fail-closed 策略处理敏感动作。

授权域可以用“资源范围 + 动作范围 + 观察范围”理解：资源范围限制节点能处理哪些资源或资源类型，动作范围限制节点能执行注册、同步、索引或管理中的哪些动作，观察范围限制节点能够向哪些查询方或网络区域返回结果。三类范围共同决定节点的有效权限，不能用一个宽泛的 `active` 状态替代。

3.11 信任边界与责任矩阵

OAN 的责任矩阵应沿着“产生事实、验证事实、保存状态、对外声明”四个维度建立。资源控制方产生资源身份和控制证明，注册服务节点验证接入材料并提交登记，根平台验证节点授权和资源版本并形成发布事实，内容分发平台传递已验证材料，发现服务节点验证并按授权域索引，链下信任索引器投影治理状态，调用方验证候选资源并执行本地调用策略。每一项动作都应能指向对应的签名、凭证、哈希、治理事件、接口响应或持久化记录。

3.11.1 资源控制方与资源提供方

资源控制方对资源 DID、控制密钥和更新授权负责，资源提供方对能力描述、资源内容、端点和业务可用性负责。二者可以是同一主体，也可以通过明确的委托或运营关系分离。注册服务节点和根平台只应依据可验证控制证明判断资源操作权限，不能因资源提供方提交了服务地址或因某个账号登录成功，就推定其取得资源 DID 控制权。

3.11.2 注册服务节点、根平台与发现服务节点

三者的 HTTP 接口也对应不同的信任边界：资源发布者向注册服务节点提交 `ResourceRegistrationSubmission`，注册服务节点向根平台提交包含 `registrarDid` 和 `upstreamAuth` 的 `ResourceVerifyAndPublishRequest`，发现服务节点向调用方返回 `ResourceDiscoveryResponse`。前一个接口负责接收资源控制材料，中接口负责证明授权节点在代表网络提交，后一个接口负责提供本地索引候选；接口可达、请求可解析和候选可见分别是不同事实，不能在角色之间相互替代。

注册服务节点负责受理、校验和上游提交，根平台负责节点授权校验、资源验证、可信发布和分发协调，发现服务节点负责对已发布材料进行同步、验证、授权域过滤和查询服务。注册服务节点的登记结果不能替代根平台发布事实，发现服务节点的搜索结果不能改写根平台状态；三者之间的接口、证明和状态应保持可追踪。

3.11.3 调用方与验证方

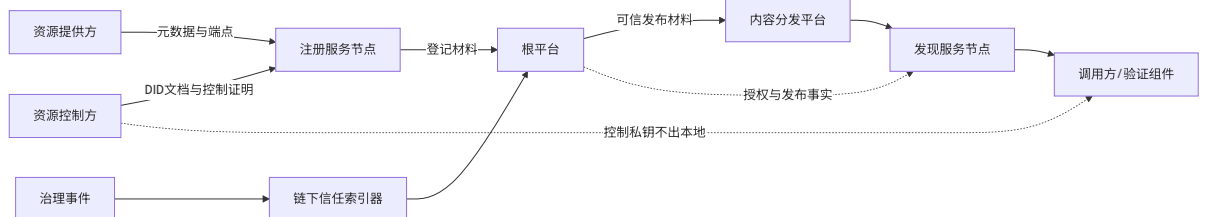
调用方可以由用户、应用或用户助手智能体承担，负责提出查询、接收候选资源并决定是否建立业务连接；验证方可以是调用方自身、SDK 或受其信任的验证组件，负责检查 DID、签名、VC、根平台证明、治理状态、版本、端点和调用策略。验证组件只能提供验证结果，不能未经调用方策略授权直接执行高风险业务动作。

3.11.4 关键动作的责任归属

资源创建和更新归资源控制方，注册受理和登记凭证归注册服务节点，根平台接受和可信发布归根平台，材料传输归内容分发平台，授权域内索引和发现响应归发现服务节点，治理事件投影归链下信任索引器，最终业务调用授权归资源服务端点和调用方本地策略。任何公共网站、统计接口或人工维护的事件表都属于观察和展示层，不能承担上述关键动作的协议权威。

角色之间的边界可以按“谁产生事实、谁验证事实、谁保存状态、谁对外声明”四个问题检查。下表中的“负责”是协议职责，不等同于对全部业务内容或运行环境承担责任。

角色	主要产生的事实	主要验证的事实	对外提供的结果	不替代的职责
资源控制方	DID 文档、控制证明、资源更新	自身密钥和待发布材料	受控资源材料	不能自行声明根平台已发布
资源提供方	能力描述、端点、版本和使用条件	资源业务内容	可供登记的资源说明	不必然拥有 DID 控制权
注册服务节点	登记受理、字段校验、注册凭证	控制证明、字段和接入资格	向根平台提交的登记材料	不替代根平台发布
根平台	节点授权、发布事实、根平台可信发布证明	节点授权、资源版本、哈希和控制证明	可分发的可信发布材料	不替代业务端点授权
内容分发平台	传输、缓存和分发记录	发布材料完整性	可取得的发布产物	不授予资源或节点权限
发现服务节点	本地同步、验证和索引记录	发布证明、授权域和生命周期	授权范围内的发现结果	不改变根平台事实
链下信任索引器	治理状态读模型	链上治理事件和顺序	节点授权状态查询	不创造链上治理事实
调用方/验证组件	本地验证和调用决策	DID、VC、VP、端点和业务策略	是否建立连接的判断	不替代资源服务端业务授权



关键动作发生时，应能够定位到对应的签名、凭证、哈希、治理事件、同步游标或接口响应。若一个结果只有“某节点返回了成功”这一层证据，而无法说明事实来源和验证范围，则只能作为运行状态观察，不能作为跨角色信任证明。

实际排查一个资源或节点问题时，可以按以下顺序建立证据链：先确认标识对象和版本，再确认控制证明或节点授权，然后核对发布事实和治理状态，最后检查发现索引、端点和调用方策略。这个顺序既适用于注册失败、发现结果过期，也适用于节点授权撤销后的传播排查；任何一步缺少来源或无法验证，都应把结论限制在可观察范围内。

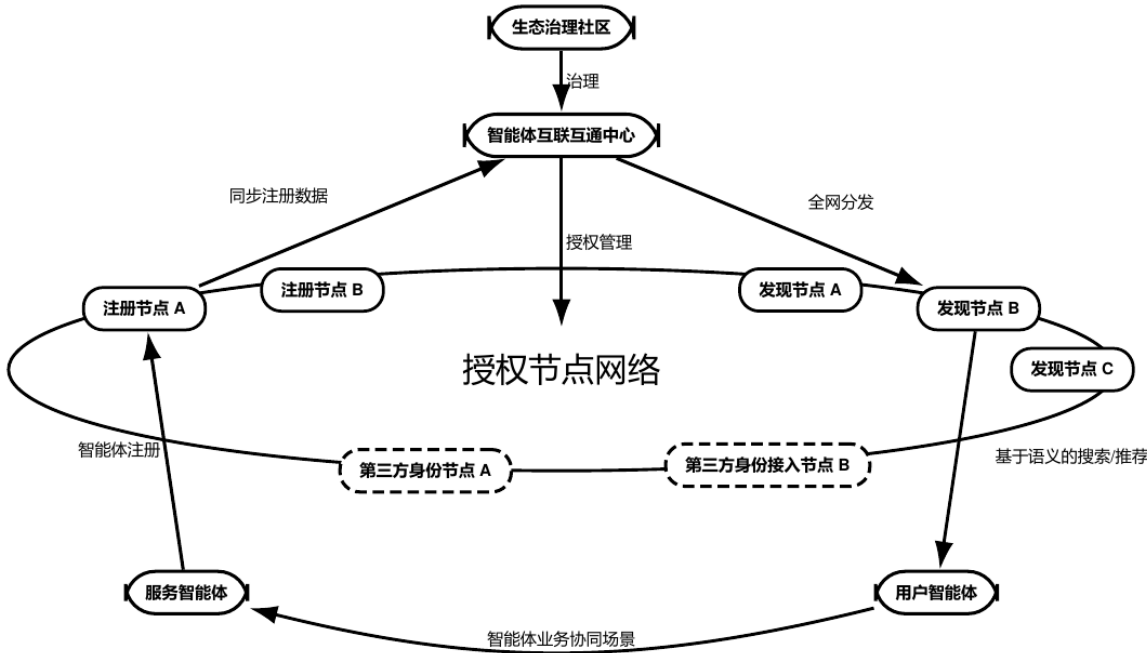
3.12 参考来源

来源	类型	链接
oan-protocol-common	代码仓：共享协议类型、身份对象和信任材料	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：角色、信任边界和验证机制	https://arxiv.org/abs/2606.03163

来源	类型	链接
did:oa DID Method Specification	标准/方法规范: DID 方法和身份对象	https://github.com/OpenAgetnet/oa-public-docs/blob/main/did-oa-specs/doc/OA%20DID%20Method%20Specification.md

第四章 总体架构与网络拓扑

本章说明 OpenAgenet (OAN) 采用“根平台协调、多注册服务节点接入、内容分发平台传播、多发现服务节点索引、链下信任索引器提供治理读模型”的联邦式架构。OAN 网络可以同时接入官方和第三方运行的注册服务节点、发现服务节点，各节点以独立 DID、角色、授权域和运行状态参与网络。资源控制方可以选择合适的注册服务节点提交资源，调用方也可以选择不同发现服务节点查询；根平台统一验证节点资格、资源版本和发布事实，内容分发平台把已验证材料传播给获得授权的发现服务节点，各发现服务节点据此维护自己的结构化与语义索引。



三类流在同一条资源生命周期中相互衔接但不相互替代。数据流承载 DID 文档、资源包、元数据、哈希和证明；控制流承载节点授权、请求认证、发布接受、生命周期和治理状态；语义流承载能力标签树、标签别名、能力描述、用例以及查询解释。任何独立部署都可以调整存储、消息和网络实现，但不得让内容分发平台取得治理权威、让发现服务节点改写根平台发布事实，或让语义相关性替代身份和授权验证。

4.1 逻辑架构

客户端是资源控制方和调用方的主要操作边界，保存本地身份和私钥，生成或导入资源控制材料，并向任一符合其授权域和服务要求的注册服务节点或发现服务节点发起请求。每个注册服务节点独立保存本节点的登记记录、注册凭证和提交状态，将经过校验的资源登记请求封装为提交给根

平台的请求；根平台保存全网授权状态、接受记录、资源版本、发布证明和待处理任务。内容分发平台向多个获得授权的发现服务节点提供根平台已验证的资源包、DID 文档和元数据；每个发现服务节点分别保存自己的同步游标、已验证资源包、拒绝记录、结构化索引和语义索引。链下信任索引器保存链上治理事件到节点授权状态的投影，为第三方节点接入、停权和授权域变更提供统一读模型。

逻辑连接可概括为：客户端到注册服务节点是资源接入流，注册服务节点到根平台是受保护的上游提交流，根平台到内容分发平台是发布数据流，内容分发平台到发现服务节点是同步流，发现服务节点到客户端是查询响应流，链上治理系统到链下信任索引器再到执行节点是授权状态流。根平台同时向发现服务节点提供发布通知或摘要，使发现服务节点能够按游标取得当前资源包，而不是依赖注册服务节点直接把资源写入发现索引。

一个部署实例可以用以下逻辑配置理解组件边界。字段名称仅用于说明角色关系，实际环境中的域名、端口、数据库和消息组件属于部署配置，不是协议固定值。

```
network:
  root_platform:
    role: root
    publishes: [node-authorization, resource-publication, capability-vocabulary]
  registrar_nodes:
    - id: registrar-a
      role: registrar
      submits_to: root_platform
      operator: official
    - id: registrar-b
      role: registrar
      submits_to: root_platform
      operator: third-party
  discovery_nodes:
    - id: discovery-a
      role: discovery
      syncs_from: root_platform
      operator: official
      authorization_domain: domain-a
    - id: discovery-b
      role: discovery
      syncs_from: root_platform
      operator: third-party
      authorization_domain: domain-b
  trust_indexer:
    role: governance-read-model
    projects: governance-events
```

该拓扑以节点角色和授权关系组织网络，而不是以某一组固定服务实例限定网络规模。第三方运营者完成节点身份、授权域、接口、安全和符合性检查后，可以运行新的注册服务节点或发现服务节点；新节点通过根平台认可的共同发布事实和治理状态与现有节点协作，同时保留独立部署、存储、索引和服务策略。

4.2 根平台作为信任授权枢纽

根平台的信任授权枢纽职责针对基础设施参与者，而不是针对资源控制方的业务质量。根平台应维护能够验证的节点身份、节点角色、授权域和治理状态，并在处理注册提交、发布任务、内容分发和发现通知时检查相关节点是否仍处于有效授权状态。链下信任索引器为根平台提供链上事件的读侧投影，但投影存在同步延迟；涉及高风险写入时，根平台应依据明确的新鲜度、缺失和失效规则决定接受、拒绝或暂停处理。

节点凭证证明“该节点具有某类网络资格”，资源控制证明证明“请求方能够代表某个资源 DID”，根平台发布证明证明“某个资源版本已经通过根平台验证”。三者覆盖的事实不同，不能由其中任何一项替代另外两项。授权检查至少应绑定节点 DID、角色、授权域、状态、有效期或治理序列，并将验证结果写入可审计的接受、拒绝或待处理状态。

根平台授权判断的最小输入可以抽象为：

```
{
  "nodeDid": "did:oon:example-registrar",
  "role": "registrar",
  "authorizationDomain": "example-domain",
  "governanceStatus": "active",
  "requestedAction": "submit-resource-registration",
  "observedAt": "2026-09-09T00:00:00Z"
}
```

输出应至少能区分 `allow`、`deny` 和 `indeterminate`。`indeterminate` 表示授权状态、治理投影或关键证明暂时无法确认，不应被静默转换为允许；具体接口状态码和错误结构由注册、治理与 API 章节说明。

4.3 根平台作为数据分发枢纽

根平台在资源版本通过验证后，组织 DID 文档、资源元数据、资源包、DID 文档哈希、元数据哈希、资源包哈希、资源类型、版本、生命周期状态、授权域和根平台可信发布证明，形成可被内容分发平台承载的发布产物。发布产物必须具有明确的版本和哈希边界，发现服务节点取得材料后能够独立重算哈希、校验证明声明并判断当前版本，而不依赖内容分发平台的口头或接口状态。

不同发现服务节点的一致性不是要求它们拥有相同的本地数据库，而是要求它们对同一根平台发布事实使用相同的资源版本、哈希、证明和治理状态作校验。根平台通过发布游标、资源 DID 和版本建立可追踪顺序；内容分发平台可采用缓存、批量或重试机制提高传输效率，但不得把旧版本冒充当前版本，也不得在未取得新发布事实时修改资源包内容。

```
{
  "publicationArtifact": {
    "resourceDid": "did:oon:example-resource",
    "version": "1.2.0",
    "didDocumentDigest": "sha256:did-document",
    "metadataDigest": "sha256:metadata",
    "packageDigest": "sha256:package",
    "publicationSequence": 108,
    "status": "published"
  },
}
```



```
"consumerCheck": ["digest", "version", "rootProof", "governanceStatus"]
}
```

该对象表示可分发布产物的逻辑组成，不代表所有部署必须采用完全相同的字段名。关键是接收方能够把资源 DID、版本、各类摘要和根平台可信发布证明绑定起来，并在本地校验后再写入读模型。

4.4 根平台作为语义治理枢纽

根平台维护能力标签树及其版本、层级、标签标识、中文或英文标签名、别名和父子关系，并将这套受治理词汇同时提供给注册服务节点和发现服务节点。注册时，客户端可以根据资源描述选择或获得标签推荐；发现时，发现服务节点可以把用户的自然语言任务、标签条件和结构化过滤转换为本地查询条件。标签树用于统一语义边界和授权域判断，不代表根平台替资源提供方评价业务质量。

语义治理与语义检索分工明确：根平台维护共享词汇和治理事实，发现服务节点负责本地索引、分词、向量或其它排序实现。发现服务节点可以保留本地语义别名和排序信号，但对外返回结果时仍应保留资源来源、版本、授权域和验证状态；相关性高的结果如果缺少有效控制证明、根平台发布证明或治理状态，也不能进入可信发现结果。

标签治理可以用版本化的词汇条目表达：

```
{
  "vocabularyVersion": "2026-09",
  "tag": {
    "id": "code-repository-analysis",
    "preferredLabel": "代码仓库分析",
    "aliases": ["代码检索", "项目结构总结"],
    "parent": "software-development",
    "status": "active"
  }
}
```

发现节点可以根据别名和自然语言查询生成候选条件，但返回结果仍需引用资源 DID、发布版本和验证状态。标签词汇的版本变化影响查询解释和排序，不应悄悄改变已经签署资源包中的原始身份和摘要。

4.5 注册节点到根平台的发布路径

一次正常发布由本地准备、注册服务节点接入校验、注册服务节点签名提交、根平台认证与验证、资源版本接受、内容分发和发现通知组成。注册服务节点可以返回登记结果或注册凭证，但只有根平台完成资源版本验证并产生发布事实后，资源才具备进入传播链路的条件。发布任务通常是异步的，登记响应、根平台接受、内容分发完成和发现服务节点索引完成应分别记录和展示。

失败处理必须保留失败所在阶段。客户端字段错误、DID 文档结构错误、控制证明错误属于接入或验证失败；注册服务节点未获授权、上游签名过期或请求体哈希不一致属于节点认证失败；根平台资源类型、版本、哈希或治理条件不满足属于发布拒绝；内容分发或发现索引暂时不可用属于传播阶段失败。只有可安全重试的暂时性失败才能自动重试，重试请求仍应具备新鲜度和幂等约束。

客户端 -> 注册服务节点：资源材料与控制证明
注册服务节点 -> 根平台：节点签名的上游提交
根平台 -> 注册服务节点：accepted / rejected / pending
根平台 -> 内容分发平台：已验证发布产物
内容分发平台 -> 发现服务节点：发布材料与游标

这条链路中的每个响应只说明当前阶段的结果。例如 `accepted` 可以表示根平台接受了资源版本，但不表示发现服务节点已经完成索引；`pending` 表示任务仍在异步推进，调用方和运营者应继续依据状态接口或同步记录判断后续结果。

4.5.1 注册节点接入校验

注册服务节点首先检查请求是否符合资源注册模型，包括资源 DID、DID 文档、资源类型、主体类型、资源名称和描述、能力标签、用例、协议绑定、服务端点、授权域、版本和哈希等字段的一致性。它还应验证资源 DID 与 DID 文档主体、控制密钥及控制证明的对应关系，并确认提交的资源类型与 DID 语义编码一致。推荐标签可以辅助填写，但不能替代必填字段和控制权证明。

4.5.2 上游提交与身份认证

注册服务节点向根平台提交的不是未经保护的表单，而是包含资源登记内容、请求标识、协议版本、目标路径、时间戳、随机数、请求体哈希、发送方 DID 和签名的受保护请求。根平台据此验证注册服务节点的节点身份、授权域和请求新鲜度，再验证其中的资源登记凭证、资源控制证明和资源包声明。提交方私钥只在注册服务节点自身的节点环境中使用，资源控制私钥仍由资源控制方保管。

4.5.3 根平台验证与发布确认

根平台在验证阶段同时检查节点授权、资源 DID 文档、资源控制证明、注册凭证、资源类型绑定、元数据哈希、资源包哈希、版本关系、授权域和生命周期条件。通过后，根平台记录资源版本接受事实，生成包含资源 DID、版本、哈希、状态、授权域及相关治理或发布引用的根平台可信发布证明，并将后续内容分发和发现通知任务置为可执行。该确认不等于发现服务节点已经完成索引。

4.5.4 失败、拒绝与重试

根平台应对重复请求区分相同资源 DID、相同版本和相同哈希的幂等重试，以及相同 DID 下内容或版本发生变化的更新请求。前者可以返回既有接受结果或既有发布状态，后者必须重新验证控制权、版本关系和发布条件。签名失效、随机数重放、节点授权失效、哈希不匹配和资源类型不一致不得通过简单重试绕过；网络超时、下游暂时不可用等暂时性错误可以进入带退避的任务重试，并保留原请求关联关系。

4.6 根平台到发现节点的分发路径

根平台通过内容分发平台或受保护的同步接口向发现服务节点提供发布内容和同步位置。发现服务节点依据自身保存的游标请求增量材料，也可以在游标缺失或需要恢复时执行受控回填。对每个候选资源，发现服务节点必须取得当前资源包，验证 DID 文档、元数据、哈希、根平台可信发布

证明、生命周期状态和授权域，再将通过验证的版本写入本地索引；验证失败的材料应进入可追踪的拒绝记录而不是静默丢弃。

同步链路的最终目标是让发现服务节点形成“本地可查询、来源可追溯、状态可验证”的读模型。发现索引可以使用关系表、全文索引、语义索引或缓存，但索引记录应保留资源 DID、版本、包哈希、发布时间或游标、来源和验证状态。查询响应中的排序和摘要是派生信息，不能覆盖根平台发布事实。

4.6.1 分发内容与发布检查点

可分发内容至少包括资源 DID、资源类型、资源版本、DID 文档、资源元数据、资源包哈希、元数据哈希、DID 文档哈希、授权域、生命周期状态、根平台可信发布证明及必要的发布游标或来源引用。发布检查点应能够回答资源是否已被根平台接受、内容是否已写入分发层、发现服务节点是否已经取得并校验，以及失败停留在哪一个阶段。

```
{
  "resourceDid": "did:oon:example-resource",
  "version": "1.2.0",
  "checkpoints": {
    "rootAccepted": true,
    "publishedToDistribution": true,
    "discoveryFetched": false,
    "discoveryVerified": false,
    "discoveryIndexed": false
  },
  "lastError": null
}
```

该检查点对象用于解释传播进度，不是把多个状态压缩成一个 `available` 字段。发现服务节点尚未取得或校验时，资源可以已经完成根平台发布，但不应被描述为已在该节点可发现。

4.6.2 发现节点同步与索引

发现服务节点取得材料后，先完成来源、完整性和治理校验，再写入已验证资源包表或等价存储，随后更新结构化索引和语义索引。结构化索引可以支持按 DID、资源类型、版本、能力标签和授权域查询；语义索引可以使用能力描述、标签别名、用例和协议绑定进行匹配。任何索引写入都应验证结果关联，不能先展示、后补做信任校验。

4.6.3 序列号、新鲜度与收敛

发布游标用于表达根平台发布材料的传播顺序，发现服务节点同步游标用于表达该节点已经处理到的位置，二者不等价。发现服务节点应记录最后成功应用的游标、收到材料的时间、资源版本和校验结果，并根据允许的新鲜度判断旧快照是否还能用于有限展示。游标前进表示同步位置前进，不表示每一个资源都已经可以调用。

```
{
  "nodeId": "did:oon:example-discovery",
  "rootPublicationCursor": 108,
  "localAppliedCursor": 105,
  "lastSuccessfulSyncAt": "2026-09-09T00:00:00Z",
}
```

```
"freshness": "lagging",  
"pendingRanges": [[106, 108]]  
}
```

当本地游标落后时，节点可以报告滞后并继续处理已验证旧快照，但不能把 `localAppliedCursor` 伪装成根平台最新发布序列。缺口恢复完成后，应重新校验缺失范围内的每个发布材料，再推进本地游标。

4.6.4 分发失败与恢复

内容分发失败、网络中断、资源包暂时不可获取和发现服务节点数据库故障都应支持恢复。可恢复任务应保留资源 DID、目标版本、来源游标和重试信息；遇到哈希不匹配、证明不匹配或治理状态无效等确定性失败时，应转入拒绝或隔离状态，等待新的发布材料或治理变化，而不是无限重试。恢复后应从持久化游标或受控回填点继续，避免跳过发布记录。

4.7 多个注册节点与发现节点

多个注册服务节点可以面向不同组织、资源类型或授权域提供接入，多个发现服务节点可以面向不同网络区域、用户群或查询策略提供索引。它们共享根平台对身份、授权、资源版本和发布证明的判断基础，但不必共享同一套数据库或本地排序。资源在多个注册服务节点重复提交时，根平台应依据资源 DID、版本、哈希和控制证明识别幂等请求或更新请求；发现服务节点重复收到同一发布材料时，应保持索引幂等。

联邦化不意味着任意节点之间互相写入。注册服务节点只能在其授权域内受理和提交，发现服务节点只能同步和返回其授权域内允许的已发布资源；节点之间的传播优先经过根平台发布事实和-content分发路径。一个发现服务节点故障不会使根平台事实消失，另一个已完成同步的发现服务节点可以继续提供结果，但故障节点的本地可见性和新鲜度应如实标识。

4.7.1 多注册节点接入

根平台为不同注册服务节点维护独立的节点身份、角色和授权域。不同注册服务节点可以提交同一资源，但不能通过更换注册服务节点获得新的资源控制权；同一资源的更新仍必须由资源控制方提供有效控制证明，并满足版本更新规则。注册服务节点的本地登记记录属于受理侧证据，根平台接受记录才是网络发布判断的依据。

4.7.2 多发现节点同步

每个发现服务节点拥有独立同步游标和本地索引状态，可以按自身授权域从内容分发平台取得同一根平台发布事实。节点之间不应直接把本地搜索结果当作对方的权威数据；需要比较一致性时，应比较资源 DID、版本、包哈希、根平台证明和治理状态。不同节点短时间内结果数量或排序不同，可能来自同步延迟或本地策略，但不得来自对根平台事实的任意改写。

4.7.3 节点间数据一致性

节点间一致性应区分事实一致性和读模型一致性。事实一致性要求同一发布版本的 DID 文档、元数据、资源包哈希和根平台证明一致；读模型一致性允许不同节点在索引时刻、排序、缓存和语义

解释上存在差异，但差异必须可由游标、新鲜度或授权域解释。发现服务节点发现版本冲突时，应优先依据根平台发布序列和证明进行判断，不能以本地到达时间覆盖可信来源。

比较两个发现服务节点时，可以优先比较以下稳定字段，而不是直接比较结果数量或排序：

(resourceDid, version, packageDigest, rootProof, governanceStatus)

这些字段一致时，通常说明两节点对同一发布事实的核心理解一致；索引时间、语义分数、分页位置和缓存时间不同，属于读模型差异。若稳定字段不一致，应回到根平台发布记录和治理事件序列定位原因。

4.7.4 节点故障与隔离

节点出现签名异常、授权过期、持续同步失败、发布材料篡改或接口行为不符合要求时，应能够被隔离，不再接受其受限写入或不再向其提供新的授权范围。隔离不等于删除历史事实；根平台和其它节点仍应保留可审计记录，并根据治理状态和恢复检查决定是否重新启用。普通查询节点故障可以由其它节点承接，但不能因此扩大其它节点的授权域。

4.8 第三方节点接入模型

第三方节点接入应从节点身份材料和运行角色开始，而不是从开放一个 HTTP 端口开始。申请方应提交节点 DID 文档、公开验证方法、服务端点、拟承担的节点角色、授权域和运行联系人或运维材料；治理方或根平台据此完成资格审核和授权。获得测试资格后，节点需要通过身份、签名、接口、资源包验证、同步、错误处理和状态更新测试，正式节点再按照正式授权域提供服务。

测试节点和正式节点应在授权状态、可见范围、数据保留和网络影响上隔离。测试节点可以使用测试资源和测试治理状态验证实现，不得把测试发布事实混入正式发现索引；正式节点的授权撤销、暂停或范围变化应能够在运行期被节点识别并影响受限操作。

4.8.1 接入前授权

接入前授权至少应确认节点 DID 可解析、验证方法可用、节点角色明确、端点归属清楚、请求签名可验证、授权域可表达，并且节点运营者能够说明密钥保管、数据存储和故障处理方式。授权材料应绑定节点身份和角色，不应仅凭域名、IP 地址或人工名称作为节点资格证明。

4.8.2 节点能力与授权范围

节点能力描述应区分“能够提供的接口”和“被授权执行的动作”。一个发现服务节点可能支持结构化查询和语义查询，但只被授权索引某些资源域；一个注册服务节点可能支持四类资源接入，但只获准受理其中部分类型。授权范围应能够映射到节点实际过滤、提交和响应逻辑，超出范围的请求必须拒绝或不返回。

4.8.3 协议符合性检查

符合性检查应覆盖健康检查、DID 文档解析、签名和哈希验证、节点授权状态、资源注册、根平台发布、内容分发、发现同步、查询响应和错误契约。测试结果应记录输入、节点版本、配置、预期结果、实际结果和证据位置；通过接口连通性测试不等于通过信任边界测试。

第三方节点的最小接入记录可以按以下结构组织，实际字段由接入流程确定：

```
node:
  did: did:oon:example-third-party-node
  role: discovery
  endpoint: https://example.invalid/discovery
  authorizationDomain: example-domain
  testProfile: discovery-node-v1
  checks:
    - did-resolution
    - signature-verification
    - publication-sync
    - query-response
  status: test-passed
```

该记录只能说明接入评估的范围和结果，不能把测试通过直接扩大为长期运行保证；正式运行仍需检查授权状态、依赖服务和持续同步状态。

4.8.4 运行期状态管理

正式节点运行期间应持续检查自身授权、治理状态、依赖端点、同步游标、任务队列和数据库状态。授权暂停或撤销后，节点应停止相应受限动作；内容分发或链下信任索引器暂时不可用时，节点可以按新鲜度规则使用既有状态，但不得伪造新的授权、接受或发布事实。状态恢复应具有可观察的重新同步和重新校验过程。

4.9 公开网络、私有网络与联盟网络部署

公开网络允许互联网用户访问公开注册、发现或解析入口，但公开可访问不等于所有资源或所有节点状态都对任何用户可见。私有网络可以把注册服务节点、发现服务节点、内容分发平台和资源端点限制在组织网络或身份域内，仍然使用相同的资源身份、控制证明和发布验证边界。联盟网络可以由多个组织共同运营节点，在约定的根平台、治理权威和授权域下交换资源发布事实。

三种部署形态可以共用协议对象，但应明确网络范围、域名和端点、证书、授权材料、数据留存和跨域传播条件。部署环境的域名、节点数量和数据库选型属于运行配置，不应写入资源 DID 或根平台证明作为不可变协议条件；真正需要互操作的是身份、签名、资源包、哈希、状态和节点授权。

部署形态	主要可达范围	适合的资源传播边界	仍需保持的共同事实
公开网络	互联网用户或公开客户端	公开授权域内的资源和发现结果	DID、控制证明、根平台发布证明和治理状态
私有网络	组织内部用户和系统	内部授权域或业务域	资源版本、哈希、节点角色和验证顺序
联盟网络	多个组织的授权成员	按联盟规则传播的资源和语义词汇	共享信任根、节点授权和跨域验证边界

部署形态改变的是可达性和数据传播范围，不改变资源控制权的证明方式，也不把网络入口本身变成信任权威。

4.10 可用性、隔离性与故障边界

OAN 的降级策略必须区分读侧展示和信任敏感动作。发现服务节点可以在分发服务暂时不可用时展示带新鲜度标记的旧快照，运营页面可以展示延迟的统计或历史事件；但资源注册、根平台发布、节点授权变更、撤销状态应用和高风险调用前验证在无法确认关键证据时应拒绝、暂停或进入待处理状态。任何降级都不得生成新的签名事实或改变资源的生命周期状态。

网络中断、数据库故障和索引滞后应分别记录。恢复时，组件应从持久化任务、游标或事件位置继续，重新校验跨节点材料，避免把缓存中的旧版本直接当作最新版本。隔离边界包括节点身份、授权域、资源版本和数据存储边界；一个节点的故障或错误数据不应自动扩散为其它节点的信任事实。

故障现象	读侧允许的行为	信任敏感动作	恢复依据
内容分发暂时不可用	展示标记新鲜度的旧快照	暂停新的可信发布或未验证传播	分发恢复、哈希校验和发布证明
治理投影延迟	展示延迟状态或历史数据	对授权变更采取保守处理	治理事件序列和投影游标
发现索引故障	由其他节点提供结果或显示不可用	不生成新的可信索引结论	数据库恢复和重新索引
节点身份/签名异常	可以保留审计记录	拒绝受限写入并隔离节点	授权恢复和重新接入测试

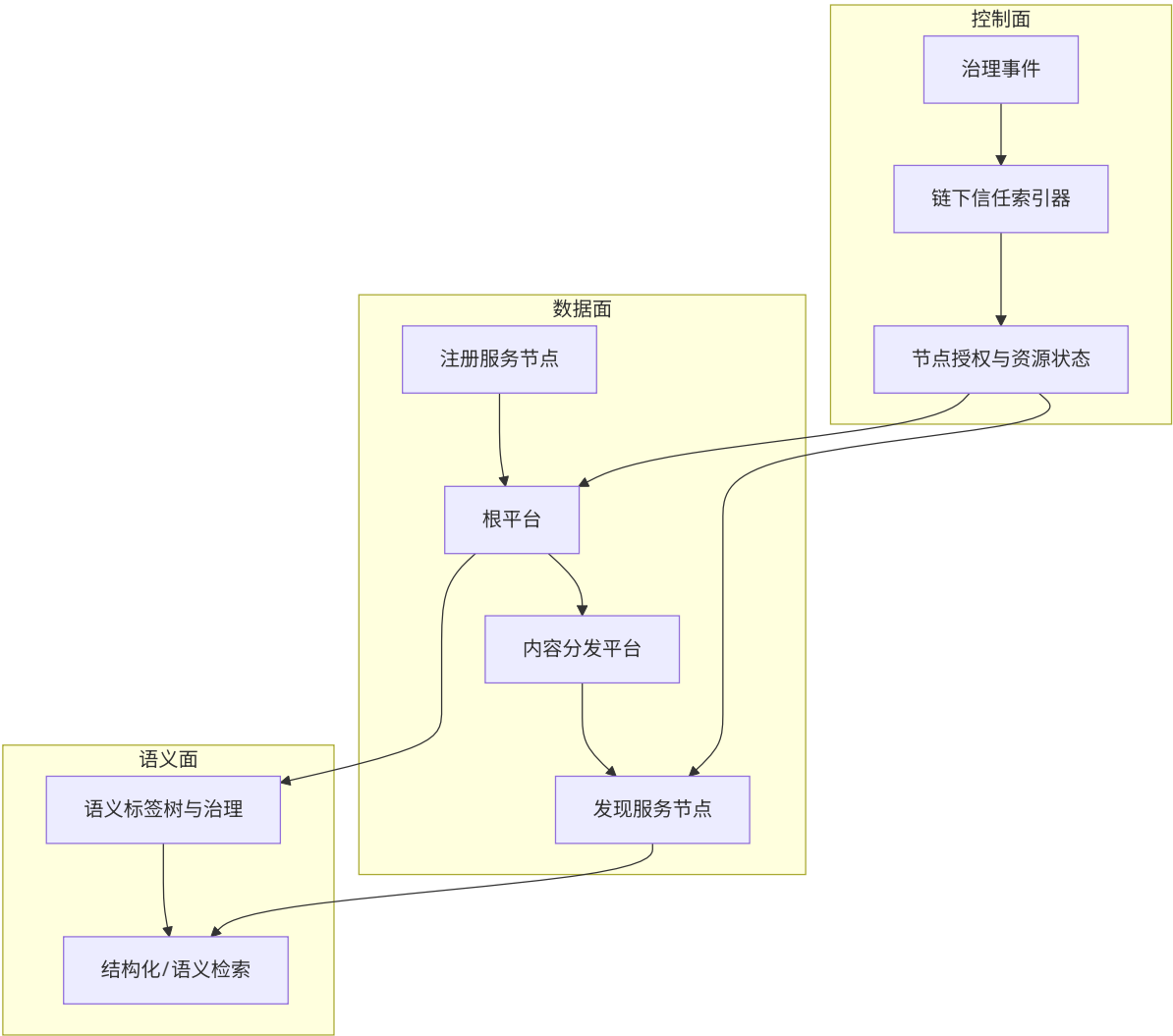
4.11 控制面、数据面与语义面的交互

控制面先决定节点和资源版本是否具备进入网络、传播和被视为有效的条件；数据面再传输控制面允许发布的 DID 文档、资源包、元数据、哈希和证明；语义面对已经通过身份和发布检查的资源建立能力标签、描述和用例索引。查询可以同时使用结构化条件和语义条件，但结果必须携带或可追溯到资源 DID、版本、来源、根平台证明和治理状态。

治理状态变化会影响数据面和语义面的继续传播：节点撤销或资源版本失效后，新的同步应停止或过滤相应材料，既有索引应在状态传播后更新；标签树变更可以影响注册推荐和查询解释，但不能追溯性地改变已签署资源包的身份和哈希。三平面之间的依赖顺序是控制事实约束数据传播，数据材料支撑语义索引，语义结果反过来不能授予控制权限。

三类平面的边界也决定了故障处理顺序：控制面状态未知时，数据面可以保留已验证材料用于有限读取，但不得继续扩大传播；数据面材料缺失或哈希不一致时，语义面不得建立或更新可信索引；语义索引不可用时，仍可提供 DID 精确查询或其它已明确支持的结构化查询。恢复后应先恢复控制状态和数据完整性，再重建或刷新语义索引。

三类平面及其依赖关系如下。实现上，三类平面之间传递的不是未定义的自由 JSON，而是由共享 Rust 类型序列化形成的 JSON/HTTP 载荷：资源注册使用 `ResourceRegistrationSubmission`，注册服务节点向根平台转发时包入 `ResourceVerifyAndPublishRequest`，内容分发使用 `ResourceCdnPublishRequest` 或 `ResourceCdnBatchPublishRequest`，发现查询使用 `ResourceDiscoveryQuery` 并返回 `ResourceDiscoveryResponse`。这些类型规定字段名称和嵌套关系，Axum 路由规定方法与路径，签名包络和资源控制证明规定不同信任边界；三者共同构成节点协作契约。



一次跨平面查询的返回值应同时说明“为什么找到”和“为什么可以被使用”。例如，发现服务节点可以先用能力描述、能力标签和结构化资源类型筛选资源，再关联资源 DID、版本、资源包哈希、根平台可信发布证明和当前治理状态：

```
{
  "resourceDid": "did:oon:SKDM:example-resource",
  "match": {
    "mode": "semantic_and_structured",
    "query": "检索代码仓库并总结项目结构",
    "matchedTags": ["code-repository", "project-structure-summary"]
  },
  "trust": {
    "governanceStatus": "active",
    "rootProof": "present",
    "packageDigest": "sha256:..."
  },
  "freshness": {
    "indexedAt": "2026-09-09T08:30:00Z",
    "sourceCursor": "publish-1842"
  }
}
```

上例中的语义匹配只能解释资源与查询的相关性，`governanceStatus`、`rootProof` 和 `packageDigest` 才分别支撑状态判断、可信发布判断和内容完整性判断。任一信任字段缺失时，发现服务节点可以返回资源，但必须降低结果状态或要求调用方继续验证，不能把语义相关性直接解释成授权或安全保证。

4.12 事实来源与派生读模型

资源控制关系的事实来源是资源 DID 文档、控制密钥和有效控制证明；注册受理事实来源是注册服务节点的登记记录或注册凭证；根平台接受与发布事实来源是根平台持久化记录和根平台可信发布证明；分发事实来源是内容分发平台保存的发布产物及其哈希；发现可见性来源是具体发现服务节点的同步、校验和索引记录；节点授权状态来源是治理事件及其链下信任索引器投影。官网页面、缓存接口和统计数据只能作为观察层，不是上述事实的唯一权威。

派生读模型必须保留来源引用、版本或游标、校验时间、新鲜度、授权域和错误状态。读模型可以为了查询效率合并字段、建立全文或语义索引，但不能丢失足以重新验证资源的 DID、版本、哈希和证明引用。读模型与权威来源不一致时，应显示同步或校验异常，不能用本地数据库的存在覆盖权威状态。

网络事实	权威来源	派生节点必须保留的引用	出现缺口时的处理
资源控制关系	DID 文档与控制证明	DID、验证方法、文档哈希	拒绝或待验证
注册受理	注册服务节点记录/注册凭证	节点身份、资源 DID、时间和状态	不得写成已发布
根平台发布	根平台记录与根平台可信发布证明	资源版本、三类哈希、证明	不得进入可信分发
节点授权	治理事件及链下读模型	事件标识、序列、当前状态	敏感动作 fail-closed
发现索引	发现服务节点本地记录	同步游标、校验时间和新鲜度	保留旧快照并标记滞后

发现服务节点中的资源索引可以采用如下派生结构。它不是新的权威记录，而是把多个来源组织成便于查询的读模型；其中 `source` 用于回溯事实来源，`sync` 用于判断本地数据是否已经追上发布位置：

```
{
  "resourceDid": "did:oan:SKDM:example-resource",
  "version": "2026-09-09T08:20:00Z",
  "metadata": {
    "resourceType": "skill",
    "capabilityTags": ["code-repository", "summary"]
  },
  "source": {
    "registrarRecord": "registrar-record-901",
    "rootPublication": "publish-1842",
    "rootProof": "root-proof-1842",
    "packageDigest": "sha256:..."
  },
  "sync": {
    "localCursor": "publish-1842",
    "checkedAt": "2026-09-09T08:31:00Z",
```



```
    "freshness": "current",
    "error": null
  }
}
```

当 `localCursor` 落后、来源校验失败或治理投影尚未更新时，读模型仍可保留用于有限展示，但应将 `freshness` 改为 `stale` 或 `verification_required` 并保留错误原因。索引数据库中的一行记录只能说明本地曾经接收过材料，不能单独证明资源仍处于有效、可传播或可调用状态。

4.13 网络拓扑与序列一致性

根平台发布序列由根平台产生，用于标识资源发布材料在数据分发链路中的位置；链上治理事件序列由链上治理系统产生，用于表达节点授权和状态变化的顺序；发现服务节点同步游标由发现服务节点维护，用于表达该节点已处理到的分发位置。三者可能数值相似，但没有可互换关系。实现必须分别保存和检查它们，避免用一个全局编号推断资源发布、治理最终性和本地索引状态。

发现服务节点遇到游标缺口、乱序或回退时，应暂停受影响范围的应用，向根平台或内容分发平台补取缺失材料，并在重新验证后继续。重复材料可以按资源 DID、版本和包哈希幂等处理；治理事件重复应依据事件标识或序列去重；发布序列回退不得覆盖已经确认的较新版本。跨节点延迟期间，可以保留旧快照用于有限读侧展示，但必须标记新鲜度，并在无法确认撤销、暂停或授权状态时停止敏感动作。

三类序列应在实现和运维记录中分开保存。下面的配置式示例只表达它们的来源和处理边界，不表示三者共享同一种编号格式：

```
resource_publication:
  producer: root-platform
  latest: publish-1842
  applies_to: did-document-and-resource-package
  gap_action: pause-affected-publication

governance_events:
  producer: governance-ledger
  latest: event-771
  applies_to: node-authorization-and-resource-status
  gap_action: fail-closed-for-sensitive-actions

discovery_sync:
  producer: discovery-service-node
  local_cursor: publish-1839
  applies_to: local-validation-and-indexing
  gap_action: fetch-missing-publications-and-revalidate
```

例如，发现服务节点的同步游标落后于根平台发布序列，只能说明本地索引可能不完整，不能据此推断治理事件也落后同样数量；治理事件出现缺口时，即使资源发布材料完整，也应暂停依赖治理状态的应用。只有在对应序列各自连续、材料校验通过且状态投影完成后，节点才可以把资源标记为当前可发现结果。

4.14 参考来源

来源	类型	链接
oan-root-services	代码仓：根平台、发布协调和节点目录	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务节点和接入链路	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现服务节点、同步和索引	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理事件同步和链下状态投影	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv 黄皮书：网络架构和信任链路	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF 草案：智能体覆盖网络架构	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

第五章 OAN 资源模型

本章说明 OpenAgenet (OAN) 将可被注册、发布、分发和发现的能力单元统一建模为资源。当前核心资源类型包括智能体服务、技能包、MCP 服务和工具 API；基础设施节点、组织和开发者也可以作为身份或主体类型出现在关联关系中，但本章重点讨论四类可发现资源。每个资源至少具有独立的资源 DID、资源类型、描述、能力标签、用例、协议绑定、服务端点、版本和生命周期状态，并可通过资源包承载某一版本的可验证表示。

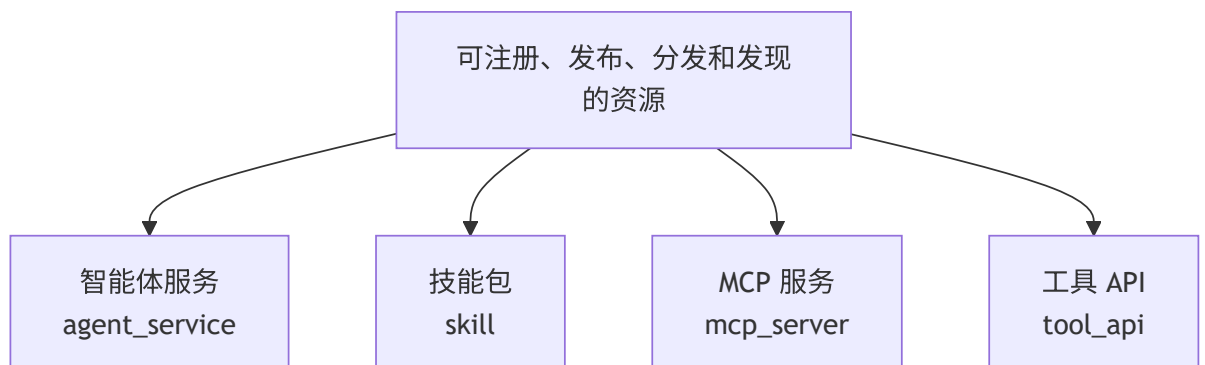
资源模型只统一跨类型互操作所需的共性，不把四类资源强行变成同一种业务对象。资源类型决定字段解释、端点形态、协议绑定和调用方式；资源 DID 确定控制和注册边界；资源元数据用于注册和发现；资源包将 DID 文档、元数据、哈希和根平台可信发布证明绑定为可分发对象。后续章节分别规定身份、注册、发现、发布和调用，避免在本章重复定义其全部协议行为。

5.1 资源模型概览

资源模型的公共身份字段包括资源 DID、资源类型、主体类型，以及可选的控制方 DID、发布方 DID 和签发方 DID；公共描述字段包括名称、描述、能力描述、能力标签和用例；公共连接字段包括协议绑定、服务端点和认证或凭证要求；公共状态字段包括版本、生命周期状态、授权域、网络范围和更新时间。`oanMetadata` 是 DID 文档中承载 OAN 资源语义和生命周期信息的主要元数据对象，资源包中的资源元数据则为发布和发现提供版本化表示。

资源类型枚举至少包括 `agent_service`、`skill`、`mcp_server` 和 `tool_api`。资源 DID 的方法特定编码应与资源类型保持一致，资源类型、主体类型、DID 文档中的 `oanMetadata` 和资源包元数据必须能够相互校验。字段未出现、为空或仅存在于网页展示层时，不得据此推断资源具备对应能力；实际调用许可也不由资源模型中的端点或描述字段自动授予。

四类资源共享资源 DID、资源元数据、版本和生命周期等公共模型，同时根据自身的能力形态使用不同的协议绑定、端点和包内容：



下面的示例只展示公共字段如何组合，具体资源仍需根据类型补充协议、端点、包信息和凭证要求。`resourceType`、`subjectType` 和 `resourceDid` 是机器判断资源边界的字段，名称、描述和能力标

签则服务于人类理解与发现检索：

```
{
  "resourceDid": "did: oan: SKLG: example- skill",
  "resourceType": "skill",
  "subjectType": "skill",
  "publisherDid": "did: oan: ORLG: example- publisher",
  "name": "Repository Structure Summary",
  "description": "Summarizes the structure of a code repository.",
  "capabilityTags": ["code- repository", "project- structure- summary"],
  "authorizedDomains": ["public"],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}
```

该对象可以作为资源元数据的最小阅读样例，但不能单独替代 DID 文档、控制证明、资源包哈希或根平台可信发布证明。发现服务节点可以使用其中的结构化字段建立索引，调用方仍需取得对应证据并根据资源类型执行后续验证。

5.2 智能体服务

智能体服务是提供可被调用的智能体能力的资源类型，对应资源类型值 agent_service。它通过资源 DID 表达独立的控制和发布边界，通过资源描述表达能力、能力标签、用例、输入输出和适用场景，通过服务端点和协议绑定表达如何建立连接。智能体服务可以声明 A2A、MCP 或其它业务协议，但声明协议只说明资源的接口语境，不代表调用方已经获得业务授权。

与 A2A Agent Card 的关系属于互操作映射关系。OAN 的能力描述、能力标签、用例、服务端点、协议绑定、版本和认证声明可以为 A2A Agent Card 的相应信息提供可信身份和发布来源；A2A Agent Card 的字段也可以通过适配器映射为 OAN 资源元数据。OAN 额外提供资源 DID、控制证明、注册凭证、根平台发布证明、治理状态和授权域，使“能力描述”能够与来源、版本和网络治理事实关联起来，但不改变 A2A 本身的交互协议语义。

互操作信息	OAN 资源模型中的承载位置	映射时应保留的边界
服务名称和描述	resourceDescription.name、resourceDescription.description	说明资源是什么，不等同于调用授权
能力和适用任务	capabilityDescription、capabilityTags、useCaseExamples	说明可发现能力，不替代运行时能力检查
服务入口	DID 文档的 service 与协议绑定的 serviceRef	入口地址可公开，但端点仍需独立认证
协议和版本	protocolBindings.protocol、version、transport	OAN 记录接入语境，不修改 A2A 的交互语义
身份和发布证据	资源 DID、控制证明、注册凭证和根平台可信发布证明	不能只把 A2A Agent Card 的内容当作 OAN 发布事实

5.3 技能包

技能包是可独立发布、安装、加载或复用的能力单元，对应资源类型值 `skill`。它应通过资源描述说明能力、输入输出、用例、依赖和使用条件，通过服务端点、包信息或实现链接说明获取和使用入口。技能包可以被智能体服务或其它编排系统组合使用，但技能包的可发现状态、版本和来源仍然应以自身资源 DID、资源包和根平台发布事实为依据。

技能包的“可复用”表示调用方能够在满足其依赖、版本和授权条件时重复使用，不表示它可以绕过资源控制、安装安全或业务权限。技能包描述中的安装命令、下载地址、脚本或依赖只能作为公开声明，调用方仍应验证资源包哈希、发布证明、版本状态和本地执行策略。

例如，技能包的公开元数据可以把能力、适用任务、依赖和版本放在同一资源边界内：

```
{
  "resourceDid": "did:oa:SKLG:example-skill",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Repository Structure Summary",
  "description": "Summarizes a code repository and its project structure.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "implementationLinks": [
    {
      "relation": "depends_on",
      "targetDid": "did:oa:TlFI:example-tool-api",
      "targetType": "tool_api",
      "versionConstraint": ">=1.0.0"
    }
  ],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}
```

这里的 `implementationLinks` 表达可验证的资源关系，版本约束表达组合条件；它不表示目标工具 API 已经被安装，也不表示调用方已经获得访问权限。实际资源包发布时，还需把该元数据与 DID 文档哈希、元数据哈希和资源包哈希绑定。

5.4 MCP 服务

MCP 服务是以 Model Context Protocol 提供工具或资源访问能力的服务资源，对应资源类型值 `mcp_server`。它的资源 DID 标识服务发布和控制边界，资源元数据说明能力标签、用例、协议版本、传输方式、服务端点和认证要求，MCP 能力描述可以通过协议绑定或专用能力对象表达。工具清单或资源清单属于服务能力声明，不能单独替代服务端点认证、调用授权和运行状态检查。

OAN 与 MCP 元数据的映射应保留两层边界：MCP 负责描述和执行协议层的工具、资源及交互语义，OAN 负责为 MCP 服务提供资源身份、控制证明、可信发布、治理状态和发现入口。MCP 服务的端点可访问，不代表该端点已经被根平台发布或对所有调用方开放；发现结果也不应把 MCP 能力声明直接解释为调用许可。

一个 MCP 服务的资源描述可以同时给出协议绑定和服务端点，例如：

```
{
  "resourceType": "mcp_server",
  "subjectType": "mcp_server",
  "protocolBindings": [
    {
      "id": "mcp-http",
      "protocol": "MCP",
      "version": "2025-06-18",
      "transport": "streamable-http",
      "serviceRef": "#mcp-service"
    }
  ],
  "services": [
    {
      "id": "#mcp-service",
      "type": "MCPServer",
      "serviceEndpoint": "https://example.invalid/mcp",
      "protocol": "MCP"
    }
  ]
}
```

该示例中的地址、版本和标识符均为示例值。`protocolBindings` 说明资源按何种协议接入，`services` 说明网络入口，两者仍需与资源 DID、发布版本、哈希和认证要求关联；发现服务节点不应仅因存在 MCP 工具清单就把资源标记为可直接调用。

5.5 工具 API

工具 API 是通过网络接口提供具体操作能力的资源，对应资源类型值 `tool_api`。它应描述接口地址、操作或能力名称、输入参数、输出结果、协议、版本、认证方式和调用约束；复杂参数可以通过输入和输出 schema、协议 schema 引用或外部文档表达。一个工具 API 可以被智能体服务、技能包或 MCP 服务引用，但引用关系不等于调用方已经通过目标端点的认证和授权。

工具 API 的资源描述与实际服务实现应保持可验证关联。资源控制方负责声明接口和版本，发布链路负责绑定 DID 文档、元数据、哈希和根平台证明，调用方负责在连接前检查端点、版本、证据和自身策略。接口存在、HTTP 响应正常或在发现索引中出现，都不能单独证明工具业务结果正确或调用一定被允许。

工具 API 的能力描述可以用输入、输出 schema 和认证要求把“能做什么”与“如何调用”分开表达：

```
{
  "resourceType": "tool_api",
  "resourceDescription": {
    "name": "Repository Structure Summary API",
    "capabilityDescription": "Accepts a repository reference and returns a project structure summary.",
    "inputSchema": {
      "type": "object",
```

```
{
  "required": ["repository"],
  "properties": {
    "repository": { "type": "string" }
  }
},
"outputSchema": {
  "type": "object",
  "required": ["summary"],
  "properties": {
    "summary": { "type": "string" }
  }
}
},
"credentialRequirements": [
  {
    "credentialType": "Bearer",
    "presentationMode": "header",
    "required": true
  }
]
}
```

schema 约束输入输出形状，凭证要求说明调用前需要准备的认证材料；真正的权限判断、限流、业务校验和结果质量仍由目标 API 及调用方策略负责。资源模型只为这些声明提供可登记、可发布和可发现的身份边界。

5.6 资源关系与组合

资源关系用于表达资源之间可验证的依赖、提供、调用、包含或组合。当前实现以 `implementationLinks` 等结构化链接承载关系，链接至少应包含关系类型、目标资源 DID，并可包含目标资源类型、目标服务和版本约束。关系的可信性来自双方资源身份、发布证据和当前状态，而不是来自一段无法校验的自然语言说明。资源包可以承载某一版本关系的发布表示，但不应把一个版本的关系自动延伸到所有未来版本。

组合系统在运行前应验证每个被引用资源的 DID、资源类型、版本或包哈希、根平台发布证明、生命周期状态和授权域。被引用资源不可发现、版本不满足约束、证明缺失、被撤销或暂停时，调用方或编排系统应根据风险选择拒绝组合、等待同步或仅作不具备调用资格的展示，不得把索引中存在摘要当作组合完整性证明。

资源关系在数据上可以表示为一个明确的链接对象，而不是把依赖写进自由文本：

```
{
  "relation": "provides",
  "targetDid": "did: oan:TLFI:example-tool-api",
  "targetType": "tool_api",
  "targetService": "#repository-summary",
  "versionConstraint": "~1.0.0"
}
```


关系验证至少需要检查目标 DID、关系类型、目标类型、版本约束和目标资源的当前发布状态。`targetService` 只是定位目标服务的引用，不能替代端点可达性和认证；当目标资源的版本或包哈希变化时，原关系需要重新匹配和验证。

5.6.1 资源之间的依赖关系

依赖关系表示一个资源的正常使用需要另一个资源或外部能力。依赖声明应指向目标资源 DID，并在必要时给出目标类型、服务引用和版本约束；依赖的业务含义、安装方式和运行时授权可以通过资源描述或协议绑定补充，但身份和版本边界必须结构化表达。依赖目标更新时，调用方应重新检查版本约束和发布状态。

5.6.2 智能体服务与技能包组合

智能体服务可以依赖或组合多个技能包，将技能包作为任务执行、信息处理或工具编排的可复用能力。组合记录应区分智能体服务 DID、技能包 DID、技能包版本或包哈希以及关系类型；智能体服务的发布不自动覆盖技能包的控制和发布事实。调用方需要同时验证智能体服务和技能包的证据及兼容条件。

5.6.3 MCP 服务、工具 API 与调用关系

MCP 服务可以提供工具集合，也可以通过协议绑定引用一个或多个工具 API；工具 API 也可以被智能体服务或技能包直接调用。关系声明应说明目标 DID、目标服务或操作、协议及版本，避免只写“支持某工具”。MCP 协议层可发现的工具能力仍需叠加 OAN 的资源身份、发布状态、端点认证和调用方授权判断。

5.6.4 组合完整性与版本约束

组合完整性至少要求关系双方的 DID、资源类型、版本或包哈希、关系声明、根平台可信发布证明和当前生命周期状态一致，并检查目标资源是否处于调用方可见的授权域。版本约束不满足、包哈希变化、证明过期或资源暂停时，不得继续使用旧组合声明作无条件调用依据；必要时应重新发现、重新验证或回退到明确兼容的版本。

5.7 作为独立可标识对象的资源

四类资源均可拥有独立资源 DID，并以该 DID 作为创建、登记、发布、发现和更新的身份边界。主体 DID 可以表达资源控制关系，发布方 DID 可以表达登记或运营来源，但不能用主体或发布方 DID 替代资源自身 DID。资源 DID 的方法特定编码应与资源类型一致，DID 文档中的公钥、服务端点和 `oanMetadata` 应共同支持资源身份和能力声明的校验。

独立 DID 使技能包、MCP 服务或工具 API 可以脱离某个智能体服务单独被发现和复用，也使它们能够独立更新、暂停、撤销和形成版本记录。资源之间存在组合关系时，仍应分别验证各自 DID 和发布证据；一个资源的可信状态不能自动传递给其依赖资源。

资源类型与 `did:oan` 方法特定编码之间存在代码级校验关系。当前公共库用 AG、SK、MC 和 TL 分别对应智能体服务、技能包、MCP 服务和工具 API；下面的 DID 仅用于说明形态：

资源类型	机器值	方法特定编码示例	独立身份边界
智能体服务	agent_service	did:oan:AG...:<method-specific-id>	服务本身的控制、发布和版本
技能包	skill	did:oan:SK...:<method-specific-id>	技能包内容、依赖和版本
MCP 服务	mcp_server	did:oan:MC...:<method-specific-id>	MCP 服务入口和工具集合
工具 API	tool_api	did:oan:TL...:<method-specific-id>	API 操作、schema 和端点

编码前缀是当前实现用于资源类型一致性检查的标识，不应被理解为脱离 did:oan 方法规则的独立 DID 方法。资源类型与 DID 文档 oanMetadata.resourceType 不一致时，节点应拒绝将其当作该类型资源继续处理。

5.8 资源元数据与能力描述

资源元数据是资源身份之外用于注册、发布和发现的结构化说明。公共字段包括资源 DID、资源类型、主体类型、控制方或发布方 DID、名称、描述、能力描述、能力标签、授权域、协议绑定、服务端点、凭证要求、包信息、网络范围、生命周期状态、版本和更新时间。DID 文档中的 oanMetadata 与资源包中的资源元数据应在资源 DID、资源类型、版本、哈希、授权域和状态等关键字段上保持一致。

能力描述回答资源能够完成什么，能力标签回答资源属于哪些受治理的能力类别，用例回答这些能力在什么任务中使用。名称和自由文本有助于人和语义引擎理解资源，但不能替代结构化标签、协议绑定、端点或控制证明。元数据是资源提供方作出的声明，根平台和发现服务节点可以验证其结构、哈希和发布来源，但不因字段齐全就替代对业务质量的独立评估。

能力字段在 oanMetadata.resourceDescription 中可以组织为一个面向检索的最小对象：

```
{
  "name": "Repository Structure Summary",
  "description": "A reusable capability for examining code repositories.",
  "capabilityDescription": "Finds repository files and summarizes the project structure.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "useCaseExamples": [
    "I need a tool that can search code repositories and summarize the project structure."
  ],
  "version": "1.0.0"
}
```

其中自然语言字段适合语义发现和用户理解，标签适合结构化筛选，版本用于与资源包和发布证据关联。它们共同描述资源的能力，但不等于节点已经验证了运行效果，也不替代调用前的身份、发布和授权检查。

5.8.1 基本身份与描述字段

基本字段至少应能够确定资源 DID、资源类型、主体类型、名称、描述、控制方或发布方关系，并与 DID 文档的 id、oanMetadata.resourceType、oanMetadata.subjectType 及相关 DID 字段一致。资源类型值应使用协议定义的枚举，展示层的中文名称或别名不能作为唯一机器判断依据。

5.8.2 能力标签、能力描述与用例

能力标签应优先使用根平台维护的能力标签树中的规范标识，也可以在明确标识来源的前提下保留扩展标签；能力描述和用例应说明可执行能力、输入输出和适用任务。注册服务节点可以根据描述提供标签推荐，发现服务节点可以结合标签、别名、描述和用例进行结构化或语义检索，但相关性结果仍需经过身份、发布和授权域校验。

5.8.3 协议、端点与认证信息

协议绑定应说明协议名称、版本、传输方式、服务引用或 schema 引用，并与一个或多个服务端点关联；服务端点应说明地址、类型、版本、协议及必要的端口或服务器信息；认证信息应表达公开的认证方式、凭证要求或调用前提。端点和认证声明是资源能力说明的一部分，不等同于调用方已经获得访问权限。

5.8.4 状态、版本与治理信息

资源元数据应区分资源生命周期状态、资源包版本、更新时间、过期时间、上一版本或版本约束，以及授权域和网络范围。生命周期状态表达资源是否 active、暂停、撤销或被替换等事实，传播进度则由根平台发布、内容分发和发现同步记录表达；二者不能用一个 `available` 字段混为一谈。

5.9 用例、协议绑定、端点与认证声明

用例、协议绑定、服务端点和认证声明共同组成“如何理解和使用该资源”的公开说明。一个资源可以有多个用例、多个协议绑定和多个端点；每个绑定应尽可能说明适用能力、协议版本、传输方式、服务引用和 schema，认证声明应尽可能关联到目标端点、操作或版本。若端点不可用、协议版本不匹配或认证信息不完整，发现服务节点可以返回带有相应状态的结果，但不应将其表述为可直接调用。

5.9.1 用例描述

用例描述应说明任务目标、输入条件、期望输出和适用资源能力，优先与能力描述和能力标签建立对应关系。自然语言用例可以用于语义检索和用户理解，但不能单独证明资源具备该能力；需要精确约束时，应结合输入输出 schema、协议绑定和版本信息。

5.9.2 协议绑定

协议绑定应记录协议名称、协议版本、传输或交互方式，以及与端点或 schema 的引用关系。A2A 和 MCP 可以作为外部协议语境进入资源描述，OAN 的绑定字段只表达该资源如何接入这些协议，不应把 OAN 资源注册状态写成外部协议的强制条款。

5.9.3 服务端点

服务端点是资源对外提供能力或获取材料的网络入口，可以包含端点标识、端点类型、地址、版本、协议、服务器类型和端口等信息。一个资源存在多个端点时，应说明端点之间的协议或版本差异；端点不可达、证书错误或服务停用时，发现结果应保留该事实或新鲜度信息，不能只看静态地址。

5.9.4 认证与调用要求

认证与调用要求用于说明调用方需要提供何种身份、凭证、签名、权限或协议上下文。资源可以公开声明凭证类型、签发方、范围和呈现方式，但最终业务授权由资源端点和调用方策略共同决定；OAN 的 VC、根平台可信发布证明和节点治理状态是调用前验证材料，不是对所有业务操作的自动授权。

资源模型中的认证声明主要回答“调用方需要准备什么”，例如凭证类型、签发方、作用域和呈现方式；端点收到请求后仍需检查凭证有效期、请求主体、业务权限和运行策略。可以把同一资源的公开声明理解为两层：

层次	资源模型可以表达的内容	不能由该字段单独推出的结论
公开声明	凭证类型、签发方、作用域、呈现方式和适用端点	调用方一定拥有该凭证
调用判断	端点认证结果、授权策略、协议上下文和业务条件	资源一定可用或业务结果一定正确

5.10 资源包与发布产物

资源包是绑定某一资源版本的可分发对象，通常包含资源 DID、资源类型、资源版本、DID 文档、DID 文档哈希、元数据哈希、资源包哈希、哈希算法、资源元数据、根平台可信发布证明和创建时间。DID 文档表达身份、公钥、服务端点和 OAN 元数据；资源元数据表达能力、用例、协议、端点、授权域和生命周期；控制证明表达资源控制方对操作的授权；注册 VC 表达注册服务节点处理过的登记事实；根平台可信发布证明表达根平台已验证并接受该资源版本的事实。

在节点接口中，资源模型分别对应不同的载荷类型：客户端提交的是 `ResourceRegistrationSubmission`，其中的 `metadata` 仍是动态 JSON，因为注册服务节点需要按当前资源元数据规则校验其结构；根平台验证通过后，把这些字段组织为 `ResourcePackage`，再由 `ResourceCdnPublishRequest` 或批量请求承载并分发。因此，“资源对象”“登记请求”和“可分发资源包”不是同一个 JSON 对象，客户端不应把注册请求直接当作已经完成根平台证明的资源包。

这些对象形成分层验证链路而不是平行附件。调用方或发现服务节点应先解析 DID 文档并验证控制关系，再校验元数据与 DID 文档的资源类型、DID、授权域和版本一致性，重算 DID 文档哈希、元数据哈希和资源包哈希，验证注册凭证、根平台可信发布证明及当前治理状态。注册受理、根平台发布和发现索引分别由不同组件产生，任一阶段成功都不能替代其它阶段。

一个资源包可以按以下结构理解。示例中的哈希、DID、时间和证明内容均为占位示例，用于展示对象之间的引用关系，不代表官方固定值：

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did:oan:SKLG:example-skill",
  "resourceType": "skill",
  "didDocument": {
    "@context": ["https://www.w3.org/ns/did/v1"],
    "id": "did:oan:SKLG:example-skill",
    "verificationMethod": [],
    "authentication": [],
    "assertionMethod": [],
    "service": [],
```

```

    "oanMetadata": {
      "subjectType": "skill",
      "resourceType": "skill",
      "capabilityTags": ["code-repository", "project-structure-summary"],
      "authorizedDomains": ["public"],
      "lifecycleState": "active"
    }
  },
  "didDocumentHash": "sha256:example-did-document",
  "metadataHash": "sha256:example-metadata",
  "packageHash": "sha256:example-package",
  "hashAlgorithm": "sha256",
  "metadata": {
    "resourceDid": "did:oan:SKLG:example-skill",
    "resourceType": "skill",
    "subjectType": "skill",
    "name": "Repository Structure Summary",
    "description": "Summarizes a code repository.",
    "capabilityTags": ["code-repository", "project-structure-summary"],
    "authorizedDomains": ["public"],
    "services": [],
    "lifecycleState": "active",
    "packageVersion": "1.0.0",
    "packageHash": "sha256:example-package",
    "metadataHash": "sha256:example-metadata",
    "hashAlgorithm": "sha256",
    "updatedAt": "2026-09-09T08:30:00Z"
  },
  "rootProof": {
    "rootDid": "did:oan:INLG:example-root",
    "packageClaims": {
      "resourceDid": "did:oan:SKLG:example-skill",
      "resourceType": "skill",
      "version": "1.0.0",
      "didDocumentHash": "sha256:example-did-document",
      "metadataHash": "sha256:example-metadata",
      "packageHash": "sha256:example-package",
      "hashAlgorithm": "sha256",
      "lifecycleState": "active",
      "authorizedDomains": ["public"]
    }
  },
  "createdAt": "2026-09-09T08:30:00Z"
}

```

验证方可以先用 `resourceDid` 和 `resourceType` 判断对象归属，再重算三类哈希，检查 `metadata` 与 DID 文档的关键字段一致性，最后验证 `rootProof.packageClaims` 与当前包声明是否一致。该顺序有助于把“内容完整”“资源可控”“注册受理”和“根平台发布”分别定位，不把一个下载文件误解为全部网络事实。

5.10.1 ResourcePackage 内容

资源包内容应以一个资源版本为中心组织，至少绑定 `resourceDid`、`resourceType`、`packageVersion`、`DID` 文档、`DID` 文档哈希、元数据哈希、资源包哈希、哈希算法、资源元数据和根平台可信发布证明。资源包中的 `rootProof` 应能够通过其声明关联资源 `DID`、类型、版本、哈希、生命周期状态、授权域和必要的治理或发布引用。

5.10.2 元数据与资源内容

元数据是对资源内容、能力和运行入口的结构化说明，资源内容可以是技能包文件、服务描述、工具接口或 MCP 能力声明。元数据中的包信息、下载地址、清单地址、实现链接和版本字段应指向或描述当前资源版本，不得让历史元数据覆盖当前资源包的身份和哈希。

5.10.3 哈希与完整性证明

资源包应分别表达 `DID` 文档哈希、元数据哈希和资源包哈希，并声明使用的哈希算法。验证方应按规范化表示重算相应哈希，检查资源包内部字段和根平台证明中的声明是否一致；哈希匹配只能证明内容完整性，不能单独证明控制权、节点授权或业务端点可信。

5.10.4 发布、下载与验证产物

注册服务节点可以返回登记结果、注册凭证或下载材料，根平台产生接受记录和根平台可信发布证明，内容分发平台提供可下载的资源包和相关材料，发现服务节点保存已验证的索引对象。下载到资源包不等于资源已经完成调用授权；调用方应根据来源、版本、新鲜度、治理状态和本地策略重新验证。

5.11 资源生命周期与可见状态

资源生命周期描述资源版本本身的状态，网络传播进度描述该版本是否经过注册受理、根平台接受、内容分发和发现索引。一个资源可以已登记但尚未根平台发布，可以已发布但尚未被某个发现服务节点同步，也可以在发现索引中存在但因授权域或治理状态变化而不再向特定调用方可见。状态判断应同时引用产生状态的组件和相应证据。

资源更新应区分同一版本的幂等重试与新的版本发布。相同资源 `DID`、版本和哈希的重复提交可以复用既有结果；内容变化、版本变化、控制密钥变化或生命周期变化必须进入新的验证和发布判断。旧快照可以用于有限展示，但不能覆盖当前撤销、暂停、替换或授权域限制。

资源版本和传播状态可以用两个维度观察：

资源版本/生命周期	注册服务节点	根平台与内容分发平台	发现服务节点	对调用方的含义
草稿	尚未受理	无发布事实	不应进入可信索引	只能本地编辑或预览
已登记	有登记记录或注册凭证	可能尚未发布	可能不可见	不能据此判断已全网发布
已发布	保留登记证据	有根平台可信发布证明和分发材料	可能仍在同步	可按授权域等待发现或直接验证材料
已索引	保留登记证据	发布材料可验证	已完成本地校验和索引	仍需检查当前治理状态和调用授权

资源版本/生命周期	注册服务节点	根平台与内容分发平台	发现服务节点	对调用方的含义
暂停/撤销	状态记录需保留	新传播或使用受限	查询结果应更新或标记过期	不得继续把旧快照当作当前可用

5.11.1 注册状态

本地草稿表示资源材料尚未被注册服务节点受理；已提交表示注册服务节点已收到并处理请求；已登记表示注册服务节点形成了登记记录或注册凭证。上述状态不能直接说明根平台已发布或发现服务节点已索引。资源控制方应保存本地 DID 文档、密钥和提交材料，以便在失败或更新时重新验证。

5.11.2 发布与索引状态

根平台已发布表示根平台接受了特定资源版本并形成发布证明；内容已分发表示资源包已进入内容分发平台的承载范围；发现服务节点已索引表示某个发现服务节点完成资源包验证并写入本地读模型。三个状态按异步流程推进，可能短暂不一致，应通过游标、状态和新鲜度表达。

5.11.3 暂停、撤销与恢复状态

暂停表示资源或其版本在规定范围内暂时不得继续按 active 状态传播或调用；撤销表示相应资源、凭证、发布事实或节点授权不再有效；恢复表示经过新的治理或发布判断后重新允许相应动作。状态变化应关联版本、资源 DID、治理事实或发布记录，不能只改网页显示文字；发现服务节点应在状态传播后更新索引和查询结果。

5.11.4 公开可见与可信可发现状态

公开可见表示某个门户或节点能够向特定查询方返回资源信息，可信可发现表示该发现服务节点已验证资源身份、发布证明、哈希、生命周期和授权域，并允许在其授权范围内返回。资源可能公开显示但尚未具备可信可发现资格，也可能已被根平台发布但对某个授权域不可见；调用方仍需在连接前执行自己的验证。

5.12 资源所有权、控制权与委托关系

资源所有权是业务或组织层面的权利关系，DID 控制权是通过密钥和签名证明能够代表资源 DID 执行规定操作的密码学关系，运营管理权是对端点、内容或运行环境的管理责任，发布代理权是注册服务节点或其它授权参与者代为提交或处理材料的资格，调用授权是资源端点和调用方策略共同决定的业务许可。它们可以属于同一主体，也可以由不同主体承担。

资源元数据可以通过 controllerDid、publisherDid、subjectDid、implementationLinks 或凭证要求表达部分关系，但字段本身不是关系真实性的充分证明。验证方应分别检查资源 DID 控制证明、发布方或注册服务节点的凭证、根平台发布证明、治理状态和业务端点认证，不能因某一方能够上传文件、提供地址或签发登记结果，就推定其拥有其它权限。

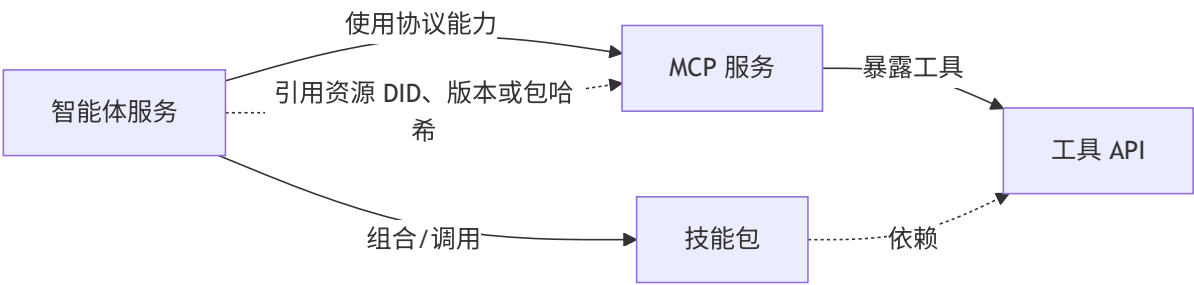
5.13 资源依赖与组合完整性

依赖完整性要求组合中的每个资源都能被独立识别、定位、验证并按版本使用。最低检查集合包括目标资源 DID、资源类型、关系类型、目标版本或包哈希、DID 文档和元数据一致性、根平台可信发布证明、生命周期状态、授权域和必要的协议兼容条件。发现服务节点只返回索引摘要时，调用方还应取得完整资源包或可验证证明后再建立高风险组合。

当发现节点落后、目标资源暂不可获取或证明暂缺时，可以将组合标记为待同步或不可验证；当目标资源版本漂移、被暂停、撤销、超出授权域或哈希不一致时，应拒绝继续使用该组合，除非存在明确满足约束的替代版本。组合完整性是调用前的验证条件，不是资源登记或搜索排名的附带结果。

四类资源共享资源 DID、资源类型、能力描述、版本、端点、协议绑定和生命周期等共性信息，但其内容形态和调用方式不同。资源模型的目标是提供共同的登记、发布和发现骨架，而不是把四类资源强行转换成同一种运行时对象。

资源类型	主要承载内容	常见发现依据	调用或使用前重点检查
智能体服务	面向任务或会话的智能体能力	能力标签、用例、交互协议和服务端点	服务身份、协议、认证和服务权限
技能包	可复用的指令、流程或能力组件	能力描述、适用场景、依赖和版本	包哈希、来源、兼容性和执行边界
MCP 服务	面向 MCP 的工具集合或服务入口	MCP 协议绑定、工具能力和端点	MCP 版本、服务器身份、认证和工具声明
工具 API	可直接调用的接口能力	API 能力、输入输出 schema 和端点	API 版本、schema、认证和调用授权



资源关系必须显式表达目标 DID、关系类型以及版本或包哈希。仅有名称、搜索相关性或端点可达性，不能证明依赖完整；被引用资源暂停、撤销、版本漂移或超出授权域时，组合方应拒绝继续使用或转入待验证状态。

资源模型的边界：OAN 负责资源的身份、描述、发布、发现和验证关联；资源内部的模型推理、业务流程、工具执行和最终业务授权仍由相应资源及调用方策略负责。

5.14 参考来源

来源	类型	链接
oan-protocol-common	代码仓：资源类型、核心领域对象和资源包	https://github.com/wolfbrother/oan-protocol-common

来源	类型	链接
oan-sdk-ts	代码仓：资源 DID、资源包和客户端材料构造	https://github.com/OpenAgenet/oan-sdk-ts
OAN White Paper	arXiv 白皮书：资源生态和互操作定位	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv 黄皮书：资源模型与可信发布机制	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	标准/方法规范：资源身份表达	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

第六章 did:oan 方法与资源身份

本章规定 `did:oan` 作为 OpenAgenet (OAN) 资源身份方法的稳定规则。方法层负责回答“标识符是否合法、标识了什么对象、如何得到对应的 DID 文档、谁能够证明控制权”，不负责回答资源是否已经登记、根平台是否已经发布或发现服务节点是否已经索引。资源网络中的注册、发布、分发和发现状态应引用本规范其它章节的对象和证据。

`did:oan` 方法实现必须同时满足语法可解析、语义编码可解释、DID 文档结构可验证和控制证明可验签四个条件。其 DID 文档还通过 `oanMetadata` 将资源身份与能力语义放在同一个可解析对象中：能力标签提供受治理的分类键，能力描述说明资源能够完成的任务，用例给出典型使用情境，协议绑定和服务引用说明如何接入。这种设计使解析 DID 后即可取得语义发现所需的核心描述，并通过文档哈希、控制证明和资源包把语义声明与资源身份及版本绑定，是 `did:oan` 面向智能体资源发现的重要创新。

6.1 方法目的与标识符范围

`did:oan` 可以标识智能体服务、技能包、MCP 服务、工具 API，也可以标识主体、组织和基础设施节点等方法定义允许的对象。方法特定语义编码的前两位区分对象类别，后两位表示应用或语义域；资源 DID、主体 DID、注册服务节点 DID、发现服务节点 DID 和根平台 DID 的用途不同，不能因为都使用同一 DID 方法就混为一谈。

标识符的唯一性首先是方法命名空间内的字符串唯一性；随机生成依赖密码学随机数，确定性派生则由语义编码、控制材料和 `nonce` 共同决定。资源跨不同注册服务节点或发现服务节点传播时，资源 DID 不因入口变化而改变；同一 DID 的历史版本应通过版本、哈希、发布序列和历史 DID 文档记录关联，而不是为每次更新随意生成新 DID。

在 OAN 的网络拓扑中，入口节点和身份对象是两条不同的关系：客户端可以通过一个注册服务节点提交资源，也可以从多个发现服务节点查询同一资源，但这些服务节点的 DID 只标识节点自身。资源 DID 仍由资源控制方生成或持有，解析器应根据资源 DID 取得对应 DID 文档，而不是把提交入口拼接到 DID 中。

对象	DID 的主体	主要用途	不应替代的对象
主体 DID	组织、开发者或控制主体	表达控制关系和签名语境	具体资源 DID
资源 DID	智能体服务、技能包、MCP 服务或工具 API	表达资源的注册、版本和控制边界	主体 DID 或节点 DID
注册服务节点 DID	注册基础设施节点	表达登记入口及其节点身份	资源控制权
发现服务节点 DID	发现基础设施节点	表达索引和查询入口	资源发布事实
根平台 DID	根平台基础设施身份	表达根平台相关证明和发布来源	资源所有权

6.2 DID 语法与方法特定标识符

方法特定标识符采用 `did: oan: <semantic-code>: <suffix>` 形式。`semantic-code` 必须是 4 个 ASCII 大写字母或数字，其中前两位为主体代码，后两位为应用域代码；`suffix` 必须是 32 个 Base58 字符，使用 OAN 定义的 Base58 字符表，不允许 0、o、I 和 l 等易混淆字符。方法实现对整体 DID、分段数量、语义编码和后缀分别校验。

例如，`did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu` 是语法上可解析的技能包 DID，而大小写错误、后缀长度不为 32、包含非法 Base58 字符或缺少分隔段的字符串必须拒绝。SDK 的规范化只将语义编码按方法规则转换为大写并保持后缀不变；规范化不能修复错误字符，也不能改变 DID 的对象类别。

方法库把解析结果拆分为完整值、语义编码、主体代码、应用域代码和后缀，便于后续校验资源类型，而不是让调用方自行按字符串位置猜测含义：

```
{
  "input": "did: oan: skfi: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "normalized": "did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "semanticCode": "SKFI",
  "subjectCode": "SK",
  "appDomainCode": "FI",
  "suffixLength": 32,
  "parseResult": "valid"
}
```

这里的规范化只适用于语义编码大小写可以按实现规则转换的输入；分段数量错误、后缀长度错误或含非法字符时，解析应直接失败，不能以“尽量修复”的方式生成另一个 DID。

6.3 资源 DID 与主体/控制者关系

方法特定编码只能说明 DID 所属对象类别，不能单独证明主体、控制方、提供方或运营方之间的法律和业务关系。资源 DID 的控制关系应在 DID 文档的验证方法、`oanMetadata.controllerDid`、`publisherDid`、`issuerDid` 及资源控制证明中表达，并由验证方检查这些字段与签名之间的对应关系。

主体可以控制多个资源 DID，资源提供方也可以与控制方不同；注册服务节点的 DID 代表基础设施资格，不能替代资源 DID。资源更新仍使用同一资源 DID 时，必须提交能够由当前控制关系验证的更新证明；DID 字符串本身不携带所有权，也不因被公开解析就转移控制权。

6.3.1 主体 DID

主体 DID 标识控制资源或代表组织、开发者的主体对象。它用于建立控制关系和签名主体的语境，不等同于某个具体资源的 DID，也不等同于用户登录账号。主体 DID 的 DID 文档应公开验证所需的公钥和服务信息，私钥由主体在本地身份中保管。

6.3.2 资源 DID

资源 DID 为一个可独立注册、发布、发现、更新和撤销的资源建立身份边界。四类产品资源都可拥有独立资源 DID；资源类型必须与 DID 语义编码及 DID 文档中的 `oanMetadata.resourceType`、`subjectType` 一致，否则注册服务节点和根平台应拒绝该材料。

6.3.3 控制关系与签名关系

控制关系由 DID 文档中的验证方法和针对具体操作的可验证控制证明共同支撑。验证方应根据证明中的签名者、验证方法、目标 DID、用途、挑战、时间和 DID 文档哈希检查签名，不能因为证明中的公钥与文档中某个公钥相同，就忽略用途、目标或新鲜度校验。

6.3.4 主体、资源与提供方分离

主体负责控制身份和资源操作，资源提供方负责能力、内容和服务端点，运营方负责运行环境和可用性。三者可以是同一主体，也可以分离；`publisherDid` 或 `subjectDid` 只是关系声明，关系真实性仍需由相应凭证、委托材料和控制证明支撑。

6.4 DID Document 结构

OAN 的 DID 文档以 DID Core 数据模型为基础，至少包含 `@context`、`id`、`verificationMethod`、`authentication`、`assertionMethod`、`service` 以及面向 OAN 资源的 `oanMetadata`。`id` 必须是规范化后的 `did:oan`，验证方法的控制者、服务端点的引用和元数据中的资源身份必须能够相互校验。对于智能体服务、技能包、MCP 服务和工具 API，`oanMetadata` 是资源类型、能力语义、协议关系和发布引用的结构化承载位置。

`oanMetadata.resourceDescription` 和 `oanMetadata.agentDescription` 可以表达能力描述、能力标签和用例；资源描述还可携带输入输出模式、示例、受众、领域、语言和版本。顶层的 `authorizedDomains`、`protocolBindings`、`implementationLinks`、`credentialRequirements` 和 `packageInfo` 分别补充可见范围、交互协议、组合关系、调用凭证要求和发布材料。发现服务节点可以使用这些字段建立标签、词法和向量索引，把自然语言任务映射到候选资源；调用方则可以在取得候选后继续核对 DID 控制关系、资源版本和发布证明。

一个用于说明字段关系的最小 DID 文档如下。示例中的公钥、服务地址和资源 DID 均为占位值；实际文档还应使用与控制方匹配的真实公钥和证明材料：

```
{
  "@context": [
    "https://www.w3.org/ns/did/v1",
    "https://w3id.org/oan/v1"
  ],
  "id": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "verificationMethod": [
    {
      "id": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
      "type": "Ed25519VerificationKey2020",
      "controller": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "cryptoSuite": "Ed25519Sha256",
      "publicKeyMultibase": "zExamplePublicKey"
    }
  ],
  "authentication": [
    "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"
  ],
  "assertionMethod": [
```

```
"did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"
],
"service": [],
"oanMetadata": {
  "subjectType": "skill",
  "resourceType": "skill",
  "controllerDid": "did: oan: DVFI: 7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "resourceDescription": {
    "name": "Repository structure analyzer",
    "description": "Searches code repositories and summarizes project structure.",
    "capabilityDescription": "Locate repository content, identify modules and summarize their
relationships.",
    "capabilityTags": ["code-repository", "project-structure-summary"],
    "useCaseExamples": [
      "Inspect an unfamiliar repository before development",
      "Summarize modules, entry points and dependencies"
    ]
  }
},
"lifecycleState": "active"
}
}
```

验证方至少应检查 `id`、验证方法的 `controller`、认证和断言引用是否属于同一文档，并检查资源类型是否与 `SK` 主体代码一致。能力标签、能力描述和用例应描述同一资源能力，发现服务节点可以分别用它们完成分类过滤、语义召回和结果解释。由于这些语义属性位于 `DID` 文档中，资源控制方可以对包含语义声明的文档生成控制证明，资源包也可以通过 `didDocumentHash` 锁定该版本，避免发现索引把身份与来源不明的外部描述拼接在一起。

6.4.1 DID 标识与方法元数据

`id` 是 `DID` 文档的主体标识，必须与请求或资源包中的资源 `DID` 完全一致；`@context` 应包含 `DID Core` 上下文和 `OAN` 方法上下文。方法特定语义编码可由解析器解释资源类别，但解析器不得仅据此替代 `oanMetadata` 和资源包的一致性校验。

6.4.2 verificationMethod

`verificationMethod` 描述可用于验证签名的公钥方法，每项至少需要方法 `ID`、类型、控制者和一种公开密钥表示。`OAN` 核心类型支持 `publicKeyMultibase`、`publicKeyJwk` 等表示，并可通过 `cryptoSuite` 和 `publicKeyFormat` 明确密码套件及编码；私钥绝不能出现在 `DID` 文档中。

6.4.3 authentication 与 assertionMethod

`authentication` 指定可用于身份认证或请求证明的验证方法，`assertionMethod` 指定可用于签发或验证声明的验证方法。引用值必须能定位到同一 `DID` 文档中的方法 `ID`，不能把仅用于断言的密钥无条件当作认证密钥，也不能把存在于 `verificationMethod` 中的公钥自动视为所有用途均有效。

6.4.4 service 端点

`service` 中的每个服务项应包含唯一服务 ID、服务类型和 `serviceEndpoint`，并可说明版本、协议、服务器类型和端口。端点可以指向资源业务服务、解析服务、资源包或其它协议描述，但端点可解析只证明地址存在，不能证明业务可用或调用方已获授权。

6.4.5 资源元数据与扩展字段

`oanMetadata` 用于承载 OAN 资源类型、主体类型、控制方、发布方、能力、授权域、协议绑定、实现链接、凭证要求、包信息和生命周期等语义。扩展字段不得覆盖 DID Core 的身份和验证含义；资源元数据与资源包、注册提交之间出现 DID、类型、版本、授权域或哈希冲突时，应按冲突处理规则拒绝或隔离。

`oanMetadata` 与 DID Core 核心字段的关系可以按以下方式理解：

字段区域	主要回答的问题	校验重点
<code>id</code> 、 <code>verificationMethod</code>	这是哪个 DID，使用什么公钥验证	DID 方法语法、方法 ID、控制者和公钥格式
<code>authentication</code> 、 <code>assertionMethod</code>	哪些方法可以用于认证或断言	引用是否存在、用途是否匹配
<code>service</code>	从哪里获取服务或协议入口	服务 ID、协议、版本和端点格式
<code>oanMetadata</code>	资源是什么，具有什么能力和状态	资源类型、主体类型、授权域、版本和包绑定

因此，解析器可以保留未知的 OAN 扩展字段，但不能让扩展字段改变 `id`、验证方法和用途引用的含义；资源网络验证器则应在方法解析成功后继续检查元数据与资源包的一致性。

6.5 验证方法与公钥

验证方法的 `id` 应属于对应 DID，`controller` 应与控制关系一致，公钥表示和密码套件必须能被验证方解析。控制签名应明确 `proofPurpose`、`verificationMethod`、目标对象和签名覆盖范围；验证方先解析 DID 文档取得公钥，再检查签名、用途、哈希和时间条件。

多个验证方法可以分别承担认证、断言、更新或恢复用途。密钥轮换必须通过当前有效控制关系授权，并保留旧密钥失效或历史验证所需的状态；旧密钥不能继续签发新的有效更新。仅把攻击者的公钥写入一份未被当前控制关系接受的 DID 文档，不能取得该 DID 的控制权。

一个验证方法的可接受性不是“公钥存在”这一单一条件，而是多个字段共同成立：

```
{
  "verificationMethod": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
  "expectedController": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "proofPurpose": "authentication",
  "targetDid": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "documentHash": "sha256:example-did-document",
  "signatureVerification": "required"
}
```

验证方应先确认 `verificationMethod` 能从目标 DID 文档解析出来，再确认其控制者、用途、目标 DID 和签名覆盖的文档哈希相符。若公钥属于另一个 DID、方法未被相应用途引用或文档已被替换，验证失败；不能只比较公钥字节是否相同。

6.6 服务端点与发现引用

DID 文档中的服务端点是资源或协议的定位信息，典型用途包括业务服务、注册服务、发现服务、DID 解析和资源包获取。服务项的 `id` 应可由资源 DID 关联，`serviceEndpoint` 应符合声明的地址或对象格式；发现响应可以引用端点，但调用方仍须验证根平台发布事实、端点新鲜度和业务认证要求。

注册服务节点、发现服务节点和解析器的接口属于网络组件行为，不因某个端点被写入 DID 文档就改变其授权边界。端点缺失、协议不匹配、地址不可达或引用对象不兼容时，解析可以成功返回 DID 文档，但资源发现或调用前验证应将相应问题明确标记。

解析结果应把“文档取得成功”和“端点可用”分开表达。一个面向客户端的结果摘要可以包含：

```
{
  "did": "did: oan: SKFI: 5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resolution": {
    "status": "resolved",
    "documentRetrieved": true,
    "methodSyntaxValid": true,
    "documentIdMatchesRequest": true
  },
  "services": [
    {
      "id": "#resource",
      "type": "OANResource",
      "serviceEndpoint": "https://example.invalid/resource",
      "availability": "not_checked"
    }
  ],
  "nextVerification": ["control-proof", "publication-proof", "endpoint-authentication"]
}
```

`availability: not_checked` 表示解析阶段没有对业务端点做可用性判断；解析器不应把 HTTP 地址存在或 DID 文档返回成功改写为端点健康、资源已发布或调用已授权。

6.7 资源元数据绑定

资源 DID 文档通过 `oanMetadata` 和包信息与资源元数据、`ResourcePackage` 建立关联。对于同一资源版本，资源 DID、资源类型、能力标签、能力描述、用例、授权域、协议绑定、版本和哈希应在 DID 文档、注册提交和资源包之间保持可验证一致；资源包的 `didDocumentHash` 用于绑定具体文档内容，不能只绑定 DID 字符串。语义发现因此能够同时引用稳定 DID、受控语义声明和发布版本，而不是只对脱离身份的自由文本建立索引。

解析器返回 DID 文档和 `oanMetadata` 后，发现服务节点可以提取能力标签、能力描述、用例、输入输出模式和协议绑定，结合共享能力词汇及本地语义搜索引擎建立索引。查询时，自然语言任务用于语义召回，能力标签和资源类型用于结构化过滤，用例与能力描述用于解释匹配原因；随后再叠加授权域、生命周期和发布状态检查。这一链路把 DID 方法中的身份语义扩展与 OAN 的语义治理、发现协议连接起来。

同一资源版本的绑定关系可以用如下摘要检查。三个哈希的用途不同，资源 DID 只能定位对象，不能替代对文档和内容的完整性检查：


```
{
  "resourceDid": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "packageVersion": "1.0.0",
  "didDocumentHash": "sha256:example-did-document",
  "metadataHash": "sha256:example-metadata",
  "packageHash": "sha256:example-package",
  "checks": {
    "didDocumentIdMatchesResourceDid": true,
    "resourceTypeMatchesSemanticCode": true,
    "metadataHashRecomputed": true,
    "packageHashRecomputed": true
  }
}
```

如果 DID 文档内容变化，即使资源 DID 不变，`didDocumentHash` 也应变化并触发新的控制证明和资源网络验证；如果只查询 DID 字符串而不比较文档哈希，调用方无法判断拿到的是哪个版本的文档。

6.8 控制证明与 DID Document 证明

资源控制证明由控制方使用与 DID 文档中验证方法对应的私钥生成，证明其对特定 DID、文档哈希、操作目的和挑战具有控制权。注册服务节点验证该证明，根平台在接受资源版本前再次验证；注册凭证证明登记处理事实，根平台可信发布证明证明网络发布事实，三者不能互相替代。

DID 文档证明应覆盖规范化后的文档内容或规定的文档哈希，并包含可定位的验证方法、创建时间、证明用途和签名值。缺少证明、证明过期、目标 DID 不匹配、文档哈希不匹配、用途不匹配或签名无法验证时，验证必须失败且不得产生已发布状态。

OAN 控制挑战的验证输入应把请求上下文绑定到签名，而不是只验证一段脱离上下文的签名。安全库的检查重点可以抽象为：

```
{
  "challenge": {
    "challengeId": "challenge-example",
    "draftId": "draft-example",
    "subjectDid": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "registrarDid": "did:oan:INRG:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "purpose": "register_resource",
    "verificationMethod": "did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
    "didDocumentHash": "sha256:example-did-document",
    "expiresAt": "2026-09-09T09:00:00Z"
  },
  "verification": {
    "signatureValid": true,
    "challengeMatchesRequest": true,
    "notExpired": true,
    "proofPurposeMatches": true
  }
}
```

注册服务节点应把挑战中的主体 DID、目标注册节点、用途、文档哈希和有效期与当前请求逐项比较。挑战通过只说明控制方授权了这一次具有关联上下文的操作，不等于注册服务节点已经形成登记记录，也不等于根平台已经发布资源。

6.9 DID 生成与本地密钥处理

客户端是 DID 和控制密钥的首要生成与保管边界。随机 DID 由密码学随机数生成，确定性 DID 由语义编码、控制材料和 nonce 派生；无论采用哪种方式，生成结果都必须经过方法解析和资源类型一致性校验。注册服务节点只应接收公开 DID 文档、签名和证明，不应取得主体或资源控制私钥。

本地身份备份是恢复控制关系的敏感材料，不是公开凭证。导入时应校验 DID、密钥、公钥、控制关系和文件结构的一致性；恢复后生成的控制证明必须重新绑定当前操作和文档哈希，不能把历史签名当作所有未来版本的无条件授权。

客户端处理本地身份时，可以按以下顺序减少密钥泄露和错误绑定风险：

1. 在本地生成或导入主体及资源控制密钥，并检查公私钥是否匹配。
2. 根据资源类型生成或解析 `did:oan`，检查语义编码与 `resourceType` 是否一致。
3. 构造 DID 文档，确认验证方法、用途引用和 `oanMetadata` 指向同一资源 DID。
4. 对当前文档和操作生成新的挑战控制证明，不复用与旧文档绑定的签名。
5. 仅向注册服务节点提交公开 DID 文档、证明和必要的资源元数据，不上传私钥。

浏览器下载的本地身份备份属于用户侧恢复材料，不能作为服务器端保存密钥的替代品。页面显示“已创建”或下载完成只表示客户端完成了文件操作，用户仍需确认文件内容完整并按本地安全要求保管。

6.9.1 DID 生成

客户端可调用方法库的随机生成或确定性派生接口创建 DID。生成参数中的语义编码必须合法；确定性派生只有在控制材料和 nonce 均相同且实现使用同一方法版本时才产生相同结果，不能把派生算法当作从 DID 反推出私钥的机制。

6.9.2 密钥生成与存储

控制密钥应在用户可控制的客户端环境生成或导入，私钥以本地安全存储或加密备份方式保存。DID 文档、注册请求和发现响应只公开公钥及验证所需材料；服务器日志、请求体和错误响应不得包含私钥或可恢复私钥的明文材料。

6.9.3 本地身份备份

本地身份备份可以包含主体 DID、资源 DID、密钥材料、资源配置和必要凭证，应明确其敏感级别、创建时间和恢复用途。备份文件名、下载位置和页面展示文本不承担身份验证功能，用户必须独立保管备份并防止未经授权复制。

6.9.4 导入、恢复与密钥使用边界

导入或恢复后，客户端应确认公私钥对应、DID 文档主体一致、资源类型一致，并在签名前重新生成当前挑战 and 哈希。注册服务节点、根平台和发现服务节点可以验证公开证明，但不应从导入流程获得私钥；节点运行私钥与用户资源控制私钥也必须分离。

6.10 DID 解析要求

解析请求首先验证 DID 方法和语法，再由解析服务取得当前可用的 DID 文档或明确的历史版本。成功响应应包含文档及其可验证元数据；未找到、方法不支持、格式非法、资源已停用或历史版本不可用应使用可区分的错误或状态，而不能返回空文档并让调用方猜测原因。

解析成功只说明解析器能够返回一个 DID 文档，不说明资源已登记、根平台已发布、凭证仍有效或业务端点可用。当前资源详情或 DID 文档 HTTP 接口的返回类型由具体节点决定：内容分发平台的 GET /cdn/documents/{did} 直接返回 DidDocument，而某些目录接口会把 DID 放在 did 字段并将文档放在 document 字段中；客户端必须按实际接口契约解析，不能把所有 DID 读取接口假定为同一种包装结构。调用方应根据资源网络证据、治理状态、版本和本地策略继续判断。

解析错误应使调用方能够区分输入错误、方法能力和网络状态。可采用如下结果分类表达，具体 HTTP 映射由解析服务接口决定：

结果类别	方法层判断	调用方处理
invalid_did	DID 前缀、分段、语义编码或后缀不合法	修正输入，不发起业务调用
method_not_supported	当前解析器不支持 did:oan	更换支持该方法的解析器
not_found	方法合法但没有可返回的 DID 文档	区分未登记与暂时不可达
resolved	已取得并通过基本结构检查	继续验证控制、发布和端点
status_restricted	文档或资源状态限制当前使用	保留历史证据，停止敏感动作

解析器不应通过返回一个空对象来掩盖 not_found 或 method_not_supported，也不应把发现节点的旧快照直接当作当前 DID 方法解析结果。

与 A2A、MCP 等外部协议的互操作属于解析结果的消费和适配问题，不改变 did:oan 的标识符、DID 文档或控制关系规则。解析器可以保留 oanMetadata、服务端点和协议绑定，供适配器映射为外部协议描述；调用方仍须独立验证 DID、资源包、控制证明和根平台发布证据。

6.11 DID 更新、版本管理、暂停、恢复与停用

同一资源的更新通常保持资源 DID 不变，通过新的 DID 文档版本、资源包版本、哈希和控制证明表达变化。更新方必须使用当前有效控制关系签名，根平台和相关节点应检查版本顺序、上一版本引用、文档哈希及资源生命周期状态；方法库本身只提供 DID 解析和类型校验，不替代网络更新协议。

同 DID 更新的最小判断可以归纳为以下关系：

```
current DID + current control proof + new document hash
-> verify current control and version relation
-> accept a new resource version or reject the update
```

更新请求中的 DID 与旧记录不一致时，应按新资源或非法替换处理，不能由接收时间决定覆盖对象；DID 相同但控制证明无效时，也不能因为资源名称、端点或提交方相同而覆盖旧文档。

暂停、恢复和停用是资源网络状态动作，必须绑定明确的控制证明、治理事实或授权条件。历史文档可以为审计和历史验证保留，但解析当前状态时不得把旧文档或旧发布证明当作最新状态。

6.11.1 同 DID 更新

同 DID 更新要求新材料中的 `id` 与既有资源 DID 相同，并由当前控制密钥对新文档或规定的文档哈希签名。控制关系发生变化时，应先满足密钥轮换或恢复规则；仅提交相同 DID 和一把新公钥不能覆盖现有文档。

6.11.2 版本选择

当前版本应由资源版本、文档哈希、包哈希和发布序列共同确定。解析器或发现服务节点选择最新版本时，应依据可信发布事实和生命周期状态，而不是仅按接收时间或本地数据库自增编号选择；历史版本查询需要明确版本标识或发布引用。

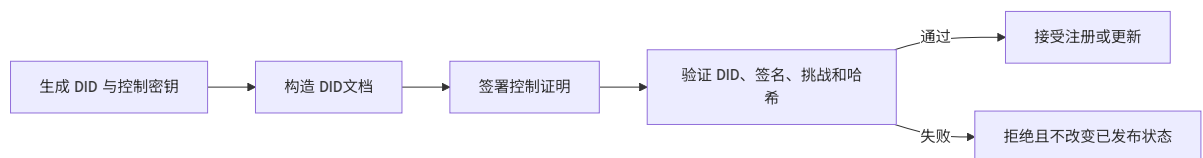
6.11.3 暂停与恢复

暂停应使规定范围内的发现、发布或调用前验证停止把资源当作 `active`；恢复需要新的有效状态依据，并重新检查版本和发布证据。缓存的旧文档可以用于历史展示，但不能绕过当前暂停状态。

6.11.4 停用与历史状态

停用表示 DID 或资源在规定范围内不再接受新的有效使用，历史文档和证明仍可为审计保留。停用、资源撤销、注册凭证吊销和端点下线应分别记录，不能用一个页面状态字段代替全部事实。

同 DID 更新、暂停、恢复和停用的基本控制路径如下。图示只表达方法层与资源网络层之间的验证顺序，不表示方法库单独完成注册或发布：



6.12 DID 方法安全与隐私考量

安全性来自私钥保密、签名覆盖范围、DID 文档公钥绑定、挑战和新颖度检查，而不是来自 DID 字符串不可见或格式复杂。攻击者可以复制公开 DID 文档和公钥，但在没有有效控制私钥、当前挑战和相应授权的情况下，不能产生可被接受的更新证明。随机 `nonce`、请求时间、文档哈希和用途字段应参与重放防护。

方法实现应避免将主体关系、资源内容和端点隐私无条件集中暴露。DID 文档中只放置公开验证和发现所需的信息；敏感凭证、私钥和内部地址应通过受控端点或外部授权机制处理。失败校验不得写入已发布状态，错误日志应保留足够审计信息但不得泄露私密材料。

6.13 DID 方法符合性要求与测试向量

符合性测试至少应覆盖：合法 DID 解析；错误前缀、段数、语义编码、后缀长度和非法字符拒绝；四类资源代码映射；随机生成结果可重新解析；确定性派生在相同输入下稳定、在控制材料或 nonce 变化时改变；DID 与资源类型不一致时拒绝；规范化前后 DID 语义不变。

签名测试还应覆盖验证方法用途不匹配、目标 DID 不匹配、文档哈希不匹配、过期挑战、重复 nonce、错误公钥和旧密钥更新等情况。测试证据应记录固定输入、方法版本、预期错误和实际输出；方法层测试通过不等于注册、发布或发现链路已经通过。

方法实现可以使用下列最小测试向量作为输入分类的示例：

输入	预期结果	失败或通过依据
did: oan: SKFI: 5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	通过	四段结构、4 位语义编码和 32 位 Base58 后缀成立
did: oan: skfi: 5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	规范化后再判断	仅语义编码大小写可按 SDK 规则规范化
did: oan: SKFI: 0 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	拒绝	后缀含 Base58 禁止字符 0
did: oan: SK: 5 HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu	拒绝	语义编码不是 4 位
合法 DID + resourceType: tool_api	拒绝	SK 与资源类型不一致

测试向量中的 DID 和后缀仅用于验证方法实现，不应直接作为生产资源身份。测试还应记录解析器版本、规范化前后输入、错误类型和是否产生任何持久化副作用；单纯返回错误字符串不足以证明发布状态没有被修改。

6.14 DID 方法互操作与解析器行为

did: oan 应遵守 DID Core 对 DID、DID 文档、验证方法和服务的基本数据模型，同时由方法实现规定自己的方法特定标识符和解析行为。外部解析器至少应能识别方法名并拒绝不符合方法语法的标识符；支持 OAN 方法的解析器还应保留 oanMetadata 及其扩展字段，不能在转换过程中丢失资源类型、能力和发布引用。外部系统消费公开信息时，仍须按 OAN 规则验证控制关系、资源包和根平台发布证据。

did: oan 标识符由方法名和方法特定标识符组成。下例只用于说明结构和字段关联，示例值不代表真实网络对象，也不应被当作可直接发布的控制材料。

did: oan: <主体类型或资源类型编码>: <方法特定标识符>

```
{
  "id": "did: oan: SKDM: example-resource-id",
  "verificationMethod": [
    {
      "id": "did: oan: SKDM: example-resource-id#key-1",
```

```

    "type": "JsonWebKey2020",
    "controller": "did: oan: SKDM: example-resource-id",
    "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "..." }
  },
],
"authentication": ["did: oan: SKDM: example-resource-id#key-1"],
"service": [
  {
    "id": "#resource",
    "type": "OANResource",
    "serviceEndpoint": "https://example.invalid/resource"
  }
]
}

```

检查对象	必须回答的问题	失败后果
DID 字符串	方法名、编码、字符和类型是否符合规则	拒绝解析或进入无效状态
DID 文档 id	是否与请求中的资源 DID 完全一致	拒绝绑定
公钥与控制关系	验证方法是否由 DID 控制方声明并可验签	不接受控制证明或更新
签名覆盖范围	签名是否覆盖目标 DID、文档/包哈希、挑战和用途	拒绝签名事实
service 与元数据	端点、协议和资源描述是否一致	不进入可信发现或调用流程

方法层符合性测试应把合法解析、非法输入拒绝、DID 类型映射、文档绑定、签名用途、挑战新鲜度和更新控制权分别记录。方法测试通过，只能证明 DID 方法行为符合测试向量，不能替代注册服务节点、根平台和发现服务节点的业务链路验证。具体生成、签署、验证和拒绝路径已经在 5.9、5.10、5.11 和 5.13 中说明，本节只规定跨解析器消费时不得丢失方法语义。

6.15 参考来源

来源	类型	链接
oan-protocol-common	代码仓: did: oan 核心类型和方法实现	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓: TypeScript DID 解析、生成和验证	https://github.com/OpenAgenet/oan-sdk-ts
OAN Yellow Paper	arXiv 黄皮书: 身份、控制权和资源发布机制	https://arxiv.org/abs/2606.03163
did: oan DID Method Specification	标准/方法规范: did: oan 标识格式和解析规则	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

来源	类型	链接
OAN Resource Identity and Discovery	IETF 草案：资源身份和发现	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

第七章 资源元数据与资源包类型

本章规定资源元数据和 ResourcePackage 的数据边界、字段关系、确定性哈希和接收方验证方式。资源元数据描述资源是什么、能够做什么以及如何连接，ResourcePackage 将特定资源版本的 DID 文档、元数据、哈希和根平台可信发布证明组织为可分发对象。注册提交是进入验证流程的输入，根平台验证资源包是经过网络级验证后的发布产物，三者不能互相替代。

本章以 oan-protocol-common 的资源类型、序列化实现和资源包校验代码为主要依据。当前注册 payload schema 只提供对象级入口约束，具体字段语义由协议类型和校验逻辑共同确定；当 schema、实现和部署配置存在差异时，应标明适用版本和差异，不能把单一示例直接视为所有实现的强制行为。

7.1 资源元数据模式

资源元数据在 Rust 实现中由 ResourceMetadata 表达，核心字段如下：

字段	类型	语义与校验重点
resourceDid	字符串	资源 DID，必须与 ResourcePackage 和 DID 文档的 id 对应
resourceType、subjectType	枚举	资源及其主体类型，当前资源包校验要求二者一致
publisherDid、subjectDid	可选字符串	发布方和主体关系声明，不单独证明控制权
name、description	字符串	资源名称与概要描述
capabilityTags	字符串数组	能力标签，供结构化和语义发现使用
authorizedDomains	字符串数组	允许传播或发现的授权域
protocolBindings	对象数组	协议、版本、传输方式及服务或 schema 引用
services	服务端点数组	资源业务入口或协议入口
lifecycleState	字符串	当前资源版本的生命周期状态
packageVersion	字符串	当前资源包版本
packageHash、metadataHash	字符串	资源包和元数据完整性引用
hashAlgorithm	字符串	哈希套件标识，须与哈希引用保持一致
updatedAt	UTC 时间	当前元数据版本的更新时间

这些字段可以公开用于注册、分发和发现，但控制私钥、内部访问凭证和不应公开的运行细节不得放入元数据。未知扩展字段只有在不改变核心字段含义、哈希输入和验证结果时才可保留；核心字段类型错误、资源 DID 缺失或枚举值不受支持时，接收方应拒绝。

下面的片段展示一个 `ResourceMetadata` 实例的字段关系；哈希值使用示例值，不能直接作为生产材料。`packageHash` 和 `metadataHash` 在元数据哈希计算阶段需要先清空，之后再回填计算结果。

```
{
  "resourceDid": "did:oon:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "subjectType": "skill",
  "publisherDid": "did:oon:ORLG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "subjectDid": "did:oon:DVLG:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "name": "Repository Structure Summary",
  "description": "Summarizes the structure of a code repository.",
  "capabilityTags": ["code-repository", "project-structure-summary"],
  "authorizedDomains": ["public"],
  "protocolBindings": [],
  "services": [],
  "lifecycleState": "active",
  "packageVersion": "1.0.0",
  "packageHash": "sha256:example-package",
  "metadataHash": "sha256:example-metadata",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-09T08:30:00Z"
}
```

7.2 必填字段与可选字段

`ResourcePackage` 的结构性必需字段包括 `packageVersion`、`resourceDid`、`resourceType`、`didDocument`、`didDocumentHash`、`metadataHash`、`packageHash`、`hashAlgorithm`、`metadata`、`rootProof` 和 `createdAt`。元数据中的资源 DID、资源类型、主体类型、包版本、三个关键哈希和哈希算法是形成可验证发布对象的核心字段；能力标签、授权域、协议绑定和服务端点可以为空数组或按资源类型不声明，但省略后不得宣称具备相应能力。

四类资源共享公共身份、描述、版本、完整性和治理字段。智能体服务通常需要能力描述、用例、交互协议和服务端点；技能包需要包入口、安装或加载信息及依赖；MCP 服务需要传输方式、工具或资源清单和认证要求；工具 API 需要操作、参数、输出和调用约束。具体必填性以对应版本的协议模型为准，网页输入项不能直接替代底层 schema。

协议要求的字段为空时必须拒绝，可选字段缺失时按“未声明”处理；未知扩展字段可以在兼容模式下保留，但不得被用于无法解释的权限判断。注册服务节点应在登记前检查，`ResourcePackage` 接收方应再次检查。

四类资源共享资源 DID、资源类型、主体类型、描述、能力标签、版本、授权域和完整性字段，但资源类型决定哪些能力描述和端点信息具有实际意义。字段缺失时，应区分“未声明”与“结构不完整”：例如没有可选服务端点可以表示尚未声明调用入口，但缺少 `resourceDid`、资源类型、DID 文档或资源包哈希则无法形成可验证资源包。

字段范围	四类资源共同要求	类型相关的补充
身份和类型	<code>resourceDid</code> 、 <code>resourceType</code> 、 <code>subjectType</code>	三者需要与 <code>did:oon</code> 语义编码一致

字段范围	四类资源共同要求	类型相关的补充
能力描述	名称、描述、标签和用例可以用于发 现	智能体服务强调任务和交互，技能包 强调复用与依赖
连接信息	协议绑定和服务端点按需声明	MCP 服务和工具 API 通常需要更 明确的协议或接口说明
完整性和版本	包版本、三类哈希和哈希算法	内容变化必须形成新的验证和发布判 断

7.3 资源类型与主体类型约束

资源类型与 did:oon 语义编码应保持一致：agent_service 对应 AG，skill 对应 SK，mcp_server 对应 MC，tool_api 对应 TL。资源元数据中的 resourceType 和 subjectType 在当前资源包校验中必须相同，并且都应 与资源 DID 的主体代码一致。基础设施节点、组织和开发者可以作为方法层标识对象，但不能把节点 DID 直接填充为产品资源 DID。

一个主体可以控制多个资源 DID，一个资源提供方也可以运营多个资源；主体 DID、资源 DID 和节点 DID 的身份边界不因关系字段而合并。资源类型、主体类型、DID 文档 id 或包内元数据不一致时，验证方必须拒绝该资源包。

7.4 名称、描述、标签与用例

名称和描述是资源提供方对资源能力的公开声明，能力标签和用例是供注册推荐、语义治理和发现检索使用的信息。标签应优先引用根平台能力标签树中的标识，扩展标签应标明来源或命名空间。自然语言可使用中文、英文或其它声明语言，但字段类型和数组结构必须稳定，不能混入 HTML、脚本或不可解析对象。描述详细不等于能力已验证，相关性排序也不构成控制权或业务质量证明。

名称和描述是资源提供方对资源能力的公开声明，能力标签和用例是供注册推荐、语义治理和发现检索使用的信息。标签应优先引用根平台能力标签树中的标识；扩展标签应标明来源或命名空间。自然语言可使用中文、英文或其它声明语言，但字段类型和数组结构必须保持稳定，不能将 HTML、脚本或不可解析对象混入文本字段。

描述和用例越详细并不等于能力已经验证，发现节点的相关性排序也不构成控制权、端点可用性或业务质量证明。长度、数量和内容过滤应由部署配置或符合性配置明确，并在接入和索引前执行。

能力标签和用例进入发现流程时，可以同时作为结构化条件和语义检索文本，但结果应保留原始值及其来源。例如，查询“我需要一个可以检索代码仓库并总结项目结构的工具”可以命中相应的能力描述和用例；是否可调用仍需继续检查资源 DID、发布证明、端点和凭证要求。标签命中只是相关性证据，不是资源控制权或业务效果的证明。

7.5 语义标签与协议绑定

capabilityTags 表达资源能力分类，authorizedDomains 表达传播和发现权限范围，二者不能互换。protocolBindings 中的绑定对象至少应说明协议名称，并可包含版本、传输方式、serviceRef 和 schemaRef；协议绑定描述接入方式，不自动赋予调用权限。注册服务节点可以据此提供标签推荐，发现服务节点可以据此进行结构化或语义检索，但返回结果仍须保留来源和验证状态。

`capabilityTags` 表达资源能力分类，`authorizedDomains` 表达传播和发现权限范围，二者不能互换。`protocolBindings` 中的绑定对象至少应说明协议名称，并可包含版本、传输方式、`serviceRef` 和 `schemaRef`；协议绑定描述接入方式，不自动赋予调用权限。标签树版本、标签标识和别名应保留来源，以便不同节点解释同一标签。

注册服务节点可以根据能力描述和标签树提供推荐，发现服务节点可以据此进行结构化或语义检索，但对外返回结果时仍须保留原始标签、协议绑定、来源和验证状态。标签匹配成功而根平台发布证明缺失时，不得进入可信发现结果。

协议绑定的最小表达可以写成如下对象，`serviceRef` 和 `schemaRef` 是资源内部引用或外部 schema 引用，不能被解释为新的身份：

```
{
  "id": "mcp-http",
  "protocol": "MCP",
  "version": "2025-06-18",
  "transport": "streamable-http",
  "serviceRef": "#mcp-service",
  "schemaRef": "https://example.invalid/schema/mcp.json"
}
```

7.6 服务端点与调用要求

资源端点由服务端点数组表达，核心结构包括端点 ID、服务类型和 `serviceEndpoint`，并可包含版本、协议、服务器类型和端口。端点可以是智能体交互入口、MCP 服务入口、工具 API 地址、资源包地址或协议描述地址；端点类型和协议应能解释其用途，多个端点之间的版本和认证差异应明确。认证声明可以表达凭证类型、签发方、范围、呈现方式或调用前提，但只是公开调用要求；端点存在或 HTTP 可达不等同于调用方已经获得访问许可。敏感 token、私钥和内部网络凭据不得作为公开元数据发布。

7.7 资源版本与兼容性字段

`packageVersion` 标识资源包版本，资源描述中的 `version` 或协议绑定中的版本可以表达资源能力或协议版本，不能无说明地相互替代。`previousVersion`、版本约束或发布说明可以建立更新链和兼容关系，但版本字符串本身不证明内容来源。每个版本都必须绑定资源 DID、DID 文档哈希、元数据哈希、资源包哈希和哈希算法；同一资源 DID 的新版本应通过新的资源包和控制证明进入验证流程。

版本比较至少要把资源包版本、协议版本和资源生命周期分开记录：

信息	示例	判断对象
资源包版本	1.2.0	当前资源材料及其哈希是否为新版本
协议版本	MCP 2025-06-18	调用方和端点能否按同一协议交互
版本约束	>=1.0.0	组合关系允许使用哪些目标版本
生命周期状态	active	当前版本是否仍处于可继续传播的状态

版本字符串相同也不代表内容相同，接收方仍应检查资源 DID、三类哈希、根平台证明和治理状态；内容变化却沿用旧哈希时，应按完整性失败处理。

7.8 包哈希与元数据哈希

当前实现分别计算 DID 文档哈希、资源元数据哈希和资源包哈希。哈希值以算法前缀表达，例如 `sha256:<digest>; hashAlgorithm` 必须与哈希引用的算法前缀一致。元数据哈希计算前会清空元数据自身的 `metadataHash` 和 `packageHash`，避免循环依赖；资源包哈希只对包版本、资源 DID、资源类型、DID 文档哈希、元数据哈希和哈希算法等固定声明计算。验证方应使用相同密码套件和序列化方式重算三类哈希；哈希匹配不能单独证明控制权、节点授权或端点可用性。

资源包哈希的输入不是整个资源包的递归序列化，而是当前实现明确选取的包声明字段：

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did:oon:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "didDocumentHash": "sha256:example-did-document",
  "metadataHash": "sha256:example-metadata",
  "hashAlgorithm": "sha256"
}
```

该输入用于说明 `verify_package_hash` 的字段边界；实际哈希值应由实现按统一密码套件计算，不能把示例摘要写入生产包。元数据哈希则针对清除自身两个哈希字段后的 `ResourceMetadata` 计算，二者的输入集合不能混用。

7.9 规范化序列化与确定性哈希

确定性哈希要求实现使用相同字段名、枚举序列化、数组顺序、时间格式、空值处理和字符编码。当前 Rust 实现通过 `serde` 序列化对象后交由统一哈希函数处理；跨语言实现不得改变字段命名、补入任意默认值或在哈希前排序数组，除非规范版本明确规定。元数据和包哈希均不应把自身输出字段再次纳入输入；规范化失败、对象无法序列化或算法不支持时，应停止构造并返回错误。

跨语言复现时应把哈希前对象记录下来，便于定位“字段不同”和“序列化不同”两类问题：

```
ResourceMetadata for hashing
-> clear metadataHash and packageHash
-> serialize with protocol field names and stable enum values
-> hash with the selected crypto suite
-> write metadataHash back to the package

Package hash input
-> packageVersion, resourceDid, resourceType
-> didDocumentHash, metadataHash, hashAlgorithm
-> serialize and hash
```

如果只改变元数据中的描述、标签顺序、授权域或时间字段，规范化后的元数据输入就会变化，接收方应得到不同的元数据哈希；如果只回填计算结果而不改变其它输入，已有哈希不应再次变化。

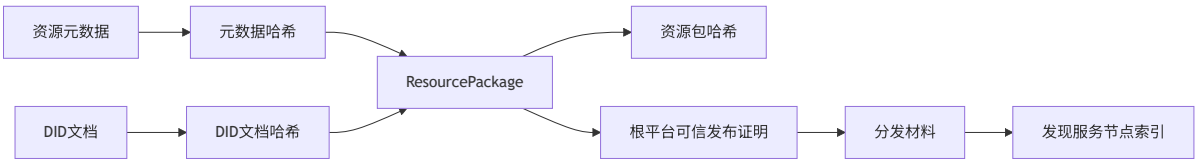
7.10 ResourcePackage 构造规则

ResourcePackage 的构造顺序是：确定资源 DID、资源类型、DID 文档和元数据；计算 DID 文档哈希和元数据哈希；计算资源包哈希；组织版本、哈希、生命周期状态和授权域声明；由资源控制方提交控制证明，并由注册服务节点和根平台分别处理其登记与发布职责。根平台可信发布证明只能在根平台验证通过后产生，不能由客户端预先构造为已发布事实。

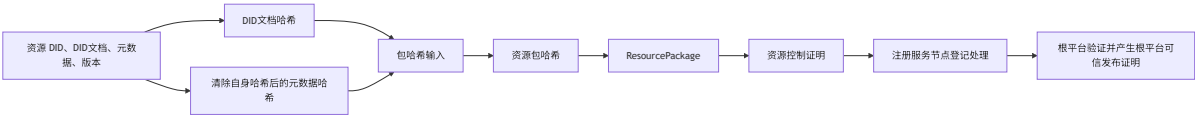
资源元数据、DID 文档、资源包、根平台可信发布证明和发现索引记录的责任边界如下：

对象	主要内容	产生方	接收方的核心检查
资源元数据	类型、描述、标签、用例、端点、版本和授权域	资源提供方/控制方	字段、类型、语义和元数据哈希
DID 文档	DID、验证方法、公钥和服务	资源控制方	DID 解析、控制关系和文档哈希
资源包	DID 文档、元数据、版本、哈希和状态	构造方	结构、三类哈希和字段一致性
根平台可信发布证明	根平台对特定版本的验证与发布声明	根平台	签发者、覆盖范围、签名和状态
发现索引记录	本地验证、授权域、新鲜度和同步位置	发现服务节点	来源引用、治理状态和索引时效
内容分发材料	已验证资源包及其传播引用	内容分发平台	来源、哈希、版本和发布授权

构造和发布之间的关系可以表示为：



构造过程可以用一条可复核的依赖链表示：



图中的登记处理和根平台验证属于资源网络阶段，不能提前写入客户端构造的普通资源包。每一步都应保留输入版本和输出哈希；任一步输入改变，都应重新执行后续依赖步骤。

7.10.1 输入材料

输入材料包括资源 DID、资源类型、DID 文档、资源元数据、资源包版本、哈希算法、资源控制证明和必要的注册凭证或来源引用。DID 文档 id、元数据 resourceDid、包 resourceDid 和控制挑战目标 DID 应一致，资源类型和主体类型应与 DID 语义编码一致。

7.10.2 字段规范化

构造方应使用协议字段名和确定性序列化规则，固定时间为可解析的 UTC 表示，保持数组顺序和枚举值稳定。计算元数据哈希前清除元数据中的 metadataHash、packageHash，计算包哈希时只纳入规定的包声明字段。

这些字段通过 `serde` 映射到 HTTP JSON，而不是在编译时自动生成一份独立的接口模式。`ResourceRegistrationSubmission` 的 `didDocument` 使用 DID 文档类型，`metadata` 使用动态 JSON；`ResourcePackage` 的 `metadata` 使用资源元数据类型，`rootProof` 使用根平台证明类型。接收方因此既要依据 Rust 类型完成反序列化，也要执行资源类型、哈希和证明的业务校验；能够反序列化不等于能够发布。

7.10.3 哈希计算

依次计算 DID 文档哈希、元数据哈希和资源包哈希，并将算法前缀写入相应字段。回填哈希后不得改变参与哈希的其它字段；任一重算结果变化都表示当前版本内容发生变化。

7.10.4 签名与证明生成

资源控制方对控制挑战和规定的资源控制材料签名，注册服务节点可以对登记事实签发注册凭证，根平台在验证节点授权、控制证明、哈希和发布条件后生成根平台可信发布证明。证明的签发者、验证方法、目标 DID、用途和覆盖哈希必须可核对。

7.10.5 构造后的校验

构造完成后应依次检查资源类型一致性、元数据一致性、DID 文档哈希、元数据哈希、资源包哈希和根平台声明绑定。任何检查失败都不得将对象标记为可分发或可信发布；失败对象可以作为本地诊断材料保存，但不应进入公开索引。

构造后的检查结果可以记录为机器可读的诊断摘要：

```
{
  "resourceDid": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "packageVersion": "1.0.0",
  "checks": [
    { "name": "resource_type_consistency", "status": "passed" },
    { "name": "metadata_consistency", "status": "passed" },
    { "name": "did_document_hash", "status": "passed" },
    { "name": "metadata_hash", "status": "passed" },
    { "name": "package_hash", "status": "passed" },
    { "name": "root_claim_binding", "status": "passed" }
  ],
  "publishable": true
}
```

`publishable` 只表示本地对象通过当前包校验，不能替代注册服务节点受理或根平台发布。若任一检查失败，诊断结果应包含对应错误类别，包不得进入可信分发或发现索引。

7.11 根平台验证的 ResourcePackage 结构

根平台验证后的 `ResourcePackage` 仍然以资源控制方提供的 DID 文档、资源元数据和哈希为基础，但新增或绑定根平台对该特定资源版本的验证结果。`rootProof` 中的根平台 DID、`packageClaims`、签名或数据完整性证明，以及必要的公告事件引用，共同表达根平台已经检查并接受哪些事实。发布序列、发布时间和同步游标属于发布或传播层信息，不能倒写成资源提供方原始声明。

7.11.1 资源身份部分

资源身份部分包括 `resourceDid`、`resourceType`、包内 DID 文档及 `didDocumentHash`。接收方应检查包字段与 DID 文档 `id` 一致，解析 `did: oan` 并验证资源类型映射；身份部分有效只说明包绑定了一个可解析资源 DID，不说明资源已被根平台发布。

7.11.2 资源元数据部分

资源元数据部分包括名称、描述、能力标签、用例、授权域、协议绑定、服务端点、生命周期和版本信息。接收方应检查元数据哈希、资源类型、资源 DID、包版本、授权域和生命周期与包外声明一致，并保留元数据来源和更新时间。

7.11.3 根平台验证证明

根平台验证证明至少应能关联根平台 DID、资源 DID、资源类型、包版本、DID 文档哈希、元数据哈希、包哈希、哈希算法、生命周期状态和授权域。`packageClaims` 与包内容不一致、证明签名无法验证或证明来源不在信任范围内时，包不得被视为根平台验证资源包。

7.11.4 分发和索引所需字段

分发和索引至少需要资源 DID、资源类型、包版本、三类哈希、根平台证明、生命周期状态、授权域、来源引用和发布或同步游标。发现服务节点还应记录本地接收时间、验证时间、当前同步位置和拒绝原因，以便解释旧快照、索引滞后和跨节点不一致。

7.12 ResourcePackage 完整性与来源验证

接收方应先验证结构和资源身份，再按 DID 文档哈希、元数据哈希、资源包哈希、字段一致性和根平台证明的顺序完成验证，最后结合生命周期、授权域、版本和治理状态决定是否写入分发缓存或发现索引。哈希或证明失败必须阻止进入可信读模型；网络暂时不可用可以进入待重试状态，但不得把未完成验证的对象标记为已发布。

接收方可以把验证结果划分为“通过”“拒绝”和“待重试”三类：

结果	典型原因	是否进入可信读模型	后续动作
通过	结构、哈希、来源和状态均可验证	可以	按版本和授权域写入分发或索引
拒绝	哈希不一致、证明无效、类型冲突或来源不可信	不可以	保存错误和证据，要求修正或重新提交
待重试	上游暂时不可达或治理投影尚未同步	不可以标记为已发布	保留待处理任务，恢复后重新验证

“待重试”不是一种已验证状态。即使包的 JSON 结构正确，只要根平台证明或必要来源无法确认，接收方也只能保留未完成状态。

7.13 ResourcePackage 来源链与来源归属

资源来源链至少包含五类事实：资源控制方产生 DID 文档、元数据和控制证明；注册服务节点产生登记受理和注册凭证；根平台产生资源版本接受、根平台可信发布证明和发布引用；内容分发平台转发或缓存根平台允许传播的材料；发现服务节点产生本地验证和索引记录。每类事实都应标识来源主体、目标资源 DID、版本、时间或游标，并保持签名或哈希关联。

来源链可以按“声明、登记、发布、传输、索引”五个阶段阅读：

资源控制方

-> DID文档、资源元数据、控制证明

注册服务节点

-> 登记记录、注册凭证

根平台

-> 接受记录、根平台可信发布证明、发布引用

内容分发平台

-> 已验证材料的传输或缓存记录

发现服务节点

-> 本地校验、同步位置和发现索引记录

后一个阶段只能补充自己的处理事实，不能替前一个阶段签发其没有产生的证明。例如发现服务节点的索引记录可以说明它验证并保存了某版本材料，但不能替代资源控制方的控制证明或根平台的发布证明。

7.13.1 资源控制方来源

资源控制方来源由 DID 文档控制关系、资源控制证明及其签名验证方法确定。publisherDid 或 subjectDid 可以补充业务关系，但不能替代控制证明；接收方应将资源方声明与当前 DID 文档和证明一起核对。

7.13.2 注册节点来源

注册服务节点来源由登记请求的接收记录、节点 DID、节点授权材料、注册凭证和处理状态表达。注册凭证的有效范围是登记事实，不自动证明根平台已经接受或发现服务节点已经索引。

7.13.3 根平台发布来源

根平台发布来源由根平台 DID、根平台可信发布证明、资源包声明和发布或公告引用表达。根平台证明必须绑定具体资源版本和哈希；同一资源 DID 的其它版本不能直接复用该证明。

7.13.4 来源链验证

来源链验证要求沿控制证明、注册凭证、根平台可信发布证明和分发记录逐层检查。内容分发平台只证明传输或缓存行为，发现服务节点只证明本地取得、验证和索引行为；任一层缺失都应在响应中反映，不得用下一层的存在掩盖上一层失败。

7.14 包大小、内容限制与校验策略

大小、长度、嵌套深度、端点数量、标签数量和协议绑定数量应由部署或符合性配置明确，并在注册服务节点和 ResourcePackage 接收方执行相同或不低于网络要求的限制。当前 schema 未给出统一业务数值上限时，不应在本章虚构固定数字；实现必须在超限时返回可识别错误，避免超大文本、深层嵌套或大量端点耗尽解析、索引和哈希资源。限制检查应在哈希计算前完成，避免因截断、丢弃或自动压缩产生不同哈希。

限制检查应针对原始输入执行，并把失败原因与字段路径关联起来，而不是静默截断内容：

```
{
  "validation": "rejected",
  "errors": [
    {
      "path": "metadata.capabilityTags",
      "code": "limit_exceeded",
      "message": "capability tag count exceeds the active profile"
    }
  ],
  "hashingStarted": false
}
```

示例中的限制值由实际部署配置决定。只要输入被截断、字段被丢弃或数组被自动重排，计算对象就已发生变化，不能继续使用原始签名或哈希。

7.15 模式演进与向后兼容

模式演进应区分新增可选字段、改变既有语义、删除字段，以及改变哈希输入或签名覆盖范围的升级。新增字段若不影响旧实现解析和既有哈希规则，可以通过版本化扩展使用；若改变资源身份、资源类型、版本、哈希输入、根平台声明或验证流程，则必须提升模式或协议版本，并明确旧节点的处理方式。旧节点遇到未知可选字段时可以忽略，但不得在重新计算已签名对象时静默丢弃被覆盖的字段；遇到未知必需字段、无法解释的哈希算法或证明无法验证时，应拒绝或进入待兼容状态。

本节只规定模式变化对解析和完整性校验的影响。资源元数据、DID 文档、资源包、根平台可信发布证明和发现索引记录的职责边界，分别在 6.1、6.4、6.10 至 6.13 以及第 9、10 章中说明；模式版本变化不自动改变资源控制权、发布状态或发现可见性。接收方仍应按照既有校验顺序重新计算相关哈希并保留失败原因。

7.16 参考来源

来源	类型	链接
oan-protocol-common	代码仓：资源元数据、资源包和 schema	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓：资源包构造、哈希和形状校验	https://github.com/OpenAgenet/oan-sdk-ts

来源	类型	链接
OAN Yellow Paper	arXiv 黄皮书：资源包、哈希和可信发布	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF 草案：资源身份和发现材料	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

第八章 身份、密钥、签名、VC 与 VP

本章统一描述 OpenAget (OAN) 身份材料和密码学证明的语义边界。身份材料回答对象是谁以及谁能够代表对象作出声明；签名回答指定内容是否由相应私钥产生；可验证凭证（VC）回答签发者对哪些事实作出可验证声明；可验证呈现（VP）回答持有者为特定验证目的呈现哪些凭证。四者相互关联，但不能互相替代。

资源注册通常同时涉及主体 DID、资源 DID、资源控制密钥、注册服务节点密钥、节点授权 VC、资源控制证明、注册 VC 和根平台可信发布证明。客户端可以生成和保管私钥，节点可以验证公开公钥和证明，根平台可以验证节点授权及发布条件；上述验证能力不意味着节点或根平台取得资源控制私钥，也不意味着资源已经获得业务调用许可。

8.1 主体身份与资源身份

主体是能够管理一个或多个资源控制关系的身份对象，资源身份是可以独立注册、发布、发现、更新和撤销的对象边界。主体 DID、资源 DID、注册服务节点 DID、发现服务节点 DID 和根平台 DID 都可以采用 `did:oa` 方法，但对象类别、密钥用途和信任来源不同，不能仅因方法相同而混用。

对象	主要证明的问题	典型材料	不直接证明的事实
主体	谁管理资源控制关系	主体 DID 文档、主体控制密钥	资源端点一定可用
资源	哪个资源及其版本被登记	资源 DID 文档、资源包、版本和哈希	调用方已经获准调用
注册服务节点	节点是否有资格受理登记	节点 DID 文档、节点授权 VC	资源业务质量
发现服务节点	是否已同步、索引并按当前节点状态返回资源	节点 DID 文档、索引记录、发现响应和新鲜度信息	资源控制权、根平台发布事实或业务质量
根平台	是否形成网络级发布事实	根平台可信发布证明、治理状态	资源提供方的法律关系

注册时应披露完成验证所需的最小公开材料，例如资源 DID 文档、公开公钥、资源元数据、哈希和控制证明。主体姓名、联系方式、本地身份备份和私钥不是资源发现所必需的公开字段，不能因提交登记而默认进入节点日志、索引或公开响应。

同一注册请求中的身份材料可以按其作用分成三层：

层次	典型材料	说明
身份层	主体 DID、资源 DID、DID 文档和公钥	说明对象是谁，以及验证方从哪里取得公钥

层次	典型材料	说明
控制层	控制挑战、控制证明和文档哈希	说明当前控制方是否授权本次具体操作
网络事实层	注册 VC、根平台可信发布证明和治理状态	说明哪个节点处理或发布了什么事

验证方应从身份层开始，逐层核对控制层和网络事实层。把资源 DID 文本、注册 VC 或某个节点的签名单独拿出来，都不能还原完整的控制与发布关系。

8.2 密钥生成、保管、备份与恢复

密钥应在控制者可管理的客户端或节点安全运行环境中生成、导入和使用。当前 `oan-crypto` 提供 Ed25519 及 SM2/SM3 相关密钥处理，并支持多重编码公钥和 JWK 公钥表示。密码套件、公开密钥表示和证明类型必须能够被接收方识别；遇到未知、截断或不兼容材料时，应明确失败，不得静默改用另一套算法。

密钥材料	持有者	典型用途	对外材料
主体控制私钥	主体或授权控制者	管理主体及相关资源	主体 DID、公钥和控制证明
资源控制私钥	资源控制方	创建或更新资源 DID 文档和版本	资源 DID、公钥和操作证明
注册服务节点私钥	注册服务节点	节点认证、上游请求签名、注册 VC 签发	节点 DID、公钥和授权 VC
根平台密钥	根平台受控运行边界	节点授权、发布证明和服务签名	根平台公钥和可验证证明

本地身份备份可能同时包含主体配置、资源配置、公钥、私钥和凭证，属于敏感材料。导入或恢复时应检查文件结构、DID、公钥与私钥对应关系、资源类型及控制关系；恢复成功只表示客户端重新取得本地控制材料，不表示资源已经登记、发布或重新索引。

本地身份备份与公开 DID 文档的边界可以用最小结构表示：

```
{
  "publicIdentity": {
    "did": "did:oan:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "verificationMethod": "did:oan:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
    "publicKey": "public-key-material"
  },
  "localSecret": {
    "privateKey": "private-key-material",
    "storage": "browser-local-backup"
  },
  "controlledResources": ["did:oan:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu"]
}
```

该结构仅用于说明边界，实际备份字段以客户端实现为准。公开身份材料可以发送给验证方，`localSecret` 只能留在控制者管理的本地安全边界内；服务器取得公钥并不能反推出私钥。

8.3 资源控制签名

资源控制签名必须绑定具体操作，而不是对任意文本作孤立签名。`DidControlChallenge` 将挑战标识、草稿标识、目标资源 DID、DID 文档哈希、注册服务节点 DID、操作目的、验证方法、随机数和有效时间组合起来；`SubjectControlProofBundle` 再关联控制证明、验证时间、实际验证方法和证明哈希。注册服务节点和根平台均应检查这些绑定。

至少应检查以下条件：

1. `subjectDid` 与提交资源 DID 一致。
2. `didDocumentHash` 与提交的 DID 文档一致。
3. `verificationMethod` 属于目标 DID 文档，并且用途允许当前操作。
4. `purpose` 与当前注册、更新或发布操作一致。
5. `nonce`、签发时间和失效时间满足新鲜度和防重放要求。

证明验签成功只能说明相应私钥签署了指定载荷；资源提供方是否有权代表某组织、端点是否安全、调用方是否获得业务授权，仍须由委托材料、治理状态和业务策略分别判断。

控制证明的核心不是签名值本身，而是签名所覆盖的上下文。验证方可以按照下面的关系检查一次注册操作：

```
{
  "operation": "register_resource",
  "subjectDid": "did:oon:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "registrarDid": "did:oon:INRG:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "didDocumentHash": "sha256:example-did-document",
  "challengeId": "challenge-example",
  "nonce": "nonce-example",
  "proofPurpose": "assertionMethod",
  "signature": "signature-example"
}
```

其中 DID、目标注册服务节点、文档哈希、用途和随机数都属于待核对的绑定关系，示例值不代表真实请求。任何一个绑定字段与当前请求不一致，都不能仅因签名算法验证成功而接受注册或更新。

8.4 节点身份密钥与节点签名密钥

节点 DID 文档表达节点身份，通信签名密钥证明请求来自相应节点运行边界，节点授权 VC 证明节点角色和授权域，运行凭据支撑服务启动、密钥加载或受控基础设施访问。实际部署可以由同一安全边界托管多类材料，但协议语义仍应按用途分开。

向根平台发送资源验证请求时，`SignedRequestEnvelope` 将 `requestId`、协议版本、用途、HTTP 方法、路径、受众、请求时间、请求随机数、请求体哈希和 `proof` 绑定。接收方应检查受众、路径、时间窗口、`nonce` 是否重放、节点 DID 文档、公钥和节点授权，再处理业务内容。

`SignedRequestEnvelope` 的验证顺序可概括为：

1. 读取请求中的节点 DID、受众、HTTP 方法、路径和请求体哈希。

2. 解析节点 DID 文档，定位请求使用的验证方法和公开公钥。
3. 校验 proof 的用途、创建时间、请求 ID、随机数和签名载荷。
4. 检查节点授权 VC 的角色、授权域和当前状态。
5. 通过后才处理资源验证、登记或发布业务。

这样可以把“请求来自某个节点”与“该节点有权执行当前动作”分开判断。

8.5 基础设施节点的根平台授权 VC

节点授权 VC 的核心关系是：**issuer** 是根平台或治理认可的签发者，**subject** 是被授权节点 DID，**role** 表示节点角色，**claims** 承载授权域和能力范围，**status**、**issuedAt**、**expiresAt** 描述状态和时间窗口。**NodeAuthorizationCredential** 的 **proof** 必须由 issuer 对无 proof 凭证内容签发，验证方应从 issuer DID 文档取得公钥并验证。

```
节点 DID文档 -> 取得节点验证公钥
-> 验证授权 VC proof
-> 检查 issuer、subject、role、status、issuedAt、expiresAt
-> 检查授权域和能力范围
-> 决定当前节点动作是否获准
```

缺少 proof、issuer 无法解析、subject 不匹配、角色不允许、授权已过期、状态不是 active 或授权域不覆盖请求范围时，受限动作必须拒绝。节点授权 VC 证明基础设施资格，不证明资源控制权。

节点授权 VC 的字段关系可以用如下示例核对：

```
{
  "type": "NodeAuthorizationCredential",
  "issuer": "did:oa:INRT:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "subject": "did:oa:INRG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "role": "registrar",
  "status": "active",
  "issuedAt": "2026-09-09T08:30:00Z",
  "expiresAt": null,
  "claims": {
    "authorizedDomains": ["public"],
    "capabilities": ["resource_registration"]
  },
  "proof": {
    "type": "DataIntegrityProof",
    "proofPurpose": "assertionMethod",
    "verificationMethod": "did:oa:INRT:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
    "proofValue": "signature-example"
  }
}
```

该示例中的角色、授权域和能力值用于说明校验关系。验证方需要确认 issuer、subject、proof 和治理状态相互匹配，再将授权范围与当前请求比较；授权 VC 的有效性不改变资源 DID 的控制者。

8.6 向资源控制方签发的注册 VC

当前注册服务节点流程先调用根平台的资源验证与发布接口；上游成功后，由注册服务节点签发注册 VC，并将登记记录写入存储。凭证包含 OAN 凭证上下文、凭证类型、issuer、资源 credential subject、资源 DID、资源类型、DID 文档哈希、元数据哈希、资源包哈希、资源包版本、哈希算法、授权域、生命周期状态和 credential status。

说明性结构如下，字段顺序不构成额外语义：

```
{
  "type": ["VerifiableCredential", "OANResourceRegistrationCredential"],
  "issuer": "did:oa:...",
  "issuanceDate": "2026-09-06T00:00:00Z",
  "credentialSubject": {
    "id": "did:oa:SKFI:...",
    "resourceDid": "did:oa:SKFI:...",
    "resourceType": "skill",
    "didDocumentHash": "sha256:...",
    "metadataHash": "sha256:...",
    "packageHash": "sha256:...",
    "packageVersion": "1.0.0",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active"
  },
  "credentialStatus": {"status": "active"},
  "proof": {"type": "DataIntegrityProof", "proofValue": "..."}
}
```

注册 VC 证明注册服务节点对指定登记材料完成了处理以及凭证中明确列出的登记事实；它不单独证明根平台已完成全网发布、发现服务节点已完成索引或调用方已经获得业务许可。

注册 VC 中最重要的绑定关系是 issuer、subject、资源版本和三类哈希。验证方应把凭证声明与收到的 DID 文档、资源元数据和资源包重新比较：

```
{
  "type": "AgentRegistrationCredential",
  "issuer": "did:oa:INRG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
  "subject": "did:oa:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "status": "active",
  "issuedAt": "2026-09-09T08:31:00Z",
  "claims": {
    "resourceDid": "did:oa:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "resourceType": "skill",
    "packageVersion": "1.0.0",
    "didDocumentHash": "sha256:example-did-document",
    "metadataHash": "sha256:example-metadata",
    "packageHash": "sha256:example-package",
    "hashAlgorithm": "sha256",
    "lifecycleState": "active"
  },
  "proof": {
    "type": "DataIntegrityProof",

```

```

    "proofPurpose": "assertionMethod",
    "proofValue": "signature-example"
  }
}

```

示例值不代表真实凭证。若 `subject`、`resourceDid`、版本或任一哈希与当前材料不一致，注册 VC 不能用于证明当前材料已经登记；即使凭证签名本身有效，也只能说明签发者签署了另一组声明。

8.7 治理与授权证据

证据	产生者	验证者	证明事实
资源控制签名	资源控制方	注册服务节点、根平台	控制方对特定 DID、文档、版本和操作的授权
节点授权 VC	根平台或治理签发者	节点、根平台和受信组件	节点角色、授权域和有效状态
注册 VC	注册服务节点	控制方、审计方和客户端	登记处理及凭证所列材料关系
根平台可信发布证明	根平台	内容分发平台、发现服务节点和调用方	指定版本通过根平台验证并进入发布流程
治理事件	治理系统或链上记录	链下信任索引器、节点和审计方	授权或治理状态的时间序列事实

验证方必须按事实类型组合证据。注册 VC 有效只能说明登记处理完成；根平台可信发布证明缺失时，不能把登记结果写成全网可发现；资源控制证明不匹配时，不能因节点授权有效而接受资源更新。

8.8 VC 与 VP 结构要求

VC 是签发者对声明事实的可验证表达，VP 是持有者围绕一次验证目的组织的呈现。当前核心代码直接建模节点授权 VC 和资源注册 VC；采用 VP 时应补充 `holder`、呈现目的、`challenge`、有效期和呈现 `proof`，不能把任意 VC JSON 集合直接称为 VP。

8.8.1 VC 基本结构

VC 应至少能定位上下文、类型、`issuer`、`credentialSubject`、签发时间、可选失效时间、状态和 `proof`。OAN 的 Rust 凭证结构使用 `type`、`issuer`、`subject`、`status`、`issuedAt`、`expiresAt`、`claims` 和 `proof`；HTTP JSON 结构可使用 `credentialSubject`、`issuanceDate` 等互操作命名，但序列化映射必须稳定，且不能改变声明语义。

8.8.2 VP 基本结构

VP 应包含呈现类型、`holder`、一个或多个 `verifiableCredential`、呈现目的和针对本次验证的 `proof`。需要防重放时，还应绑定验证方提供的 `challenge` 或 `nonce`。没有 `holder` 证明、用途绑定或本次验证上下文的凭证集合，不应作为新鲜 VP 接受。

VP 的最小示例可以表示为：

```
{
  "type": ["VerifiablePresentation"],
  "holder": "did:oon:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "verifiableCredential": [
    {
      "type": ["VerifiableCredential", "AgentRegistrationCredential"],
      "issuer": "did:oon:INRG:8LcR3Vn5YpQw2Tx7Mb9Zd4Fa6GhKsEuJ",
      "subject": "did:oon:SKFI:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "status": "active",
      "claims": { "packageVersion": "1.0.0" },
      "proof": { "proofValue": "credential-signature-example" }
    }
  ],
  "purpose": "resource_verification",
  "challenge": "verifier-challenge-example",
  "proof": {
    "proofPurpose": "authentication",
    "proofValue": "presentation-signature-example"
  }
}
```

该示例用于区分 VC 的签发证明和 VP 的持有者呈现证明。验证方仍需按实际凭证结构校验 issuer、subject、VC proof、holder、VP proof、challenge 和状态，不能把示例中的凭证对象直接作为可用凭证。

8.8.3 签发者、持有者与主体

issuer 对 VC 声明负责，credentialSubject 是声明所描述的对象，holder 是当前控制呈现材料的一方，controller 是 DID 文档中控制验证方法的主体。四者可以相同，也可以不同；注册服务节点可以是注册 VC 的 issuer，资源 DID 是 credentialSubject，资源控制方可以是 holder，而节点运营者不会因此自动成为资源 controller。

8.8.4 凭证状态与验证

验证不能止于 proof 验签，还应检查凭证类型、issuer 与 DID 文档、公钥、subject、用途、有效期、状态和必要 claims。状态源不可用时，应依据凭证用途采取明确的失败或降级策略；“暂时无法查询”不能被解释为 active。

对于不同材料，验证结果应保留“签名有效”和“业务事实有效”两个层次：

检查结果	可以确认的事实	仍需继续检查的内容
proof 验签通过	指定密钥签署了指定载荷	签发者身份、用途、有效期和状态
VC 状态为 active	凭证当前未被该状态源标记为失效	声明内容是否与当前资源材料一致
节点授权有效	节点具备声明范围内的基础设施资格	当前请求的角色、授权域和资源控制证明
资源控制证明有效	控制方授权了特定资源和操作	注册受理、根平台发布和业务调用许可

8.9 签发者、持有者、控制者与凭证主体关系

这些字段表达的是不同关系，不能根据字段出现就推导完整的所有权和授权链。验证时应同时检查字段关系、DID 文档、公钥、证明用途以及治理和业务策略。

8.9.1 签发者

issuer 使用签发密钥对无 proof 的凭证内容签名，并对声明承担可追溯责任。验证方应从 issuer DID 文档取得验证方法，检查 proof creator、verification method 与 issuer 的一致性。

8.9.2 持有者

holder 是当前控制凭证呈现材料的一方，可以是资源控制方、客户端或受委托的用户助手智能体。持有公开 VC 不等于取得 credentialSubject 的控制权，代表关系仍需由控制证明、委托或业务授权支撑。

8.9.3 资源控制者

资源控制者控制资源 DID 对应的注册、更新或其它受保护操作。控制者可以与资源提供方、注册 VC holder 或端点运营方不同，其权限判断依赖目标 DID、验证方法、操作绑定证明和当前版本规则。

8.9.4 凭证主体

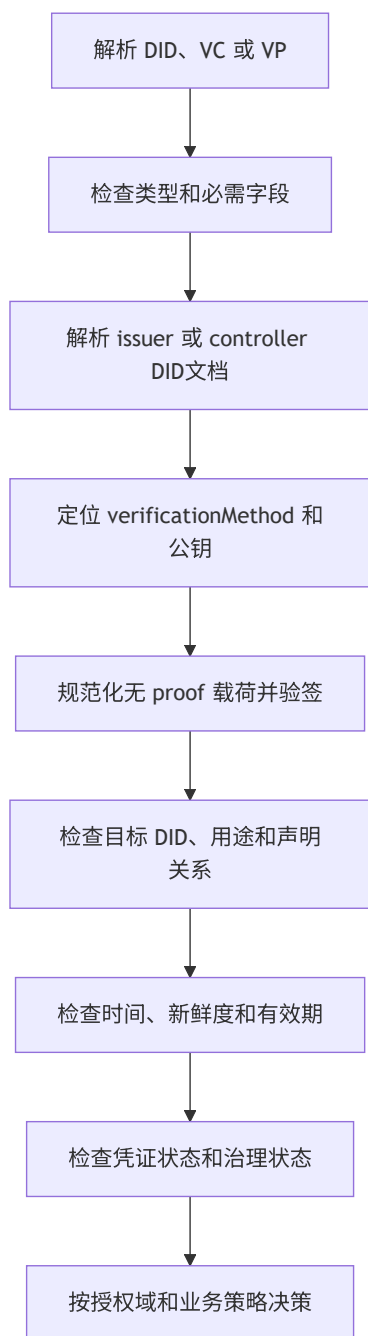
credentialSubject 是 VC 所描述的对象。节点授权 VC 的主体通常是基础设施节点 DID，注册 VC 的主体通常是资源 DID；主体被替换、DID 文档不匹配或 claims 与主体类型冲突时，凭证不得用于相应决策。

8.10 签名验证与状态验证

验证服务接收到 JSON 后，应先按接口声明的类型或字段结构解析，再分别验证签名覆盖范围和状态来源。节点间请求的 upstreamAuth 用于证明请求来自具备相应授权的节点；资源登记中的 subjectControlProof 用于证明资源控制方授权本次登记或更新；注册 VC 用于说明注册服务节点产生的登记事实；根平台可信发布证明用于说明特定资源版本通过根平台验证。它们可能在同一 HTTP 业务链路中连续出现，但不能混为一个凭证对象。

请求可解析

- > 请求签名与受众有效
- > 资源控制证明有效
- > 注册 VC 的签发者和主体匹配
- > 根平台发布证明与版本、哈希匹配
- > 治理状态和新鲜度可接受



任何一步失败都应在发布、授权或调用等副作用动作前终止，并返回可定位的失败原因。验证记录可以保存验证对象、proof 类型、verification method、验证时间、状态来源和错误代码，但不得保存私钥或可恢复私钥的材料。

8.11 凭证隐私与披露边界

资源发现所需的资源 DID、公钥、能力标签、资源描述、端点、版本、哈希和根平台发布证据可以进入公开资源包或发现响应。主体个人信息、私钥、本地身份备份、未公开委托材料和不必要的内部配置应保持在受控边界内。

材料	默认范围	处理要求
资源 DID 文档	可公开解析	不得包含私钥，校验核心字段和扩展字段
注册 VC	按登记和审计需要	检查 issuer、subject、状态和哈希关系
节点授权 VC	面向节点和验证组件	公开可验证部分，保护运行凭据
根平台可信发布证明	面向分发和发现验证	绑定资源版本和资源包哈希
VP	仅向指定验证方	绑定 holder、用途、challenge 和时效
本地身份备份	仅控制者保存	高敏感材料，禁止进入日志和公开索引

最小披露不能删除验证所需的绑定字段。若隐藏字段会使验证方无法确认 issuer、subject、版本、哈希或 proof，应采用受控呈现或可验证摘要，而不是不可核验的展示文本。

8.12 密钥轮换与失陷响应

轮换应先建立新公钥与既有 DID 的可验证控制关系，再使旧公钥退出新操作。资源 DID 更新应在新 DID 文档中说明新验证方法、用途和生效版本；节点密钥轮换还应同步节点授权 VC、节点配置和上游信任材料；根平台密钥轮换则必须考虑下游缓存、历史证明和验证窗口。

密钥失陷时，应暂停受影响密钥，保留验证历史材料所需的旧公钥和状态，并通过新的控制关系发布撤销、替换或恢复事实。若控制私钥完全丢失且没有恢复密钥或转移机制，系统不能仅凭 DID 字符串恢复控制权，应进入治理介入或重新注册流程。

8.13 注册 VC 与资源发布证据

两类证明构成连续但独立的事实层。注册服务节点在根平台接口成功返回后生成注册 VC；根平台则对资源控制证明、元数据、资源包、版本、哈希和发布条件完成验证后产生根平台可信发布证明。HTTP 成功码或 rootResponse 只能作为传输结果，不能替代对证明内容和状态的验证。

资源控制证明 -> 注册服务节点受理

- > 根平台验证资源、版本、哈希和发布条件
- > 根平台可信发布证明
- > 内容分发平台分发
- > 发现服务节点验证和索引
- > 调用方再次验证并按策略调用

登记受理、根平台接受、材料分发、发现节点索引和调用方授权必须分别表达。任何单一 VC 或发布证明都不得将这些状态压缩为“资源已经可信可用”。

8.14 节点授权 VC 与基础设施权限

节点动作由角色、授权域、能力范围和治理状态共同限定。注册服务节点只能处理授权域覆盖的资源，发现服务节点只能同步和返回获准范围；节点能够验证某份材料不等于节点被授权执行所有网络动作。授权域应检查格式、排序、重复、层级覆盖和与节点 VC 的一致性。

```
allow(action) = valid(node authorization VC)
    && active(governance status)
    && role permits(action)
    && granted domains cover(requested domains)
    && freshness is acceptable
```

该表达式表示判定关系而非固定实现代码。错误应区分节点授权失败、资源控制失败、资源内容失败和上游服务失败，便于审计、客户端处理和运维定位。

8.15 凭证吊销、暂停与状态传播

签名有效的凭证仍可能因治理事件、状态记录或 `expiresAt` 而不能用于新的授权决策。状态传播可能经过链上治理事件、根平台状态、链下信任索引器和节点缓存，因此验证结果应带有状态来源、观察时间和新鲜度信息。

8.15.1 凭证状态变更

状态变更应关联凭证标识、主体、旧状态、新状态、原因、事件序列和时间，并由有权主体产生。恢复 `suspended` 与重新签发新凭证不是同一操作；撤销后能否重新签发，应由治理和签发策略规定。

8.15.2 节点授权 VC 状态

节点授权 VC 被暂停或撤销后，注册服务节点和发现服务节点不得继续执行受限的新动作。已有材料是否继续提供，只能按缓存新鲜度和风险策略处理，不能以历史 VC 推导当前节点仍有权提交或索引新内容。

8.15.3 资源注册 VC 状态

资源注册 VC 状态变化影响登记证明的当前有效性，但不必然等同于删除资源 DID。历史文档和审计材料可以保留，同时将当前版本标记为暂停、撤销或不可发现；验证方应分别检查 VC 状态、资源生命周期和根平台发布状态。

8.15.4 状态传播与缓存更新

节点收到较新的治理事件或状态游标后，应验证序列连续性、事件完整性和证明，再更新本地缓存。游标落后、事件缺口、状态源不可用或缓存超过允许新鲜度时，敏感动作应暂停或返回状态不确定，而不是继续沿用旧的 `active` 判断。

8.16 签名格式、规范化与验证测试向量

跨语言实现必须统一待签名原文、字段命名、字段排序、空值处理、时间格式、哈希算法、公钥编码和 `proof` 字段。参考实现中的 `signature_input`、`build_data_integrity_proof` 和 `verify_payload_with_proof` 体现了载荷构造、证明生成和验证边界；其它语言实现应产生等价字节序列，而不能只比较内存对象。

向量类别	正常样例	失败样例	合格判据
公钥解析	多重编码和 JWK 还原同一公钥	前缀、长度或坐标非法	明确返回公钥编码错误
载荷签名	同一规范化原文验签成功	修改任一字段或 proofValue	验签失败
控制证明	DID、哈希、用途和 challenge 一致	任一绑定字段不一致	发布前拒绝
VC 状态	active 且未过期	revoked、suspended 或过期	授权决策不通过
节点请求	受众、路径、nonce、时间有效	重放 nonce 或错误受众	请求拒绝且无发布副作用

测试向量应版本化，并覆盖 Rust 参考实现、TypeScript SDK 和节点 HTTP 层。测试私钥只能使用专用材料；生产私钥不得进入测试文件或日志。日志可保留公钥、哈希、请求 ID 和错误代码等非敏感证据。

本章验收清单

- ☐ 主体 DID、资源 DID、节点 DID 和根平台 DID 的控制关系能够分别解释。
- ☐ 私钥不会出现在 DID 文档、注册请求、VC、VP、日志或发现响应中。
- ☐ 资源控制证明、节点授权 VC、注册 VC 和根平台可信发布证明可以分别验证。
- ☐ 验证流程同时检查签名、DID、公钥、用途、时间、状态和授权域。
- ☐ 合法、篡改、过期、撤销、重放和用途不匹配输入得到可区分结果。

8.17 参考来源

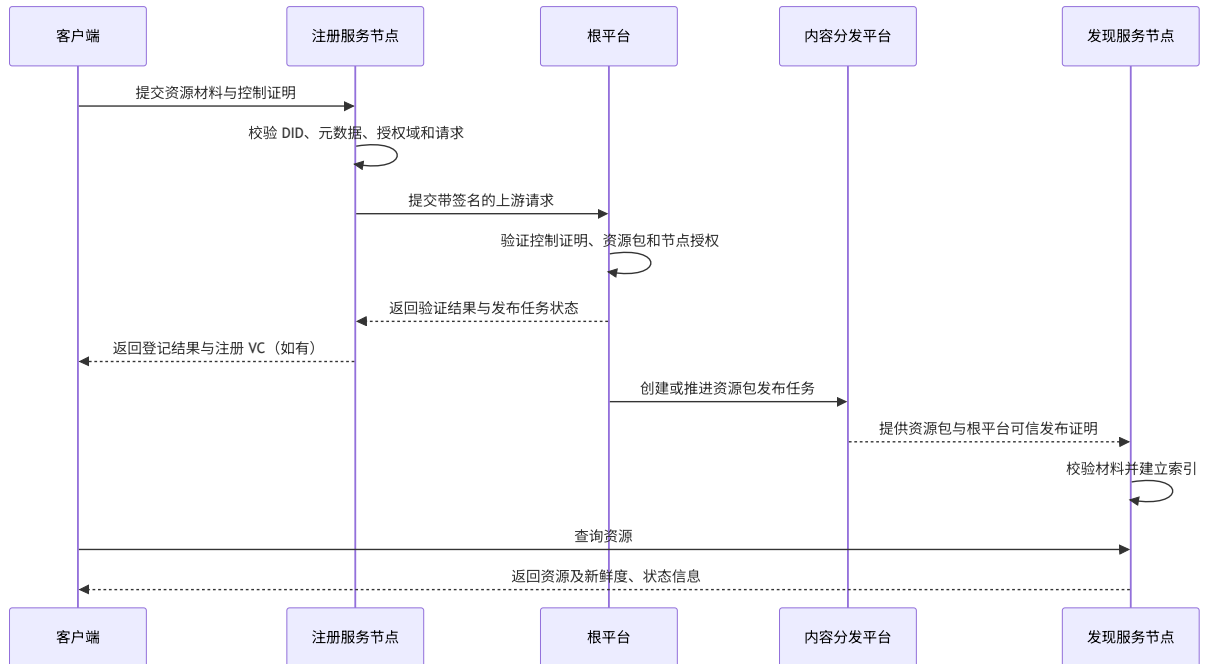
来源	类型	链接
oan-protocol-common	代码仓：密码学、凭证、签名和身份结构	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓：身份包、密钥和凭证客户端处理	https://github.com/OpenAgenet/oan-sdk-ts
OAN Yellow Paper	arXiv 黄皮书：身份、VC、证明和验证边界	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	标准/方法规范：DID 文档和验证方法	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

第九章 注册协议

本章规定资源从客户端进入 OpenAgenet (OAN) 的注册流程。客户端形成资源 DID、DID 文档、资源元数据、资源包和资源控制证明后，提交给注册服务节点；注册服务节点执行入口校验和授权域校验，再以签名的上游请求调用根平台的资源验证与发布接口。当前参考实现使用 `/resources/register` 和兼容的 `/resources/submit` 入口，成功路径包括根平台请求成功、注册 VC 签发、登记记录写入和响应返回。

注册服务节点返回 `status: submitted` 只表示本次登记流程已获得该节点及其上游调用的成功结果，不等于资源已经完成全网发布、内容分发或发现索引。客户端应把登记结果、根平台可信发布证明和发现服务节点的索引状态分开保存和展示。

注册链路可以概括为“客户端准备材料，注册服务节点受理并校验，根平台验证并发布，注册服务节点返回登记结果”。客户端不应直接根据页面上的成功提示推断发现可见性；应以资源登记记录、根平台返回内容和发现服务节点查询结果分别判断三个阶段。注册服务节点的公开接口是资源提交入口，节点之间的上游调用属于受保护的服务间协议，二者的请求格式和签名主体不能混用。

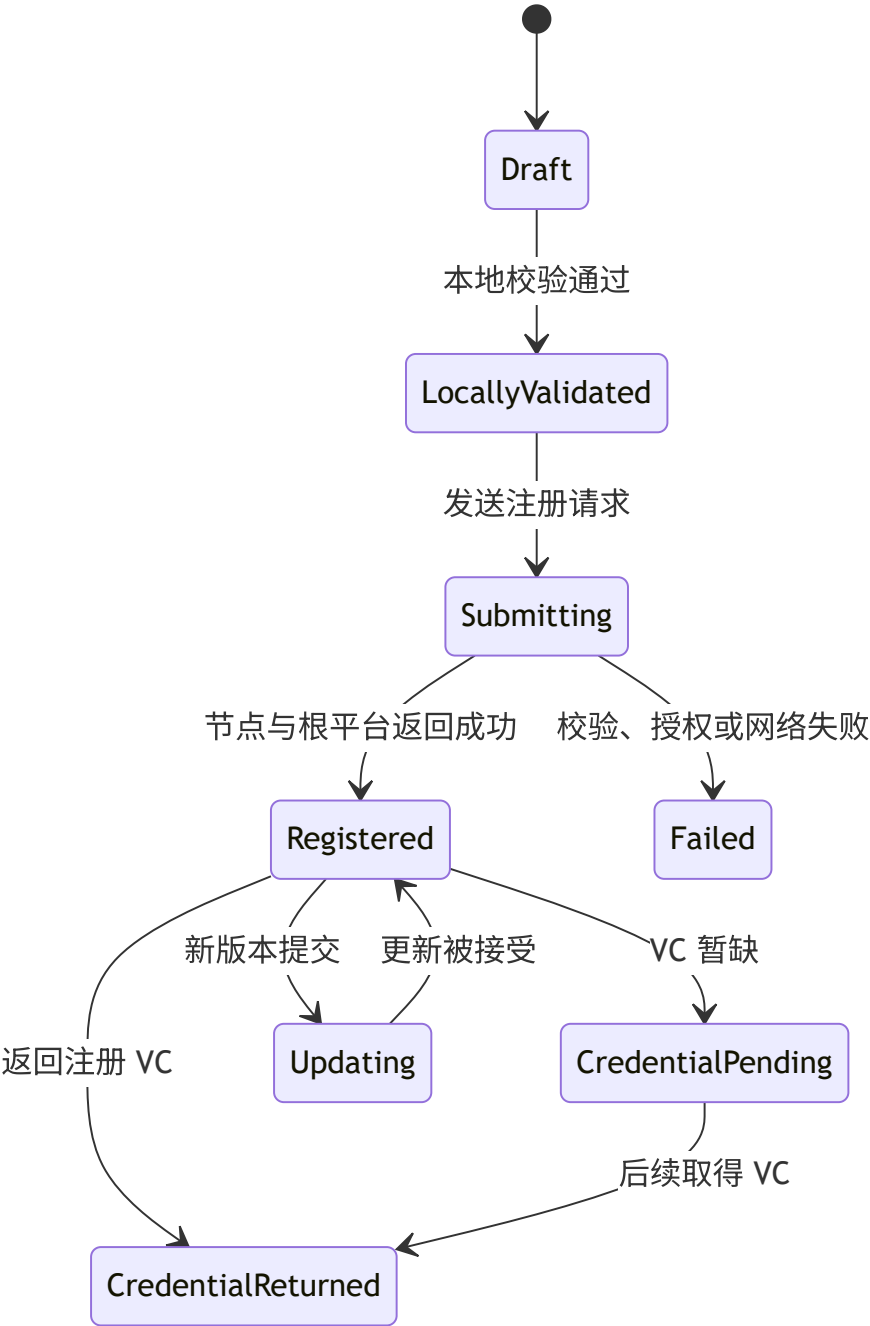


上图中的根平台验证、内容分发和发现索引是连续但可异步完成的阶段。`resource-verified-and-queued` 或注册服务节点返回的 `status: submitted` 只代表相应登记或排队事实，不能推导内容分发平台已经完成托管，也不能推导发现服务节点已经建立可查询索引。

9.1 注册目标与状态

注册状态应区分客户端本地状态、注册服务节点处理状态和根平台发布状态。推荐使用以下逻辑状态：

状态	进入条件	允许操作	终止或转换条件
草稿	表单或 SDK 资源材料尚未提交	编辑、预览、重新生成	材料通过本地校验或被修改
本地已校验	本地身份、字段、哈希和控制证明通过	提交或继续编辑	任一绑定字段变化后回到草稿
提交中	客户端已发送注册请求	等待响应或查询	收到成功、失败或超时结果
登记成功	注册节点获得规定的上游成功结果	保存 VC、查询发布状态	后续根平台发布和发现流程独立推进
凭证待取得	登记结果存在但 VC 缺失或为空	按接口规则查询或重试	取得并验证 VC，或进入异常处理
重复	DID、版本和哈希与已有材料一致	返回既有结果	不应产生新的业务版本
冲突或失败	校验、控制、授权、版本或网络条件不满足	修正材料或按可重试性处理	不得显示为全网发布成功



浏览器预览或本地校验通过不创建注册记录；网络超时也不应直接生成新的 DID 或新版本。相同业务材料的重试应沿用原资源 DID、版本和哈希，并由服务端或查询接口确认真实结果。

9.2 注册请求结构

注册入口的核心载荷是 ResourceRegistrationSubmission。主要字段与用途如下：

字段	来源	校验和用途
resourceDid	客户端生成的资源身份	作为资源身份和登记记录主键
resourceType	客户端选择	与 DID 语义编码及 DID 文档元数据一致

字段	来源	校验和用途
<code>didDocument</code>	客户端草稿	提供公钥、验证方法、服务端点和 OAN 元数据
<code>didDocumentHash</code>	客户端按协议计算	绑定控制 challenge 和发布材料
<code>metadata</code>	客户端表单或资源描述	校验资源声明和授权域
<code>packageVersion</code>	客户端资源版本	区分同 DID 的不同版本
<code>packageHash</code> 、 <code>metadataHash</code>	客户端计算	绑定资源包和元数据完整性
<code>hashAlgorithm</code>	协议和客户端配置	解释哈希字段的算法前缀
<code>registrationCredential</code>	协议兼容字段	向根平台提交时由注册节点清空
<code>subjectControlProof</code>	资源控制方签名	证明本次注册或更新获得控制授权

这些字段可按验证顺序分为四组：

字段组	字段	主要判断
身份与类型	<code>resourceDid</code> 、 <code>resourceType</code> 、 <code>didDocument</code>	资源身份、资源类型和 DID 文档是否相互一致
完整性绑定	<code>didDocumentHash</code> 、 <code>metadataHash</code> 、 <code>packageHash</code> 、 <code>packageVersion</code> 、 <code>hashAlgorithm</code>	本次登记材料是否对应同一版本、同一份内容
业务声明	<code>metadata</code>	能力、端点、授权域和生命周期等资源描述是否完整
授权证明	<code>subjectControlProof</code> 、 <code>registrationCredential</code>	资源控制方是否授权本次操作；注册节点向根平台转发时不携带已有注册 VC

说明性请求片段如下：

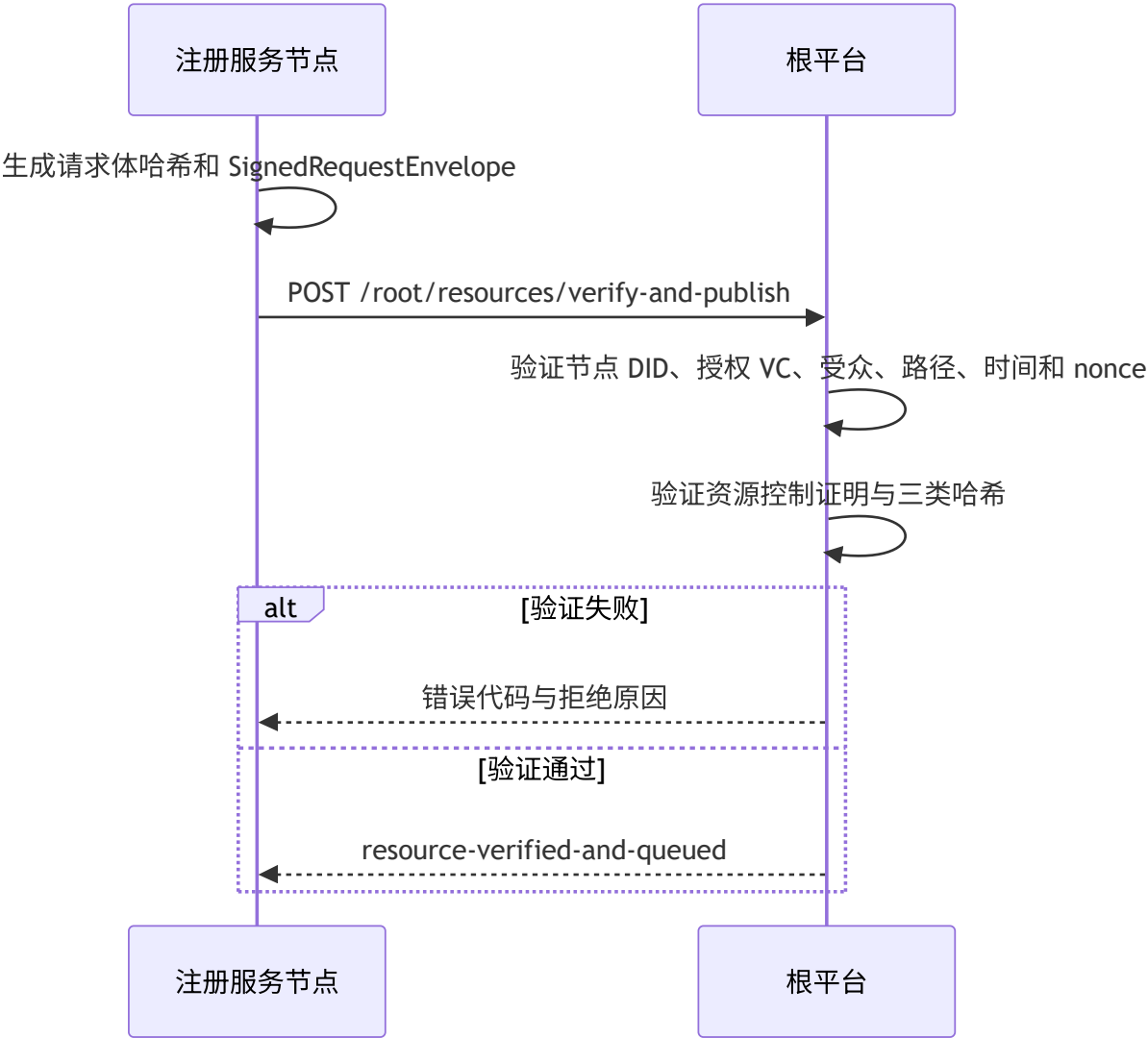
```
{
  "resourceDid": "did:oon:SKFI:...",
  "resourceType": "skill",
  "didDocument": {"id": "did:oon:SKFI:...", "verificationMethod": []},
  "didDocumentHash": "sha256:...",
  "metadata": {"resourceType": "skill", "authorizedDomains": ["software"]},
  "packageVersion": "1.0.0",
  "packageHash": "sha256:...",
  "metadataHash": "sha256:...",
  "hashAlgorithm": "sha256",
  "registrationCredential": null,
  "subjectControlProof": {"challenge": {}, "proof": {}}
}
```

注册服务节点应在调用根平台前拒绝资源 DID、DID 文档标识、challenge 目标或哈希不一致的请求。未知扩展字段是否保留由协议版本决定；未知核心字段、错误类型或无法解析的结构不得被当作合法请求。

合法请求至少需要同时满足以下关系：`didDocument.id == resourceDid`；DID 文档声明的资源类型与 `resourceType` 一致；三个哈希字段能够按 `hashAlgorithm` 解释；控制 challenge 的主体和文档哈希分别对应资源 DID 和 `didDocumentHash`。例如，只有把 `resourceType` 从 `skill` 改成 `agent_service` 而不同时更新 DID 文档和元数据的请求，应在注册节点本地校验阶段被拒绝。`registrationCredential`

保留在共享请求模型中用于兼容和记录，但注册节点向根平台构造上游请求时会将其清空，避免把注册节点此前签发的凭证当成根平台验证依据。

注册服务节点向根平台转发的关键交互可以概括为：



这段交互只描述注册服务节点与根平台之间的受保护上游调用。客户端提交入口、注册 VC 签发和后续内容分发仍按本章其它小节分别处理；根平台返回成功也不改变发现服务节点尚未完成索引这一事实。

9.3 本地身份初始化与导入

本地身份是客户端保存主体配置、资源控制密钥和必要凭证的材料集合，不是注册服务节点上的用户账户。它用于生成或恢复控制关系，并为具体注册操作签发控制证明。注册节点只能接收验证所需的 DID 文档、公钥、证明和登记材料，不应接收主体或资源控制私钥。

资源控制关系可表示为“本地身份持有控制密钥，资源 DID 文档发布验证方法，控制证明把一次具体操作绑定到该资源和该文档版本”。因此，本地身份与资源不是同一个对象：一个主体配置可以控制多个资源，但每次注册请求仍应明确一个 `resourceDid`，并使用与该资源 DID 文档匹配的证明。注册节点验证的是公开材料和签名结果，而不是客户端声称“这是我的资源”。

9.3.1 新建本地身份

客户端生成主体控制密钥和主体 DID，保存公开验证材料及本地配置；随后可为每个资源创建独立的资源 DID 和资源控制材料。新建本地身份本身不产生注册节点记录，也不代表资源已获得网络发布状态。

9.3.2 导入本地身份

客户端解析本地身份备份，校验主体 DID、公钥、私钥、资源配置和凭证之间的对应关系。导入成功后可以继续控制已有资源，但本次注册、更新或重试仍应针对目标 DID、版本、哈希和操作目的生成新的证明。

9.3.3 私钥与主体配置

私钥只在客户端签发控制证明或其它受保护签名时使用；主体配置描述主体 DID 与资源的关联。服务端请求和日志可包含公开 DID、公钥、签名、哈希和凭证，但不得包含私钥或可恢复私钥的明文材料。

9.3.4 身份备份与恢复

身份备份应包含恢复控制关系所需的结构化信息，并标明敏感性和版本。恢复后应重新检查密钥对应关系和 DID 文档一致性；备份文件存在不等于资源已经登记，恢复失败也不应覆盖客户端已有的有效身份。

9.4 DID Document 草稿生成

客户端根据资源类型、名称和描述、能力标签、授权域、端点、版本、本地身份和控制密钥生成 DID 文档。id 必须与 resourceDid 相同，oanMetadata.resourceType 和 subjectType 必须与请求一致，验证方法必须提供可解析公钥，服务端点和资源包信息必须与资源元数据绑定。

预览按钮可以生成或复用本地 DID 文档并展示校验结果，但不得因为展开预览而发起注册网络请求。若表单、本地身份、高级访问或资源包信息未变化，客户端可以复用已生成文档；影响 DID、文档哈希、版本或控制证明的输入发生变化后，必须重新生成并重新签名。

为避免用户未改变材料时反复得到不同资源身份，客户端应把生成 DID 文档所依赖的表单值、本地身份主体 DID、资源控制配置、端点、授权域、版本和资源包摘要形成内部快照。快照相同且已有预览文档时，点击按钮只切换预览区域的展开状态；快照变化时才重新生成文档和相关哈希。该缓存只属于客户端体验优化，不能替代注册节点对请求材料的再次校验。

9.5 提交前本地校验

提交前至少检查以下条件：本地身份已创建或导入；资源类型已选择；名称、描述、端点、授权域、能力标签和版本等必填项完整；DID 语法和资源类型匹配；DID 文档、元数据、资源包及哈希一致；控制 challenge 的目标、用途、文档哈希和 verification method 正确；控制证明可由 DID 文档中的公钥验证。错误提示应指出具体缺失项，例如“缺少本地身份”“缺少资源端点”“未选择授权域”或“DID 文档哈希不一致”。

预览按钮只执行本地生成和校验，提交按钮在发起请求前复用校验结果；两者都不应在页面打开时自动提交。校验失败不得写入注册节点记录、根平台发布记录或成功注册 VC。

本地校验结果宜按字段返回，便于界面和 SDK 采取确定动作：

```
{
  "valid": false,
  "errors": [
    {"field": "localIdentity", "code": "required", "message": "缺少本地身份"},
    {"field": "serviceEndpoint", "code": "required", "message": "缺少资源端点"}
  ]
}
```

`valid: false` 时不得进入提交请求；网络不可用属于提交阶段错误，不应被伪装成本地字段错误。预览成功只证明当前浏览器能够生成并核验草稿，不能证明注册节点或根平台已经接受该材料。

9.6 资源控制证明验证

注册服务节点应同时验证证明载荷、签名和业务上下文。目标资源 DID、DID 文档哈希、验证方法、操作目的、challenge 中的注册服务节点、时间窗口和 nonce 必须与当前请求一致；公钥和签名算法必须可解析。证明缺失、过期、用途不符、目标不符、哈希不符或重复使用时，应在调用根平台前拒绝。

控制证明失败不得产生成功登记状态或注册 VC。可保存挑战 ID、资源 DID、文档哈希、验证方法、验证时间和错误代码用于审计，但不得保存签名私钥。

验证顺序应先检查结构和上下文，再验证签名，最后进入发布调用：

```
请求可解析
-> challenge.subjectDid == resourceDid
-> challenge.didDocumentHash == request.didDocumentHash
-> challenge.registrarDid == 当前注册服务节点 DID
-> purpose、issuedAt、expiresAt 和 nonce 合法
-> verificationMethod 位于 DID文档的 assertionMethod
-> 解析公钥并验签
-> 允许继续执行注册流程
```

任一步失败都应停止后续根平台调用。验签成功只说明签名者掌握对应私钥且签名内容未被修改；节点授权域、资源类型、版本和根平台发布条件仍需分别验证。

9.7 注册节点授权与范围验证

当前注册服务节点会读取自身 DID 文档中的授权域，将资源 DID 文档中的授权域与请求元数据中的授权域进行一致性检查，再判断节点授权域是否覆盖资源域。授权域必须非空、格式合法、顺序稳定且没有越权；不一致或越权时，应在根平台调用前返回错误。

节点签名密钥只证明请求来自该节点运行边界，节点授权 VC 才证明该节点具有相应角色和范围；资源控制证明则是另一个独立条件。

注册节点向根平台发送的请求由 `SignedRequestEnvelope` 绑定请求 ID、协议版本、用途、HTTP 方法、路径、受众、时间戳、随机数、请求体哈希和节点 proof。根平台可以据此核对哪个注册服务

节点在什么时间、以什么用途提交了哪一份材料。这层节点间签名不替代资源控制证明，也不把注册节点自动变成资源控制者。

9.8 资源类型、元数据、端点与资源包校验

资源类型、DID 语义编码、DID 文档元数据、资源元数据和资源包应形成一条一致性链。资源 DID、资源类型、版本、DID 文档哈希、元数据哈希和资源包哈希必须相互引用；端点应符合声明协议和格式；能力标签应能由能力标签树或允许的扩展规则解释；授权域不能由自由文本绕过。

当前 `validate_shape()` 已覆盖 DID 语法、资源类型关系、DID 文档标识、资源结构、资源包版本、哈希算法、三类哈希引用、challenge 目标 DID、文档哈希和操作目的。端点是否可达、业务是否安全和资源质量是否满足调用需求，属于后续发布或调用前判断，不能由形状校验替代。

9.9 重复注册与幂等性

业务幂等应以资源 DID、资源类型、版本、DID 文档哈希、元数据哈希和资源包哈希判断，而不能以提交时间或 VC 的签发时间判断。请求 ID 和 nonce 用于请求关联、新鲜度和重放防护，不能单独决定两个请求是否属于同一资源版本。

9.9.1 同 DID 重复注册

同 DID 再次提交时，服务端应先验证控制权，再比较版本和哈希。材料完全相同应返回既有登记结果或明确幂等结果；DID 相同但版本或任一哈希不同，应进入同 DID 更新决策；控制证明无效时不得覆盖已有记录。

9.9.2 幂等判断依据

幂等判断至少依赖资源 DID、资源类型、版本、DID 文档哈希、元数据哈希和资源包哈希，并应保存请求 ID、处理时间和结果以区分业务重试与新版本。当前注册节点通过按资源 DID 的 upsert 方式写入 SQLite、PostgreSQL 或文件存储，这可以保持当前记录，但不足以单独证明已实现完整的版本级幂等历史。

9.9.3 VC 重发

同一业务版本的重试可以返回已保存注册 VC，也可以按明确策略重新签发内容等价但时间不同的 VC。客户端应依据资源 DID、版本、哈希和状态识别凭证对应版本，不能把新的签发时间当作资源版本更新。

9.9.4 根平台重复发布控制

根平台应使相同 DID、版本和哈希的重复发布产生单一业务结果；不同哈希或版本应进入更新、冲突或拒绝路径。注册服务节点收到上游 HTTP 成功并不足以证明根平台已完成重复发布去重，发布证明和序列记录才是判断依据。

9.10 资源更新与同 DID 版本更新

同 DID 更新保留资源身份，但必须产生新的版本、文档或资源包哈希，并通过 `previousVersion` 等字段表达前序关系。新版本必须使用当前控制关系生成新的控制证明；旧注册 VC 或旧资源包不能自动授权未来更新。版本递增、并发冲突、历史保留和回滚由注册节点与根平台共同执行，具体发布替换规则见第十章。

9.10.1 更新授权验证

验证方必须确认提交者仍由当前 DID 控制关系授权，并检查控制证明绑定新文档哈希、版本、用途和 `challenge`。只有旧凭证而没有新版本控制证明时，应拒绝更新。

9.10.2 DID Document 更新

更新后的 DID 文档仍使用原资源 DID，文档哈希必须进入注册提交和资源包。新增、替换或停用验证方法都需要可验证控制链；无法证明更新由当前控制者授权时，不得替换当前文档。

9.10.3 资源元数据和端点更新

资源描述、能力标签、授权域、协议绑定或端点变化都应形成新的元数据哈希和资源包哈希。扩大授权域不能只修改文字字段，端点可达也不能替代业务安全和调用授权判断。

9.10.4 新版本发布与旧版本处理

新版本被接受后，根平台应明确当前版本、历史版本和旧版本状态，发现服务节点按发布序列同步。旧版本可用于审计或回滚，但不能在查询结果中与最新版本混淆。

9.11 注册 VC 签发与获取

当前参考实现只有在根平台资源验证与发布接口返回成功后，才调用注册凭证生成逻辑，并把注册 VC 写入登记记录和成功响应。注册 VC 的 `issuer` 是注册服务节点，`credential subject` 是资源 DID，声明包含资源类型、版本、哈希、授权域和生命周期等登记事实。

9.11.1 VC 签发条件

签发前应完成入口形状校验、资源授权域校验、控制证明处理和根平台请求。客户端本地预览通过、点击提交或注册节点收到请求，都不是单独的签发条件。

9.11.2 VC 返回方式

成功响应当前包含 `status`、`resourceDid`、`resourceType`、`registrationCredential` 和 `rootResponse`。客户端应保存 VC 原文并与 DID 文档、版本和哈希关联；`rootResponse` 仍需按根平台证明和状态规则解析。

9.11.3 VC 缺失或为空的处理

若登记结果成功但 VC 缺失或为空，客户端应分开记录登记状态和凭证状态，明确提示凭证尚未取得，不能伪造空 VC，也不能把空值当作有效凭证。查询或重试应保留原 DID、版本和哈希。

9.11.4 VC 本地保存

注册 VC 应与资源 DID 文档和资源版本建立文件关联。客户端打包下载 VC、DID 文档和本地身份备份时，私钥仍属于敏感材料，只能由控制者保管，不得发送到节点或公共存储。

9.12 注册响应与错误处理

当前注册服务节点的处理函数接收 `Json<ResourceRegistrationSubmission>`，成功时返回动态 JSON。成功响应的稳定核心字段是 `status`、`resourceDid`、`resourceType`、`registrationCredential` 和 `rootResponse`；其中 `rootResponse` 保留根平台返回的动态内容，不能被客户端假定为固定的 VC 或全网发布完成标志。

阶段	典型结果	客户端解释
注册节点本地校验	4xx 或结构化错误	请求未进入根平台发布流程，先修正字段、哈希或控制证明
根平台验证并排队	<code>status: resource-verified-and-queued</code>	根平台已验证并建立后续任务，不代表内容分发和发现索引已完成
注册节点登记成功	<code>status: submitted</code>	注册节点已记录登记结果，响应中可能包含注册 VC 和根平台响应
网络或上游暂时失败	5xx、连接错误或超时	依据资源 DID、版本、哈希和请求标识查询状态后再决定是否重试

下面的 handler 关系说明输入和输出的实际类型；它是接口契约的缩略表示，不是可直接替换的完整实现：

```
async fn register_resource(
  State(state): State<AppState>,
  Json(submission): Json<ResourceRegistrationSubmission>,
) -> ApiResult<serde_json::Value>;
```

注册节点与根平台之间的 `ResourceVerifyAndPublishRequest` 还包含 `registrarDid`、原始登记提交和 `upstreamAuth`。客户端不应自行构造该内部转发对象，也不应把注册节点的 `registrationCredential` 当作根平台可信发布证明；错误响应中的 `error`、状态码和请求关联标识应共同用于判断修正、查询、重试或停止。

结果	含义	客户端处理
成功且有 VC	注册节点完成当前流程并返回注册凭证	保存 VC，另行确认根平台发布和发现状态
成功但 VC 暂缺	登记结果存在但凭证不完整	保留 DID、版本和哈希，查询或重试
校验失败	请求结构、类型、哈希或证明不合法	修正具体字段，不盲目重试
授权失败	节点范围或资源控制权不足	检查授权 VC、授权域和控制关系
上游失败	根平台拒绝或不可用	按可重试性处理，不声明发布成功

结果	含义	客户端处理
网络超时	客户端没有确定响应	保留原材料，查询或使用相同业务材料重试
重复或冲突	已有相同版本或存在不同内容	返回既有结果，或进入更新和冲突处理

当前实现的错误主要通过 HTTP 状态和 `error` 文本返回；后续稳定协议应补充 request ID、错误码、登记状态、上游状态和可重试标志，使客户端不依赖自由文本猜测网络状态。

建议将响应处理归纳为以下确定性规则：

```
{
  "status": "submitted",
  "resourceDid": "did:oon:SKFI:...",
  "resourceType": "skill",
  "registrationCredential": null,
  "rootResponse": {"status": "published"}
}
```

当 HTTP 请求超时而没有响应时，客户端只能记录“结果未知”，保留原 DID、版本和哈希，并通过查询或相同材料重试，不能据此生成新 DID。收到非成功 HTTP 状态时，不得保存为登记成功；收到成功状态但 VC 为空时，应将登记状态与凭证状态分开显示，只有实际拿到并验证的凭证才能作为 VC 文件下载。

9.13 注册审计记录

登记记录至少应关联资源 DID、资源类型、版本、DID 文档哈希、元数据哈希、资源包哈希、授权域、注册服务节点、登记 VC、根平台响应摘要、提交时间和处理结果。当前 `write_resource_record()` 按资源 DID 写入 SQLite、PostgreSQL 或文件存储，因此完整审计实现还应保留操作类型、请求 ID、前后版本和失败原因，避免覆盖记录丢失历史上下文。

审计记录不得保存资源控制私钥、本地身份备份或可恢复秘密。DID、公钥、哈希、挑战 ID、请求 ID、状态和错误代码可以作为追踪证据；审计读取权限必须与公开发现接口分开。

9.14 资源更新授权与同 DID 替换

服务端收到同 DID 新材料后，应依次确认控制证明、版本关系和哈希关系。相同版本同哈希属于幂等重试；相同版本不同哈希属于冲突；新版本带有效前序关系才可进入更新；版本倒退、前序缺失、授权变化未经批准或控制证明无效时应拒绝或转人工处理。注册节点可以保存受理记录，但根平台决定网络发布事实。

替换判断可用下表核对：

DID	版本	内容哈希	控制证明	处理
相同	相同	相同	有效	按同一业务版本幂等处理
相同	相同	不同	有效	作为冲突拒绝，不覆盖当前版本

DID	版本	内容哈希	控制证明	处理
相同	更新	不同	有效且绑定新材料	按更新流程处理
相同	更新或相同	任意	无效	拒绝，不产生替换副作用
不同	任意	任意	有效	按新的资源登记处理

9.15 提交重试、幂等键与重复处理

节点到根平台的上游请求使用 `SignedRequestEnvelope` 绑定 request ID、协议版本、用途、方法、路径、受众、请求时间、请求随机数、请求体哈希和 proof；安全层通过 nonce 存储和时间窗口实施重放防护。业务幂等则依赖资源 DID、版本和内容哈希。客户端超时后应保留原材料，优先查询既有结果或使用相同业务材料重试，不能立即生成新 DID 来掩盖未知结果。

9.16 注册速率限制与滥用控制

公开注册入口应限制单位时间请求量、请求体和资源包大小、单来源失败次数、同 DID 冲突次数以及上游转发并发量。控制证明、节点授权和授权域检查应尽早执行，以减少伪造材料造成的存储、网络和计算压力。限流不能把合法的同 DID 幂等查询误判为新版本，也不能通过放宽控制证明换取吞吐量。

当前参考实现明确包含结构校验、授权域覆盖检查、上游请求签名和 nonce 重放防护；反向代理、WAF 或服务级限流若在部署中启用，应在部署文档中注明实际策略、响应码和恢复方式。

注册流程验收清单

- ☐ 空表单、缺少本地身份、缺少端点或缺少授权域时，客户端能指出具体条件。
- ☐ 注册服务节点在根平台调用前完成请求形状、资源授权域和控制证明校验。
- ☐ 根平台失败或网络超时时，客户端不会显示全网发布成功，也不会丢失原 DID 和版本材料。
- ☐ 成功响应中的注册 VC 能与资源 DID、版本和三个哈希建立对应关系。
- ☐ 相同 DID、版本和哈希的重试与不同哈希或版本的更新得到不同处理。
- ☐ 登记、根平台发布、内容分发和发现索引状态能够分别观察和验证。
- ☐ 审计记录可追踪请求而不包含私钥、本地身份备份或其它敏感秘密。

9.17 参考来源

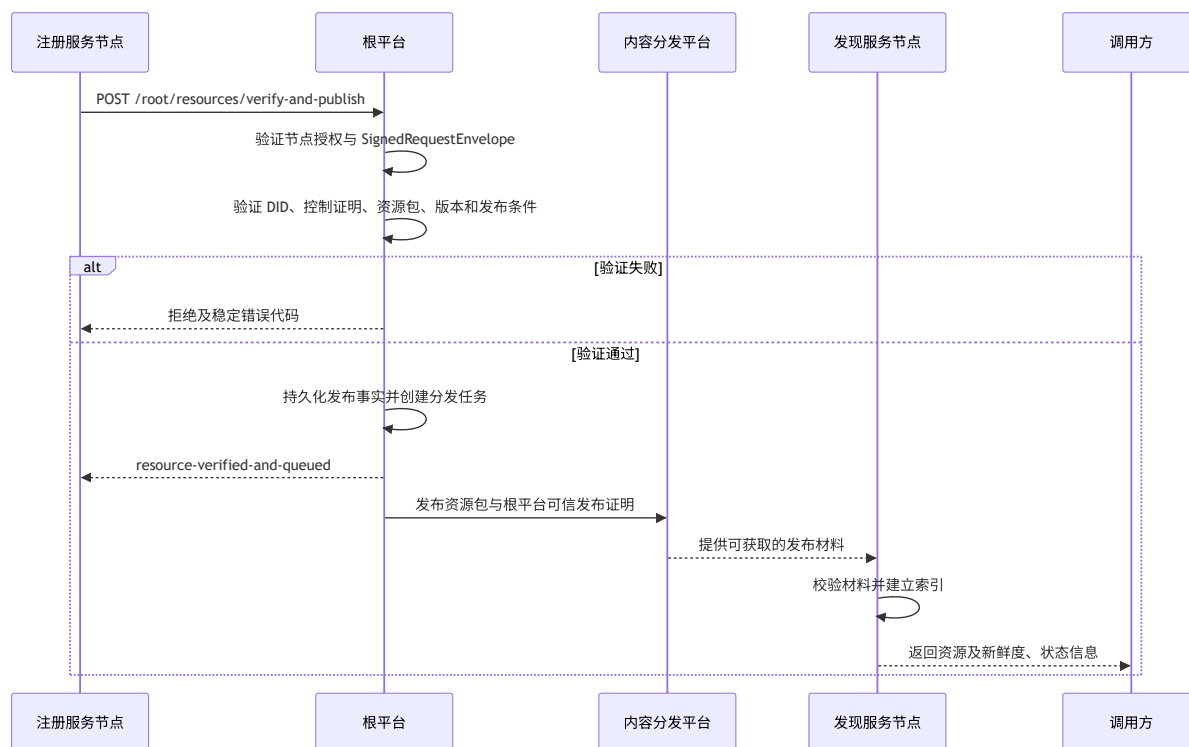
来源	类型	链接
oan-registrar-node	代码仓：注册入口、校验、登记记录和注册 VC	https://github.com/OpenAgenet/oan-registrar-node
oan-root-services	代码仓：根平台资源验证与发布接口	https://github.com/OpenAgenet/oan-root-services

来源	类型	链接
oan-protocol-common	代码仓: 注册载荷、错误码和跨节点协议类型	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓: 客户端注册材料和注册调用	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓: 注册流程脚本和开发者调用示例	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv 黄皮书: 注册、发布和验证流程	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF 草案: 资源身份与注册发现关系	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

第十章 根平台验证与可信发布

本章说明根平台如何把注册服务节点提交的资源材料转换为可验证的网络级发布事实。内容覆盖上游请求认证、资源 DID 和控制证明验证、节点授权、资源包完整性、发布策略、根平台可信发布证明、版本选择、分发任务和失败恢复，并持续区分“已验证并排队”“已形成发布事实”“内容已分发”和“发现服务节点已索引”等不同状态。

10.1 根平台发布职责



该流程中的“验证并排队”、内容分发和发现索引分别对应不同完成条件。根平台的成功响应只说明资源已通过根平台处理并进入分发任务链路；内容分发平台和发现服务节点仍需完成各自的校验、发布或索引，调用方也应独立验证 DID、注册 VC、根平台可信发布证明和状态信息。

根平台是资源进入网络级发布链路前的独立验证边界。它接收授权注册服务节点提交的登记材料，重新验证资源 DID、DID 文档、资源控制证明、资源包（ResourcePackage）、节点授权、版本和发布条件，并将通过验证的资源版本写入发布记录。注册服务节点的前置校验属于接入层检查，不能替代根平台对发布事实的独立确认。

从运行结果看，根平台的“接受”包含两个连续但可区分的事实：一是资源材料通过根平台验证并被持久化，二是面向内容分发平台的发布任务被建立。当前实现的成功响应为 **resource-verified-and-queued**，因此注册服务节点或客户端应把它解释为“已验证并排队”，不能直接显示为“已完成”。

全网发布”。

10.2 注册节点的上游提交

注册服务节点向根平台提交的是带有上游认证的资源登记请求，而不是一个无来源的资源包上传。请求应关联注册服务节点 DID、请求标识、协议版本、目标路径、时间、随机数、请求体哈希和签名，并携带资源登记提交及其控制证明。根平台应先验证提交来源和请求完整性，再处理资源内容；请求重复、过期或受众不匹配时不得进入发布状态。

当前入口为 `POST /root/resources/verify-and-publish`。根平台收到请求后，先依据注册服务节点 DID 加载授权材料，再验证 `SignedRequestEnvelope` 的签名、方法、路径、受众、时间、随机数和请求体哈希，随后才执行资源提交的结构和内容验证。上游请求的签名证明“哪个注册服务节点提交了这份材料”，不等同于资源控制证明，也不等同于根平台可信发布证明。

该入口的请求类型是 `ResourceVerifyAndPublishRequest`，其最小关系可以表示为：

```
{
  "registrarDid": "did:oon:...",
  "submission": {
    "resourceDid": "did:oon:SKDM:...",
    "resourceType": "skill",
    "didDocument": {},
    "metadata": {},
    "packageVersion": "1.0.0",
    "didDocumentHash": "sha256:...",
    "metadataHash": "sha256:...",
    "packageHash": "sha256:...",
    "hashAlgorithm": "sha256",
    "subjectControlProof": {}
  },
  "upstreamAuth": {}
}
```

成功响应当前是动态 JSON，核心结果为 `status: resource-verified-and-queued`，并带有资源 DID、资源类型、包版本、三类哈希、生命周期状态和 `cdnDispatchStatus: queued`。这表示根平台已完成当前验证并建立内容分发任务；内容分发平台的接收以及发现服务节点的索引需要通过各自状态或同步记录确认。

10.3 资源 DID 与控制证明验证

根平台应先解析资源 DID 并确认资源类型与 DID 语义编码相符，再确认 DID 文档 id、资源类型、验证方法和服务声明满足方法规则，最后使用文档中声明的公钥验证控制证明。控制证明必须绑定本次挑战、目标资源 DID、DID 文档哈希、用途和有效时间；验证失败应阻止该版本进入发布记录，且不得覆盖已有有效版本。

可复现的负向测试是：先准备一个能通过验证的提交，记录根平台当前资源版本和发布任务数量；然后仅修改 `resourceDid`、`didDocument.id`、`didDocumentHash` 或控制挑战的 `subjectDid` 其中一项。根平台应返回对应错误，资源接受记录和任务分发任务不应因该失败请求新增或覆盖；这能验证根平台没有把“请求可达”误当成“资源控制已证明”。

10.4 注册节点授权验证

注册服务节点授权检查的对象是基础设施节点资格，不是资源控制权。根平台应根据节点 DID、目标角色、授权 VC、授权域、DID 文档稳定哈希、签发者和当前治理状态判断该节点是否有权提交相应资源。授权过期、撤销、角色不匹配或授权域不覆盖目标资源时，提交应被拒绝或进入隔离，不得因请求来自已知地址而放宽检查。

根平台的判断至少包含“节点是否仍处于可用治理状态”和“节点授权范围是否覆盖资源授权域”两步。前者处理节点被撤销、暂停或授权过期，后者处理注册服务节点只能代为接入部分资源域的情况。即使注册服务节点已经成功签发注册 VC，根平台仍应以当前授权状态重新判断是否接受发布；注册 VC 不能替代根平台的节点授权检查。

10.5 ResourcePackage 与哈希验证

根平台应按统一规范化规则重算 DID 文档哈希、元数据哈希和资源包哈希，并检查哈希算法前缀、资源 DID、资源类型、包版本和嵌套字段之间的一致性。任一哈希不匹配、循环字段处理不一致或包内容与声明不一致，都属于完整性失败；失败材料可以保存为诊断记录，但不得进入可分发资源集合。

根平台成功重算后，会把资源 DID、资源类型、版本、三个哈希、生命周期和授权域组织为 `packageClaims`，并将根平台 `proof` 写入资源包的 `rootProof`。验证方应以这些声明与资源包实际字段的一致性作为判断依据；仅看到客户端提交的 `packageHash`，或者只比较字符串格式，都不足以证明资源包完整。

10.6 元数据与端点策略检查

策略检查用于判断已通过身份和完整性检查的材料是否适合进入网络发布链路。根平台应检查资源类型、能力标签、授权域、协议绑定、服务端点、生命周期和版本声明是否一致，并阻止私钥、访问令牌和不应公开的内部凭据进入公开资源材料。策略检查通过不代表端点业务质量已经得到保证，端点调用仍由调用方和资源服务共同完成授权。

策略检查的结果应与后续传播阶段分开记录。通过检查的材料可以进入根平台接受记录和内容分发任务；端点是否在线、内容分发平台是否完成托管、发现服务节点是否完成索引，则由后续服务分别确认。资源中若含有私钥、访问令牌或不应公开的内部地址，应在形成公开分发材料前拒绝或隔离，而不能依赖发现服务节点在索引阶段补救。

10.7 根平台可信发布证明签发

10.7.1 签发前提

根平台可信发布证明只能在提交来源、注册服务节点授权、资源控制证明、DID 文档、三类哈希、资源版本和发布策略均通过检查后签发。证明签发前还应确认当前治理状态没有阻止该节点或该资源版本进入发布链路；待验证、待复核或分发失败不能直接转化为已发布。

10.7.2 证明内容

证明内容应至少绑定根平台 DID、资源 DID、资源类型、资源包版本、DID 文档哈希、元数据哈希、资源包哈希、哈希算法、生命周期状态和签发时间。若证明包含授权域、发布游标或公告引用，也必须明确这些字段覆盖的是发布事实还是传播事实，不能混入资源控制方未声明的业务结论。

10.7.3 证明签名

根平台应使用自身受保护的签名密钥对规定的证明内容进行签名，并明确密码套件、签名覆盖范围和验证方法引用。签名验证方应先确认签发者属于信任的根平台，再核对签名输入是否与当前 ResourcePackage 完全一致；仅能解析 JSON 或看到根平台 DID 不足以证明签名有效。

10.7.4 证明获取与验证

资源提供方、内容分发平台和发现服务节点可以通过根平台发布接口、发布材料或关联地址取得证明。验证方应检查证明状态、目标资源 DID、版本、哈希、签发时间和根平台授权状态，并保留验证结果；证明取得成功不表示发现服务节点已经完成索引。

根平台可信发布证明应与一个确定的资源版本绑定使用。验证方至少应关联根平台 DID、资源 DID、包版本、DID 文档哈希、元数据哈希、资源包哈希、proof 和验证时间；再次取得同一资源时仍需重新核对哈希和证明状态。注册服务节点签发的注册 VC、根平台可信发布证明和发现服务节点的索引记录分别证明不同阶段，不能互相替代。

10.8 发布序列与版本选择

10.8.1 发布序列

发布序列由根平台用于标识资源发布事实在分发链路中的先后位置。序列应保持单调推进并与资源 DID、资源版本、包哈希绑定；重复读取或重试同一发布任务不得产生新的逻辑发布事实，发现服务节点同步游标也不得被当作根平台发布序列。

10.8.2 同 DID 版本选择

同一资源 DID 的版本选择应结合资源包版本、前序版本关系、包哈希、控制证明和根平台发布序列判断。内容相同的重复材料可以幂等确认；内容不同但缺少有效控制证明或版本关系的材料应拒绝或隔离，不能按请求到达时间覆盖已有发布版本。

10.8.3 重复提交处理

根平台收到与既有资源 DID、包版本和包哈希完全相同的提交时，可以返回已有发布结果或重复提交状态，并保持原发布游标和证明不变。重复提交的处理记录仍应关联新的请求标识，便于审计和区分网络重试与真实版本更新。

10.8.4 冲突版本处理

同一 DID 下出现同版本不同哈希、同发布序列不同内容、前序版本不匹配或证明覆盖范围不一致时，应进入冲突或隔离处理，不得自动选择先到版本。根平台应保留冲突摘要和拒绝原因；只有控制方提交满足版本与控制条件的新材料后，才可重新验证。

排查版本问题时，应同时查看资源 DID、包版本、三个哈希、前序关系和发布游标，不能只按请求到达时间选择版本。发布游标是根平台发布记录在分发链路中的顺序证据，不是资源控制权，也不是发现服务节点的同步游标；发现服务节点即使看到游标推进，仍需校验具体资源包和根平台证明。

10.9 发布状态与拒绝原因

当前根平台成功响应的状态为 `resource-verified-and-queued`，表示资源已通过根平台处理并进入内容分发任务链路，不表示内容分发平台已完成发布。对外处理时至少应区分以下状态：

状态	语义	是否允许进入分发	失败或后续动作
待验证	已接收但必要检查尚未完成	否	查询处理结果
已验证并排队	资源验证完成，发布任务已建立	是	等待内容分发平台处理
已发布	发布事实已持久化且证明可验证	是	继续确认传播和索引
重复提交	相同 DID、版本和哈希再次到达	复用已有事实	返回既有结果
已拒绝	控制、授权、哈希或策略检查失败	否	修正材料后重新提交
待重试	外部发布链路暂时失败	使用已有任务	保留任务并按策略重试

拒绝原因应优先使用稳定代码，例如 `registrar_not_authorized`、`did_document_hash_mismatch`、`metadata_hash_mismatch` 和 `package_hash_mismatch`。控制证明无效、节点未授权和哈希不一致属于阻止发布的验证失败；事件投递失败或内容分发平台暂时不可用属于可恢复的后续失败，两者不能都只显示为“发布失败”。

10.10 内容分发准备

根平台接受资源后，会持久化资源包、发布任务和事件发布意图，并通过内容分发事件把任务交给内容分发平台。事件可以只携带资源 DID、包版本、发布游标、根平台 DID 和哈希等可重放引用，由内容分发平台再向根平台取得资源包。当前设计通过事务性 outbox 使资源接受记录、发布任务和事件意图保持一致，事件发送失败时由 outbox relay 重试。

```
{
  "jobKey": "did:oon:SKDM:example-resource:1.0.0",
  "resourceDid": "did:oon:SKDM:example-resource",
  "packageVersion": "1.0.0",
  "publicationCursor": 123,
  "rootDid": "did:oon:ROOT:example-root",
  "packageHash": "sha256:<example-package-hash>",
  "didDocumentHash": "sha256:<example-did-document-hash>",
  "metadataHash": "sha256:<example-metadata-hash>"
}
```

事件被消息系统接受不等于资源已经完成内容发布。内容分发平台应在取得资源包、验证根平台 proof 和哈希、完成自身发布后确认任务；发现服务节点收到传播材料后，也应独立校验资源包和证明。

10.11 根平台发布审计轨迹

审计链应能从请求标识或任务键追溯到完整发布决定：注册服务节点提交了什么，根平台验证了哪些字段，资源包和哈希是什么，生成了哪个发布游标，是否创建内容分发任务，事件是否投递，以及任务最终如何处理。建议至少保留以下字段组：

字段组	典型字段	追溯用途
请求关联	requestId、注册服务节点 DID、接收时间	定位一次上游提交
资源版本	资源 DID、packageVersion、三个哈希	固定被处理的材料
验证结果	控制证明、节点授权、策略检查结果	解释接受或拒绝
发布证据	根平台 DID、proof、publicationCursor	证明根平台发布事实
分发状态	jobKey、事件状态、重试次数、最后错误	追踪后续传播

审计记录可以保存摘要和引用，不应写入主体私钥、资源控制私钥、身份备份或访问令牌。公开资源字段和内部诊断字段应分开授权读取，避免审计接口成为敏感材料的旁路出口。

10.12 重新发布、回滚与恢复

分发失败时，优先重试同一资源 DID、包版本、哈希和任务键的发布任务，不要为重新发送而生成新的资源身份或把传输重试解释为新版本。数据恢复后，应先恢复根平台接受记录、发布任务和 outbox 状态，再重建未完成的事件投递。回滚到历史版本时，历史版本的 DID、版本、哈希和根平台证明仍对应原事实；如果资源内容发生变化，则必须重新生成控制证明、重算哈希并重新执行根平台验证。

分发失败

-> 保留原 jobKey、资源 DID、版本和哈希

-> 检查任务租约、重试次数和最后错误

-> 重新取得同一资源包并验证根平台证明

-> 内容分发平台成功后确认任务

-> 发现服务节点按发布游标继续同步

10.13 同 DID 更新授权与版本接受

同 DID 更新不能只比较版本字符串。根平台应确认新材料仍由当前控制关系授权，DID 文档、资源元数据和资源包的哈希分别对应新材料，并能与已有版本建立前序关系。相同版本相同哈希应作为幂等重试处理；相同版本不同哈希应作为冲突处理；新版本若控制证明仍绑定旧文档哈希，应在接受记录写入前拒绝。

可复现的验收可以固定一个已接受的 v1，分别提交完全相同的 v1、哈希不同的 v1、带有效控制证明的 v2 和缺少前序关系的 v2。合格判据是：第一种不产生第二个逻辑发布事实，第二种不覆盖 v1，第三种进入新版本处理，第四种被拒绝或隔离，并且每种结果均能从请求、版本和哈希字段解释。

10.14 发布拒绝、隔离与重新处理

明确验证失败应直接拒绝并返回原因；材料暂时无法判定、依赖外部治理状态或需要复核时，才适合进入隔离。隔离记录应保留原始提交摘要、资源 DID、版本、哈希、注册服务节点、失败阶段和重新处理条件，但不得出现在内容分发清单或发现索引中。重新处理必须重新检查当前授权、控制证明有效期、哈希和治理状态，不能只把原状态改成“已发布”。

10.15 根平台发布状态机

10.15.1 待验证

待验证表示根平台已接收提交或发布任务，但尚未完成所有必要检查。该状态可以允许查询处理状态和诊断信息，不得进入可信分发清单，也不得被对外解释为注册成功、根平台已发布或发现可见。

10.15.2 验证通过

验证通过表示资源材料、控制证明、节点授权和发布条件已经通过根平台检查，但如果根平台证明尚未签发或发布记录尚未持久化，仍不能等同于已发布。该状态的转换应具有明确的提交标识、验证时间和证据引用。

10.15.3 已发布

已发布表示根平台已经形成与具体资源版本绑定的发布事实和根平台可信发布证明，并允许相关材料进入内容分发链路。该状态不表示内容分发平台已完成托管、发现服务节点已完成索引，或调用方已经获得业务调用许可。

10.15.4 已拒绝、隔离与恢复

已拒绝表示当前提交不满足必要条件；已隔离表示材料需要复核、补齐或等待外部状态变化；恢复处理必须重新检查当前授权、签名、哈希和治理状态。分发失败可以触发待重试，不应篡改原有资源版本或伪造新的根平台证明。恢复时应以原提交的资源 DID、版本和材料摘要为关联依据，重新执行当前授权、控制证明、哈希和治理状态检查。只有重新验证通过，隔离材料才能回到发布流程；网络重试不得改变资源版本或生成新的根平台可信发布证明。根平台的“已发布”状态仍只表示已形成发布事实，不表示所有发现服务节点已经索引。

10.16 参考来源

来源	类型	链接
oan-root-services	代码仓：根平台资源验证、发布证明和分发任务	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务节点上游提交和登记结果	https://github.com/OpenAgenet/oan-registrar-node

来源	类型	链接
oan-protocol-common	代码仓：资源包、证明和发布事件共享类型	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：根平台、可信发布和安全边界	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF 草案：资源身份、发布和发现关联	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

第十一章 内容分发与网络传播

本章说明根平台形成发布事实后，资源包、DID 文档、元数据、哈希、凭证和发布证明如何经由内容分发平台传播到获得授权的发现服务节点。章节重点区分发布任务、传播材料、同步游标、接收校验和本地索引等状态，说明批量传输、重试、部分失败、幂等和恢复如何保证传播过程可追踪，而不把内容可取得误认为资源已经被所有发现节点应用。

11.1 分发模型与内容类别

内容分发采用“发布事实先形成、传播材料后传输、发现节点再次验证”的模型。一个资源版本的传播集合通常包括资源包（ResourcePackage）、DID 文档、资源元数据、哈希、注册凭证、根平台可信发布证明、生命周期状态、授权域和必要的摘要或索引提示。摘要可以优化同步效率，但不能替代完整资源包和证明的校验。

传播链路中有三类容易混淆的对象：根平台产生发布任务和发布游标，内容分发平台保存可取得的资源材料，发现服务节点将验证后的材料转换为本地索引。它们分别代表“根平台决定传播什么”“内容在哪里取得”和“哪个节点已经应用什么”，不能因为同一资源 DID 贯穿三者，就把三种状态合并成一个全网状态。

11.2 内容分发服务职责

内容分发平台负责发布材料的托管、获取、批量传输、缓存和传输状态反馈。内容分发发布者可以从根平台取得发布任务，按发布游标组织资源包并写入内容分发平台，再把成功或失败结果回报根平台。该服务不产生资源控制权、节点授权或根平台可信发布证明，发现服务节点仍需以自身验证结果决定是否应用材料。

当前参考实现的批量发布响应包含已发布项目、失败项目 and 对应发布游标。发布者只有在响应可解析且项目数量与请求一致时，才确认这一批任务的处理结果；请求成功但响应缺少游标或项目状态不完整时，应按异常处理，不能直接确认整批发布完成。

11.3 分发清单与发布检查点

发布清单是发现服务节点确定同步范围和验证对象的入口，发布检查点则把传播状态拆分为已发送、已接收、已校验和已应用。清单中的资源 DID、包版本、包哈希和发布游标用于关联资源版本，通知批次中的目标节点和序列范围用于关联一次传播操作。发现服务节点只有完成校验并写入索引后，才可以报告该项目已应用。

11.4 资源包、元数据、证明与索引数据分发

11.4.1 资源包分发

资源包是面向发现服务节点的主要完整载荷，至少应通过资源 DID、资源包版本和包哈希与发布清单关联。内容分发平台可以提供完整包或按清单提供增量材料，但接收方必须在写入可信索引前重算哈希并验证根平台可信发布证明。

当前内容分发平台的单项发布入口为 `POST /cdn/resources`，批量发布入口为 `POST /cdn/resources/batch`。单项请求由 `ResourceCdnPublishRequest` 承载，包含 `ResourcePackage` 和 `upstreamAuth`；批量请求由 `ResourceCdnBatchPublishRequest` 承载，包含带 `publicationCursor` 的条目和公共上游认证。发布成功的单项响应包含 `status: published`、资源 DID、资源类型、包版本和 `publicationCursor`；批量响应还区分 `acceptedCount`、`failedCount`、成功项目和失败项目，不能用批次 HTTP 成功替代所有项目已应用。

发现服务节点可以通过 `POST /cdn/resources/batch-get` 按资源 DID 批量读取资源包，或通过资源索引接口按 `afterCursor` 增量读取。批量读取返回请求数量、命中数量和资源包项目；增量索引返回 `items`、`count`、`afterCursor`、`nextCursor` 和 `hasMore`。这些游标分别服务于批量读取、列表分页和传播同步，不能相互替代。

11.4.2 DID Document 与元数据分发

DID 文档和资源元数据可以随资源包传输，也可以按资源 DID、版本和哈希单独取得。分开发送时，发现服务节点不得先应用其中一部分而丢失关联关系；DID 文档、元数据和包外摘要不一致时，应拒绝组合并记录缺失或冲突字段。

11.4.3 VC 与根平台证明分发

注册凭证用于说明注册服务节点的登记事实，根平台可信发布证明用于说明根平台接受特定资源版本。两者应按资源 DID、版本和哈希关联传输，接收方应分别验证签发者、签名、覆盖范围和当前状态，不能因注册凭证存在就跳过根平台证明检查。

11.4.4 索引数据分发

索引数据是发现服务节点根据已发布材料形成的派生读模型，可以包含资源摘要、能力标签、端点引用、生命周期状态和新鲜度信息。索引数据不应被回传为新的根平台事实；发现服务节点必须能够说明索引记录对应的发布游标、校验时间和授权域。

11.5 分发边界完整性验证

发现服务节点接收材料后，应先校验通知目标、清单范围和载荷结构，再按资源 DID、资源类型、版本、三类哈希、根平台可信发布证明、生命周期和授权域完成验证。校验失败的对象不得写入可信发现索引；可以写入拒绝记录以支持诊断和重试。传输成功但内容损坏属于完整性失败，材料缺失属于可恢复的获取失败，治理状态已撤销属于阻止应用的状态失败。

接收方的负向验收可以分别只改变包内容、证明绑定的资源 DID 或当前发现服务节点的授权域。三种情况下，前两者应分别在哈希或证明绑定阶段拒绝，后一种应阻止索引应用；失败可以留下包含资源 DID、游标、失败阶段和错误代码的记录，但不应推进受影响的同步游标。

11.6 版本新鲜度与同步游标

发布游标描述根平台发布事实在传播链路中的位置，同步游标描述发现服务节点已处理的位置，资源版本描述资源内容本身的版本，新鲜度描述材料从发布或接收到当前观察时刻的时间关系。它们分别服务于顺序、恢复、版本选择和延迟解释，不能用一个字段代替其它字段。

11.6.1 同步游标

同步游标由发现服务节点维护，用于表示该节点已经处理到的内容分发位置。游标推进必须以相应材料完成结构、哈希、证明和状态校验为前提；游标推进到某个位置不代表该位置之后的所有资源都已成功应用。

11.6.2 版本新鲜度

版本新鲜度应同时参考根平台发布游标、资源包版本、发布或更新时间、本地接收时间和索引应用时间。时间差用于解释传播延迟，不能单独决定版本优先级；同一 DID 的内容选择仍以可验证版本、哈希和根平台发布事实为准。

11.6.3 过期数据判断

发现服务节点应根据允许延迟、治理状态和资源生命周期判断数据是否过期。普通信息展示可以保留带新鲜度标记的旧快照；涉及撤销、暂停、节点授权变化或高风险调用前验证时，无法确认最新状态应暂停敏感动作，而不是继续使用未确认的旧状态。

11.6.4 缺口补偿

检测到游标缺口、批序列不连续或资源材料缺失时，节点应暂停受影响范围的游标推进，按清单或更新接口补取缺失内容，并在重新校验后继续。缺口补齐前可以报告同步滞后，但不得把后续项目无依据地标记为已应用。

例如节点已应用游标 100，收到游标 102 而缺少 101 时，可以保存 102 的待处理材料并报告缺口，但本地“已应用位置”仍应保持 100。补取并验证 101 后，再按顺序处理 102；若 101 被明确拒绝，也应保留可追踪的拒绝记录，并依据实现定义决定是否允许游标跨过该项目。收到更大的游标值本身不是推进游标的充分条件。

11.7 初始同步与增量同步

11.7.1 初始全量同步

初始同步应先取得可验证的发布清单、起始游标和目标授权域，再按清单取得资源包及其证明。节点应逐项完成校验、落库和索引应用，并在整个范围完成或明确记录拒绝后，报告该范围的同步结果。

11.7.2 增量同步

增量同步从已持久化的同步游标或通知批序列继续，获取其后的发布材料并按发布顺序处理。重复批次和重复资源包应幂等处理，新的游标只能在对项目达到已应用或明确拒绝状态后推进。

11.7.3 同步确认

同步确认应至少区分已接收、已验证、已建立索引和对外可查。确认响应、日志和数据库记录应能够关联目标发现服务节点、序列范围、成功/拒绝项目、当前游标和最后错误，不能以 HTTP 请求成功代替内容收敛确认。

11.7.4 中断后恢复

节点重启或网络中断后，应从最后一个持久化游标或未完成批次恢复，重放必要材料并按幂等规则处理。若游标失效、清单发生变化或治理状态无法确认，应重新获取清单或进入待复核，而不是直接跳过缺失区间。

11.8 重试、退避与中断恢复

重试只针对可恢复的传输、超时、服务不可用或暂时性锁定失败。发布器和发现服务节点应保存任务或批次标识、目标游标、已处理项目、重试次数和最后错误，并使用退避避免持续冲击上游。签名无效、哈希不一致、资源类型错误和授权域不匹配属于材料或权限失败，应记录并停止对同一未改变载荷的无限重试。

重试记录至少应能回答三个问题：重试的是哪一个资源版本，失败发生在内容取得、校验、写入还是结果回报阶段，下一次重试依赖什么条件。批量处理时，应根据成功项目和失败项目分别更新结果；不能因为批次整体响应成功就重复应用已成功项目，也不能因为一项失败就丢弃其它项目的处理证据。重复投递同一任务应使用资源 DID、版本、哈希和发布游标进行幂等判断。

11.9 缓存行为与过期数据处理

11.9.1 浏览器或客户端缓存

浏览器或客户端缓存保存的是页面或客户端已经取得的公开材料、网络快照或资源摘要，不是根平台的权威发布记录。缓存键应包含接口范围、资源 DID 或页面数据版本等必要上下文；清空浏览器数据后缓存可以消失，服务端发布事实不受影响。客户端读取缓存时应带上取得时间、来源和新鲜度标记，避免把本地命中误显示为刚刚从网络确认。

11.9.2 节点侧缓存

内容分发平台和发现服务节点可以缓存已经通过相应校验的资源包或索引读模型。内容分发平台缓存的是可传播材料，发现服务节点缓存的是本地应用后的查询数据；节点侧缓存应保留资源版本、包哈希、发布游标和校验时间，更新时按可验证顺序替换。缓存不应绕过根平台可信发布证明、资源包哈希或当前治理状态检查。

11.9.3 旧快照优先展示

当页面或查询接口的新数据尚未取得时，可以先展示最近一次已保存且可识别来源的旧快照，同时标注快照时间、数据来源和可能的同步滞后。旧快照适合改善普通读操作的等待体验，不适合作为撤销、暂停、授权变化或高风险调用的实时依据。快照展示应与后台刷新状态分开，不能把旧数据的展示时间更新成当前确认时间。

11.9.4 后台刷新与失败降级

后台刷新成功后，客户端或节点应以新材料完成结构、哈希、证明和状态检查，再原子替换旧快照；刷新失败时保留旧快照并记录失败原因、最后成功时间和新鲜度。若失败涉及撤销或授权状态无法确认，应降低可用状态或暂停敏感操作，而不是无条件继续使用旧快照。刷新逻辑不能阻塞页面基本渲染，也不能在失败时清空已有有效数据造成不必要的空白。

11.10 传播可观测性与审计记录

传播审计应将资源 DID、包版本、包哈希、发布任务、发布游标、通知批次、目标发现服务节点、发送/接收/校验/应用时间、重试次数和错误原因关联起来。根平台和发布者应能说明哪些项目完成了内容分发，发现服务节点应能说明哪些项目被拒绝或仍待同步；这些记录用于追溯传播过程，不改变资源的原始证明。

对批量传播，应同时记录批次整体结果和每个资源项目结果。整体响应为成功不代表所有项目均已应用，部分成功也不应抹去已完成项目的证据。运维排查时可以沿 `jobKey`、请求 ID 和发布游标关联根平台任务、内容分发平台历史和发现服务节点同步记录；这些字段用于追踪传播，不等同于资源控制方签名或治理授权。

对批量传播，应同时记录批次整体结果和每个资源项目结果。整体响应为成功不代表所有项目均已应用，部分成功也不应抹去已完成项目的证据。运维排查时可以沿 `jobKey`、请求 ID 和发布游标关联根平台任务、内容分发平台历史和发现服务节点同步记录；这些字段用于追踪传播，不等同于资源控制方签名或治理授权。

11.11 发布顺序与冲突解决

传播顺序以根平台发布游标和通知序列为基础，资源版本选择以资源包版本、前序版本关系、包哈希和根平台可信发布证明为基础。相同材料的重复批次应幂等处理；同一位置出现不同哈希、旧版本试图覆盖新版本或前序材料缺失时，应暂停相关应用并记录冲突，不能按到达时间任意覆盖。

11.12 分发一致性与新鲜度保证

一致性指标应分别说明根平台已发布、内容分发平台可取得、发现服务节点已接收、已验证和已应用的范围；新鲜度指标应说明从哪个发布或接收时间点计算到哪个观察或应用时间点。指标必须附带样本范围、失败项目和授权域口径，不能以单个节点成功或缓存命中代表全网收敛。

11.13 部分失败与恢复语义

部分失败按资源项目和目标节点隔离处理。已完成校验和应用的项目可以推进自身状态，失败项目应保留独立错误、重试和缺失信息，不得因批次整体响应成功而跳过失败项目。恢复时先补齐缺失材料，再完成单项验证和落库，最后推进相应游标或记录明确拒绝；已应用项目不因同批次其它项目失败而回滚，失败项目也不能因旧快照可用而隐藏失败。

最终收敛要求目标序列范围内每个项目都达到“已应用”，或具有明确、可追踪的拒绝记录。恢复过程中若发现哈希、证明、生命周期或授权域发生变化，应重新进入相应校验流程；旧快照只能用于带新鲜度标记的有限读侧结果，不能掩盖撤销、暂停或授权变化。

11.14 参考来源

来源	类型	链接
oan-root-services	代码仓：发布任务、内容分发协调和发布状态	https://github.com/OpenAgenet/oan-root-services
oan-discovery-node	代码仓：传播材料接收、验证和索引	https://github.com/OpenAgenet/oan-discovery-node
oan-protocol-common	代码仓：资源包、发布事件和哈希结构	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：可信发布、传播和发现链路	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF 草案：节点协作和覆盖网络传播	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

第十二章 发现协议与查询语义

本章说明发现服务节点如何从已验证并完成授权范围过滤的本地索引中处理 DID 精确查询、结构化查询、标签查询、文本查询和语义查询。内容覆盖查询输入、版本选择、来源和新鲜度、结果去重、分页、解释、错误和降级边界，明确发现响应是节点索引产生的候选结果，调用方仍需依据 DID、凭证、根平台可信发布证明、治理状态和端点信息完成调用前验证。

12.1 发现目标与查询类型

发现服务节点面向已经进入本地索引的资源提供五种查询路径：完整资源 DID 的精确查找，按资源类型、版本、协议和能力标签执行的结构化查询，按能力标签执行的标签查询，基于资源名称、描述、标签、用例和协议材料的文本或自然语言查询，以及自然语言条件与结构化条件结合的组合查询。查询结果是发现服务节点的派生响应，不是根平台发布记录本身；响应应保留资源 DID、资源类型、版本、生命周期、来源和必要的验证引用。

当前参考实现通过 `POST /discovery/resources/query` 接收统一查询对象。查询文本若是完整且无空白的 `did:oon:` 标识符，会进入精确 DID 分支；其它文本与显式字段进入结构化、词法或语义路径。查询节点只从本地已索引材料构造候选，不能把查询请求直接转发到注册服务节点或根平台来临时补造结果。

查询输入在进入检索前应先完成边界校验，再转换为内部查询对象。`resourceType`、`capabilityTags`、`protocol` 和 `version` 用于形成显式条件，`versionMode` 用于说明版本选择方式，`limit` 用于限制本次响应规模，`query` 承载 DID 或文本/自然语言查询。对同一个请求同时提供文本和结构化字段时，文本负责召回与排序，结构化字段负责收缩候选范围；这两类条件不能被互相覆盖。

```
{
  "query": "我需要一个可以检索代码仓库并总结项目结构的工具",
  "resourceType": "tool_api",
  "capabilityTags": ["code-repository"],
  "protocol": "MCP",
  "versionMode": "latest",
  "limit": 10
}
```

上例中的查询文本、标签、协议和数量用于说明字段组合，具体资源类型和标签必须以目标发现服务节点接受的协议值为准。响应候选中的 `score` 是查询相关性结果，不是资源信任等级。

12.1.1 DID 精确发现

完整资源 DID 应被解释为精确查找条件，不得退化为名称、前缀或语义相似搜索。节点应读取该 DID 对应的本地索引对象，并继续检查显式条件、生命周期和本节点授权域。

12.1.2 结构化条件发现

资源类型、版本、协议和能力标签等字段应转换为可复核的过滤条件。多个显式条件默认采用交集关系；非法枚举、无法解释的版本模式或不支持的协议值应返回明确请求错误。

12.1.3 能力标签发现

能力标签查询使用资源元数据中的 `capabilityTags`。标签树中的父子关系、别名和扩展标签可以参与查询投影，但必须保留原始标签及其来源，标签命中不代表控制权或调用授权。

12.1.4 文本与语义发现

文本和自然语言查询可以使用词法检索、向量检索或二者结合的混合检索。语义索引不可用时可以使用词法和结构化路径降级，但应在查询解释或诊断信息中说明实际使用的后端。

12.1.5 组合条件发现

组合查询应先执行授权域、生命周期和显式结构化条件过滤，再对剩余候选进行词法或语义排序。相关性不能解除硬过滤条件，也不能让未发布或未授权资源进入可信结果。

12.2 DID 精确查询

精确查询首先验证 DID 的方法语法和资源类型语义，再读取本地索引中的资源包。命中结果必须使用同一个资源版本的 DID 文档、元数据、端点、哈希和根平台证明引用；未命中不能简单解释为资源不存在于整个网络，因为资源可能尚未传播到本节点或被授权域过滤。节点应区分以下结果：

结果	解释	调用方处理
返回候选	本节点存在满足条件且可见的索引对象	继续验证 DID、证明和端点
空候选	不存在、尚未索引或不在本节点授权域内	不推断全网不存在
已暂停/撤销	索引对象存在，但当前状态限制使用	不按当前有效资源调用
服务错误	数据库、索引或治理读模型暂不可用	按临时故障重试或降级

当前精确查询实现以 `query` 字段识别 DID，并将其它显式条件继续交给资源类型、版本、能力标签和协议匹配逻辑；因此，带有 DID 的查询仍可能因版本或授权域条件不满足而返回空候选。精确命中也只说明本发现服务节点有一条可见索引记录，调用方仍应取得完整 DID 文档和证明材料进行调用前验证。

精确 DID 查询的结果边界可以用下表核对。这里的“未找到”是本节点当前索引和授权域下的查询结果，不是对整个智能体互联网状态的断言。

查询阶段	核对象	结果含义
输入识别	<code>query</code> 是否为完整的 <code>did:oon:</code> 字符串	决定进入精确路径还是普通检索路径
索引读取	本节点是否有对应资源包索引记录	只反映本节点是否已保存该记录

查询阶段	核对对象	结果含义
条件匹配	版本、资源类型、协议、能力标签等显式条件	任一硬条件不满足即可得到空候选
可见性判断	本节点授权域和资源生命周期	不可见、暂停或撤销的资源不能按正常候选返回
调用前判断	DID 文档、注册凭证、根平台可信发布证明和端点	由调用方完成，不由 <code>score</code> 或索引命中替代

12.3 结构化资源查询

结构化查询将请求中的字段映射为数据库或内存过滤条件。当前参考实现对资源生命周期、资源类型、版本、能力标签和协议执行过滤，并对未指定资源类型的普通查询使用面向用户消费资源的默认范围。显式能力标签要求资源全部包含相应标签；协议条件要求资源服务端点或协议绑定能够解释该协议。空查询在带有结构化条件时可以执行，完全没有条件的查询应受到返回条数和资源范围限制。

过滤完成后，结果仍需经过本节点授权域检查和最新有效版本选择。展示名称、描述和检索分数是索引字段，不能作为身份、发布或治理证据。

显式过滤应在候选进入相关性排序前执行。当前实现对 `resourceType`、`version`、`capabilityTags` 和 `protocol` 执行资源级匹配；能力标签按请求集合的交集判断，协议可以由协议绑定或服务端点声明命中。字段缺失不会自动被解释为通配符，无法解析的请求值不应悄然转换为另一个条件。

结构化过滤的可复核性要求是：请求字段、资源字段和判定结果能够逐项对应。比如请求同时指定 `resourceType = "mcp_server"` 和 `capabilityTags = ["file.search", "desktop.automation"]` 时，候选必须先属于指定资源类型，并同时满足两个标签，再参与文本相关性排序；仅有“文件搜索”语义相近但资源类型不符的候选不能通过排序进入结果。字段没有出现在资源包中时，应按该条件不满足处理，不能把缺失值当作任意匹配。

对于未知枚举、空白字段、重复标签和互相冲突的版本条件，节点应在请求解析阶段给出确定处理。重复标签可以先规范化为一个条件；未知资源类型或无法解释的版本模式应作为请求错误；版本和生命周期冲突时，应由版本选择与状态规则收紧结果，而不能由检索分数放宽。

12.4 能力标签查询

能力标签是资源提供方对资源能力分类的声明，根平台能力标签树为标签标识、层级、别名和版本提供治理语境。发现节点可以将查询中的别名投影为规范标签，也可以在本本地建立检索别名，但响应应保留资源原始标签和实际命中的标签。父标签是否覆盖子标签、扩展标签是否进入可信索引，应由标签树版本和节点配置明确，不能由模型相关性隐式决定。

标签查询应区分三种信息：资源原始的 `capabilityTags`，查询文本或显式参数经过别名展开后的检索词，以及最终用于解释命中的标签。别名展开可以提高召回，但不能修改资源原始声明；如果一个扩展标签没有得到当前节点的标签树或配置支持，应作为普通未命中处理，而不是推断为父标签或同义标签。

12.5 文本与语义查询

语义查询的可检索内容包括资源名称、描述、能力标签、用例、协议绑定和资源类型等公开信息。参考实现可以将资源材料组织为标签视图、上下文视图和意图视图，并结合向量分数、词法分数和结构化分数形成候选排序。`score` 只表示当前查询和索引快照中的相关性，不表示信任等级、服务质量或安全等级。

语义索引、嵌入服务或 `pgvector` 不可用时，可以降级到词法和结构化查询；降级不得放宽授权域、生命周期或显式条件。空文本不应强行调用语义后端，过长、无法编码或不支持语言的输入应返回明确错误或可解释的降级结果。

当前语义实现会把资源类型、协议、能力标签和查询文本投影为检索词，并从资源名称、描述、标签、上下文、示例和用例等公开材料构造搜索文档。语义后端可用时，结果可结合 BM25 类词法分数、标签分数、上下文分数、意图分数和结构化分数；这组内部得分用于排序和解释，不应被外部调用方当作经过治理的质量评价。

语义检索的索引文本必须来源于资源已公开的描述材料。索引构建时应保存资源 DID、资源包版本和语义源内容的关联，以便资源更新后重建对应索引，避免旧描述继续产生高相关性结果。多语言查询可以通过规范化、别名或嵌入模型提高召回，但不能把模型推断出的能力写回资源元数据，也不能以模型相似度替代资源控制权、注册凭证或根平台可信发布证明。

发现节点应能够从运行日志或诊断接口区分语义后端、词法路径和结构化路径。对于同一组硬条件，语义服务失败时允许只改变候选的召回和排序质量；如果索引数据库、授权域读模型或资源包读取失败，则必须把故障作为数据不可用或服务错误处理，不能返回看似正常但实际不完整的空结果。

12.6 语义与结构化条件组合查询

组合查询可抽象为以下处理顺序：

```
visible = authorizedDomain AND active AND explicitFilters
ranked  = rank(queryText, visible)
result  = stableSort(ranked)[0:limit]
```

其中 `explicitFilters` 包括资源类型、版本、协议和显式能力标签；`rank` 可以使用词法、向量或混合检索；`stableSort` 需要保留稳定次级键。若没有符合硬条件的资源，应返回空候选，而不是返回“相似但不满足条件”的资源。

组合查询的一个实际边界是：语义检索结果仍需经过本节点的资源包有效性、生命周期和授权域判断；语义索引中存在一条记录，不等于该记录在当前查询时刻仍可见。对自然语言中隐含的“最新”“可信”“可调用”等要求，节点可以将能够落到协议字段的部分转换为结构化条件，无法落到字段的部分只能作为排序或解释信息，不能自动生成授权结论。

组合查询的处理可以用一个最小可复现例子表示：

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure .",
  "resourceType": "tool_api",
  "protocol": "MCP",
  "capabilityTags": ["code-repository"],
  "versionMode": "latest",
```

```
"limit": 10
}
```

这个请求的合格结果必须同时满足资源类型、协议和能力标签条件；自然语言只用于补充名称、描述、用例等字段的相关性判断。若查询文本表达了“可信”或“可调用”，节点只能在存在对应结构化字段和可验证事实时执行相应判断，否则应把它作为检索意图保留，交由调用方在后续验证环节处理。

12.7 授权域过滤

发现节点从自身 DID 文档或治理读模型取得节点授权域，再将其与资源元数据中的 `authorizedDomains` 比较。资源域必须被节点获准的域覆盖；域不匹配的资源不应进入查询结果。该过滤决定的是本节点向查询方提供的发现可见性，不改变资源 DID 控制关系，也不代表资源在其它发现节点不可见。治理状态或节点授权无法确认时，应在普通展示中标明不新鲜，在敏感查询和调用前验证场景采取收紧处理。

12.8 最新有效版本选择

默认查询按资源 DID 选择当前满足本节点授权域、生命周期为有效、已经完成必要校验且同步位置可确认的最新版本。同一 DID 的版本选择应结合资源包版本、根平台发布游标、包哈希、前序版本关系和治理状态，不能只用更新时间或查询到达顺序判断。发现节点本地落后时可以返回已验证旧版本用于有限展示，但必须携带新鲜度和同步位置，不能宣称其为网络最新版本。

版本选择至少要验证三个条件：候选属于同一资源 DID，候选版本满足请求的 `versionMode` 和 `version` 条件，候选对应的生命周期与治理状态仍允许发现。对于 `latest` 查询，选择的是当前可验证的最新有效版本，而不是数据库中最后写入的记录；对于指定版本查询，若版本记录不存在、哈希不一致或状态不可确认，应返回空结果或明确的数据不可用错误，不得静默替换为另一个版本。

可复现的检查可以使用两条同 DID 记录：记录 A 的发布序列较早但状态有效，记录 B 的更新时间较晚但已暂停。合格实现应选择记录 A 或返回受限结果，不能因为 B 的时间更新而把暂停版本当作最新有效版本。检查还应验证重复查询在同一索引快照上返回相同的资源 DID、版本和状态，并保留发布序列、包哈希与状态来源作为判断证据。

12.9 搜索排序与确定性结果顺序

12.9.1 匹配条件

匹配条件是候选进入排序的第一层依据。资源类型、版本、协议和能力标签等显式条件属于硬过滤；名称、描述、用例和协议材料中的文本命中属于相关性依据。候选必须先满足显式条件，再参与相关性排序；一个资源只因文本相近而不满足资源类型或版本条件时，不得进入结果。

12.9.2 信任和治理状态

已完成根平台可信发布、治理状态有效且节点授权可确认的候选，才能进入可信可发现结果。信任和治理状态不是把检索分数加高的普通特征，而是决定候选是否具备返回资格的约束。状态未知、证明缺失、资源暂停或撤销的候选应被过滤，或以明确受限状态返回，不能通过高相关性掩盖。

12.9.3 版本和新鲜度

同一资源 DID 默认只保留当前有效版本；在多个候选相关性相近时，应优先选择发布序列和治理状态均可确认的较新有效版本。新鲜度用于说明索引距离发布事实和治理事实的程度，不应直接等同于检索相关性。旧快照可以用于有限展示，但必须带有 `stale` 或等价状态。

12.9.4 排序稳定性

在同一索引快照、同一授权域和同一查询条件下，排序应得到可重复结果。排序首先依据实际启用的相关性组合，其后使用确定性的资源 DID、版本或发布序列作为稳定次级键。节点应避免使用数据库自然顺序、并发完成顺序或随机扰动作为隐含排序依据；否则分页和客户端去重将无法可靠工作。

12.9.5 重复结果处理

排序前或排序后都必须执行资源身份去重，不能让同一 DID 的重复同步记录占用多个结果位置。去重保留当前有效版本、较新发布序列和可验证证据更完整的记录，并保留来源节点和重复诊断信息。重复记录之间出现哈希或治理冲突时，应进入异常处理而不是任意选择一条作为可信结果。

12.10 搜索解释与证据展示

12.10.1 匹配原因

匹配原因应尽量指向可观察字段，例如能力标签交集、资源类型命中、协议命中、文本字段命中或自然语言意图匹配。解释可以是摘要，不要求暴露内部模型或索引实现细节，但不得编造资源未声明的能力。若查询经过词法、向量或混合路径处理，应能说明实际路径或至少说明解释的确定性范围。

12.10.2 资源来源

结果应标明发现服务节点的身份、索引更新时间、资源来源节点或来源引用，以及可用的发布序列和资源包哈希。来源字段用于追踪事实从何处同步，不表示来源节点对资源业务质量作担保。通过内容分发平台取得的资源，还应保留根平台发布材料的引用关系，不能只记录缓存地址。

12.10.3 DID、VC 与根平台证明

资源 DID 用于识别资源身份，DID 文档用于提供公钥、服务端点和资源元数据，注册凭证用于表达注册服务节点处理过的登记事实，根平台可信发布证明用于表达根平台对特定资源版本及其材料的验证和发布事实。发现响应可以返回这些证据的摘要或引用，但调用方在连接前应取得并验证完整材料，不能把任何一个引用字段当作已经完成验证。

发现响应的字段应按“索引展示”和“可验证证据”分层理解。`resourceDid`、`resourceType`、`score`、版本、标签和端点是候选展示信息；DID 文档、注册凭证、根平台可信发布证明及其签名材料才是调用方进行身份、控制权和发布事实核验的依据。发现节点返回一个资源候选，不等于节点替调用方完成了端点连通性测试、业务适配性评估或授权授予。

12.10.4 治理状态与版本信息

解释区域应同时展示资源生命周期、资源版本、发布序列、治理状态和索引新鲜度，使调用方能够区分“相关性较高”和“当前可验证”。治理状态来自治理事件投影或其它明确事实源，版本信息来自资源发布链；二者出现矛盾时，应标记冲突并按限制性规则处理，不得以较新的展示时间覆盖撤销或暂停事实。

搜索解释应把可验证事实和检索推断分开保存。`matchReason`、标签交集、资源类型命中和协议命中属于查询解释；DID 文档、公钥、注册凭证、根平台可信发布证明、发布序列和治理状态属于验证材料。前者回答“为什么出现在结果中”，后者回答“调用方可以依据哪些事实继续判断”，两者不能合并为一个推荐分数。

12.11 发现响应模式

发现响应至少应能让调用方识别四件事：查询由哪个发现服务节点处理，候选资源对应哪个资源 DID，候选来自哪个索引状态，以及为什么被返回。参考实现的响应可包含 `discoveryDid`、`candidates`、`createdAt` 和 `proof` 等顶层字段；候选项应包含资源 DID、资源类型、检索分数、版本、生命周期、能力标签、授权域、服务端点、协议绑定、资源包信息和根平台证明引用。具体部署可以增加字段，但不应把展示字段误认为证明字段。

```
{
  "discoveryDid": "did: oan: ...",
  "createdAt": "2026-09-07T10:00:00Z",
  "candidates": [
    {
      "resourceDid": "did: oan: ...",
      "resourceType": "skill",
      "score": 0.87,
      "version": "1.0.0",
      "lifecycle": "active",
      "capabilityTags": ["code-repository", "project-structure"],
      "serviceEndpoints": ["https://example.invalid/endpoint"],
      "protocolBindings": ["MCP"],
      "rootProofRef": "...",
      "freshness": {
        "publicationSequence": 42,
        "indexUpdatedAt": "2026-09-07T09:59:30Z"
      }
    }
  ]
}
```

`proof` 或 `rootProofRef` 只表示响应或候选所引用的证明材料，调用方仍需按第 7、9 章规则取得并验证相应内容。字段缺失时，节点应明确返回“不可用”或省略该字段，不能填入未经验证的默认值；未知扩展字段应允许兼容客户端忽略，但核心身份、状态和来源字段不得被改名后改变含义。

与当前协议结构对应的响应最小骨架是 `discoveryDid`、`candidates`、`createdAt` 和可选的 `proof`。候选项中的 `resourceDid`、`resourceType`、`score`、`version`、`lifecycleState`、`capabilityTags`、`authorizedDomains`、`services`、`protocolBindings`、`packageInfo` 和 `rootProof` 分别承担身份、检索、状态、能力、范围、

服务和证明引用的展示职责。当前接口未必返回独立的 `matchReason` 或完整新鲜度对象，客户端不能把缺省字段补成已经验证的事实。

当前发现查询入口为 `POST /discovery/resources/query`，请求体对应 `ResourceDiscoveryQuery`。该类型可包含 `query`、`resourceType`、`capabilityTags`、`protocol`、`version`、`versionMode` 和 `limit`；其中 `limit` 默认值为 10，`versionMode` 默认值为 `latest`。响应对应 `ResourceDiscoveryResponse`，候选项使用 `ResourceDiscoveryCandidate`，返回的 `score` 是匹配相关性分数，不是授权等级或可信等级。

`POST /discovery/query/explain` 也接收 `ResourceDiscoveryQuery`，但返回动态解释 JSON，包括匹配资源 DID、资源类型、匹配标记、分数、标签重合、资源类型/协议是否匹配以及语义搜索状态等。解释响应用于诊断检索过程，不是普通发现结果，也不能作为资源已经通过控制权或发布证明验证的替代品。

当前公共协议中的字段名称应作为客户端兼容的基准。资源候选使用 `lifecycleState`、`services`、`packageInfo` 和 `rootProof` 等字段时，客户端应按字段是否存在处理可选信息；示例中的 `lifecycle`、`serviceEndpoints`、`rootProofRef` 或 `freshness` 只能作为说明性扩展，不能据此声称当前所有部署都会返回这些名称。新增响应字段时，应保持旧客户端能够忽略未知字段，并为字段的事实来源和验证方式给出说明。

12.12 发现结果新鲜度与状态

发现结果的新鲜度由索引更新时刻、已处理的发布游标、治理事件序列和本地错误状态共同说明，而不是由 HTTP 响应时间单独决定。响应应尽可能提供 `createdAt`、索引更新时间、发布序列或同步游标、治理状态读取位置和来源节点 DID。调用方据此判断结果是当前快照、可接受的旧快照还是部分可用结果。

状态	含义	允许的使用方式
current	已完成必要发布和治理状态同步，且未发现更新缺口	可进入调用前验证流程
stale	本地仍有可验证快照，但发布或治理同步落后于允许范围	仅作带新鲜度标记的只读参考；调用前必须重新确认状态
partial	部分资源或部分索引分片可用，完整性范围已缩小	只能按响应明确的范围使用，不得宣称全量结果
unknown	无法确认同步位置、治理状态或证明新鲜度	不得作为敏感动作的可信依据

版本状态和治理状态发生冲突时，撤销、暂停或节点授权失效等限制性状态优先于相关性和时间排序。发现服务节点可以保留历史快照以改善网络异常时的访问体验，但必须显式标记快照状态，不能把缓存命中描述为网络最新状态。

发现结果的状态判断应以本地索引材料及其同步记录为依据。`createdAt` 只表示本次响应生成时间，不等于资源发布或索引应用时间；发布游标表示传播位置，索引更新时间表示本节点处理时间，治理事件序列表示治理读模型位置。调用方需要比较这些来源和位置，才能判断结果是否可能落后。

错误响应应让调用方区分请求错误、资源未命中、索引暂不可用和后端暂时失败。DID 格式错误、未知资源类型、无法解析的版本模式、超过查询限制等属于请求侧错误；合法查询但没有可见候选属于空结果；数据库、语义索引、治理读模型或上游同步失败属于服务或数据侧错误。错误信息应包含机器可判断的错误类别、请求标识和必要的重试提示，不应泄露受限资源的详细内容。

```
{
```

```
"error": {
  "code": "INDEX_STALE",
  "message": "The discovery index is temporarily behind the publication cursor.",
  "requestId": "...",
  "retryable": true
},
"freshness": {
  "status": "stale",
  "lastKnownPublicationSequence": 41
}
}
```

语义后端不可用时，节点可以回退到词法和结构化路径；回退只改变排序能力，不得放宽资源类型、生命周期、授权域、版本和显式标签条件。若基础索引本身不可用，节点不得用空数组伪装成“没有资源”，应返回可识别的服务错误或带有明确范围的部分结果。重试应由客户端依据 `retryable`、退避提示和自身请求预算执行，节点不得因自动重试造成请求放大。

错误处理应保持“请求错误”和“数据暂不可用”的边界：格式错误、未知枚举和超限参数不应重试原请求；语义后端暂时不可用可以在同一硬条件下走词法降级；基础索引或治理状态不可读时，返回部分结果必须明确范围和状态。发现节点可以记录查询统计和后端名称用于运维，但不应在公开错误中泄露授权域外资源的存在。

发现接口的错误处理应保持响应结构稳定，并使客户端能够决定是否修正请求、等待后重试或停止使用结果。可按以下边界处理：

错误类别	典型原因	客户端动作
请求无效	DID 格式、资源类型、版本模式或 <code>limit</code> 不合法	修正请求后重新发起，不重复原请求
无可见结果	条件合法，但本节点没有满足条件且可见的候选	可调整查询条件，但不得据此断言全网不存在
语义降级	语义索引或嵌入服务暂不可用，基础索引仍可读	接受词法/结构化结果，并降低对相关性的预期
索引不可用	资源索引、数据库或必要治理读模型无法读取	按服务错误处理，等待恢复或切换已知节点
同步滞后	本地发布游标落后或快照不完整	只按响应标记的范围使用，必要时稍后重试

语义降级与基础索引故障必须在实现和运维观测中区分：前者通常仍能返回满足硬条件的候选，只是相关性排序能力下降；后者无法证明候选集合完整或可见性判断有效，不应伪装成普通空结果。错误日志可以记录后端、耗时、候选数量和失败原因，但公开响应只返回客户端完成处理所需的最小信息。

12.13 发现错误与降级响应

资源能够通过某个 HTTP 端点访问，只说明该端点对当前请求可达；进入发现索引，说明发现服务节点已经保存了相应索引记录；具有根平台可信发布证明，说明根平台对特定版本和材料形成了可验证发布事实；通过调用前验证，则还要求调用方重新检查 DID 控制关系、证明、治理状态、端点和自身业务策略。四者是逐级增加的事实，任何一级都不能自动替代其它级别。

事实	主要来源	不能推出的结论
端点可访问	资源服务端点	资源已登记或值得信任
已进入发现索引	发现服务节点索引	资源是全网最新或允许调用
有根平台可信发布证明	根平台发布材料	业务端点一定可用或适合特定场景
通过调用前验证	调用方及其验证组件	调用方已经获得全部业务权限

公开发现响应可以展示必要的 DID、资源摘要、端点和证明引用；对受限资源应仅返回授权范围内的最小摘要。调用方在建立连接前，应根据资源风险重新取得或核验完整 DID 文档、注册凭证、根平台可信发布证明、最新治理状态和端点声明。

公开可见性由发现服务节点当前授权域和索引状态共同决定。资源没有出现在公开结果中，可能是尚未同步、已被过滤、已暂停或当前索引不可用，不能仅凭空结果判断资源不存在。调用方如果需要确定性结论，应使用精确 DID 查询、读取状态信息并执行完整的调用前验证，而不是枚举标签或分页结果推断隐藏清单。

可见性判断至少涉及三个独立问题：资源是否已经登记，资源是否已经传播到本节点，当前查询方是否能够看到该资源。发现服务节点只能依据本地可验证的索引材料和授权域作答，因此“返回候选”表示该资源在当前节点的可见范围内具备发现记录，“未返回”只表示当前查询没有获得可见候选。涉及敏感资源时，成功、空结果和权限过滤的响应不应通过错误码、字段数量或响应时延泄露额外信息。

可信可发现性还要求候选具备与当前版本对应的 DID 文档、注册凭证或根平台可信发布证明引用，并且没有已知的暂停、撤销或授权失效状态。发现节点提供的是索引和证据入口；调用方是否接受该资源，还要结合自身风险策略验证控制权、证明有效性、端点安全性和业务授权。

12.14 公开可见性与可信可发现性

去重的首要键是完整资源 DID。来自多个注册服务节点、内容分发平台或同步批次的记录，只要指向同一资源 DID，默认视为同一资源身份；如果记录还包含版本，则以资源 DID 加版本标识区分版本。包哈希、根平台发布序列和来源节点用于确认内容是否一致及保留证据，不能把名称相同或描述相似的资源错误合并。

去重决策应保留“合并依据”和“被舍弃记录的来源”，以便解释为什么一个候选只出现一次。默认查询可以采用以下优先级选择代表记录：先排除生命周期受限、授权域不匹配或证明不可核验的记录；再比较版本模式要求和发布序列；最后以包哈希、来源节点和确定性 DID 顺序消除并列。这个优先级只用于构造发现结果，不改变注册服务节点的原始受理记录，也不删除历史审计材料。

记录关系	发现结果处理	必须保留的依据
同 DID、同版本、同包哈希	合并为一个候选	来源节点和同步批次
同 DID、不同版本	按 <code>versionMode</code> 选择或分别返回	版本、发布序列和前序关系
同 DID、同版本、不同包哈希	不自动合并为可信记录	冲突哈希、来源和诊断状态
不同 DID、内容相同	作为不同资源返回	各自 DID、控制关系和证明

去重必须发生在分页之前，否则同一资源的重复记录会占用多个结果位置，并在客户端跨页合并时造成遗漏。历史版本查询是例外：它可以返回同一 DID 的多个版本，但每个版本都必须带有明确版本和状态，不能让客户端误把历史候选当成当前有效版本。

12.14.1 DID 级去重

同一查询响应中，同一资源 DID 默认只返回一个代表记录。代表记录应来自当前有效且发布序列较新的版本，并保留可追溯的来源节点、根平台证明引用和同步位置。若不同记录的治理状态相互冲突，应保留冲突诊断信息并采用限制性状态，不得通过任意排序隐藏冲突。

12.14.2 版本级去重

同一 DID 的不同有效版本只有在查询明确要求历史版本或指定版本约束时才同时返回。默认查询返回当前有效版本；历史查询应将版本标识、前序版本和发布序列作为结果字段，避免客户端把历史记录误当成当前版本。相同版本标识但包哈希不同属于一致性异常，应拒绝合并并进入诊断或隔离处理。

12.14.3 注册节点重复记录

同一资源重复向一个或多个注册服务节点提交，不应产生多个资源身份。发现节点应依据根平台发布事实、资源 DID、版本和包哈希去重；注册服务节点的本地受理记录可以作为来源审计信息保留，但不能单独生成公开候选。重复提交是否重新签发注册凭证属于注册协议行为，不改变发现侧的资源身份规则。

12.14.4 资源内容相同但 DID 不同的处理

内容、名称或端点相同而 DID 不同的记录仍是不同资源身份，不能仅因内容哈希相同就合并。节点可以在解释字段中提示内容相似或共享包哈希，但必须分别展示控制关系、注册状态、版本和根平台证明。是否允许同一内容被多个 DID 代表，应由资源控制方和治理规则决定。

12.15 结果去重与资源身份匹配

节点应限制查询文本长度、结构化条件数量、单次返回条数和语义检索资源消耗。limit 必须经过服务端上限校验，客户端不得通过重复请求或异常参数绕过该限制。无效游标、过期游标、与原查询不匹配的游标应返回明确错误，而不能静默从第一页重新返回，避免客户端误以为分页结果完整。

在同一索引快照中，分页必须沿用同一套过滤、去重和排序规则。排序至少应由检索分数、确定性的资源 DID 或其它稳定键组成；不能使用每次请求随机数、未定义的数据库自然顺序或不稳定的并发完成顺序。索引快照在分页期间发生变化时，响应应通过快照标识、游标失效或新鲜度字段告知客户端，避免跨快照重复或遗漏被误认为协议错误。

情形	推荐处理
结果少于 limit	返回全部可见且满足条件的候选，并标明没有更多结果
结果达到上限	返回稳定的分页游标和当前快照信息
查询超过长度或资源预算	返回不可重试的请求错误
游标无法解析或不匹配查询	返回游标错误，要求客户端重新发起查询
索引在分页期间更新	继续使用原快照或明确使游标失效

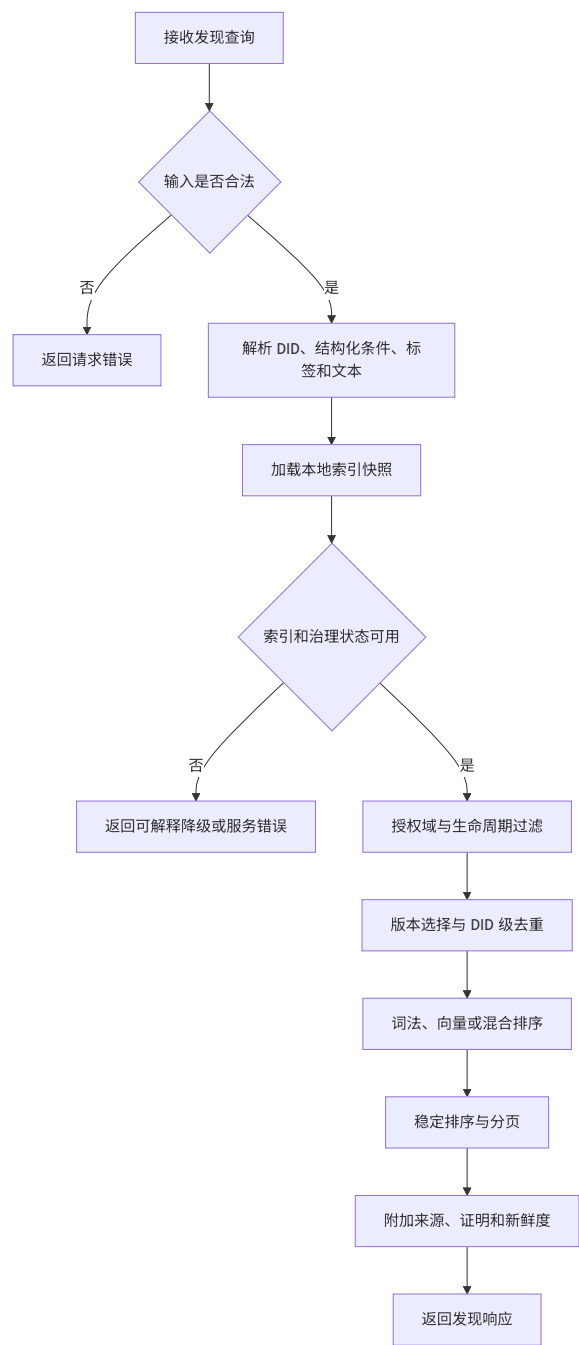
分页只解决传输和展示规模问题，不改变授权域和可信可发现性的判定。客户端合并多页时应以资源 DID 和版本作为去重键，并保留每页来源和快照信息。

12.16 查询限制、分页与结果稳定性

授权域过滤应在结果返回前完成，而不是先返回候选再依赖客户端自行隐藏。对于不在请求方或节点授权范围内的资源，节点不应通过名称、标签、端点、证明引用或错误差异泄露其存在；在需要区分权限不足和资源不存在的管理接口中，才可以向已授权管理方返回更详细的原因。公开查询默认只返回已发布、可发现且属于本节点服务域的最小必要字段。

索引材料应按字段敏感性划分：资源 DID、资源类型和公开能力摘要可以进入公开结果；私有端点、内部标签、控制公钥以外的敏感配置、提交人的私密身份信息和未公开证明材料不得因语义索引被暴露。向量索引、词法索引和查询日志都应遵守同一授权域边界，不能因为检索后端需要文本而复制不应公开的内容。

查询日志应最小化记录原始自然语言输入、调用方标识和过滤条件，并按照部署方的隐私策略设置保留期限、访问控制和脱敏规则。请求方不能通过枚举 DID、标签组合、分页游标或错误响应推断受限资源的完整清单。授权域变更、资源撤销和节点停权传播期间，发现节点应优先收紧可见性，并保留状态来源和处理时间供审计。



该流程表达的是处理责任顺序：输入校验和硬过滤先于相关性排序，版本选择和去重先于分页，来源、证明和新鲜度随响应一起返回。语义后端的使用是排序能力，不得改变前置身份、授权和生命周期判断。任何降级路径都必须说明它实际完成了哪些验证以及没有完成哪些验证。

检查项	合格判据
DID 查询	完整资源 DID 按精确路径处理，不退化为模糊搜索
结构化过滤	资源类型、版本、协议、能力标签和生命周期条件按明确交集执行
授权域	未获授权的资源不出现在公开结果，域状态不明时按收紧规则处理
相关性解释	检索分数只表达相关性，不被称为信任等级或调用许可
版本选择	默认返回当前有效版本，历史版本查询不混淆当前状态
去重	同一 DID 不因多节点同步或重复注册产生重复候选

检查项	合格判据
响应来源	结果能指向发现服务节点、索引状态、发布序列和证明引用
新鲜度	旧快照、部分结果和治理状态不明均有明确标记
错误降级	语义后端故障不放宽硬条件，基础索引故障不伪装为空结果
分页稳定性	同一快照中排序稳定，游标与查询和快照绑定
隐私边界	查询、索引和错误响应不泄露授权域外资源的敏感信息
调用前验证	发现结果只作为候选输入，调用方仍重新验证身份、证明和业务权限

12.17 发现隐私与授权域可见性

授权域是发现可见性的边界，不是资源控制权或调用权限的替代物。发现服务节点应先依据本节点获授权的服务域和资源包中的 `authorizedDomains` 判断候选是否可见，再对可见候选进行结构化和语义处理。资源域与节点服务域不相交时，候选应在公开响应前被过滤；查询方不能通过名称、标签、端点、响应数量或错误差异推断被过滤资源的详细信息。

索引和展示应遵循最小披露原则。公开候选可以提供资源 DID、资源类型、公开能力标签、必要的服务端点、版本和证明引用；控制私钥、提交人的本地身份备份、内部治理材料、未授权端点和不属于公开资源包的配置不能进入索引或查询响应。语义索引使用的文本、标签别名和向量表示也应继承原始字段的授权边界，不能因为进入检索后端就扩大可见范围。

当授权域、生命周期或治理状态正在同步，且本地无法确认最新状态时，应优先收紧公开可见性。对管理和审计用途，可以在受控接口中记录过滤原因、来源节点、治理事件序列和处理时间；对普通发现请求，只返回不泄露受限资源存在性的统一结果。查询日志也应按最小必要原则保存，并限制原始自然语言、调用方信息和查询条件的访问范围与保留期限。

信息类别	可进入公开发现结果	处理要求
资源身份	资源 DID、资源类型	与当前可见版本绑定
能力描述	已公开的能力标签、名称和摘要	不得扩展为资源未声明的能力
服务信息	授权范围内的服务端点和协议绑定	调用前仍需验证端点
信任材料	根平台可信发布证明的必要引用	不把引用当作已完成验证
私密材料	私钥、本地身份备份、内部配置和未公开证明	不进入公开索引、响应或错误详情

12.18 参考来源

来源	类型	链接
oan-discovery-node	代码仓：同步、索引、结构化查询和语义查询	https://github.com/OpenAgenet/oan-discovery-node
oan-protocol-common	代码仓：发现请求、响应和查询类型	https://github.com/wolfbrother/oan-protocol-common

来源	类型	链接
oan-sdk-ts	代码仓：发现客户端和查询结果处理	https://github.com/OpenAgenet/oan-sdk-ts
GRAIL Semantic Discovery Paper	arXiv 论文：混合语义检索和发现解释	https://arxiv.org/abs/2605.02489
Efficient Agent Discovery Profile	IETF 草案：智能体发现元数据和查询配置	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第十三章 语义治理与语义发现

本章说明 OAN 如何治理能力标签、别名、授权域和语义配置，并将资源名称、描述、用例、协议绑定等语义信息用于注册建议、结构化过滤、文本召回、语义排序和结果解释。章节同时划清语义相关性与身份、控制权、发布证明、生命周期和治理状态之间的边界，确保语义发现提升资源匹配能力，而不改变可信发布和授权判断。

13.1 语义模型与能力词汇

在 OpenAgenet (OAN) 中，语义模型把资源的“能做什么”拆成几类可以被注册和发现复用的信息：资源名称和描述提供自然语言上下文，能力标签提供相对稳定的分类键，用例说明适用任务，协议绑定说明交互方式，资源类型说明对象类别。它们共同参与推荐和检索，但承担的事实范围不同。能力标签树回答“这个词属于哪一类能力”，资源包回答“某个具体资源声明了哪些能力”，DID 文档和凭证材料回答“该资源由谁控制、是否完成登记以及发布事实是否可验证”。

因此，语义匹配的结果应被理解为候选发现线索，而不是可信结论。注册服务节点可以依据名称、描述、用例和协议生成标签建议，提供方确认后再写入资源元数据；发现服务节点可以复用这些字段进行过滤、词法检索或语义排序。无论推荐分数多高，都不能替代控制权验证、注册凭证、根平台可信发布证明、生命周期状态和授权域检查。

13.2 根平台管理的能力标签树

能力标签树由根平台发布其版本和快照，供注册推荐器和发现服务节点作为共同语义材料使用。当前公共实现中的标签树对象包含 `version`、可选的 `snapshotHash` 和 `tags`；标签节点至少包含稳定的 `id`、展示用 `label`，并可包含 `aliases` 和递归的 `children`。下面是根据该数据结构整理的最小示例，值均为说明性占位值：

```
{
  "version": 1,
  "snapshotHash": "sha256:example",
  "tags": [
    {
      "id": "code.repository.search",
      "label": "代码仓库检索",
      "aliases": ["检索代码库"],
      "children": []
    }
  ]
}
```

标签树的节点结构用于组织语义和复现查询，不等于资源授权清单。根平台新增或调整标签后，注册推荐可以使用新版本，发现服务节点也可以据此重建索引；已经提交的资源原始描述和原始标签不能因为标签树变化而被静默改写。若节点使用旧快照，应记录所用版本，以便解释推荐或发现结果的差异。

13.2.1 标签树的组织方式

能力标签树采用“稳定标识 + 父子关系 + 展示名称 + 词汇版本”的组织方式。树节点表达能力分类，不直接表达资源是否已经注册、发布或可信。资源可以关联多个叶节点，也可以只关联较高层级节点，但注册服务节点应保存资源实际选择的标签，而不是仅保存推荐路径。

```
{
  "id": "code.repository.search",
  "parent": "developer.tools",
  "labels": {"zh-CN": "代码仓库检索", "en-US": "Code Repository Search"},
  "vocabularyVersion": "v1"
}
```

13.2.2 标签标识与层级关系

标签 ID 是跨节点关联键，父子关系用于组织检索范围，不能在没有词汇规则的情况下自动推导能力继承。查询结果应同时保留命中的标签 ID、匹配路径和词汇版本，避免只返回本地化名称导致跨节点无法复现。

信息	用途	是否可作为可信依据
标签 ID	跨节点关联和稳定查询	否
父子路径	分类和过滤	否
标签别名	查询扩展	否
资源控制证明、发布证明	控制权和发布事实	是，需独立验证

13.2.3 标签推荐与发现使用

注册阶段的推荐结果由资源提供方确认后进入资源元数据；发现阶段可以使用同一标签 ID、别名和层级路径进行筛选或排序。推荐命中、标签命中和资源可信验证应在响应中分开表达。

```
资源描述 -> 推荐候选 -> 提供方确认 -> 写入资源包
                                         -> 发现索引
查询文本 -> 意图与别名解析 -> 标签条件 -> 候选排序
```

13.2.4 标签树版本管理

标签树发布时应有可比较的版本标识和生效时间。旧版本标签可以在迁移期保留解析映射，但不得静默改写资源原始声明；索引重建完成前，查询结果应标注所使用的词汇版本。

13.3 标签层级、标识符、别名与本地化

标签对象至少应能区分规范标识、显示名称、别名、语言和版本：

字段	示例	处理规则
id	code.repository.search	稳定，不因翻译改变
label	代码仓库检索	用于展示
aliases	检索代码库	用于查询扩展
language	zh-CN	说明文本语言
vocabularyVersion	v1	用于复现和迁移

未知标签可以作为未治理的提供方文本保存，但不得被伪装成根平台正式标签。

标签 ID、显示名称、别名和层级路径的职责应保持分离。查询输入可以先通过本地化名称或别名找到规范 ID，再使用规范 ID 进行结构化过滤；返回结果则应同时保留实际资源声明和命中的规范标签。这样既能支持中文、英文和同义词查询，也不会把展示层的翻译误当成新的能力声明。

13.4 注册过程中的标签选择与推荐

13.4.1 资源提供方选择

资源提供方可以接受、取消或修改推荐标签。最终提交内容应同时保留提供方原始描述和规范标签选择，便于后续解释“资源声明了什么”与“系统推荐了什么”。

13.4.2 根平台推荐

根平台或其治理材料提供标签树、别名和版本；推荐器根据资源类型、名称、描述、用例和协议生成候选。推荐器不负责签发资源控制证明，也不替代注册校验。

13.4.3 自定义标签补充

自定义标签用于表达正式词汇尚未覆盖的业务术语。自定义标签必须带来源或命名空间，并与正式标签分开存储；它可以参与文本检索，但不能直接改变资源类型、授权域或发布状态。

13.4.4 标签校验与保存

保存前至少校验标签 ID、词汇版本、父节点关系、语言、重复项和资源类型适配性。校验结果应与资源 DID、资源版本和资源包摘要关联，推荐失败或标签过期不得导致静默改写。

13.5 自定义标签与提供方描述

自定义标签适合表达具体业务术语、组织内部叫法或暂未进入公共词汇的能力。它应与根平台正式标签分开保存，并带有提供方命名空间或来源信息；名称、描述、用例等自由文本也应保留原文和语言标记。正式标签 ID 需要来自当前可用的标签树，自定义词汇不能借助格式相似或语义相近自动升级为正式标签。

发现服务节点可以把自定义标签和提供方描述用于词法或语义召回，但结果解释必须区分“提供方声明”“正式标签命中”和“检索模型推断”。重复标签、超长文本、无效字符、与资源类型明显冲突的标签以及试图操纵排序的关键词，应按节点的输入校验和索引策略拒绝、截断或降级为普通文本；处理结果应保留可追溯的原因，不得覆盖原始提交内容。

13.6 语义治理变更流程

语义治理变更应以新版本材料表达，而不是直接改写旧标签或历史资源。一个变更记录至少需要能够说明变更类型、影响的标签 ID、旧版本与新版本、别名或层级映射、生效时间以及对注册推荐和发现索引的影响。新增和改名通常可以通过新增映射兼容；合并和拆分需要明确旧标签如何查询；废弃标签则应停止新推荐，同时保留历史资源的原始值和可解析关系。

变更发布后，注册推荐器可以加载新词汇，发现服务节点可以按资源 DID 和资源版本逐步重建相关索引。重建未完成时，查询结果应能说明使用的词汇或索引版本；如果旧标签无法映射，节点可以保留其原文用于文本检索，但不得把它伪装为新标签的确定命中。标签治理只改变语义解释和检索材料，不改变资源 DID、控制关系、注册凭证或根平台发布事实。

13.7 发现节点的语义索引构建

13.7.1 索引输入

发现服务节点的索引输入分为三类：

1. **资源事实：**资源 DID、版本、资源类型、公开 DID 文档和资源包元数据。
2. **语义材料：**规范标签、别名、名称、描述、用例、协议和语言。
3. **治理材料：**生命周期、根平台可信发布证明引用、授权域、标签树版本和来源节点。

私钥、未发布草稿和内部配置不得进入索引。

13.7.2 标签索引

标签索引至少支持按规范标签 ID、层级路径、语言和词汇版本过滤，并保留标签来源。索引命中只能说明资源声明或索引记录匹配，仍需检查资源是否处于可发现状态。

13.7.3 文本和语义索引

文本索引可以覆盖名称、描述和用例；语义索引可以使用推荐器或可选向量实现。两者都应返回命中字段、索引版本和匹配依据，不能补写资源没有声明的能力。

13.7.4 索引更新与删除

资源版本、生命周期、发布证明或标签树版本变化时，应按资源 DID 和版本执行增量更新。撤销、暂停或删除事件到达后，索引可以保留审计记录，但不得继续作为正常可发现候选返回。

13.8 标签、文本、协议与资源类型匹配

四类条件在查询中的作用不同：资源类型和明确指定的版本通常用于限制候选类别，能力标签和协议可以作为显式过滤条件，名称、描述和用例中的文本主要用于相关性排序。多个显式条件一般按交集处理；标签查询要求候选满足请求的标签集合，协议可以从协议绑定或服务声明中匹配。语义相似只能改善排序，不能让不满足硬条件的候选重新进入结果。

候选资格 = 可见性与生命周期 + 显式结构化条件
相关排序 = 标签/协议命中 + 文本相关性 + 语义相似度
可信判断 = DID文档 + 凭证/发布证明 + 调用方策略

其中，前两行属于发现检索范围，最后一行属于独立的身份、发布和调用前验证。语义服务不可用时，发现节点可以退回词法或结构化路径，但不得因此放宽授权域、生命周期和显式条件。

13.9 查询解释与匹配说明

13.9.1 查询意图解析

查询解析应输出可审计的中间条件，例如资源类型、能力标签、协议、语言和状态过滤。无法确定的词语可以作为文本条件保留，不得被强行转换成正式标签。

```
{
  "query": "我需要一个可以检索代码仓库并总结项目结构的工具",
  "parsed": {
    "resourceType": "tool",
    "capabilities": ["code.repository.search", "code.structure.summarization"]
  }
}
```

13.9.2 标签和文本匹配

标签命中、别名命中、原文命中和语义推断命中应分类返回。硬过滤条件不满足时，相关性得分不得使候选重新进入结果集。

13.9.3 结构化条件匹配

资源类型、协议、授权域、生命周期和发布状态属于结构化条件。结构化条件的结果应明确为满足、不满足或未知，未知不能按满足处理。

13.9.4 结果解释

当前发现服务节点的查询解释接口接收与普通发现查询相同的 `ResourceDiscoveryQuery`，但返回的是动态 JSON，而非 `ResourceDiscoveryResponse`。解释结果可包含 `items`、`candidateCount`、`matched`、各类匹配分数、标签重合、结构化条件命中和语义搜索状态；这些字段是检索过程的可观察结果，不是新的资源证明。客户端应把解释用于回答“为什么返回”，仍以候选资源的 DID 文档、资源包、根平台可信发布证明和治理状态回答“是否可以信任或调用”。

结果解释至少应包含资源 DID、命中的字段或标签、词汇/索引版本和可信状态来源。可用以下方式区分两类信息：

“相关”说明查询与资源描述存在匹配；“可信”还需要 DID、控制证明、发布证明、授权状态和生命周期等独立验证。

查询解释应保留从自然语言到结构化条件的可观察链路。例如，用户输入“我需要一个可以检索代码仓库并总结项目结构的工具”时，系统可以识别“工具”作为资源类型候选，把“检索代码仓

库”和“总结项目结构”作为能力或文本检索线索；如果当前词汇没有可靠的正式标签映射，就应保留原句和普通文本命中，而不是凭模型推断创建新标签。解释内容可以展示命中的标签、字段、别名和索引版本，但不应暴露不必要的模型内部细节。

13.10 多语言语义发现

13.10.1 中英文资源描述

中文和英文描述可以同时保存，原文应带语言标记。翻译或机器生成文本只能作为检索投影，不得覆盖提供方原始描述。

13.10.2 多语言标签

不同语言的标签名称应映射到同一规范标签 ID；如果没有可靠映射，应保留为普通文本而不是创建未经治理的新同义关系。

13.10.3 查询语言识别

语言识别只选择词法、别名和分词路径，不能改变资源类型、状态或信任过滤条件。混合语言查询应保留各片段的解析依据。

13.10.4 跨语言匹配

跨语言命中应返回规范标签 ID、实际命中的语言和别名，并标明是标签映射还是文本翻译命中。跨语言命中不等于资源具备额外能力。

13.11 面向向量检索与混合检索的扩展性

当前发现节点的语义扩展可以使用 PostgreSQL/pgvector 和 HTTP 嵌入服务构建资源级语义索引；本地语义推荐器则提供能力标签、资源类型和协议等推荐辅助。采用哪一种后端属于部署和参考实现选择，不能改变资源 DID、资源包、授权域、生命周期及根平台可信发布证明的处理规则。语义索引还应记录模型名称、模型版本、向量维度、索引版本和语义源哈希，使重建前后的结果能够解释。

阶段	需要保留的信息	失败时的边界
资源投影	DID、版本、公开描述、标签、用例和协议	不把私密材料写入语义索引
嵌入生成	模型标识、模型版本、维度和源内容关联	单个资源失败不应伪造向量
查询检索	查询文本、硬条件、索引版本和后端	语义失败可降级，但不得放宽硬条件
结果解释	相关性、命中字段和证据引用	相关性不等于信任或调用许可

当模型、维度或语义词汇发生不兼容变化时，应先重建相应索引，再将新结果作为同一索引版本提供给调用方。旧索引可以在迁移期保留，但不能把不同模型产生的分数直接横向比较。

13.12 语义治理兼容性与迁移

迁移时应同时保留三种信息：资源提供方提交的原始标签和描述、当时使用的词汇版本、以及新版本中的可解析映射。兼容映射可以让旧查询继续找到资源，但不能改写历史资源包中的原始字段。发现服务节点完成索引迁移后，应更新索引版本；迁移未完成或存在无法映射的标签时，结果可以继续作为有限的文本参考，但需要说明其语义治理状态。

13.13 语义词汇版本管理与变更影响

词汇版本与语义索引版本应分开记录：前者说明标签树和别名材料的版本，后者说明某个发现节点实际用哪些输入构建了检索数据。两者变化对系统的影响可以按以下方式理解：

变更类型	注册推荐	既有资源原始值	发现索引
新增标签或别名	可用于新推荐	不受影响	可增量更新
改名或本地化调整	保留规范 ID	原始文本保留	更新显示和检索映射
合并标签	建立旧到新的查询映射	不自动改写	重建相关资源的投影
拆分标签	需要提供方补充信息	不自动猜测	无法确认时保留旧值并标记
废弃标签	停止新推荐	保留历史可解析值	可降级或从正常推荐中移除

发生回滚时，应恢复到上一套可验证的词汇和索引版本，而不是删除资源历史记录。查询结果和运维诊断至少应能指出使用的词汇版本、索引版本和更新时间，便于解释“同一查询为何在不同节点返回不同排序”。

13.14 语义治理可审计性

语义治理的审计记录需要回答“谁在什么时间对哪一个词汇版本做了什么改变，以及哪些推荐和索引受到了影响”。建议至少关联以下对象：操作者或治理主体、操作时间、变更类型、旧值和新值、词汇版本、原因、影响范围、索引重建状态和回滚依据。审计记录用于解释语义材料的变化，不应把标签变更记录误当成资源注册凭证或资源控制证明。

注册推荐和发现查询也应保留必要的引用关系。例如，推荐结果可以关联输入摘要、词汇版本和推荐候选；发现解释可以关联命中的标签 ID、别名或文本字段、索引版本和资源 DID。对于自然语言原文、调用方信息和内部模型输出，应遵守部署方的访问控制和最小留存策略，公开接口只暴露解释所需的最小内容。

13.15 自然语言查询与结构化条件映射

自然语言映射应分为“可以落到结构化字段的条件”和“只能作为检索线索的表达”。例如，“找一个可以检索代码仓库并总结项目结构的工具”可能得到资源类型候选、能力标签候选和文本检索词；“最新”“可信”“可调用”等词只有在协议和治理材料提供对应字段或事实时，才能参与确定性过滤，否则只能作为排序或解释提示。无法确认的词语必须保留其原文，不得强行映射为正式标签或授权条件。

```
{
  "query": "我需要一个可以检索代码仓库并总结项目结构的工具",
  "structuredConditions": {
```

```
    "resourceType": "tool_api",
    "capabilityTags": ["code.repository.search"]
  },
  "textTerms": ["总结项目结构"],
  "mappingStatus": "suggested"
}
```

上述对象是解释性中间结果示例，不是当前发现接口固定返回格式。正式请求仍应使用第十二章规定的发现查询字段；中间结果中的 `suggested` 表示语义建议，调用方或资源提供方仍需确认标签、资源类型和协议条件。映射失败、词汇版本不兼容或语言不匹配时，可以退回原文检索或返回明确的无可映射结果，但不能把失败隐藏为错误的正式条件。

13.16 参考来源

来源	类型	链接
oan-discovery-node	代码仓：语义别名、索引和语义查询	https://github.com/OpenAgenet/oan-discovery-node
oan-protocol-common	代码仓：能力标签、授权域和语义辅助材料	https://github.com/wolfbrother/oan-protocol-common
GRAIL Semantic Discovery Paper	arXiv 论文：语义发现、混合检索和排序解释	https://arxiv.org/abs/2605.02489
Efficient Agent Discovery Profile	IETF 草案：发现元数据与查询 profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第十四章 节点授权与链上治理

本章说明 OpenAgenet (OAN) 如何把链上治理事实转换为基础设施节点可以使用的授权状态。治理合约记录节点授权、授权域、能力范围和状态变化；链下信任索引器读取并保存这些事件，生成可查询的主体状态；根平台、注册服务节点和发现服务节点据此决定是否接受请求、传播资源或使用某个节点产生的治理材料。链上授权、节点授权凭证、资源登记和资源发布分别证明不同事实，不能相互替代。

本章中的“当前状态”均指某个组件基于其已同步读模型作出的判断。读模型可能暂时落后于链上事件，因此涉及新授权、暂停、恢复、撤销或高风险动作时，应同时关注事件序列、同步位置、状态有效期和证据来源。

14.1 治理目标与链上、链下边界

链上治理层和链下业务层通过“读取、验证、投影、执行”衔接，而不是共享同一个状态表。链上治理合约产生可复核的原始事件，索引器保存事件和主体状态，根平台或节点服务读取该投影并结合节点 DID、授权凭证和请求内容执行运行时判断。资源控制证明、资源包和发布状态属于资源业务链路，不能因其已存在而跳过节点治理状态检查。

14.1.1 链上治理事实

链上治理事实包括节点授权、暂停、恢复、撤销及其时间、序号和来源。链上事实是治理状态的权威来源，但链下服务可能在同步完成前保留旧读模型；响应中应区分链上位置、索引时间和服务当前采用的状态。

在当前治理合约中，治理事件由 `GovernanceExecutionEvent` 表达，包含动作类型、事件序号、目标节点 DID、节点类型、状态、授权域、生效时间、到期时间、策略哈希、元数据哈希以及前一事件和当前事件摘要等字段。事件的原始载荷和交易、事件序号、checkpoint 等链上位置应一并保存。这样，读侧既可以得到某节点的当前状态，也可以回溯该状态由哪一次治理执行产生。

链上事件与链下服务的关系可以概括为：链上记录证明“治理动作已经发生以及其顺序和参数”，索引器根据这些事件构造可查询状态，根平台和其他节点再按照自身策略使用该状态。索引器不能修改链上事件、重新签发治理权限或凭空补齐缺失事件；当原始事件和读模型不一致时，应以原始位置和事件摘要为核对入口。

字段	作用
节点 DID	标识被治理节点
事件类型	表示授权、暂停、恢复或撤销
授权域与能力	限定允许执行的操作
链上位置	支持顺序和复核

字段	作用
生效时间	区分链上发生时间和服务观察时间

14.1.2 根平台运行状态

根平台运行状态表示服务是否已读取并执行当前治理结果，包括是否接受节点提交、是否允许发布和是否允许向下游分发。它是链上治理事实的运行时投影，不得反向替代链上事实。

根平台运行时至少要区分“治理状态已知”和“本次请求已获准”两个判断。前者依赖链下信任索引器或其他治理读侧提供的状态、序列和新鲜度，后者还要结合请求节点 DID、请求动作、授权域和能力范围进行匹配。即使节点状态为 `active`，也只能说明它可能具备相应基础设施职责，资源控制权、资源内容、版本和发布材料仍需单独验证。

14.1.3 注册和发布状态

注册受理、根平台发布、内容分发和发现可见是不同状态。节点授权有效是执行资格条件，不等于某个资源已经通过控制权、内容和版本验证。

这些状态由不同组件产生：注册服务节点记录资源登记请求及其校验结果，根平台决定是否形成根平台可信发布事实，内容分发平台负责已批准材料的传播，发现服务节点把可见资源纳入本地索引。链上节点授权只决定基础设施节点是否有资格执行某类动作，不能替代资源提供方的签名、资源包完整性检查、发布流程或发现索引更新。

节点获授权 -> 可以提交请求
请求通过验证 -> 可以进入发布流程
根平台发布 -> 可以进入分发流程
发现节点索引 -> 资源可能对查询可见

14.1.4 发现节点读模型

发现节点使用链下信任索引器或治理同步结果过滤资源。读模型必须记录来源和同步进度；治理事件尚未同步时，应采用保守状态，不能把旧缓存直接解释为当前授权结论。

发现节点实际消费的是治理读模型，而不是在每次查询时直接扫描链上历史。读模型至少需要能够关关节点 DID、节点类型、当前状态、授权域、最后治理序号、事件摘要和更新时间；查询结果中的资源来源状态还应与资源自身的生命周期和发布证明分开表示。索引器滞后时，发现节点可以保留只读快照改善访问连续性，但不能把快照标为当前治理事实。

14.2 基础设施节点授权模型

基础设施节点以节点 DID 作为治理对象，并通过节点类型区分注册服务节点、发现服务节点和 VC 颁发者。当前索引器的 `SubjectState` 保存节点类型、节点 DID、状态、授权域、策略哈希、元数据哈希、生效时间、到期时间、状态版本、最后更新时间、最后治理序号和最后事件摘要。节点授权凭证可以作为链下证明材料使用，但它与链上状态是两条需要组合核验的证据链：凭证说明颁发者对节点资格的声明，链上读模型说明当前治理状态和授权范围。

一个最小的状态判断可表示为：


```
{
  "subjectType": "discovery",
  "subjectDid": "did:oon:INDS:example",
  "status": "active",
  "authorizedDomains": ["technology.software_engineering"],
  "effectiveFromMs": 0,
  "expiresAtMs": 0,
  "lastSequence": 12
}
```

上例中的 DID、序号和授权域是说明性值。运行时只有在节点类型匹配、状态为有效、当前时间已达到生效时间且尚未超过到期时间时，才可把该状态作为“治理层面允许”的输入；还必须继续检查请求动作是否落在授权域和能力范围内。

14.3 授权域与能力范围

授权域回答“节点可以处理哪些业务或资源范围”，能力范围回答“节点可以执行哪些基础设施动作”。例如，一个发现服务节点可以被授权服务某个能力域，但这不等于它可以代表资源控制者修改资源 DID 文档，也不等于它能够颁发注册凭证。授权域应使用规范化、可比较的域值；合约当前还对授权域的非空、数量、长度、规范顺序、重复项和通配符混用进行输入限制。

多个授权域通常表示节点服务范围的并集，具体资源是否可被处理还要经过节点状态、资源授权域和治理状态的共同判断。授权域变更应产生新的治理事件和状态版本；缩减范围后，节点不得继续以旧范围接受新请求，已经形成的历史发布事实仍应保留其来源和时间，不能被简单删除。

14.4 治理事件模型

治理合约将提案创建、投票和最终执行分开处理；索引器关注已经发出的初始化事件和治理执行事件。事件解析后，索引器同时保存原始事件记录和结构化治理事件：原始记录保留交易摘要、事件序号、checkpoint、包标识、事件类型、时间和原始 JSON，结构化记录则便于按节点 DID、治理序号、状态和授权域查询。页面上的事件名称和说明是投影字段，不能替代原始事件载荷。

治理执行事件的关键字段可按下表理解：

字段类别	典型字段	作用
身份关系	subjectDid、subjectType、targetAddress	关联被治理节点及其链上目标
动作状态	eventType、eventLabel、subjectStatus	表示授权、暂停、恢复、撤销或域变更
范围时间	authorizedDomains、effectiveFromMs、expiresAtMs	约束节点适用范围和有效窗口
顺序证据	sequence、previousEventDigest、eventDigest	支持事件链和状态重放核对
变更依据	proposalId、policyHash、metadataHash	关联治理提案和策略材料
链上来源	交易摘要、事件序号、checkpoint	指向原始链上记录和读取位置

官网 Network 页面通过公开接口 `/api/public/network/governance-events` 展示当前读模型中的事件列表。本次核对时接口的 `generatedAt` 为 `2026-09-12T02:01:05.463553057+00:00`。以下表格按接口当前返回顺序记录时间、节点和页面说明；它用于帮助读者直观理解已经公开观测到的治理动作，不替代治理合约中的原始事件载荷、交易和 checkpoint。该列表会随链上治理和索引器同步状态变化：

时间（UTC）	节点	事件说明
2026-08-25 03:11:30	Test Discovery	更新测试发现节点授权域为全域。
2026-06-29 02:16:44	Test Discovery	更新测试发现节点授权域为 28 个行业/领域集合。
2026-06-29 02:15:47	Registrar 1	更新注册节点授权域为 28 个行业/领域集合。
2026-06-29 01:26:06	Test Discovery	更新测试发现节点授权域为 16 个行业/领域集合。
2026-06-29 01:25:10	Registrar 1	更新注册节点授权域为 16 个行业/领域集合。
2026-06-29 01:12:12	Test Discovery	更新测试发现节点授权域为 technology 。
2026-06-29 01:11:05	Registrar 1	更新注册节点授权域为 technology 。
2026-06-28 22:48:58	Official Discovery	更新官方发现节点授权域为全域。
2026-06-28 22:47:41	Test Discovery	更新测试发现节点授权域为全域。
2026-06-28 22:46:14	Test Registrar	更新测试注册节点授权域为全域。
2026-06-28 22:45:59	Official Registrar	更新官方注册节点授权域为全域。
2026-06-28 22:45:35	Registrar 1	更新注册节点授权域为全域。
2026-06-04 09:16:16	Official Discovery	恢复官方发现节点为 active 。
2026-06-04 09:15:47	Official Discovery	暂停官方发现节点授权。
2026-06-04 09:15:17	Test Discovery	恢复测试发现节点为 active 。
2026-06-04 09:14:48	Test Discovery	暂停测试发现节点授权。
2026-06-04 09:12:10	Test Registrar	恢复测试注册节点为 active 。
2026-06-04 09:11:41	Test Registrar	暂停测试注册节点授权。
2026-06-04 09:11:12	Official Registrar	恢复官方注册节点为 active 。
2026-06-04 09:10:42	Official Registrar	暂停官方注册节点授权。
2026-06-04 09:10:14	Registrar 1	恢复注册节点为 active 。
2026-06-04 09:09:46	Registrar 1	暂停注册节点授权。
2026-06-04 07:21:26	Legacy Discovery 2	撤销发现节点授权。
2026-06-04 07:19:09	Legacy Discovery 1	撤销发现节点授权。
2026-06-04 07:13:42	Legacy Registrar 3	撤销注册节点授权。
2026-06-04 07:12:51	Legacy Registrar 2	撤销注册节点授权。
2026-06-04 07:11:21	Legacy Registrar 1	撤销注册节点授权。
2026-06-04 07:06:52	Official Discovery	授权官方发现节点为 active ；授权域为 genesis.openagenet.local 、 openagenet.local 。
2026-06-04 07:05:45	Test Discovery	授权测试发现节点为 active ；授权域为 genesis.openagenet.local 、 openagenet.local 。
2026-06-04 07:04:36	Test Registrar	授权测试注册节点为 active ；未设置显式授权域。

时间（UTC）	节点	事件说明
2026-06-04 07:03:36	Official Registrar	授权官方注册节点为 active ；未设置显式授权域。
2026-06-04 07:02:43	Registrar 1	授权注册节点为 active ；未设置显式授权域。
2026-06-04 06:24:16	Legacy Discovery 2	授权发现节点为 active ；授权域为 genesis.openagenet.local 、 openagenet.local 。
2026-06-04 06:23:35	Legacy Discovery 1	授权发现节点为 active ；授权域为 genesis.openagenet.local 、 openagenet.local 。
2026-06-04 06:21:59	Legacy Registrar 3	授权注册节点为 active ；未设置显式授权域。
2026-06-04 06:21:40	Legacy Registrar 2	授权注册节点为 active ；未设置显式授权域。
2026-06-04 06:19:58	Legacy Registrar 1	授权注册节点为 active ；未设置显式授权域。

14.4.1 节点授权事件

授权事件使节点进入可用状态的候选集合。接收方仍需检查事件最终性、授权期限、能力范围和节点 DID 绑定，不能只依据事件名称放行。

当前动作编码中，注册服务节点授权为 21，发现服务节点授权为 31，VC 颁发者授权为 41。这些编码来自治理合约和索引器的动作映射，事件的实际生效仍由状态、时间窗口、授权域以及读模型是否已经同步共同决定。授权事件中的目标 DID 与目标地址、节点 DID 文档和运行时连接身份不一致时，应停止自动放行并进入核验流程。

14.4.2 节点暂停与恢复事件

暂停事件优先于普通业务请求，恢复事件只能在满足治理条件并被索引器按正确顺序观察后生效。服务应保留暂停和恢复的原始事件，避免只覆盖当前布尔状态。

暂停和恢复不是对历史事件的删除或覆盖。索引器应把它们作为连续治理序列保存，并根据最后一个有效序列投影当前状态。恢复后，系统只重新允许后续符合条件的新动作；暂停期间失败的请求是否重新提交，由调用方依据请求标识和业务结果决定，不能由状态恢复自动补发。

14.4.3 节点撤销事件

撤销事件表示节点不应继续执行授权范围内的新操作。已完成的资源发布和历史查询不应被无条件改写，但下游服务必须停止采用撤销节点产生的新治理结论。

撤销影响的是节点后续的治理资格，不是把该节点曾经产生的所有数据从历史中抹去。下游在处理撤销事件后，应停止接受该节点的新登记、发布、分发或治理状态更新，并对来自该节点且尚未完成验证的材料重新检查来源。已经由其他事实独立确认的历史资源记录可以保留，但必须保留其来源节点、事件位置和当时的状态。

14.4.4 事件时间、序列与来源

事件排序应优先使用链上 checkpoint、交易序号、事件序号或实现明确的复合键；事件发生时间、索引时间和页面展示时间必须分开记录。

当前事件源在分页读取后按 checkpoint、交易摘要和事件序号排序，并将读取游标与最后处理位置保存到索引器状态。emittedAtMs 表示事件载荷中的链上时间，索引器写入时间表示读侧观察和提交时间，网页生成时间只表示展示时刻。三者不能互相替换；如果事件没有可用时间，应依靠链上位置和索引时间进行排序和诊断。

14.5 节点授权、暂停、恢复与撤销

节点状态是治理序列对主体状态的当前投影。授权、暂停、恢复和撤销都通过治理事件改变主体的可用性，但具体生效还受授权域、节点能力、有效时间、到期时间、事件顺序和索引新鲜度影响。服务节点在执行动作前，应把状态判断与请求来源、请求类型和资源事实结合起来；不能把一次成功的历史请求当作当前授权仍然有效的证明。

14.5.1 授权

授权完成后，节点获得明确的授权域和能力范围。根平台在接收节点请求时应校验当前状态和范围，不能把授权事件解释为资源内容已经通过验证。

14.5.2 暂停

暂停状态下，服务可以保留只读状态查询和审计能力，但不得继续接受需要该节点授权的新提交或传播动作，除非治理策略明确允许。

14.5.3 恢复

恢复应重新确认节点身份、授权范围和治理事件顺序。恢复不自动补发暂停期间被拒绝的请求，客户端需要根据请求 ID 和状态重新确认。

14.5.4 撤销

撤销后应停止新授权范围内的动作，保留撤销原因、时间、操作者和链上位置。发现节点和分发服务需要刷新治理读模型并逐步收敛。

14.5.5 状态对注册、分发和发现的影响

节点状态	注册提交	内容分发	发现索引
active	按授权域处理	处理已批准材料	提供符合过滤条件的结果
suspended	拒绝或暂停新提交	暂停相关传播	不采用其新增治理结论
revoked	拒绝新提交	停止其授权传播	撤销或隔离其来源记录
unknown	保守拒绝高风险动作	不推进不确定状态	只读结果需带不确定性标记

状态转换的判定不应只依赖一个布尔字段。索引器在投影时还要检查事件序列是否连续、前一事件摘要是否匹配、事件是否重复以及生效和到期时间是否有效。运行时授权判断还需把当前时间

与 `effectiveFromMs`、`expiresAtMs` 比较：生效时间到达前不能放行，非零到期时间到达时不再视为有效。状态未知、序列存在缺口或摘要链不一致时，应优先阻止高风险新动作，而不是猜测下一状态。

14.6 治理通知与授权状态

治理通知适合向服务节点和运维界面传达“状态发生变化”及其核验入口，但通知本身不是新的授权凭证。通知或快照至少应带节点 DID、节点类型、当前状态、授权域、最后治理序号、事件摘要、链上位置和索引器观察时间。接收方收到通知后，应根据自身的安全要求读取状态接口或原始链上证据复核，而不能仅凭通知文本执行高风险操作。

当通知传递失败、读模型落后或状态快照与原始事件不一致时，服务可以暂时保留已知状态用于低风险只读展示，但应停止推进新的高风险授权动作，并记录待恢复的同步位置。通知恢复后，应按照治理序列重新核对，而不是只处理最后一条通知。

14.7 信任索引器作为治理读侧投影

链下信任索引器是治理信息的读取和投影组件。它从配置的链端点读取目标治理事件，将原始事件、结构化事件和主体当前状态保存到本地存储，并通过 HTTP 接口为根平台和节点服务提供查询入口。索引器通过链适配和统一状态投影将区块链实现与 OAN 的节点和业务逻辑解耦，使同一套节点授权、治理读取和状态判断机制可以适配星火链网等联盟链以及以太坊等公链；具体链的事件模型、最终性、查询接口和部署适配仍需分别实现和验证，当前参考实现主要覆盖 Sui。索引器的价值在于把链上事件转化为可操作的读模型，边界在于它不拥有治理密钥、不提交治理交易，也不凭自身判断创建或撤销节点授权。

14.7.1 链上事件读取

事件读取应支持初次同步、按游标增量读取和断点恢复。读取失败时保存最后一个已确认位置，不得用空结果覆盖已有有效读模型。

索引器的事件源支持按事件类型分页查询，并把事件类型对应的 GraphQL 游标、交易摘要和事件序号组合成可恢复游标。配置中的事件类型包、可接受的发出包和同步模式需要与实际链上升级历史保持一致；否则可能出现“接口可访问但读不到目标事件”的假健康状态。首次同步用于建立完整历史，增量同步从已保存游标继续，断点恢复必须从安全位置重读并依靠幂等写入消除重复。

14.7.2 状态投影

状态投影按照事件顺序更新节点状态、授权域和能力范围，并保留必要的事件历史。重复事件应幂等处理，旧事件不得覆盖已确认的新状态。

当前存储层以治理序列作为治理事件主键，并以“节点类型 + 节点 DID”作为主体状态主键。相同序列和相同事件摘要重复到达时可以视为幂等重放；相同序列却出现不同摘要时，应报告治理事件冲突并停止静默覆盖。只有新事件序列大于主体状态最后序列时，才允许更新该主体的当前投影。

14.7.3 同步游标与延迟

游标用于恢复和观测，不等同于链上最终性。接口应区分当前已读取位置、已确认位置和服务对外采用的位置，并可计算同步延迟。

`/v1/status` 可提供包标识、治理对象标识、最后处理 checkpoint、事件游标、事件源、最新治理序列、最新事件摘要、序列缺口标志、摘要缺口标志和最后同步时间等运维信息。`latestSequence` 只能说明索引器已写入的最高治理序列，不能单独证明已经追平链上最新 checkpoint；判断延迟时还要结合事件源返回的最新位置和同步结果。

14.7.4 查询接口

查询接口应返回状态来源、链上位置、索引时间和状态原因。调用方可以用其做筛选，但高风险授权仍应根据原始事实或可验证证明复核。

索引器公开的主体查询、授权状态查询和治理事件查询分别服务于不同用途：主体查询适合列出节点状态，授权状态查询适合运行时判断，事件查询适合审计和故障追踪。查询接口返回的是索引器读模型，调用方应结合最后治理序列、事件摘要和同步状态判断其新鲜度；“authorized”类结果不能被理解为索引器替调用方验证了节点全部密钥、端点或资源行为。

当前链下信任索引器提供 `GET /v1/status`、`GET /v1/summary`、`GET /v1/subjects`、`GET /v1/registrars`、`GET /v1/discoveries`、`GET /v1/vc-issuers` 和 `GET /v1/events` 等读接口。主体类接口通常返回 `subjects` 数组，治理事件接口返回 `events` 数组，状态接口返回索引状态结构；这些响应使用动态 JSON，因为索引状态和链上事件字段由索引器读模型序列化产生。`POST /v1/admin/sync-once` 是同步管理操作，不应与公开查询接口混用。

索引器的链访问实现使用 Sui 官方 gRPC/protobuf 类型，例如 checkpoint 和服务信息请求；这些类型属于外部 Sui RPC 协议，不是 OAN 自有的节点协作 `.proto`。因此，OAN 节点之间仍以共享 Rust 类型序列化的 JSON/HTTP 协议作为业务接口，区块链端点和 protobuf 只位于治理事实读取边界。

14.7.5 同步失败与恢复

同步失败时应记录错误、重试次数和最后成功游标；恢复后从安全位置重放并执行一致性检查。不能因为索引器暂时不可用就把所有节点标记为撤销。

恢复过程应至少包括读取失败原因、保留最后成功游标、重新读取受影响页、检查事件序列和摘要链、确认主体状态没有被旧事件覆盖，并记录恢复后的新游标和处理数量。索引器不可用时，依赖方应把状态标为未知或暂不可用，而不是批量写入 `revoked`；撤销必须来源于实际治理事件或可验证的治理状态。

14.8 根平台运行时治理状态强制执行

根平台处理请求时应按以下顺序执行：

1. 识别请求节点 DID 并验证通信签名。
2. 查询当前治理状态及其同步新鲜度。
3. 检查请求动作是否属于授权域和能力范围。
4. 再执行资源控制权、版本、哈希和内容验证。

5. 将接受、拒绝或延迟原因写入可追踪日志。

授权有效只是进入资源验证流程的前置条件，不是资源发布结论。

其中第一步确认的是通信请求来自哪个节点，第二步确认的是该节点在治理读侧的状态和新鲜度，第三步确认的是动作、节点类型、授权域和能力范围是否匹配，第四步才进入资源 DID 文档、控制签名、资源包、版本和内容校验。任一步骤的证据不足，都不能仅靠后续步骤的成功来弥补。例如资源包签名正确，只能说明资源控制证明可能有效，不能证明提交它的注册服务节点仍有当前授权资格。

14.9 发现侧治理过滤

发现节点过滤至少区分来源节点状态、资源生命周期、根平台发布状态和索引新鲜度。语义相关性或标签命中只能在这些硬条件通过后参与排序。

发现节点可以把节点授权状态用于过滤来源，但不应将授权状态直接写入资源的生命周期字段。来源节点处于暂停、撤销或未知状态时，新同步材料应被拒绝、隔离或标记为受限；已经存在且由其他证据确认的资源记录可以保留历史，但不能在缺少当前治理确认时继续作为正常新鲜候选。授权域不匹配时，资源对该发现节点不可见；这不等于资源 DID 在整个网络中不存在。

14.10 治理事件历史与公开可观测性

公开事件查询应使用稳定分页和明确排序，并同时展示链上位置与索引时间：

字段	说明
事件类型	授权、暂停、恢复或撤销
节点 DID	受影响节点
授权域/能力	事件涉及的范围
链上位置	checkpoint、交易或事件序号
observedAt	索引器观察时间
状态原因	面向运维和审计的解释

公开展示可以把 `eventLabel` 作为便于阅读的动作名称，但应保留原始 `eventType`、治理序列和来源位置。事件列表中的“最近”应按治理序列或链上位置解释，而不是按网页接口返回时间解释；分页边界也应绑定稳定排序，避免新事件到达时客户端把同一事件重复展示或漏掉。

14.11 链端点、包、对象与部署配置

索引器配置至少要同时记录网络、当前操作包、事件类型定义包、允许的事件发出包、治理公告对象、事件源模式和同步参数。OAN 当前使用 Sui Mainnet：治理公告对象为 `0xb900fb0dc1ebd58fbc4d0c697d644c861578ed948e26638836e338de0d1a3b46`，初始包 `0x1b602d575ddf1ced075f9d2986bfe53cd7` 定义治理事件类型，当前操作包为 `0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5`。升级过程中，`0x2fd18248c0cd42f371409eb1003176dc0d5744ccccdf2e49e5224b4249a5ffb4` 也曾发出治理事件，因此索引器需要接受三个包产生的事件，才能连续重建升级前后的治理序列。

```
[chain]
network = "sui-mainnet"
```

```
grpc_endpoint = "https://fullnode.mainnet.sui.io"
event_query_endpoint = "https://graphql.mainnet.sui.io/graphql"
package_id = "0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5"
event_type_package_id = "0x1b602d575ddf1ced075f9d2986bfe53cd7d4e59967ec26a0d949009c7b2c9d6c"
emitting_package_ids = [
    "0x1b602d575ddf1ced075f9d2986bfe53cd7d4e59967ec26a0d949009c7b2c9d6c",
    "0x2fd18248c0cd42f371409eb1003176dc0d5744cccfdf2e49e5224b4249a5ffb4",
    "0x232f984a25ceb8358290a8bae594078deca2d670a80c8d57d291d64a491d7fd5",
]
bulletin_object_id = "0xb900fb0dc1ebd58fbc4d0c697d644c861578ed948e26638836e338de0d1a3b46"

[sync]
event_source = "event_query"
poll_enabled = true
sync_on_start = true
poll_interval_ms = 10000
event_page_size = 50
max_pages_per_sync = 2
```

上述值对应当前主网事件同步配置。`package_id` 指向执行当前治理操作的包, `event_type_package_id` 保持指向最初定义事件结构的包, `emitting_package_ids` 覆盖升级历史中实际发出治理事件的包, `bulletin_object_id` 指向持续演进的治理公告对象。四者承担不同作用: 只把 `package_id` 更新为最新包而遗漏旧事件类型包或中间发出包, 会造成事件查询缺口; 只配置公告对象而不核对事件源, 也可能出现服务正常启动但治理序列停止增长的情况。

14.12 治理升级与迁移规则

升级应先声明合约包、事件结构、索引器和节点服务的兼容关系, 再执行迁移和回放。旧事件必须可解析或有明确转换规则, 转换失败时保留原始数据并阻止不确定状态覆盖有效状态。

升级还应区分“包地址变化”和“事件类型定义变化”。Sui 可升级包的当前操作包可能与事件类型定义包不同, 因此事件查询配置、checkpoint 审计配置和索引器接受的发出包集合不能凭包名猜测。迁移完成后, 应用一组已知治理事件核对原始事件数量、结构化事件数量、最新序列、摘要链和主体状态, 确认升级没有造成重复投影或跳过事件。

14.13 治理事件顺序与最终性

事件处理可按以下状态理解:

```
observed -> ordered -> confirmed -> projected -> exposed
```

较晚到达但链上位置更早的事件应插入历史并重新计算受影响投影; 已确认的更高版本状态不得被旧事件直接覆盖。

其中 `observed` 是事件被事件源读取, `ordered` 是索引器按链上位置或治理序列建立顺序, `confirmed` 表示已通过实现定义的完整性和位置检查, `projected` 表示已写入主体状态, `exposed` 表示查询接口可以读取该投影。某一事件已经进入公开读模型, 不代表链上发生时间等于接口观察时间; 当序列缺口或摘要缺口存在时, 公开状态应携带不确定性, 避免把不完整历史当成连续治理链。

14.14 链上事实、链下发布状态与读模型一致性

一致性不是要求链上事件、索引器、根平台和发现节点在同一时刻完成更新，而是要求每一层都能说明自己采用的事实位置和延迟。链上事件是治理事实来源，索引器提供带游标和时间的读模型，根平台据此执行发布决策，发现节点据此过滤来源和资源可见性。任何一层落后或发生冲突，都应保留已确认事实、标明差异，并在高风险动作上采取收紧策略。

14.14.1 链上最终事实

链上最终事实以可复核的事件位置和合约状态为依据，至少保留对象、动作、操作者、时间和原始事件引用。索引器接口返回只能作为读取入口，不能改变原始事实。

治理合约中的 `SubjectState` 保存节点当前状态、授权域、策略和元数据哈希、生效及到期时间、状态版本和更新时间；`GovernanceExecutionEvent` 则保存导致状态变化的事件上下文。二者结合可以回答“现在是什么状态”和“由哪次治理执行形成”，但仍应通过交易、事件和治理对象的原始链上位置进行复核。

14.14.2 根平台发布状态

根平台发布状态表示链下验证和发布流程的结果。它可以滞后于链上授权，也可能因资源验证失败而未产生；因此不能从节点已获授权推导资源已发布。

根平台可以读取节点治理状态后接收注册服务节点提交，但资源发布还要经过资源 DID 文档、控制签名、资源包完整性、版本关系、内容策略和发布流程检查。发布状态应保留其资源 DID、资源版本、发布证明引用、处理时间和来源节点；它与链上节点授权状态分开保存，便于在节点治理状态变化后重新判断后续传播资格。

14.14.3 发现节点索引状态

发现索引状态表示发现节点是否已接收并索引可传播资源。它应关联资源 DID、版本、发布证明引用、索引时间和治理过滤结果。

发现节点索引状态是本地读模型的状态，不是链上资源状态。索引记录应能指出资源来自哪个发布或分发来源、使用了哪个资源版本、何时写入本地索引、采用了哪个节点治理状态以及是否因授权域或生命周期被过滤。语义索引命中、能力标签命中和治理授权命中应分别解释，不能用检索分数掩盖来源节点已经撤销或治理状态未知。

14.14.4 状态不一致与收敛

状态不一致时采用保守处理：暂停高风险动作，保留已确认事实，标记差异来源和同步延迟，并在治理读模型、根平台和发现索引完成收敛后更新对外状态。

常见不一致包括链上已有新治理序列而索引器尚未读取、索引器已读取事件但节点服务尚未刷新、根平台已完成发布而发现节点尚未索引，以及资源来源节点状态发生变化而旧资源记录仍在本地。每类不一致都应记录比较的序列、时间、来源和处理动作；恢复时按事实源重新计算，而不是用网页访问时刻或最后一次成功请求覆盖差异。

14.15 治理权威密钥管理与应急流程

应将治理密钥操作与资源提供方的本地控制私钥严格分离。应急流程至少包含：暂停高风险治理操作、轮换或隔离疑似泄露密钥、确认链上事件、重新同步索引器、通知节点并验证状态收敛。未经链上或授权流程确认的运维标记不得直接作为最终治理事实。

治理密钥用于发起或批准治理动作，资源控制私钥用于证明具体资源 DID 或资源版本的控制关系，索引器和普通运维进程不应持有前者。应急处置完成后，应分别核对治理账户或对象权限、链上事件序列、索引器状态、节点运行时决策和下游发现结果；任何一个环节仍处于未知或滞后状态，都应在对外状态中保留限制性标记，而不能仅因服务进程恢复就宣布治理恢复完成。

14.16 参考来源

来源	类型	链接
oan-trust-indexer	代码仓：链上治理事件同步和链下信任状态投影	https://github.com/OpenAgenet/oan-trust-indexer
oan-root-services	代码仓：节点授权、治理状态和根平台决策	https://github.com/OpenAgenet/oan-root-services
oan-protocol-common	代码仓：授权、治理和状态共享类型	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：治理、授权和信任边界	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF 草案：网络节点和治理协作架构	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

第十五章 网络节点协议与互操作

本章说明跨节点协议如何把一次交互送达正确节点并保留可验证上下文；它不把“请求已送达”扩展为“资源已发布”，也不把“节点已认证”扩展为“资源控制权已验证”。

包络 -> 节点认证 -> 请求签名 -> 业务验证 -> 状态处理 -> 响应记录

在 OpenAget (OAN) 的实现中，跨节点操作通常由共享协议 crate 提供结构化请求和响应类型，再由具体节点服务负责 HTTP 路由、存储和后台任务。因而阅读一条链路时，应同时核对三层：HTTP 方法与路径、JSON 包络字段、节点内部的受理或发布状态。oan-protocol-common/schemas/agent-contract/openapi.yaml 当前是契约入口和版本占位，并不是所有节点接口的完整自动生成描述；不能仅凭该文件推断某个接口已经在每个部署中启用。

15.1 通用协议信封与请求头

通用包络可以采用如下字段语义，业务正文仍由对应接口定义：

```
{
  "protocolVersion": "oan-resource-2026",
  "requestId": "req-01",
  "purpose": "cdn-publish",
  "method": "POST",
  "path": "/cdn/resources",
  "aud": "did:oan:...",
  "requestTimestamp": "2026-09-07T10:00:00Z",
  "requestNonce": "random-value",
  "bodyHash": "sha256:...",
  "proof": {
    "type": "Ed25519Signature2020",
    "creator": "did:oan:...",
    "created": "2026-09-07T10:00:00Z",
    "proofPurpose": "assertionMethod",
    "proofValue": "z...",
    "cryptoSuite": "Ed25519Sha256",
    "hashAlgorithm": "SHA-256",
    "verificationMethod": "did:oan:...#key-1"
  }
}
```

上例对应 SignedRequestEnvelope 的字段，而不是另加一套 sourceDid、targetDid、timestamp 或 sequence 字段。包络中的 aud 表示目标受众，proof 承载请求签名；资源包、注册提交或批次条目等

业务载荷在包络之外由具体请求结构承载。发送方生成 `requestId`、`requestTimestamp`、`requestNonce` 和 `bodyHash`；接收方校验受众、时间、`nonce`、摘要、签名主体和签名覆盖范围。未知字段是否接受由协议版本和具体结构的反序列化规则决定，缺少必填字段或摘要不一致必须拒绝。

15.2 节点间认证

检查项	证明内容	失败处理
节点 DID	通信主体是谁	拒绝认证
节点授权 VC/治理状态	是否具备当前通信或业务资格	拒绝或降级为只读
有效期和状态	凭证是否仍有效	不使用过期或撤销凭证
授权域和能力	请求动作是否在范围内	拒绝越权动作
资源控制证明	资源是否由提交者控制	进入业务验证失败

认证判断应按顺序完成：先确认连接目标和节点 DID，再解析节点 DID 文档中的验证方法，随后验证节点授权 VC 或治理状态，最后检查请求动作是否落在授权域内。节点授权资格与资源控制权是两条不同链路：前者说明某节点可以代表特定网络角色通信，后者由资源提交中的控制证明说明提交者能够控制资源 DID。即使节点认证成功，接收方仍必须对资源包、DID 文档和控制证明执行业务校验。

15.3 签名上游请求

签名至少覆盖规范化方法、路径、目标节点、时间、`nonce`、请求 ID 和 `bodyHash`。验证方应依据 `proof.verificationMethod` 或兼容的验证方法标识解析公钥，确认 `proof.creator`、验证方法控制者和受信任的上游节点身份一致。

当前共享协议结构使用 `proof`，而不是在 `SignedRequestEnvelope` 中定义单独的 `kid`、`signature` 字段。`proof.verificationMethod` 指向 DID 文档中的验证方法，`proof.creator` 表示签名创建者；`oan-crypto` 先按密码套件对可签名值进行规范化，再执行签名或验签。对于 `Ed25519Sha256` 和 `Sm2Sm3`，签名输入是规范化 JSON 的字节；旧版兼容套件可能先对规范化内容计算哈希，因此协议版本和密码套件必须共同解释签名输入。HTTP 头示例只有在具体部署把包络字段映射到请求头时才适用，不能替代 JSON 包络本身。

验签失败、目标 `aud` 不匹配、`bodyHash` 不匹配、验证方法不属于签名主体或请求已过期，均应在进入资源写入和发布队列前拒绝，并记录 `requestId` 与可供运维定位的错误类别。

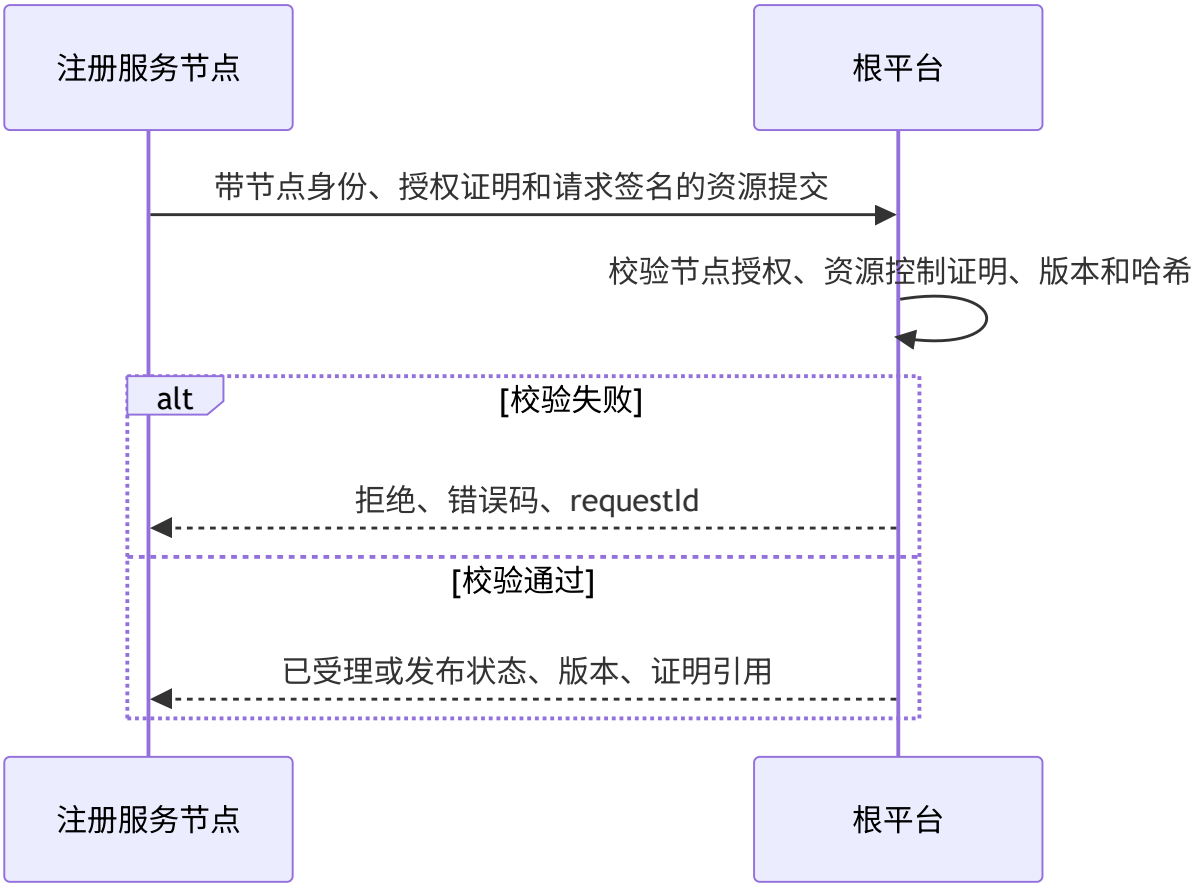
```
X-Request-ID: req-01
X-Node-DID: did: oan: ...
X-Timestamp: 2026-09-07T10:00:00Z
X-Nonce: random-value
X-Body-Hash: base64url(...)
X-Signature: jws-or-equivalent
```

15.4 节点授权 VC 交换

节点授权 VC 交换应遵循“携带或引用 → 解析签发方 DID → 验签 → 查询状态 → 检查授权域和能力”的顺序。缓存只能减少重复解析，不能绕过有效期、撤销状态和治理事件刷新。

节点授权 VC 的签发方是承担治理职责的可信机构或根平台，接收方应验证签发方 DID 文档和签名，并检查 VC 的主体节点 DID、角色、授权域、能力、有效期及撤销或暂停状态。VC 可以作为请求中的完整凭证，也可以通过双方约定的引用方式取得；后一种方式必须保证引用内容可追溯、摘要可核对，且不能因为缓存命中就跳过状态检查。资源注册 VC 属于资源登记结果，不能反过来证明注册服务节点具备节点间通信资格。

15.5 注册节点到根平台协议



根平台返回“已受理”只表示请求进入处理流程；注册 VC、根平台可信发布证明、内容分发和发现索引状态应分别表达。

注册服务节点向根平台提交的业务载荷由 ResourceVerifyAndPublishRequest 表示，其中包括注册服务节点 DID、资源登记提交内容和 upstreamAuth。登记提交内容包含资源 DID、资源类型、DID 文档、元数据、版本、各项摘要、注册凭证和主体控制证明。根平台先验证上游包络及节点资格，再校验资源控制证明、DID 文档与摘要的绑定关系；通过后才进入根平台的持久化和后续发布流程。超时或连接中断时，注册服务节点应使用原 requestId 查询受理状态，不能仅凭再次收到响应或本地重试次数判断资源是否已发布。

15.6 根平台到内容分发协议

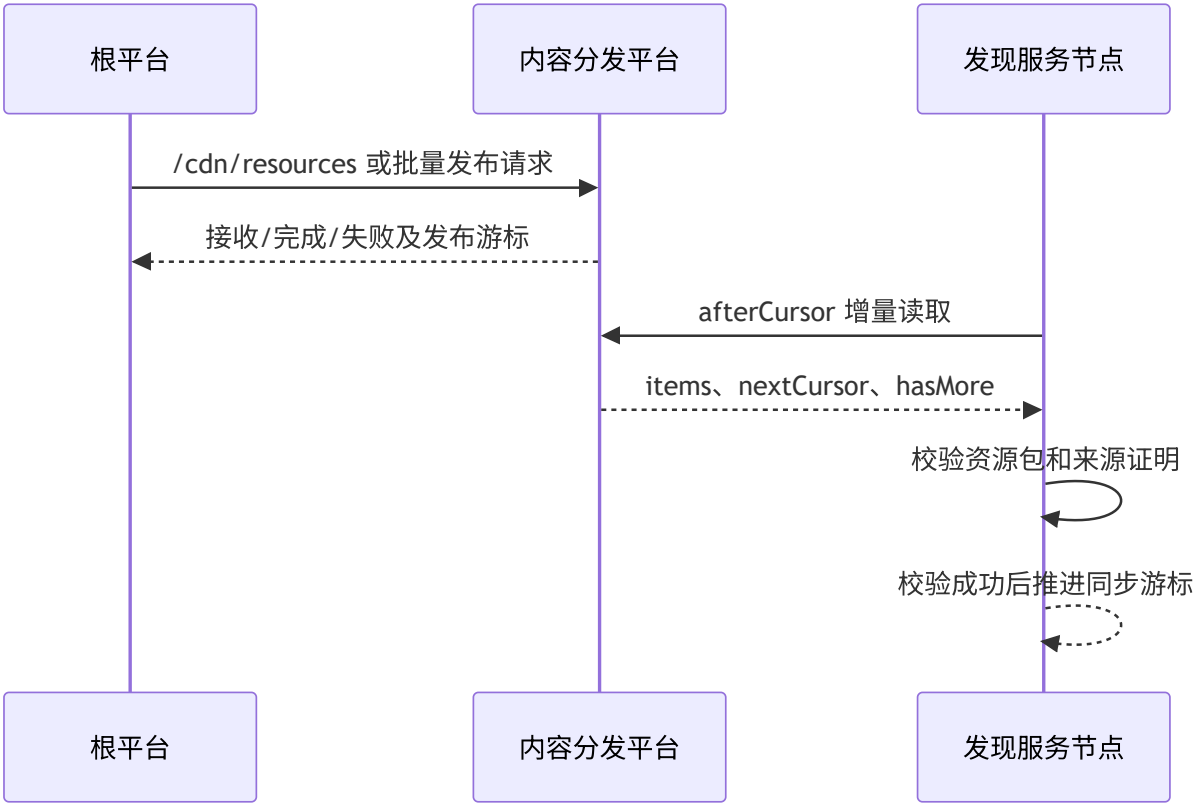
分发请求应关联资源 DID、版本、资源包摘要、根平台可信发布证明引用和分发序列。内容分发服务接收后重新计算摘要，确认来源和版本，再返回接收、完成或失败状态；接收成功不等于发现节点已索引。

共享协议中，单条内容分发请求使用 `ResourceCdnPublishRequest`，批量请求使用 `ResourceCdnBatchPublishRequest`；批量条目以 `publicationCursor` 关联根平台的发布顺序，公共包络通过 `upstreamAuth` 证明上游调用上下文。代码中对应的内容分发路径常量为 `/cdn/resources` 和 `/cdn/resources/batch`，但实际可访问地址仍取决于服务的路由前缀和反向代理配置。内容分发平台完成校验后形成自己的可读取资源或清单，根平台的发布完成状态与发现服务节点的索引完成状态应分别记录。

15.7 内容分发到发现节点协议

发现节点同步应使用游标或批序列，并对每个资源版本重新校验摘要和根平台可信发布证明引用。重复批次应幂等处理，游标推进必须在批次内容验证成功后完成。

对于从内容分发平台读取的增量数据，共享协议使用 `ResourceCdnIndexResponse` 表达 `items`、`count`、`afterCursor`、`nextCursor` 和 `hasMore`；每个条目携带当前游标和资源包。根平台向发现服务节点发送的通知则使用通知批次、起止序列和内容分发地址等信息，二者不能把字段名称直接混用。发现服务节点只有在资源包、DID 文档、摘要和发布证明关联均通过验证后，才应写入本地索引并推进下一游标；失败批次保留原游标，待修复后重试。



15.8 发现响应签名与来源证明

发现响应至少应能区分以下信息：

信息	含义
查询节点 DID	谁生成了本次响应
数据来源	数据来自哪个分发或索引来源

信息	含义
索引版本/时间	查询使用的读模型新鲜度
匹配原因	哪些字段或标签命中
根平台证明引用	发布事实的外部证据入口
响应签名	防止响应被替换

响应签名证明节点对响应内容负责，不替代资源控制证明或根平台发布证明。

当前 `ResourceDiscoveryResponse` 结构包含 `discoveryDid`、`candidates`、`createdAt` 和可选 `proof`；发现服务节点的查询实现可以返回空的 `proof`，因此响应签名属于预留的协议能力或启用后的部署能力，不应描述为所有当前实例的默认行为。若启用，签名应覆盖查询结果、创建时间、查询节点 DID 以及必要的索引或来源摘要；调用方仍需独立检查候选资源的 DID 文档、资源控制证明和根平台可信发布证明。候选项中的 `score` 只表示匹配相关性，不是信任等级或授权结论。

15.9 序列号、请求标识符与重放防护

字段	主要用途	重试时是否复用
<code>requestId</code>	关联一次业务请求及其日志	同一逻辑请求复用
幂等键	防止提交副作用重复执行	同一操作复用
<code>nonce</code>	防止签名消息重放	通常重新生成
<code>sequence</code> /游标	保证批次或事件顺序	按同步协议使用
时间窗口	限制请求新鲜度	重新签名时更新

`requestId` 用于把一次逻辑操作在注册服务节点、根平台、内容分发平台和发现服务节点之间串起来；`requestNonce` 用于阻止同一份签名包络被再次接受；`publicationCursor`、`sequenceFrom`、`sequenceTo`、`afterCursor` 和 `nextCursor` 用于传播顺序或增量读取。它们的数值空间和推进规则不同，不能把请求 ID 当作事件序列，也不能以发现查询的分页参数代替发布同步游标。重试同一逻辑请求时可以保留 `requestId`，但新的签名消息通常应使用新的 `nonce`，并由接收方依据幂等规则判断是否已处理。

15.10 分页、过滤与兼容性规则

客户端页面分页、发现查询分页和节点同步游标不可混用：页面分页面面向展示，发现分页面面向候选结果，同步游标面向增量传播。游标失效时应返回可识别错误或要求重新同步，不得静默从任意位置继续。

当前共享查询结构中的 `limit` 默认值是 10，发现查询还可携带资源类型、能力标签、协议、版本和版本模式等条件；这只是发现查询结构的默认行为，不代表所有管理列表接口的默认分页大小。同步响应中的 `hasMore` 与 `nextCursor` 服务于增量传播，查询结果的 `score`、版本模式和候选排序服务于用户检索。协议演进时，新增过滤字段可以按兼容规则处理，但改变游标含义、排序稳定性或版本模式应通过版本说明明确告知调用方。

15.11 超时、重试与幂等操作

操作	网络超时后的默认动作	原因
只读查询	有限退避后重试	通常无业务副作用
状态查询	使用原请求 ID 查询确认	区分未完成和未发生
资源提交	不盲目重放，先查询幂等状态	可能已产生登记或发布副作用
同步批次	按原游标和批次重试	保证增量顺序和幂等

重试必须区分“未收到响应”和“已收到明确失败”：网络超时只说明结果未知，资源提交、发布和状态变更类操作应先用原请求标识查询；明确的参数、授权或签名错误不应自动重试。后台发布器可以在同一发布游标上恢复处理，接收方应以资源 DID、版本、摘要和游标组合识别重复消息，避免重复写入或错误推进进度。具体超时、退避和并发度由部署配置决定，本章不把某一环境的数值固化为协议常量。

15.12 协议版本协商

版本协商至少应声明协议版本、支持的能力和不可兼容的约束。没有共同版本时应明确失败，不得按字段猜测或静默降级到不安全路径。

```
{
  "protocolVersions": ["oan-resource-2026"],
  "capabilities": ["signed-response", "cursor-sync"],
  "required": ["request-signature"]
}
```

上例是协商信息的表达示例，不表示当前所有节点都提供独立的能力协商端点。当前共享协议将 oan-resource-2026 作为资源协议版本常量，并在包络中使用 protocolVersion; openapi.yaml 的 info.version 为契约文件版本，两者不能直接等同。协商结果至少要固定字段解释、签名覆盖范围、密码套件、游标语义和错误处理方式；若对方只支持不安全或无法解释的降级路径，应拒绝建立该业务链路。

15.13 消息大小、压缩与传输限制

限制应同时考虑压缩前消息、压缩后传输体和解压后内容；接收方应先执行大小和格式保护，再进行资源写入或高成本验证。压缩只改变传输方式，不改变签名和摘要覆盖的逻辑内容。

当前共享协议类型定义了资源包、批量条目和包络字段，但没有在该公共结构文件中统一声明所有部署的消息大小、压缩算法或 HTTP body 上限。因此这些限制应由具体服务配置、反向代理和运行环境共同确定，并在节点间能力或部署文档中公开可核对的值。对批量同步尤其要同时限制单项大小、批次项数和累计解压大小；超过限制时应在入库前返回明确错误，并保留请求 ID，不能截断后继续验签或索引。

15.14 关联、可追踪性与审计标识符

一次资源传播链至少应保持以下关联字段：

```
requestId -> resourceDid -> resourceVersion -> publishSequence
           -> syncCursor -> discoveryIndexVersion -> errorCode
```


节点可以增加本地 trace 标识，但不得丢失上游 `requestId`、资源 DID 和版本；日志不得记录私钥和完整敏感凭证。

对发布链路，建议至少保存如下可检索关联：

链路阶段	关键关联字段	用途
注册受理	<code>requestId</code> 、资源 DID、资源版本	定位一次登记请求
根平台发布	资源包摘要、根平台可信发布证明引用、 <code>publicationCursor</code>	判断发布事实和顺序
增量同步	<code>afterCursor</code> 、 <code>nextCursor</code> 、批次序号	判断是否漏同步或重复同步
发现索引	发现服务节点 DID、索引时间、错误类别	解释查询结果的新鲜度

审计记录应保留验证结果和状态转换，而不是只记录 HTTP 200。对于敏感 VC，应记录其类型、摘要或引用及验证结论，避免把完整凭证内容和任何私钥写入普通业务日志。

15.15 协议兼容性与弃用规则

字段或版本弃用应经历“公告—兼容期—迁移—停止接受”阶段，并保留错误码和请求 ID。涉及签名覆盖范围、认证语义、游标含义或状态语义的变化不得仅通过增加可选字段解决，应升级协议版本或提供明确迁移规则。

兼容性测试至少应覆盖：旧版本包络能否被明确拒绝或按兼容规则解析、未知可选字段是否不会改变签名语义、旧游标是否不会被误当作新游标、签名主体和 `aud` 是否仍按同一规则验证。版本迁移期间，发送方应优先使用双方确认的共同版本，接收方应在错误响应中返回可关联的请求 ID 和不兼容原因；停止接受旧版本前，应先完成节点配置、后台发布任务和历史游标的迁移。协议文件、共享结构和服务实现必须同步更新，不能只修改文档中的版本字符串。

15.16 参考来源

来源	类型	链接
<code>oan-protocol-common</code>	代码仓：跨节点请求、响应、错误和事件类型	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	代码仓：根平台节点间接口	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	代码仓：注册服务节点接口和上游调用	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	代码仓：发现同步和查询接口	https://github.com/OpenAgenet/oan-discovery-node
<code>oan-trust-indexer</code>	代码仓：治理状态查询接口	https://github.com/OpenAgenet/oan-trust-indexer
Agentic Overlay Network Architecture	IETF 草案：节点互操作架构	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/

来源	类型	链接
OAN Resource Identity and Discovery	IETF 草案：资源身份和发现互操作	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

第十六章 公开 HTTP API

本章将 OpenAgenet (OAN) 的公开 HTTP API 组织为可实现、可测试的接口契约，覆盖官网后端、注册服务节点、根平台、内容分发平台、发现服务节点和链下信任索引器的公开及节点间调用。接口在浏览器、TypeScript SDK、社区技能包中的路径、字段和错误语义应保持一致。公开 API 只返回公开服务所需材料，不返回私钥、本地身份备份、数据库管理能力或访问统计后台数据；HTTP 成功也不自动表示资源已完成根平台发布、全网分发或发现索引。

16.1 API 约定与基础 URL

契约项	统一要求	失败时表现
方法和路径	按服务实际路由声明	返回可识别的路由或方法错误
媒体类型	JSON 请求/响应使用明确 Content-Type	不接受无法解析的正文
请求 ID	客户端可提供，服务端缺失时生成	响应和日志保持一致
时间	RFC 3339 字符串	格式错误不得静默转换
缓存	依据数据新鲜度和状态设置	过期数据必须带降级标识

OAN 接口使用 UTF-8 编码传递 JSON，时间字段采用 RFC 3339 字符串，DID、版本、哈希和游标按字符串处理，不应被客户端转换为有损数字。生产环境基础 URL 由部署配置确定，协议规范只约束路径和语义，不把官网当前域名写成所有部署的固定条件。请求应携带能够贯穿一次调用链的请求 ID；响应应以稳定的机器可读字段表达结果，展示层文字不能作为程序判断依据。

16.1.1 服务基础地址

官网当前后端的公开基础地址与各节点地址由网站前端的 API 配置、节点配置和部署环境共同决定。SDK 提供注册服务节点、发现服务节点、根平台和内容分发平台的独立端点配置，调用方不得假定所有服务必然部署在同一主机或同一域名。跨节点请求还应记录目标节点 DID，避免仅凭网络地址判断通信对象。

16.1.2 路径和版本约定

路径中的版本应通过明确的路径、协议字段或能力协商表达，不能从页面标题或服务名称推断。兼容增加字段时，客户端应忽略未知非必填字段；改变字段含义、签名覆盖范围或状态语义时必须提高协议版本或提供迁移期。接口路径、方法和媒体类型构成契约的一部分，大小写和尾部斜杠差异不得由客户端自行猜测。

16.1.3 请求认证

公开查询和健康接口可以按部署策略匿名访问；资源注册、节点间提交和治理操作分别需要资源控制证明、节点授权材料或运维认证。浏览器的 CORS 许可只决定页面是否能够读取响应，不授予资源控制权。认证失败应返回稳定错误，不应通过模糊的 404 或成功空结果掩盖权限问题。

16.1.4 响应格式

成功响应应使用 JSON 对象表达数据、状态、版本、新鲜度或来源；错误响应应至少包含机器可读错误码、请求 ID 和必要的字段定位。服务不得把 HTML 错误页当作 JSON 返回。对于异步发布和索引，响应应明确区分已受理、处理中、已完成和失败，不能用 HTTP 200 单独表示全链路完成。

16.2 健康、就绪与服务状态端点

健康检查应区分进程存活、服务就绪、关键依赖可用和业务烟测通过。官网后端的 /health 只能说明后端进程能够响应，不能证明注册服务节点、发现服务节点、根平台或数据库均正常。运维部署完成后，应分别检查节点健康、数据库连接、索引器同步状态，以及注册和发现最小业务路径；对外状态接口应避免泄露内部连接串、令牌和堆栈。

16.3 注册节点注册端点

请求阶段	必须核验	成功不代表
草稿校验	本地身份、必填字段、DID 文档和元数据	已提交或已发布
资源提交	控制证明、节点授权和资源包摘要	已完成根平台发布
状态查询	登记、发布、分发和索引阶段	所有节点已收敛
VC 获取	签发者、主体、版本、摘要和状态	资源业务质量保证
同 DID 更新	原控制关系、前序版本和新摘要	自动取得更新权

参考实现同时接受 POST /resources/register 和兼容入口 POST /resources/submit，二者进入同一资源登记处理函数。浏览器或 SDK 在本地生成资源 DID、DID 文档、元数据、版本与摘要，并以 ResourceRegistrationSubmission 提交；私钥和本地身份备份不属于请求正文。注册服务节点先检查结构、资源类型、授权域和主体控制证明，再以签名上游请求调用根平台的验证发布接口。

当前成功响应的核心字段如下，示例值仅用于说明结构：

```
{
  "status": "submitted",
  "resourceDid": "did:oa:SKLG:example",
  "resourceType": "skill",
  "registrationCredential": {},
  "rootResponse": {}
}
```

status: submitted 表示注册服务节点已完成本次处理并取得根平台响应，不等于内容分发和所有发现服务节点索引完成。参考实现会签发并返回 registrationCredential；客户端仍应允许凭证字

段为空或无法解析时保留提交结果和原始响应，以便区分“登记失败”与“凭证缺失”。GET /resources、GET /resources/{did}、GET /registrar/status 和 GET /registrar/stats 用于读取注册记录及节点状态，但公开范围仍由部署的反向代理和访问策略决定。

16.4 根平台发布与状态端点

根平台将公开读取路由与管理写入路由分开。资源发布入口为 POST /root/resources/verify-and-publish，由注册服务节点调用并携带 upstreamAuth；根平台重新验证来源节点身份、节点授权、资源 DID、DID 文档、摘要和控制证明。资源查询使用 GET /root/resources/{did}，历史版本列表和指定版本分别使用 /root/resources/{did}/versions 与 /root/resources/{did}/versions/{version}。

接口类别	典型路径	主要边界
根平台身份与状态	/root/did、/root/status	提供公开身份或运行摘要，不开放管理能力
资源读取	/root/resources/{did} 及版本路径	返回已保存的资源版本与发布材料
发布写入	/root/resources/verify-and-publish	需要注册服务节点的可信上游包络
发布队列	/root/queues/cdn-publish、 /root/queues/discovery-notify	用于观察异步传播状态，是否公开由部署策略决定
治理管理	授权、域更新和撤销路径	属于管理路由，需要独立管理员认证

发布接口成功表示根平台接受并保存了符合要求的资源版本，随后由内容分发发布器和发现通知任务继续处理。调用方应通过资源版本、发布队列或后续状态确认传播进度，不能把根平台的一次 2xx 响应解释为全网可见。

16.5 内容分发端点

内容分发平台对外提供单资源读取、批量读取、增量索引和拆分材料读取：GET /cdn/resources/{did} 返回资源包，POST /cdn/resources/batch-get 按 DID 批量读取，GET /cdn/documents/{did} 与 GET /cdn/metadata/{did} 分别返回 DID 文档和元数据。目录类接口位于 /cdn/catalog/...。根平台发布器使用受保护的 POST /cdn/resources 或 POST /cdn/resources/batch 写入；POST /cdn/purge 属于管理操作，不应作为公开写入接口暴露。

增量同步使用 GET /cdn/resources/index?afterCursor=<n>&limit=<n>。当前参考实现未传参数时可返回兼容格式；传入游标或数量参数时返回结构化页面：

```
{
  "items": [],
  "count": 0,
  "afterCursor": 42,
  "nextCursor": 42,
  "hasMore": false
}
```

发现服务节点应在完整校验本批资源包后保存 nextCursor，不能因 HTTP 请求成功就提前推进。当前服务对批量发布请求设置了实现层 body 上限，该数值属于参考实现的部署保护，不是所有 OAN 节点永久固定的协议值。

16.6 发现查询与资源详情端点

查询结果字段	用途	新鲜度或可信边界
resourceDid	精确标识资源	仍需解析和控制权验证
resourceType/version	筛选和版本选择	不替代发布证明
sourceNode	追踪结果来源	节点授权需另查
indexedAt/cursor	判断索引新鲜度	不等于链上最终性
matchExplanation	解释语义命中	不等于调用授权

发现查询入口为 POST /discovery/resources/query，请求类型允许使用自然语言描述、资源类型、能力标签、协议、版本、版本模式和数量限制。若查询文本可识别为完整资源 DID，参考实现优先执行精确 DID 查询；否则优先使用可用的语义索引，并在语义后端不可用或无结果时回退到关键词候选流程。响应中的 discoveryDid 表示本次返回结果的发现服务节点，createdAt 表示响应生成时间，候选项的 score 只表达检索相关性。

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure
    .",
  "resourceType": "skill",
  "capabilityTags": [],
  "versionMode": "latest",
  "limit": 10
}
```

索引资源读取和诊断接口包括 /discovery/index/resources、/discovery/index/resources/{did}、/discovery/index/stats 与 /discovery/query/stats；它们是否对互联网公开，应由部署暴露策略确定。空候选数组是有效查询结果，不应转换为网络错误；语义后端回退、索引滞后或快照降级应在可观测信息中体现，不能制造重复候选来掩盖数据源差异。

16.7 DID 解析端点

基础设施节点分别通过 GET /registrar/did、GET /root/did 和 GET /discovery/did 返回自身 DID 文档。资源 DID 的解析可通过根平台资源详情、内容分发平台的 /cdn/documents/{did}，或发现服务节点已经索引的资源详情取得，具体选择取决于调用方需要的是权威保存版本、分发副本还是发现读模型。

解析响应中的 id 应与请求 DID 完全一致，并保留公开验证方法、验证关系、服务端点和 OAN 元数据。客户端需要区分三类结果：DID 格式非法属于请求问题，资源不存在属于无记录，依赖或节点暂时不可用属于可恢复故障。任何解析路径都不应返回浏览器本地身份备份、私钥或服务端密钥文件。

16.8 VC 与证明获取端点

注册 VC、节点授权 VC 和根平台可信发布证明覆盖不同事实。注册服务节点在登记成功响应及资源记录中保存 registrationCredential；根平台可通过资源详情和版本接口返回资源包及其根平台可信发布证明；注册服务节点和发现服务节点的 /.../root-authorization 状态接口用于展示其根

平台授权材料或验证状态。根平台的基础设施授权 VC 签发入口属于受保护操作，不应被当作匿名自助获取接口。

证据	证明对象	获取后仍需检查
注册 VC	某注册服务节点处理了特定资源登记	签发者、主体、资源摘要、签名和状态
节点授权 VC	基础设施节点被授予角色和授权域	签发者、节点 DID、角色、有效期和撤销状态
根平台可信发布证明	特定资源版本及摘要被根平台接受	资源 DID、版本、摘要、签名和治理状态

接口返回凭证对象不等于凭证已经验证。调用方应根据 `proof.verificationMethod` 解析签发者公钥，并检查凭证时间和治理状态；无法取得状态时，应明确返回“不确定”或收紧高风险操作，而不是默认为有效。

16.9 治理状态与事件端点

根平台通过 `/root/registrars`、`/root/discovery-nodes` 及其 DID 详情接口提供基础设施节点目录，通过 `/root/bulletin/events` 和 `/root/bulletin/events/{sequence}` 提供治理事件读模型。注册服务节点与发现服务节点分别通过 `/registrar/root-authorization`、`/discovery/root-authorization` 暴露本节点观察到的授权状态；发现服务节点还通过 `/discovery/authorized-domains` 表达当前授权域。

链下信任索引器的数据属于链上治理事实的链下投影，状态响应应携带最新序列、观察时间或同步位置，使调用方能够判断新鲜度。根平台的授权、授权域更新和撤销接口属于管理路由，需要管理员认证。若链上读取失败、索引序列长期不增长或本地状态无法确认，根平台和发现服务节点应将结果标记为陈旧或不确定，并收紧发布、同步或发现可见性判断。

16.10 指标与可观测性端点

官网后端将多个节点和数据源整理为 `/api/public/home/summary`、`/api/public/network/summary`、`/api/public/network/governance-events`、节点目录与节点统计等公开读接口。节点侧另有 `/registrar/stats`、`/discovery/index/stats`、`/discovery/query/stats`、`/cdn/catalog/resources/stats` 等角色相关统计。官网汇总结果是展示读模型，不应被当作替代各节点原始状态的权威接口。

指标字段应同时给出生成时间、统计窗口、来源和缺失状态。`TODAY`、`THIS WEEK`、`THIS MONTH` 等窗口需要采用同一时区与明确起止边界；累计资源数不能和窗口新增量混为一项。公开接口只返回必要的聚合信息，访问统计明细、完整日志、数据库路径、管理令牌和内部错误堆栈仍属于运维数据。节点向官网汇报统计使用内部上报入口，其认证与暴露范围应独立于公开读取接口。

16.11 请求与响应模式

请求和响应应至少能表达成功、处理中、失败和不确定四类结果：

```
status = accepted | processing | completed | failed | unknown
requestId = 用于跨节点追踪一次调用
```

客户端必须按机器可读状态判断异步阶段，不以 HTTP 200 或页面提示单独推断发布、分发和发现已经完成。

请求和响应模式是接口字段、状态和错误的稳定边界。字段的存在性、类型、枚举、哈希算法、时间格式、DID 语法和嵌套关系应与 oam-protocol-common 及节点实际类型一致。对未知字段的处理应按协议版本确定；对缺失必填字段、重复字段、空值和不兼容版本应返回可定位的错误。

核心节点接口的实际类型关系如下。表中的 Value 表示当前 handler 返回动态 JSON，并不表示任意字段都构成稳定契约；具体字段仍需以对应 handler、共享结构和错误测试共同核对。

服务	方法与路径	输入类型	输出类型或稳定字段
注册服务节点	POST /resources/ register、 POST /resources/submit	ResourceRegistrationSubmission	Value: status、 resourceDid、 resourceType、 registrationCredential、 rootResponse
根平台	POST /root/resources/ verify-and-publish	ResourceVerifyAndPublishRequest	Value: 验证排队状态、资源版本、三类哈希、生命周期和 CDN 排队状态
内容分发平台	POST /cdn/resources、 POST /cdn/resources/ batch	ResourceCdnPublishRequest、 ResourceCdnBatchPublishRequest	Value: 发布状态、游标、 批次接受/失败项目
内容分发平台	GET /cdn/resources/{did }	路径参数 did	ResourcePackage
发现服务节点	POST /discovery/ resources/query	ResourceDiscoveryQuery	ResourceDiscoveryResponse
链下信任索引器	GET /v1/status、 GET /v1/events	可选 limit 查询参数	动态状态 JSON；事件响应 包含 events 数组

接口的业务阶段不能仅由 HTTP 状态码推断。注册服务节点返回 submitted 时，登记记录已写入；根平台返回 resource-verified-and-queued 时，资源已通过根平台当前验证并进入分发队列；内容分发平台返回 published 时，资源包已在分发平台写入；发现响应返回候选时，只表示当前发现节点的索引可见。调用方仍需按资源 DID、版本、哈希、根平台证明、治理状态和新鲜度完成后续判断。

16.11.1 注册请求与响应

注册请求至少关联资源 DID、DID 文档、资源类型、元数据、资源包、版本、哈希、注册服务节点和控制证明。服务端应在验签前确认请求路径、目标、挑战、nonce 和内容摘要，验签后再执行重复判断和写入。响应应分别表达登记受理、注册 VC、根平台提交结果和可重试错误，不能让客户从一条“success”文字推断所有阶段完成。

参考实现的公开登记入口直接接收 ResourceRegistrationSubmission；注册服务节点向根平台转发时再构造 ResourceVerifyAndPublishRequest，并在 upstreamAuth 中绑定方法、路径、目标、时间、nonce 和业务正文哈希。浏览器只提交资源材料与控制证明，不应自行构造或持有基础设施节点的上游签名身份。

16.11.2 发布请求与响应

发布请求至少关联注册服务节点来源、资源版本、资源包摘要、DID 文档哈希、元数据哈希、授权材料和请求 ID。发布响应应携带根平台可信发布证明或明确说明其尚未产生，并返回发布状态、批次或游标引用。哈希不一致、节点无权、控制证明无效和依赖暂时不可用应分为不可重试失败与可重试失败。

根平台向内容分发平台提交时，单条和批量请求分别使用 `ResourceCdnPublishRequest` 与 `ResourceCdnBatchPublishRequest`，批量条目以 `publicationCursor` 关联根平台发布顺序。响应中的接受数量、失败条目或完成游标用于发布任务恢复，不代表发现服务节点已经完成索引。

16.11.3 发现请求与响应

发现请求可以使用资源 DID、资源类型、能力标签、协议、授权域、生命周期、关键词或自然语言任务描述。响应应返回候选 DID、版本、资源类型、匹配原因、来源发现服务节点、索引版本、治理状态和证明引用；分页游标必须绑定查询条件和排序快照。空结果不是接口失败，服务不可用、索引滞后和快照降级应被明确标记。

当前 `ResourceDiscoveryResponse` 包含 `discoveryDid`、`candidates`、`createdAt` 和可选 `proof`。候选项包含资源 DID、资源类型、得分、版本、生命周期、能力标签、授权域、服务端点、协议绑定、资源包信息和根平台证明等字段；可选字段缺失时，客户端不能用展示层默认文本补造可信事实。

16.11.4 状态和错误响应

状态和错误响应使用 HTTP 状态码表达粗粒度结果，JSON 错误码表达业务原因，`requestId` 表达追踪关联。错误中可包含字段路径、是否可重试、建议等待时间和当前处理状态，但不得回显私钥、完整控制证明或内部异常堆栈。客户端应保留原始错误以便审计，同时向用户提供与敏感程度相称的说明。

当前多个 Rust 节点的错误正文采用简洁结构 `{"error": "machine_readable_reason"}`，并由 HTTP 状态码区分请求错误、权限错误或服务错误。并非所有现有响应都已经包含 `requestId`、字段路径和重试提示，因此这些字段属于接口一致化时应补齐的方向，调用方不能假定当前每个节点都会返回完整错误包络。

16.12 HTTP 状态码与常见错误

状态	典型含义	客户端动作
400	请求格式或字段非法	修正输入后重试
401/403	身份或授权不满足	不盲目重试，检查凭证和授权域
404	资源或路由不存在	区分未知资源和路径错误
409	DID、版本或幂等冲突	查询当前状态后决定是否更新
429	超过速率或并发限制	按 <code>Retry-After</code> 或退避策略重试
5xx/超时	服务或依赖异常	有限重试并保留 <code>requestId</code>

建议使用 2xx 表示请求已按接口语义处理，4xx 表示请求、认证、授权或业务前置条件不满足，5xx 表示服务端或依赖异常。202 `Accepted` 只能表示异步任务已受理，不能表示发布完成；409 `Conflict` 可用于同 DID 版本冲突或重复状态需要客户端确认的场景；429 `Too Many Requests` 应配合限流信息。各服务应避免为相同业务原因随意返回不同状态码。

客户端应首先判断 HTTP 状态，再解析 JSON 中的 `error` 或业务状态。连接失败、TLS 失败和浏览器 CORS 拒绝没有可用的 HTTP 响应，不能统一显示成“服务器返回 500”；对仅支持 GET 的端点发送 POST 时，路由层通常返回 `405 Method Not Allowed`，应优先核对调用方法和反向代理转发规则。

16.13 速率限制、缓存与安全请求头

控制项	公开查询	注册或节点间请求
速率	可按来源和接口限流	应结合主体、节点和幂等键
请求大小	限制查询描述和页大小	限制资源包和批次大小
缓存	允许带新鲜度的只读缓存	不缓存会改变授权判断的结果
安全头	TLS、CORS 和必要安全头	叠加认证和来源校验

限流应至少考虑来源 IP、认证主体、节点 DID、资源 DID 和接口类别，并对注册、发现、同步和健康请求使用不同配额。只读公开查询可以使用短时缓存，但 DID 文档、VC、根平台证明、治理状态和发现结果必须带有版本或新鲜度边界。条件请求的 ETag 必须对应响应内容版本；安全请求头、内容类型检查和响应体大小限制应在网关或服务层落实。

当前节点代码中的请求体限制、查询数量边界和 CORS 配置属于第一层保护，生产部署还需由反向代理承担连接数、速率、超时和请求大小控制。缓存测试至少应验证：首次请求取得数据和新鲜度信息，条件请求不会返回错误版本，治理撤销后旧缓存不会继续授权写入，后端不可用时旧快照被明确标记而不是伪装成实时结果。写入接口和带签名请求不应由共享缓存保存。

16.14 CORS、浏览器客户端与跨源策略

浏览器调用节点 API 时，服务端只允许必要的官网来源、方法和请求头，预检请求不得放宽业务授权。不得使用 `Access-Control-Allow-Origin: *` 配合凭据传递敏感材料。公开官网跨源读取与节点间认证是两层控制：前者服务浏览器安全模型，后者验证请求主体和权限。CORS 配置变化应作为部署检查项，避免前端出现难以区分的 `Failed to fetch`。

注册服务节点、发现服务节点和内容分发平台均从配置构造 CORS 层；官网后端优先使用配置的前端来源，在配置值无法解析时才退回更宽松策略。生产环境应避免触发该退回分支，并在部署后分别从中文和英文页面执行注册、发现及预检烟测。浏览器显示 `Failed to fetch` 时，应依次核对请求 URL、HTTPS 混合内容、证书、DNS、反向代理、预检响应和允许来源，不能只检查后端业务日志。

16.15 公开端点与仅限运维端点的边界

公开边界按“公开读取、资源控制者签名、节点间授权、运维专用”划分。公开接口可以提供健康、公开目录、已发布资源和有限指标；资源控制者接口接收控制证明但不接收私钥；节点间接口要求节点身份、授权 VC 和请求签名；运维接口只在受控网络或认证边界内开放。访问统计数据只能由后台或运维授权主体读取，普通访客不应通过网站页面或公开 API 查询明细。

16.15.1 公开查询接口

公开查询接口包括公开目录、发现查询、已发布资源详情、公开 DID 文档、公开证明和健康信息。返回数据应经过字段白名单处理，隐藏本地文件路径、私有端点、数据库管理字段和未公开凭证。公开查询仍需遵守生命周期、治理过滤和授权域可见性，不能因为接口匿名就绕过这些判断。

官网 `/api/public/...` 路由主要承担聚合展示，不等于将所有节点路由自动公开。部署时应建立公开路径白名单，并验证匿名访问只能读取经过筛选的数据。用于记录页面访问的 `POST /api/public/site-analytics/visit` 是单向采集入口，不提供统计明细读取能力。

16.15.2 节点间接口

节点间接口用于注册服务节点、根平台、内容分发平台、发现服务节点和链下信任索引器之间的协作。请求应携带来源和目标身份、授权材料、协议版本、请求 ID、时间窗口、nonce、内容摘要和签名。节点间接口不应由官网浏览器直接代替调用，浏览器提交资源应经过注册服务节点的公开入口和服务端校验。

典型节点间写入包括注册服务节点调用 `/root/resources/verify-and-publish`、根平台发布者调用 `/cdn/resources/batch`，以及根平台向发现服务节点发送授权范围内的同步通知。即使路径能够从互联网连通，接收方仍应验证可信上游包络和节点授权；网络可达性本身不是授权依据。

16.15.3 运维接口

运维接口包括部署状态、内部诊断、访问统计明细、数据库管理、密钥轮换和恢复操作。它们不得出现在公开 API 目录或浏览器端代码中，应使用独立认证、最小权限、审计日志和网络隔离。运维接口返回的错误也不得被公开代理完整透传给普通用户。

根平台的授权、撤销和发布管理路由，以及官网内部节点统计上报入口，均应在反向代理层与公开读取路由分开。服务代码中将路由合并到同一进程并不意味着它们应共享相同的互联网暴露策略；运维人员应同时核对应用认证和代理路径白名单。

16.15.4 访问控制和暴露边界

访问控制应同时检查路由、方法、认证主体、节点角色、授权域和资源范围；暴露边界应通过端口、反向代理和 CORS 配置共同落实。发布前应从匿名浏览器、资源控制者客户端、未授权节点和运维主体四类身份分别测试允许与拒绝路径，并保存状态码、响应字段和日志关联证据。

测试身份	应允许的典型操作	应拒绝的典型操作
匿名访客	健康、公开目录、发现和公开资源读取	授权、撤销、数据库与统计明细读取
资源控制者	提交带有效控制证明的资源	提交不受其控制的同 DID 更新
授权节点	在角色和授权域内执行节点间调用	越权访问其他角色的管理操作
运维主体	在独立认证下执行管理与诊断	绕过审计或取得业务主体私钥

16.16 API 示例与机器可读接口描述

```
openapi: 3.1.0
info:
```

```
title: OAN Agent Contract
version: 0.1.0
paths: {}
```

上例是当前 `oan-protocol-common/schemas/agent-contract/openapi.yaml` 的实际骨架。它标明 OpenAPI 版本和契约文件版本，但 `paths` 仍为空，因此只能作为机器可读契约入口，不能当作已经覆盖全部公开及节点间 API 的完整描述。接口核对应以各服务 Router 定义、共享协议结构、配置和部署代理规则共同为准。

面向使用者的最小可执行示例应从无需签名的健康或公开查询开始。例如以下命令中的主机名是占位值：

```
curl --fail-with-body \
  --header "Accept: application/json" \
  "https://registrar.example/health"
```

注册、根平台发布和内容分发写入示例还需要真实生成的 DID、摘要、控制证明或节点间签名包络，不能用静态假签名冒充可执行请求。后续完善 OpenAPI 描述时，应逐路由补齐方法、请求体、响应、认证、错误和示例，并通过自动化测试检查描述与实际路由是否同步。

16.17 参考来源

来源	类型	链接
oan-protocol-common	代码仓：HTTP 载荷、错误码和 OpenAPI 契约入口	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓：根平台公开和内部 API	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务 API	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现服务 API	https://github.com/OpenAgenet/oan-discovery-node
oan-sdk-ts	代码仓：客户端 HTTP 调用和类型封装	https://github.com/OpenAgenet/oan-sdk-ts
OAN Resource Identity and Discovery	IETF 草案：资源身份和发现接口方向	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF 草案：发现查询 profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第十七章 客户端 SDK、社区 Skill 与开发者体验

本章说明客户端层如何为资源提供方、调用方和节点运营者提供不同入口：`oan-sdk-ts` 面向程序化集成，社区技能包面向智能体辅助的资源整理、注册、发现和验证，官网提供浏览器交互。三者复用 OpenAgenet (OAN) 协议，不改变 DID、控制证明、注册 VC、根平台可信发布证明或治理状态的语义。主体私钥、资源控制私钥和本地身份备份留在客户端，服务端只接收公开材料和必要证明。

入口	主要使用者	核心职责	不承担的职责
TypeScript SDK	应用开发者、平台集成者	类型、请求构造、注册发现调用、验证辅助	替用户作出信任决定
社区技能包	资源提供方、智能体操作者	整理资源、辅助注册发现、解释结果	持有治理密钥或替代节点服务
官网	浏览器用户	交互式注册、发现、预览和本地下载	托管用户私钥或保证端点安全
运维辅助	节点运营者	构建、配置检查、部署与烟测	自动决定生产密钥和回滚策略

17.1 TypeScript SDK 范围

TypeScript SDK 由协议类型、HTTP 客户端、治理读取和工作流辅助组成，适合 Node.js、浏览器构建环境及兼容 `fetch` 的运行环境。调用方可分别配置注册服务节点、发现服务节点、根平台和内容分发平台地址。SDK 负责序列化、请求和响应解析，不替调用方决定资源是否可信。

代码组织上，`packages/protocol-types` 提供 `ResourceRegistrationSubmission`、`ResourcePackage`、发现查询和治理响应等共享类型；`packages/client-ts` 提供 `OanClient` 及各节点 HTTP 调用；`packages/sdk-ts` 提供 DID 文档草稿、资源包绑定、哈希格式、信任摘要和生命周期判断；`packages/governance-ts` 面向链下信任索引器的治理读取。这样的分层使应用可以只依赖 HTTP 客户端，也可以在需要时组合资源校验和生命周期辅助，而不必复制协议结构。

能力域	SDK 提供	调用方仍需负责
DID 与资源模型	类型、解析和构造辅助	确认资源内容和控制关系
注册	请求构造、提交和状态读取	保管私钥、确认提交目标
发现	DID 精确查询、语义查询和结果解析	判断相关性是否满足业务需求
验证	证明字段读取和验证辅助	制定授权策略与最终调用决策
生命周期	汇总登记、发布、分发和可见性状态	处理超时、重试和业务补偿

SDK 的端点选择有两种常见方式：面向官网和社区用户时传入一个 `baseUrl`，由客户端派生官方网关下的服务地址；进行多节点或第三方节点测试时，分别传入 `registrarEndpoint`、`discoveryEndpoint`、`rootEndpoint` 和 `cdnEndpoint`。端点配置只决定请求发送到哪里，不会替调用方完成节点授权、资源控制证明或治理判断。

17.2 SDK 中的 DID 身份与密钥操作

身份和密钥操作在客户端完成。SDK 生成主体身份、资源 DID 和验证方法，使用私钥签署控制 `challenge`、资源版本或请求，并将公钥写入 DID 文档。备份可包含主体和资源控制材料，但不得进入日志、HTTP 请求或索引；导入时应验证结构、DID 与公钥关系及签名能力。

Node.js 场景可以使用 `identity-store-node` 将主体、智能体和节点身份分桶保存到本地目录；浏览器场景则由网页状态或浏览器存储承载当前会话，并通过用户操作导出备份。两种实现的共同边界是：DID 文档可以公开传播，私钥只能用于客户端签名；服务端收到的是已经签名的控制证明或节点请求包络，而不是原始私钥。

- ☒ 私钥生成、签名和备份导出发生在客户端。
- ☒ 注册请求只携带公开 DID 文档、资源信息和必要控制证明。
- ☒ 导入备份前校验格式、版本、DID、公钥和签名能力。
- ☐ 不得把私钥写入 URL、日志、遥测或服务端存储。

17.3 注册流程示例

注册流程应依次准备本地身份、资源 DID、DID 文档、元数据、资源包、哈希和控制证明，再调用注册服务节点。下面只展示客户端调用边界，`submission` 应由经过本地校验和签名的实际数据构造：

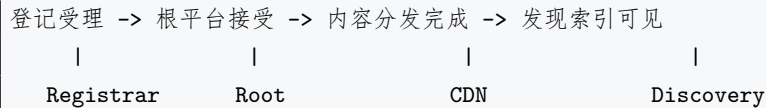
```
import { OanClient } from "@openagenet/oan-sdk-ts/client";

const client = new OanClient({
  registrarEndpoint: "https://registrar.example.org",
});
const result = await client.registerResource(submission);

console.log({
  resourceDid: result.resourceDid,
  registrationVc: result.registrationVc ?? null,
});
```

注册 VC 可能暂缺，客户端应保存资源 DID、请求 ID 和状态；网络超时不能直接判断注册未发生。应先确认明确成功或失败，再按请求 ID 或资源 DID 查询状态，并区分注册服务节点受理、根平台发布和发现索引可见三个阶段。

提交前的最小检查可以归纳为“对象一致、摘要一致、控制有效、目标明确”：`didDocument.id` 必须等于 `resourceDid`，资源类型必须与 DID 主体编码一致，DID 文档、元数据和资源包的摘要必须对应实际内容，控制证明必须由相应资源控制私钥生成，注册端点必须是用户明确选择的注册服务节点。技能包可以辅助发现缺失项，但最终提交仍由客户端或用户确认。



17.4 DID 精确发现示例

DID 精确发现以资源 DID 为定位键，不依赖名称相似度。客户端调用发现接口后，应核对返回 DID、版本、资源包和证明引用，再验证 DID 文档、版本哈希、生命周期、根平台可信发布证明和端点策略。

```
const resourceDid = "did:oon:SKDM:example";
const resource = await client.getDiscoveryResource(resourceDid);
if (resource.resourceDid !== resourceDid) {
  throw new Error("discovery_did_mismatch");
}
```

精确命中只证明发现服务节点返回了对应标识的记录，不能单独证明版本仍有效、节点仍获授权或业务端点可以安全调用。

应用在取得候选结果后，可以先按 `resourceDid` 做一致性检查，再按请求的版本模式选择候选版本，最后加载资源包或 DID 文档完成证据验证。若详情接口返回 404，通常表示发现索引中存在候选摘要但对应详情尚不可读；客户端应保留该差异并向调用方报告，而不是把候选结果直接当作完整可调用对象。

17.5 语义发现示例

语义发现可组合自然语言任务、能力标签和结构化条件。官网英文查询 “I need a tool that can search code repositories and summarize the project structure.” 可作为示例输入：

```
const discovery = await client.discoverResources({
  query: "I need a tool that can search code repositories and summarize the project structure",
  resourceType: "tool",
  limit: 10,
});

for (const candidate of discovery.candidates) {
  console.log(candidate.resourceDid, candidate.score);
}
```

响应应保留命中字段、标签、资源类型、协议、索引版本、来源和匹配原因。语义相关性回答“可能适用”，结构化条件回答“满足显式筛选”，可信证据回答“当前版本是否可验证”，三者不能互相替代。

SDK 对 HTTP 响应的封装应保留协议阶段，而不是把不同接口都压缩成一个布尔成功值。注册调用至少需要保留注册服务节点的 `status`、资源 DID、注册 VC 和根平台响应；发现调用需要保留 `discoveryDid`、候选数组、创建时间和可选响应 `proof`；动态状态或解释响应应保留原始 JSON，供

上层按实际字段处理。这样，SDK 才能区分“登记已受理”“根平台已验证并排队”“资源已被发现”和“证据已完成验证”。

语义查询的结果处理宜分成两步：先用查询文本、资源类型、标签或协议缩小候选范围，再对候选逐项检查生命周期、授权域、服务绑定和证明状态。发现服务节点可能使用语义索引，也可能在后端不可用时回退到关键词检索；调用方应关注返回的候选和解释信息，不应把某一种搜索后端名称写死为业务依赖。

17.6 验证与信任证据处理

验证应先检查 DID 文档与资源 DID、公钥和控制证明，再分别检查注册 VC、根平台可信发布证明和节点授权 VC 的签发者、主体、版本、签名、有效期、状态、授权域和能力范围。关键证明缺失或过期时，客户端应标记不可确认或拒绝使用。

证据	主要证明的事实	不能证明的事实
控制证明	提交者能使用对应控制私钥	注册服务节点已受理或资源已发布
注册 VC	注册服务节点对登记事实的声明	根平台已发布或端点安全
根平台可信发布证明	特定资源版本通过根平台验证并进入发布流程	所有发现服务节点均已索引
节点授权 VC	节点在授权域和期限内具备相应资格	节点返回的每条业务数据都正确
治理状态	当前治理侧记录的状态事实	资源功能质量和业务适用性

一个可复现的验证夹具至少包含：资源 DID、DID 文档、资源包、对应摘要、控制证明或注册 VC，以及用于校验节点资格的节点 DID 文档和授权材料。测试先使用未篡改材料确认通过，再分别修改 DID、版本、摘要、签发者或生命周期状态，确认 SDK 返回明确的验证错误。验证失败是否阻止使用取决于证据类型：控制证明或摘要失败通常应阻止登记，语义匹配不足则只影响候选排序，治理状态不确定则应阻止高风险调用或要求人工确认。

17.7 社区 Skill 包职责

社区技能包通过脚本和说明辅助资源整理、标签推荐、DID 生成、注册请求、发现查询、证据核验和结果解释。它不是注册服务节点、根平台或发现服务节点，不应持有治理密钥；用户确认、私钥使用、敏感路径和最终发布决定仍由用户或运营者控制。

典型交互遵循“读取材料 → 形成草稿 → 用户确认 → 本地签名 → 调用服务 → 解释结果”的顺序。技能包可以提示缺失字段、推荐语义标签和调用 SDK，但不得静默替换资源端点、扩大授权域或把推荐内容冒充资源提供方声明。

QanSkill 的公开工作流入口包括 `draftRegistrationFromResourceDescription()`、`registerFromResourceDescription()`、`registerBatchFromResourceDescriptions()`、`discover()`、`lifecycle()` 和只读治理辅助方法。资源描述工作流会返回候选信息、缺失输入、质量问题和建议动作；这使批量接入能够逐项修复，而不是因为一个资源失败就丢失整个批次的诊断结果。技能包默认使用官方公共网关，第三方节点只有在用户明确配置后才参与流程。

17.8 资源编写与注册辅助

资源编写辅助应统一处理智能体服务、技能包、MCP 服务和工具 API，并引导填写名称、描述、用例、端点、协议、版本、能力标签、授权域和资源包信息。推荐标签不是最终声明，生成前应检查 DID 文档、元数据和资源包哈希一致性。

输入类别	最低检查内容	产出用途
身份与控制	主体 DID、资源 DID、验证方法	生成控制证明
能力语义	名称、描述、用例、能力标签	语义治理与发现
调用信息	协议、端点、认证提示	调用前筛选和协商
版本材料	版本、前序版本、资源包摘要	更新、幂等和完整性校验
治理边界	资源类型、授权域、生命周期	准入和发现过滤

资源整理时应把 `authorizedDomains` 与 `capabilityTags` 分开处理：前者是注册服务节点用于授权范围判断的机器可读域，后者是语义发现信号。技能包可以根据描述提出标签建议，但不能从 DID 主体编码、端点主机名或自然语言描述擅自推导授权域；无法确定时，应把缺失事实返回给用户补充。

17.9 节点运营者与部署辅助

节点运营辅助可执行依赖检查、配置检查、Rust 编译、服务启动、健康检查和部署后注册发现烟测。服务器、域名、密钥、生产配置及回滚由人工确认。

- ☒ 检查进程、监听端口和健康端点。
- ☒ 检查数据库连接与必要表结构。
- ☒ 检查链下信任索引器状态及同步序列是否持续推进。
- ☒ 验证注册结果结构、DID 精确发现和语义发现最小路径。
- ☐ 不以单一健康端点成功代替完整业务烟测。

运维辅助的验收顺序应与业务传播顺序一致：先检查进程和健康端点，再检查数据库及节点授权状态，之后验证资源注册请求是否得到结构化响应，最后使用 DID 精确查询和语义查询确认发现链路。注册服务节点返回成功而根平台、内容分发平台或发现服务节点不可用时，应报告对应阶段，不应把整条链路压缩成一个“部署成功”结论。

17.10 包分发与兼容性管理

发布物必须包含运行入口、构建后的 JavaScript、类型声明、脚本、必要文档和依赖。社区技能包发布到 npm 时不得遗漏 `SKILL.md`、脚本或资源文件；SDK 发布前应从干净目录安装并验证导入、注册请求构造和发现请求构造。

发布验收至少包括：

- 预览实际入包文件，确认源码仓存在不等于发布包已包含。
- 在空目录安装待发布包，验证声明支持的模块入口。

3. 运行最小注册、DID 精确发现和语义发现请求构造测试。
4. 核对 README、类型声明、脚本和资源文件可从安装包访问。
5. 对破坏性接口或协议语义变化提高主版本，并说明迁移边界。

对于社区技能包，npm 发布物还必须保留 `SKILL.md`、`references`、`agents`、`src` 和构建后的 `dist`；不能只检查 TypeScript 编译成功。对于 SDK，应检查根入口及 `./client`、`./governance`、`./protocol-types`、`./identity-store-node` 等 exports 是否都指向实际存在的 JavaScript 和声明文件。发布验证应在未使用源码目录的空目录中执行，以便发现 `package.json` 的 `files`、入口或依赖声明遗漏。

17.11 示例资源与集成模板

最小集成流程为：创建或导入本地身份；生成资源 DID、DID 文档、元数据、资源包和控制证明；提交注册服务节点；执行 DID 精确查询；执行自然语言、标签和结构化发现；调用前验证版本、证明、治理状态和端点。



SDK 是程序化入口，技能包是操作辅助，官网是交互入口。四类模板可以共享身份、版本、语义和证明字段，但协议、端点和资源包内容必须体现各自差异。

四类资源可以复用同一套接入骨架，但其调用材料不同：智能体服务通常需要可调用服务端点，技能包需要清单或下载入口，MCP 服务需要 MCP 协议绑定，工具 API 需要 API 端点和输入输出模式。模板中的 `protocolBindings`、服务端点、资源描述和包信息应按实际资源填写；不能因为四类资源都可被发现，就把一种资源的协议字段复制到另一种资源。

17.12 客户端安全与秘密处理

私钥和身份备份只在客户端生成、使用和保存。VC、DID 文档和资源包按公开性分别处理；日志只记录 DID、版本、请求 ID、状态和错误代码，不记录私钥、完整本地身份或未脱敏签名原文。

客户端安全边界：能够被下载或导出的本地身份文件属于控制权材料，不因其由官网生成而成为公开文件。注册 VC、DID 文档和资源包也应按各自公开策略处理，不能因为共同打包下载就获得相同的披露级别。

SDK 和技能包的测试不应把真实私钥写入固定夹具或提交记录。自动化测试可以使用临时目录、临时密钥和占位 DID；测试结束后清理身份文件，并检查请求体、异常文本和日志中没有私钥字段。若应用需要长期保存本地身份，应由使用者选择加密存储、操作系统凭据库或离线备份方案，SDK 不应默认为用户上传到远程服务。

17.13 浏览器注册与本地身份备份行为

浏览器注册页面把本地身份作为用户控制材料。用户可新建并下载带时间标记的 `oan-registration-identity.bundle.<timestamp>.json`，也可导入备份；浏览器只向注册服务节点发送公开 DID 文档、资源数据和控制证明。注册结果包必须在提交成功且确认资源 DID 后生成。

用户动作	浏览器侧变化	服务端影响
新建并备份	生成身份、更新当前主体、触发下载	无
从本地导入	校验并加载用户选择的备份	无
预览 DID 文档	按当前表单和身份生成或复用预览	无注册提交
提交注册	校验当前输入并发送公开材料和证明	产生注册请求
成功后下载	打包 DID 文档、注册 VC 和本地身份	不产生第二次注册

17.13.1 浏览器端密钥生成

浏览器端使用 Web Crypto 或 SDK 实现生成密钥并建立 DID 与公钥绑定。页面不得把私钥放入 DOM、URL、公开 API 或错误消息；刷新或清理站点数据可能丢失未备份状态，应提示用户及时备份。

17.13.2 本地身份导入

导入操作读取用户明确选择的 JSON 文件，检查版本、主体配置、资源配置和私钥材料结构，并验证 DID 文档与公钥关系。导入失败不能覆盖已有有效身份；成功后预览和提交使用导入身份并重新计算控制证明。

17.13.3 本地备份下载

本地备份下载使用浏览器标准下载能力，文件名包含 `oan-registration-identity.bundle` 和生成时间，避免覆盖旧备份。服务端不参与文件生成，用户应将其作为敏感密钥备份保管。不同浏览器和移动系统对自动下载的支持存在差异，下载失败不得把空文件或未完成状态标记为有效备份。

17.13.4 注册结果打包下载

注册结果打包下载仅在服务端明确返回成功且页面确认资源 DID 后触发。压缩包名将资源 DID 中的冒号替换为连字符，包含注册 VC、DID 文档和已有本地身份文件；VC 为空可保留空项，但失败、超时或缺少 DID 时不得下载。

17.14 DID、VC 与资源包下载格式

DID 文档、注册 VC、资源包和本地身份备份使用可识别的 JSON 文件名和稳定编码。公开文件不得混入私钥；导入方按内容字段验证 DID、版本和哈希，不应仅凭扩展名或文件名信任。

文件	建议内容	敏感级别	最低校验
DID 文档	DID、公钥、服务与 OAN 元数据	公开	DID、验证方法、结构版本
注册 VC	签发者、主体、声明、签名与状态引用	按签发和披露策略	签发者、主体、签名、有效期
资源包	资源描述、端点、能力与版本材料	按资源策略	资源 DID、版本、摘要
本地身份备份	主体及资源控制私钥材料	高敏感	包版本、DID、公钥私钥对应关系

未知字段在兼容版本内可以保留，缺失必填字段、无法识别的结构版本、摘要不一致或私钥混入公开文件时必须拒绝导入或发布。

下面是一个用于说明字段关系的示例资源包，其中 DID、摘要、时间和签名值均为占位值，不代表线上资源。示例展示公开资源包如何同时引用 DID 文档、元数据摘要、资源包摘要和根平台可信发布证明；实际实现还可以保留协议类型允许的扩展字段。

```
{
  "packageVersion": "1.0.0",
  "resourceDid": "did:oa:SKDM:ExampleResourceDid00000000000000000000",
  "resourceType": "skill",
  "didDocument": {
    "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oa/v1"],
    "id": "did:oa:SKDM:ExampleResourceDid00000000000000000000",
    "verificationMethod": [
      {
        "id": "did:oa:SKDM:ExampleResourceDid00000000000000000000#key-1",
        "type": "Ed25519VerificationKey2020",
        "controller": "did:oa:SKDM:ExampleResourceDid00000000000000000000",
        "publicKeyMultibase": "zExamplePublicKey"
      }
    ],
    "authentication": [
      "did:oa:SKDM:ExampleResourceDid00000000000000000000#key-1"
    ],
    "oanMetadata": {
      "subjectType": "skill",
      "resourceType": "skill",
      "resourceDescription": {
        "name": "Example repository structure skill",
        "description": "Searches a code repository and summarizes its project structure.",
        "capabilityTags": ["code-repository", "project-structure"]
      },
      "capabilityTags": ["code-repository", "project-structure"]
    }
  },
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "hashAlgorithm": "sha-256",
  "metadata": {
    "resourceDid": "did:oa:SKDM:ExampleResourceDid00000000000000000000",
    "resourceType": "skill",
    "subjectType": "skill",
    "name": "Example repository structure skill",
    "description": "Searches a code repository and summarizes its project structure.",
    "capabilityTags": ["code-repository", "project-structure"],
    "lifecycleState": "active",
    "packageVersion": "1.0.0",
    "packageHash": "sha256:example-package-hash",
    "metadataHash": "sha256:example-metadata-hash",
```

```
    "hashAlgorithm": "sha-256",
    "updatedAt": "2026-01-01T00:00:00Z"
  },
  "rootProof": {
    "rootDid": "did: oan: INPT: ExampleRootDid00000000000000000000",
    "signature": "zExampleRootSignature",
    "hashAlgorithm": "sha-256"
  },
  "createdAt": "2026-01-01T00:00:00Z"
}
```

上述资源包可以作为公开发布和发现侧的输入，但不能替代本地身份备份。后者还包含主体或资源控制私钥，必须单独导出、导入和保管；把本地身份与资源包放进同一个注册结果压缩包，只表示下载操作集中，不改变两类材料的敏感级别和验证方式。下载格式的兼容判断应以内容结构为准，文件名中的时间戳和 DID 只用于人类识别和避免覆盖，不能作为真实性、签发者或控制权证明。

17.15 参考来源

来源	类型	链接
oan-sdk-ts	代码仓: DID、身份、资源包、注册和发现客户端能力	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓: 社区 Skill、脚本和开发者工作流	https://github.com/OpenAgenet/oan-community-skill
oan-protocol-common	代码仓: SDK 使用的协议类型和错误契约	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书: 客户端、资源发布和验证流程	https://arxiv.org/abs/2606.03163
OAN White Paper	arXiv 白皮书: 开发者生态和开放基础设施定位	https://arxiv.org/abs/2606.03161

第十八章 安全、隐私与滥用防护

本章说明 OpenAgenet (OAN) 的安全边界如何覆盖资源控制权、节点授权、根平台发布、发现结果、治理状态、调用端点和用户本地身份。攻击者可发送公开请求、提交伪造或低质量资源、重复请求、观察公开 DID 与 VC，或运行受损节点；安全控制必须说明保护对象、攻击前提、控制点、失败结果和验证证据。发现结果的可信性不等于业务端点安全。

安全要求统一使用以下判断链，后续小节中的表格和清单均应能回到这条链：

攻击前提 -> 被保护对象 -> 验证或控制点 -> 失败结果 -> 审计证据 -> 恢复动作

安全对象	主要攻击面	首要控制点	不应误判为安全的信号
资源控制权	复制 DID、伪造文档、重放签名	控制公钥、challenge、版本和签名	DID 字符串相同、HTTP 200
节点授权	冒充节点、越权、授权过期	节点 DID、授权 VC、治理状态	网络可达、历史成功
发布与分发	篡改资源包、伪造发布状态	哈希、根平台可信发布证明、批次校验	注册受理、缓存命中
发现与调用	索引污染、恶意端点、SSRF	来源、生命周期、端点策略和调用前验证	排名靠前、语义相关
可用性与隐私	洪泛、资源耗尽、凭证泄露	配额、隔离、脱敏和最小权限	单一健康端点成功

18.1 威胁模型与信任假设

威胁模型分别看待资源提供方、注册服务节点、根平台、内容分发平台、发现服务节点、链下信任索引器、浏览器和调用端点。资源控制私钥由控制方保管，治理密钥只用于治理动作，索引器是链上事实的读侧投影；协议不能自动修复已泄露密钥、完全失控主机或业务端点漏洞。

信任假设应按组件和材料拆开记录：

组件或材料	信任边界	失陷后的直接影响	仍需由其他组件重新验证的事实
资源控制私钥	由资源控制方保管	该资源控制权可能被转移或滥用	节点授权、治理状态和发布结果
注册服务节点	只在授权域内受信	可接收、篡改或拒绝请求	控制证明、资源包哈希和根平台发布
根平台	负责根验证、发布与治理衔接	发布状态或根平台可信发布证明受影响	资源控制权和外部治理事实

组件或材料	信任边界	失陷后的直接影响	仍需由其他组件重新验证的事实
发现服务节点	提供查询和索引视图	结果缺失、过期或被污染	来源授权、版本和根平台证明
链下信任索引器	提供链上治理事实的读侧投影	状态延迟或查询错误	原始链上事实和同步序列
业务端点	不属于 OAN 的安全信任根	业务调用本身可能被攻击	调用方的协议、认证和业务策略

18.1.1 资源控制权威胁

资源控制权威胁包括复制公开 DID、DID 文档和 VC 后冒充控制者，或用新文档覆盖同一 DID。控制点是 verification method、控制 challenge、签名和版本关系；错误公钥、错误 challenge、伪造签名和旧版本覆盖必须拒绝。

18.1.2 节点信任威胁

节点信任威胁包括未授权节点冒充注册服务节点、已授权节点越权，以及暂停或撤销后继续工作。控制点是节点 DID、节点授权 VC、治理事件、授权域和能力范围；网络可达或历史成功不能替代当前授权。

18.1.3 数据污染和滥用威胁

数据污染包括批量虚假资源、重复 DID、低质量描述、恶意标签、虚假端点和伪造排序信号。控制点包括控制证明、幂等规则、生命周期、根平台独立验证、标签边界、审核和限流；拒绝记录应区分具体原因。

18.1.4 可用性威胁

可用性威胁包括注册洪泛、复杂语义查询、超大资源包、并发同步、数据库锁争用和外部依赖不可用。控制点是大小限制、超时、并发配额、分页、查询降级、队列隔离和退避；不得无限重试。

18.1.5 调用端点安全威胁

调用端点威胁包括恶意 URL、内网回调、不安全协议、重定向绕过和端点被接管。控制点是协议、域名、解析地址、端口、重定向和认证校验；失败时阻止高风险访问并记录脱敏原因。

18.2 资源冒用与 DID 控制权保护

资源 DID 的控制关系由 DID 文档中的公钥和控制证明建立。公开 DID、文档、注册 VC 或根平台可信发布证明不能单独取得更新权；同 DID 更新必须使用原控制关系或规范允许的控制权转移流程，不能仅凭新文档自治就接受。

资源控制权的验证关系可以按下表逐项核对：

检查项	验证内容	失败处理
DID 与文档	文档声明的 DID 与请求资源 DID 一致	拒绝请求，不进入发布流程
公钥关系	verification method 与控制关系匹配	标记控制关系无效
签名挑战	签名覆盖当前请求、版本或 challenge	拒绝旧签名和跨请求签名
版本链	更新声明前序版本、摘要和控制权	不允许旧版本覆盖当前记录
状态约束	资源未被暂停、撤销或限制更新	按治理状态拒绝或暂停

仅有“签名格式正确”不足以证明控制权；验证者必须确认签名公钥确实由目标 DID 文档声明，并且签名覆盖了本次具体操作。

18.3 伪造注册与未授权更新防护

注册和更新分别校验来源、节点授权、控制证明、文档哈希、元数据哈希、资源包哈希、版本和幂等状态。旧版本、旧 nonce 或错误哈希不得覆盖当前记录；测试覆盖缺失证明、错签名、错版本、错哈希和重复提交。

注册与更新的拒绝条件应能被客户端区分：

情形	控制失败点	允许的结果
缺少控制证明	资源控制权未建立	拒绝注册或更新
节点授权无效	请求来源不具备接入资格	拒绝并记录授权原因
版本或前序版本错误	更新链不连续	拒绝覆盖，不改变当前版本
哈希不一致	文档、元数据或资源包被篡改	拒绝发布
已处理的有效请求重试	幂等键或请求 ID 已存在	返回稳定结果，不重复产生副作用

18.4 恶意或受损注册节点

根平台不能只信任注册服务节点转发的结论，而应重新验证节点身份、授权、控制证明、DID 文档、资源包和请求签名。越权或材料篡改时拒绝发布；注册节点历史记录不自动成为根平台发布事实。

18.5 恶意或受损发现节点

调用方应核验发现响应来源、查询节点身份、索引版本、资源 DID、DID 文档、版本、根平台可信发布证明、生命周期、授权域和治理状态。高相关性或 HTTP 成功不能替代验证；发现节点受损时可撤销授权并停止采用其索引。

18.6 根平台受损与密钥保护

根平台受损可能篡改待发布材料、伪造根平台可信发布证明、阻断传播或泄露配置。应隔离发布密钥和数据库，实行最小权限、受控签名、哈希、审计、备份校验和节点重新同步；恢复不改变资源控制私钥或链上事实。

18.7 垃圾注册与资源目录污染

目录污染治理关注资源真实性、可用性和语义质量。注册入口保留控制证明和主体，根平台独立验证并隔离可疑资源，发现节点过滤生命周期、治理状态和发布证据；清理要区分拒绝、暂停、撤销和节点授权撤销。

垃圾注册治理不应只依赖单一评分。处理前应组合控制证明、资源类型、标签命名空间、端点可验证性、重复度、来源节点和治理状态；处理后应记录动作和传播范围。

- ☐ 对批量请求实施主体、节点、接口和时间窗口多维限流。
- ☐ 对重复 DID 区分网络重试、幂等提交和真正版本更新。
- ☐ 对低质量或恶意标签保留规范标签、别名和推断命中的区别。
- ☐ 对已发布资源区分暂停、撤销、下线和节点授权撤销。
- ☐ 清理后复核根平台、内容分发平台和发现服务节点的收敛状态。

18.7.1 虚假资源注册

虚假注册由缺少控制证明、DID 文档不一致、端点不可验证或元数据伪造等信号识别，并在进入发布前拒绝。公开错误不得回显私钥或完整签名材料。

18.7.2 重复注册和版本污染

重复注册可能是网络重试、同 DID 更新或恶意提交。系统用资源 DID、版本、文档和资源包哈希及请求 ID 区分；重复有效请求返回稳定结果，真正更新遵守控制权和版本规则。

18.7.3 语义标签污染

标签污染包括占用热门标签、伪装规范标签、跨类型错配和误导关键词。注册校验标签 ID、词汇版本、类型关系和命名空间；发现区分规范、别名、原文和语义推断命中。

18.7.4 限流、审核和清理

限流、审核和清理按来源、主体、节点 DID、资源 DID、接口和风险等级实施，并保留原因、操作者和影响范围。清理后应验证根平台、内容分发平台和发现索引收敛。

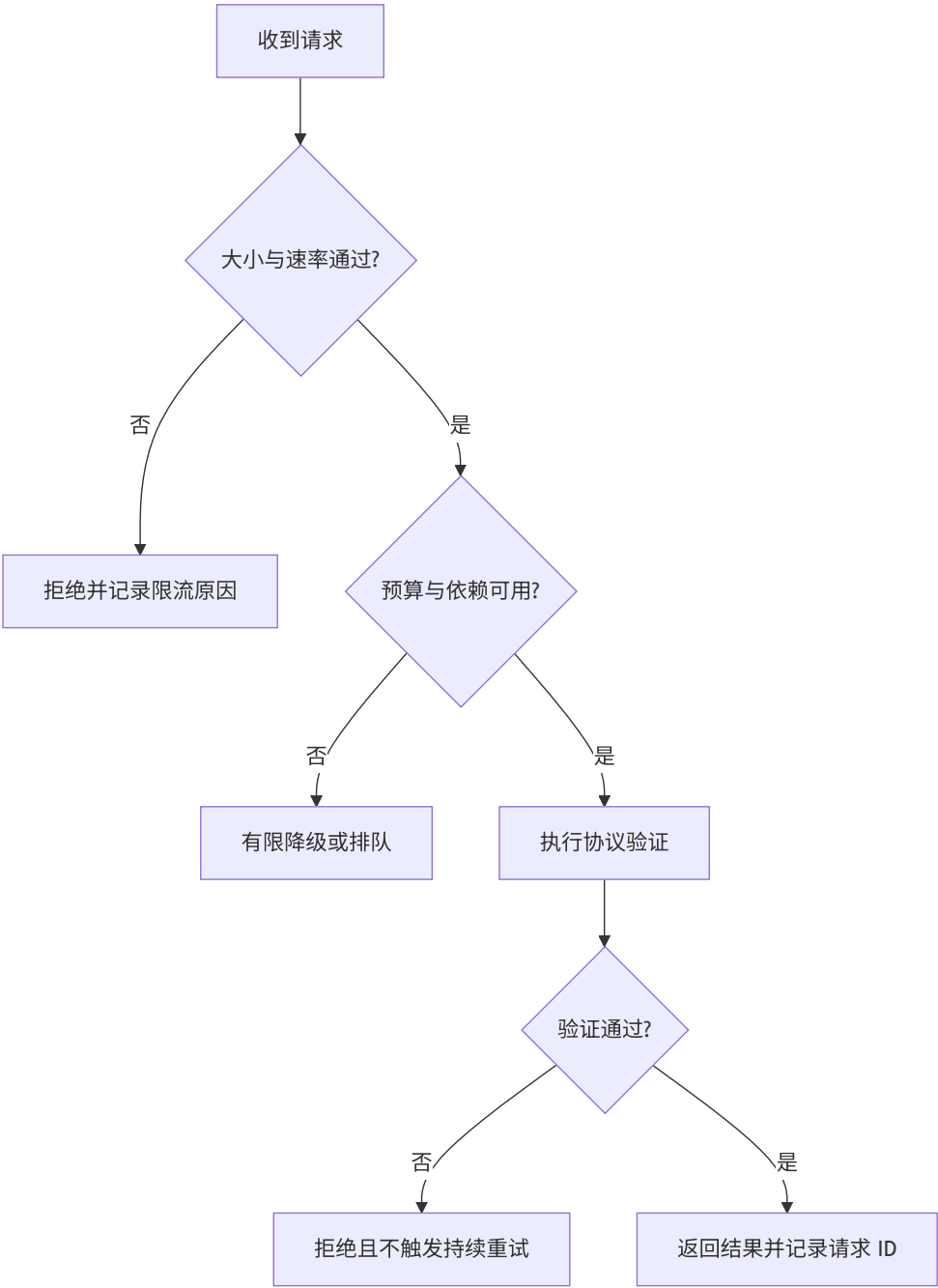
18.8 拒绝服务与资源耗尽

注册、发现、同步、健康和治理查询分开配置资源预算，限制请求体、批次、压缩展开、查询深度、并发、数据库连接和外部调用时间。语义后端失败可退回受限路径，但不能放宽可信过滤；降级需标注不完整或不可确认。

资源预算应按路径隔离，避免发现查询的高开销拖垮注册和健康检查：

路径	主要预算	超限动作	安全底线
注册	请求体、验签次数、并发和外部依赖时间	限流、排队或拒绝	不跳过控制证明

路径	主要预算	超限动作	安全底线
发现	查询长度、页大小、候选数和执行时间	截断、缓存或降级	不绕过治理过滤
节点同步	批次、游标、并行度和重试次数	退避、暂停或重置游标	不重复推进已处理序列
数据库/索引	连接、锁、磁盘和索引增长	隔离、告警或保护性只读	优先保留已验证数据



18.8.1 注册接口攻击

注册攻击包括高频提交、超大 JSON、重复 DID、昂贵验签和依赖调用。入口先做大小、速率、幂等和格式检查，再执行高成本验证；失败请求不得触发持续发布重试。

18.8.2 发现查询攻击

发现攻击包括无上限宽查询、复杂语义检索、重复查询和结果放大。限制输入长度、页大小、候选数量、执行时间和并发，使用稳定游标与缓存；降级不能绕过治理过滤。

18.8.3 节点间同步压力

同步压力包括大批次、游标重放、过多并行、重试风暴和分发不可用。同步方使用批次和并发上限、指数退避及游标幂等，重复批次不得重复写入或推进状态。

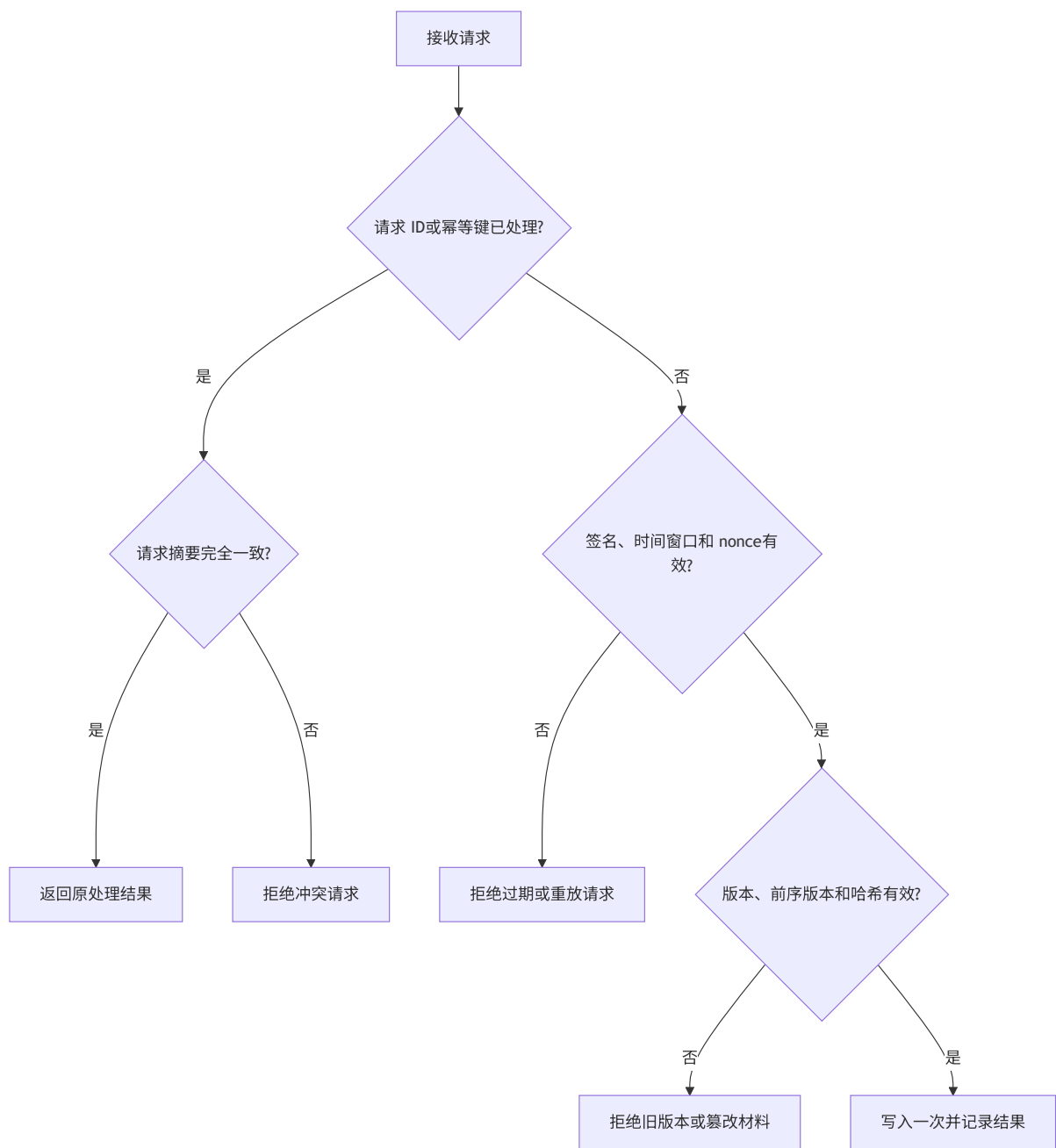
18.8.4 数据库和索引资源耗尽

数据库和索引耗尽包括连接池、锁、磁盘和索引膨胀。应有字段批次限制、索引维护、超时、隔离队列、保留策略和容量告警；资源不足时优先保护有效数据和健康查询。

18.9 重放、重复提交与过期发布

重放防护区分请求、登记、发布、同步批次和旧缓存。请求 ID、nonce、时间窗口和签名防止再次接受，DID、版本、前序版本和哈希判断业务更新，游标和批次数列保证同步幂等。

重放和重复提交的判断顺序应保持幂等优先、状态更新谨慎：



18.10 端点滥用与不安全资源调用

调用前检查端点协议、证书、域名、解析地址、认证、版本、生命周期、治理状态和发布证据，必要时进行业务权限协商、请求签名和响应验证。OAN 不保证业务端点无漏洞或无恶意行为。

调用方在使用发现结果前至少完成以下检查：

- ☐ 资源 DID、DID 文档、公钥和版本摘要一致。
- ☐ 根平台可信发布证明、生命周期和治理状态仍可验证。
- ☐ 端点协议、域名、证书、解析地址、端口和重定向符合本地策略。
- ☐ 调用所需的认证、授权域、请求签名和响应校验已明确。

□ 发现服务节点的结果新鲜度和来源不会被误当作业务服务保证。

18.11 敏感元数据与凭证暴露

私钥、本地身份备份、内部端点、数据库路径、管理令牌、未公开资源、完整签名原文和可关联个人的日志字段均属敏感信息。公开接口不得返回本地身份备份，服务端日志需白名单、脱敏和访问控制。

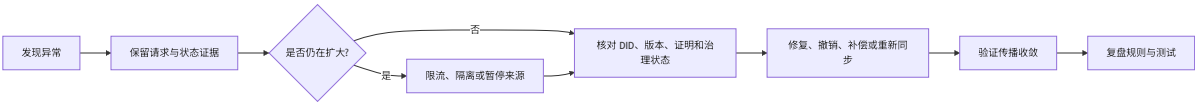
信息类别	是否可出现在公开响应	日志处理	失败后动作
DID、版本、公开元数据	按资源公开策略	可记录，保留关联 ID	发现或发布按状态处理
注册 VC、根平台可信发布证明	按披露策略	记录摘要和状态，不必记录完整敏感材料	验证失败则拒绝采用
私钥、本地身份备份	否	禁止记录	立即阻断并调查泄露路径
管理令牌、数据库路径、内部端点	否	脱敏或白名单记录	轮换凭证并限制暴露面
完整签名原文和个人关联日志	通常不应公开	脱敏、最小化和限权	删除不必要复制并复核访问

18.12 日志、审计与事件响应

安全日志关联请求 ID、资源 DID、节点 DID、版本、事件或游标、来源、结果、错误代码和时间，并区分链上证据、服务日志和手工记录。日志应限制写入权限，告警应能定位隔离、撤销、限流、补偿和恢复验证。

事件响应应保留最小但可关联的证据：

```
request_id / resource_did / node_did / version / event_or_cursor
source / result / error_code / observed_at / operator_action
```



18.13 第三方节点安全要求

第三方节点自行保护主机、节点私钥、授权 VC、配置、数据库、依赖和端口，并通过节点 DID、DID 文档、节点授权和请求签名接入。异常时可暂停或撤销授权，不得要求资源提供方交出主体或资源控制私钥。

第三方节点接入前应形成可审计的最低检查记录：节点 DID、DID 文档、节点授权 VC、授权域、有效期、公开端点、软件版本、密钥保管方式和撤销联系人。资源提供方只提交资源控制证明，不应交出主体或资源控制私钥。

18.14 端点验证、回调安全与 SSRF 考量

读取或调用端点、回调和同步地址前验证协议、主机、端口、解析地址、重定向和认证。私有、回环、链路本地和云元数据地址按策略阻止，解析与连接之间防止 DNS 重绑定，拒绝原因可审计但

不泄露拓扑。

SSRF 防护应覆盖解析前、解析后、连接前和重定向后四个时点；只在字符串层面检查 URL 不能作为充分控制。拒绝时记录策略命中类别和请求 ID，不回显内部地址、网络拓扑或解析细节。

阶段	必查对象	失败结果
解析前	scheme、主机、端口、凭据和编码	拒绝非法 URL
解析后	IPv4/IPv6、回环、私有、链路本地、拒绝受限地址 云元数据地址	
连接前	实际连接目标、证书、超时和策略	失败关闭
重定向后	每一跳协议、主机、解析地址和端口	拒绝越界重定向

18.14.1 端点格式和协议验证

只允许支持的协议和端口范围，拒绝缺少主机、非法编码、嵌套凭据或类型不符的端点。HTTPS 证书和主机名按风险校验，开发环境 HTTP 不得默认为公网安全配置。

18.14.2 内网地址与回环地址限制

每次连接前检查 IPv4、IPv6、回环、私有、链路本地、保留和云元数据地址，使用标准解析和网络策略，不能只依赖字符串黑名单。

18.14.3 回调请求安全

回调绑定一次性请求 ID、来源上下文和目标，限制方法、请求头、响应大小和超时，禁止回调改变授权或治理状态；失败重试必须有限。

18.14.4 重定向和 DNS 解析风险

DNS 解析结果、连接目标和重定向链纳入同一次判断，重定向到不同主机、协议或受限地址时重新校验；超时、证书变化或解析不一致应失败关闭。

18.15 包与依赖的供应链完整性

源码、Cargo/npm 依赖、锁文件、构建产物和资源包属于供应链边界。构建使用锁定依赖、受控来源、哈希或签名校验和可追溯提交；资源包进入根平台或发现节点前重新计算哈希。

供应链核验应覆盖“来源—锁定—构建—产物—部署”链路：

- 1. 核对源码提交、依赖锁文件和受控包源。
- 2. 在可复现或受控环境构建 Rust、TypeScript 和社区 Skill 产物。
- 3. 对二进制、npm 包、资源包和分发材料保存摘要或签名。
- 4. 部署前比较产物摘要，部署后记录版本和回滚材料。
- 5. 发现依赖或产物异常时停止发布，保留证据并重新构建。

18.16 可用性目标与优雅降级

安全状态和可用性分开表达。旧快照可支持带新鲜度标记的只读结果，但不能批准注册、发布、授权或高风险调用；依赖失败、索引滞后、治理未知和资源耗尽应有不同状态与恢复动作。

状态	可返回的内容	禁止执行的动作	必须标注的事实
正常且新鲜	当前查询、状态和证明	无额外限制	来源和时间
旧快照可用	带新鲜度的只读结果	注册、发布、授权、高风险调用	快照时间和滞后
索引滞后	已确认的部分事实	把未同步当作不存在或最新	当前序列和目标序列
治理未知	有限查询或错误状态	通过未知状态批准调用	未知原因
依赖不可用	健康错误或受限缓存	无限重试、放宽可信过滤	依赖和恢复状态

优雅降级的验收重点不是“页面仍有内容”，而是用户和调用方能够看见内容的新鲜度、来源和不确定性，并且不会因旧数据自动执行注册、发布、授权或高风险业务调用。

18.17 参考来源

来源	类型	链接
oan-protocol-common	代码仓：签名、哈希、凭证和安全请求结构	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓：根平台验证、发布和节点授权	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册入口、控制证明和滥用控制	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：查询、索引和结果安全边界	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理状态和撤销状态投影	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv 黄皮书：安全、隐私和信任边界	https://arxiv.org/abs/2606.03163

第十九章 资源生命周期、治理与版本管理

本章规定资源在 OpenAget (OAN) 网络中的时间、状态和证据关系。智能体服务、技能包、MCP 服务和工具 API 等资源以 DID 标识主体，以 DID 文档描述控制关系、能力和服务入口，以资源包承载可发布内容；注册 VC 证明注册服务节点受理过登记，根平台可信发布证明证明特定资源包通过根平台验证并进入发布链路。资源“已登记”“已发布”“已索引”和“当前可调用”是不同事实，必须分别证明。

本章中的状态关系可以用一个资源版本作为关联键来理解：

资源 DID

-> DID 文档版本

-> 资源包版本 + 文档/元数据/包哈希

-> 注册 VC

-> 根平台可信发布证明

-> 分发游标与发现索引版本

上面的链条表示证据之间的关联，不表示所有步骤同步完成，也不表示注册服务节点返回结果就等于发现服务节点已经可见。

19.1 资源生命周期状态机

状态	事实含义	默认发现行为
草稿	仅在发布者本地形成	不返回
已登记	注册服务节点已受理并记录	尚不代表全网可见
待发布	等待根平台验证或传播任务	可查询处理状态
已发布	根平台已完成验证并产生发布证据	具备索引条件
可发现	发现服务节点已验证并建立索引	可返回给查询方
暂停、撤销、停用	暂时不可用、明确撤回或主体终止	默认不作为有效资源返回

状态必须由事件、证明或持久化记录触发，不能由页面按钮或单次 HTTP 成功推断。注册成功最多说明注册服务节点完成受理；根平台验证、授权检查、资源包摘要校验和发布任务完成后才能进入已发布；发现服务节点完成验证和索引后才能进入可发现。失败应保留原因、关联标识和重试边界。

19.2 初始注册

发布者在浏览器中创建或导入本地身份，使用本地私钥签署 DID 文档和注册请求；私钥不应上传到注册服务节点。注册服务节点验证格式、控制关系、资源类型、必填元数据和节点授权，成功后

记录登记并按接口约定返回注册 VC。根平台随后独立验证资源包、注册节点授权和发布条件，内容分发平台及发现服务节点再异步接收和索引。缺少本地身份、签名无效、字段缺失、网络失败、VC 为空和传播延迟必须分别反馈，不得统称为全网注册完成。

阶段	发布者本地动作	网络侧事实	可复核证据
生成	创建或导入本地身份，生成 DID 文档	尚未登记	本地身份备份、DID 文档和控制签名
受理	发送已签名请求	注册服务节点记录登记	响应、请求标识、注册 VC（如有）
发布 可发现	等待异步处理 查询资源	根平台形成发布事实 发现服务节点完成索引	根平台可信发布证明 查询结果、来源和索引时间

19.3 同 DID 资源更新

同 DID 更新的依据是控制关系连续，而不是 DID 字符串相同。更新请求应带有前序版本、资源 DID、版本和内容摘要，并由既有控制公钥或明确授权的更新密钥签名。注册服务节点验证控制权和版本关系，根平台重新验证资源包与发布授权，发现服务节点只接受通过根平台发布流程的版本。相同内容和前序版本的重复提交应幂等处理；内容变化必须推进版本和摘要，无授权、倒退或分叉应拒绝。新版本未传播完成前，旧的有效版本可以继续返回。

更新场景	控制证明	版本关系	处理结果
同 DID、同摘要、同前序版本	有效	重复	幂等返回，不生成新有效版本
同 DID、新摘要	有效	递进	重新验证、发布并建立新版本
同 DID、新摘要	无效或越权	任意	拒绝，不覆盖旧版本
同 DID、旧版本	有效	倒退	拒绝或标记版本冲突

19.4 DID Document 版本管理

DID 文档版本表达身份文档、控制公钥、服务端点或能力元数据的变化，不等同于 did:oan 方法版本或资源包版本。当前版本应由稳定 DID、前序关系、序列或等价版本规则和控制者签名共同确定，时间戳只能用于审计。解析器不得仅按到达时间解决同一前序版本的分叉；发现服务节点默认返回已发布、状态有效且序列最高的版本，并支持按版本或内容摘要查询历史版本。

版本判定顺序应保持确定性：

生命周期状态过滤 -> 证明校验 -> 发布序列比较 -> 版本比较 -> 查询排序

资源包版本与接口兼容性不是同一个维度。技能包可以只更新文档或示例，工具 API 可能同时改变输入输出模式和端点，MCP 服务还可能改变协议绑定或暴露能力；因此版本号本身不能替代变更说明。资源包中的 packageVersion、packageHash、metadataHash 和 lifecycleState 应与 DID 文档元数据及根平台发布证明中的对应值一致。

一个用于审计的最小绑定记录可以表示为：

```
{
  "resourceDid": "did:oan:SKDM:example-resource-did",
```

```
"packageVersion": "1.2.0",
"didDocumentHash": "sha256:example-did-document-hash",
"metadataHash": "sha256:example-metadata-hash",
"packageHash": "sha256:example-package-hash",
"lifecycleState": "active",
"previousVersion": "1.1.0"
}
```

以上 DID、版本和摘要均为示例占位值；它表达的是字段之间的绑定关系，不是官方固定资源或固定版本。

19.5 资源包版本管理

资源包是根平台验证、内容分发平台保存和发现服务节点索引的发布单元。版本必须能指向一组规范化元数据、入口、协议绑定、依赖和内容摘要；根平台可信发布证明应绑定资源 DID、资源类型、资源包版本及包摘要。资源 DID 可以在正常升级中保持不变，旧资源包可标记为被替代并保留审计证据；发现服务节点默认返回最新有效包，调用方请求指定版本时仍须验证其发布证明和凭证状态。

可按变更影响范围选择更新对象：只改变发布包内容时，至少更新资源包版本和包哈希；改变 DID 文档中的服务入口、公钥或资源元数据时，应同步更新 DID 文档及其摘要；改变能力标签、授权域或生命周期时，还要重新检查语义治理和可见性。任何一种影响根平台可信发布证明绑定字段的变化，都不能只在发现节点本地修改索引。

19.6 元数据与端点变更

能力标签、描述、用例、协议绑定、端点、认证要求和依赖等会影响发现或调用判断的变化，应产生新的 DID 文档版本、资源包版本或二者组合，并重新签名、验证和索引。端点变化还应检查 TLS、认证、协议和实际能力；旧端点下线不等于资源 DID 撤销，调用方应根据新版本和状态决定是否调用。

19.7 注册 VC 生命周期

注册 VC 证明注册服务节点受理某项登记，不是资源质量保证，也不替代根平台发布证明。验证方应检查签发者授权、主体 DID、资源类型、版本或内容摘要、签发时间、有效期及吊销状态。VC 缺失、无效、签发者未授权或状态未知时，不得标记为可信注册。凭证吊销只影响其证明事实，不会自动改写 DID 文档；DID 停用也不等于历史凭证消失。

查询方处理凭证时，应将“凭证存在”和“凭证当前有效”分开记录。对于资源版本 v1 的注册 VC，即使其签名有效，也不能直接证明 v2 已登记；对于 v2 的根平台可信发布证明，也不能反向替代注册服务节点对登记事实的证明。资源更新、凭证吊销和主体停用发生后，发现服务节点应按最新可验证状态过滤结果，历史凭证仍可作为审计材料保留。

19.8 资源暂停与恢复

暂停是可逆状态，原因应区分控制者操作、治理限制、证明暂不可验证、端点维护和节点同步异常。网络超时或指标无数据不能直接改变资源生命周期。暂停应有控制签名、治理事件或其它可验证证据；恢复前重新检查控制证明、VC、根平台发布证据、端点和治理条件，并让发现节点按发布序列重新同步，而不是只修改本地状态。

19.9 资源撤销与停用

资源撤销针对发布资格或版本，DID 停用针对资源主体，凭证吊销针对某项证明，端点下线针对服务入口。撤销和停用应记录发起者、依据、目标 DID 或版本、原因、生效时间和传播范围；历史材料可以保留审计，但不能作为当前可调用资源返回。传播未收敛前，节点必须标注状态和新鲜度，而不是宣称全网立即完成撤销。

节点授权变化与资源状态的处理可以按以下关系核对：

观察到的事实	能够说明什么	不能直接说明什么
注册服务节点被撤销	该节点不再具备新的授权接入资格	已发布资源自动撤销
发现服务节点被暂停	该节点的查询或索引结果需要降级处理	资源控制权失效
资源版本被撤销	指定资源或版本不应继续作为有效结果	其它版本必然撤销
节点恢复并完成同步	节点重新具备相应运行条件	历史缺失数据已经自动补齐

19.10 节点暂停与资源可见性

节点授权属于治理对象，资源控制权属于资源 DID。注册服务节点失去授权后，根平台应停止接受其新的可信发布输入，并按治理规则重新评估已有发布；发现服务节点失去授权后，不应继续以正常授权节点身份发布结果，但其数据可为恢复和审计保留。节点恢复须先恢复身份和授权，再依游标补齐数据。客户端应区分节点故障、同步中和资源撤销。

版本选择不是简单的“最后写入优先”。例如，发现服务节点同时收到同一资源的 1.1.0 和 1.2.0，应先确认两者都具有可验证的发布来源和状态，再依据发布序列、前序关系和版本规则选择当前结果；如果 1.2.0 的摘要或证明校验失败，不能因为版本号更大就覆盖 1.1.0。选择结果还应保留被采用版本和被排除版本的原因，便于定位传播延迟与版本冲突。

19.11 发现节点的版本选择

发现节点先过滤状态、签名、注册 VC、根平台可信发布证明、版本有效性和索引新鲜度，再执行语义和结构化排序。默认对同一 DID 返回已发布、状态有效且发布序列最高的版本；版本冲突、序列缺口或证明未知时应报告异常而非随机选择。精确 DID 查询和语义查询均应返回版本、摘要、来源、证据状态和索引时间。

19.12 历史版本与可审计性

审计记录应关联 DID、文档版本、资源包版本、摘要、控制签名、注册 VC、根平台可信发布证明、发布序列、节点接收时间和索引时间。历史查询应区分当前有效、已替代、已撤销、已归档和节点尚未同步；历史证据用于还原发布事实，不得绕过当前状态。节点重建或数据库迁移不得无记录地丢失规范要求保留的证据。

历史记录至少应能回答三个问题：某个版本由谁控制并签署，根平台是否接受并发布过该版本，以及指定发现服务节点在什么时间以何种状态看到该版本。若只能找到资源包文件而没有控制签名、发布证明或索引观察记录，只能证明材料曾被保存，不能证明它曾经是网络中的有效版本。

19.13 资源 VC 与发布证据生命周期

注册 VC 和根平台可信发布证明具有不同签发主体和事实范围。前者证明登记受理，后者证明特定资源包通过根平台验证并进入发布链路；二者均须绑定明确的 DID、版本和摘要。更新产生新摘要时应重新产生相应证据，撤销、停用、过期和授权失效须分别检查。下载到发布者本地的证明文件不改变服务端的有效性判断。

证据关联可按版本逐项核对：

同一资源 DID

-> 同一资源包版本

-> 对应 DID 文档/元数据/包摘要

-> 注册 VC 的登记事实

-> 根平台可信发布证明

-> 发现服务节点的接收和索引观察

其中任何一环指向不同 DID、版本或摘要，都应按绑定不一致处理，而不是用文件名或下载时间推断它们属于同一发布。

19.14 更新传播与过期索引收敛

注册服务节点 -> 根平台验证与发布 -> 内容分发平台 -> 发现服务节点接收 -> 校验、索引与可见

各段使用 DID、版本、摘要、发布序列或事件游标关联；重复投递必须幂等，断点重试从确认游标继续。序列缺口、摘要不一致、证明无效或治理状态未知时，发现节点不得更新有效投影。旧索引的收敛以新版本完成验证和投影为准，而非以请求发出为准。

收敛检查	通过条件
内容	资源包可取得且摘要一致
证据	发布证明的目标 DID、版本和摘要一致
顺序	发布序列或同步游标无未解释缺口
状态	新版本生命周期状态已重新判断
查询	精确查询和语义查询不再返回旧的有效版本

传播检查还应记录每个节点看到的状态，而不是只比较资源数量。数量相同可能是旧版本仍在索引，也可能是新版本替换了旧版本；必须比较资源 DID、版本、摘要、生命周期状态、发布序列和观察时间。发生异常时，保留最后成功游标和原始响应，避免通过重置索引掩盖缺失或乱序事件。

19.15 资源所有权转移策略

所有权转移必须由原控制者授权、新控制者确认，并由根平台或治理流程验证后生效；重新提交同 DID 或出示新的注册 VC 都不足以证明转移。运营维护权和发布代理权可以单独授权，但范围、期限和撤销方式应与控制权分开记录。原私钥遗失时不得绕过签名强行接管，应进入可审计的治理恢复流程；当前实现未提供完整转移接口时，本节属于协议设计要求。

在控制权转移完成前，旧控制者仍是资源更新和撤销判断的关键证据来源；新控制者的声明、运营授权或资源包签名不能单独覆盖旧控制关系。转移完成后，维护记录应同时保存旧控制者授权、新控制者确认、根平台或治理流程的生效事实，以及发现服务节点完成更新后的观察结果。这样才能将“提出转移”“转移获批”和“节点已收敛”区分开来。

19.16 参考来源

来源	类型	链接
oan-root-services	代码仓：发布状态、版本和根平台证明	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：登记、更新和注册凭证	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：索引版本、生命周期和发现状态	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理、暂停和撤销状态	https://github.com/OpenAgenet/oan-trust-indexer
oan-protocol-common	代码仓：版本、生命周期和状态共享结构	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：资源生命周期、版本和治理机制	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	标准/方法规范：资源身份和更新控制	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md

第二十章 可观测性、指标与运维

本章说明可观测性如何同时回答服务是否存活、业务是否可用、数据是否新鲜以及异常是否正在收敛。根平台、注册服务节点、发现服务节点、内容分发平台、链下信任索引器、数据库和官网后端分别产生运行与业务证据；网站可以用旧快照改善访问体验，但旧快照不能替代后台真实健康状态。指标必须带来源、时间、状态和延迟，未知值不得当作零。

本章的观测证据分为四层：服务接口或源码定义的原始事实、后台聚合得到的运行指标、官网页面使用的展示数据，以及运维流程产生的烟测和告警记录。记录指标时至少同时保留 `source`、`observedAt`、`dataStatus` 和 `lag`，使“接口没有数据”“数据为零”“展示了旧快照”能够被区分。官网页面的展示结果只能说明前端获得了某次响应，不能代替节点、数据库或链下信任索引器的独立核验。

观测层	核心问题	典型证据
存活	进程是否能响应	liveness 响应、进程状态
就绪	依赖和配置是否满足	readiness、数据库和上游状态
业务	注册、发布、发现是否可用	烟测请求、响应和关联标识
数据	指标和索引是否新鲜	时间窗口、游标、最新序列
运维	故障是否被发现和收敛	日志、告警、恢复记录

20.1 服务健康与就绪模型

检查顺序应从“能否响应”逐步推进到“能否完成业务”：先核对进程和 `/health`，再核对就绪状态及数据库连接，随后检查根平台、注册服务节点、发现服务节点、内容分发平台和链下信任索引器之间的依赖。一个接口返回 200 只能作为该检查点的证据；只有依赖检查、数据状态和注册/发现业务烟测均通过，才可将服务标记为业务可用。`summary` 长期失败时，即使页面仍能显示旧快照，也应保持后台失败状态。

`liveness` 只证明进程能够响应，`readiness` 还应检查必要依赖和配置，业务烟测则验证注册、发布、发现等真实路径。健康检查应记录目标、响应码、耗时、依赖状态和时间；数据库可连接不等于索引器已追上游事件，摘要接口有响应也不等于统计完整。恢复确认至少需要连续成功的健康检查 and 对应业务烟测。

状态	能证明什么	不能证明什么
liveness	进程能够响应检查	依赖正常、业务可用
readiness	配置和必要依赖就绪	全网发布和索引完成
业务烟测通过	指定路径在测试条件下可用	所有资源和所有负载均正常
数据过期	最近数据未及时更新	资源已经撤销
指标缺失	当前无法取得指标	指标值为零

20.2 注册指标

注册指标应围绕一次资源登记建立可追踪链路。推荐将 `requestId` 绑定到注册请求，将 `resourceDid`、资源版本或内容摘要绑定到登记事实，再将发布序列、分发批次和索引版本绑定到后续阶段。客户端重复点击或网络重试可以产生多个请求记录，但不能因此把同一资源版本重复计入登记成功；VC 为空时应如实记录返回结果，不能把空值解释为有效凭证。周期统计只在聚合层按统一时区计算，前端不得根据页面刷新次数自行累加。

注册指标至少区分请求接收、校验拒绝、登记成功、VC 返回、根平台受理、发布完成和发现可见。去重应使用请求或资源版本关联标识，而不能按 IP 粗略去重；生产统计应排除明确标记的烟测。TODAY、THIS WEEK、THIS MONTH 使用统一时区和半开时间区间，周期值不能因缓存或降级与累计值混淆。

指标	计数单位	关联键	统计注意事项
注册请求	请求或登记尝试	<code>requestId</code>	区分重试和重复提交
登记成功	资源版本登记	<code>resourceDid + version</code>	不等于根平台发布
VC 返回	返回的凭证结果	<code>resourceDid + credentialId</code>	空 VC 不计为有效凭证
根平台发布	发布事实或发布批次	<code>resourceDid + sequence</code>	异步状态单独统计
可发现	索引可查版本	<code>resourceDid + indexVersion</code>	需结合新鲜度

20.3 发布与传播指标

阶段	开始时间	完成时间	主要证据
注册受理	注册服务节点接收请求	登记记录形成	请求标识、资源 DID、响应状态
根平台发布	发布任务创建	根平台可信发布证明形成	发布序列、资源包摘要
内容分发	分发批次发送	内容可校验取得	批次、游标、内容摘要
索引可见	发现服务节点接收	查询能够返回	索引时间、来源、版本

传播指标以资源 DID、版本、内容摘要和发布序列贯穿各阶段，分别记录事件发生、接收、验证、分发和索引时间。端到端延迟是最后可查时间减首次受理时间，不能用某一段耗时替代。重试和重复批次保持幂等计数；游标停滞、序列缺口和节点离线进入积压或失败指标。

传播链路至少应能回答“资源在哪一阶段停止”：注册服务节点已受理但根平台尚未发布，根平台已发布但内容分发未完成，分发完成但发现服务节点尚未索引，或索引已完成但查询条件未命中。可用如下关联关系核对，而不以某个页面上的资源数量代替链路事实：

```
resourceDid + version
-> publicationSequence
-> distributionBatch / contentDigest
-> discoveryIndexVersion / indexedAt
-> queryRequestId / visibleAt
```

同一资源版本的重试应复用幂等键或被标记为重试；如果只有某一阶段的记录，应在运维结果中明确“已受理”“传播中”或“尚未可查”，而不是简单报告为成功。

20.4 发现查询指标

查询类型	必须记录	解释边界
DID 精确查询	DID、耗时、结果数、来源	命中不等于当前可信
结构化查询	条件、耗时、命中和过滤结果	条件匹配不等于可调用
语义查询	原始描述、转换条件、排序结果	相关性不替代控制和治理验证
失败查询	状态码、错误类型、requestId	空结果不能等同于服务故障

发现指标区分 DID 精确查询、结构化条件查询和语义查询，记录请求耗时、结果数、空结果、拒绝、超时和服务端错误。相关性不等于可信度，结果仍须经过状态、凭证、发布证明和新鲜度检查；错误响应保留关联标识，便于从官网请求追到发现服务节点日志。

查询指标至少记录查询模式、标准化后的输入摘要、请求标识、服务节点、命中数量、耗时、响应状态和数据状态。语义查询的原始任务描述可以用于排障和质量分析，但日志与统计不应保存不必要的个人或敏感内容；可用摘要、长度和哈希替代全文。空结果需要结合请求是否合法、索引是否新鲜和资源是否在节点覆盖范围内判断，不能直接归为发现服务故障。DID 精确查询命中后仍须核对资源状态、发布证据和版本。

20.5 索引新鲜度与同步指标

同步状态 = 上游最新序列 - 本地游标 + 最近成功时间 + 最近错误

不确定的新鲜度 -> 标记 `unknown`，不填充为 0

重建索引、增量同步、暂停同步和同步失败应使用不同状态值；运维人员据此判断是没有新事件、处理延迟还是同步链路中断。同步观测包括当前事件游标、上游最新序列、落后距离、最近成功时间、最近失败原因、索引更新时间和待处理数量。`latest_sequence` 持续增长只说明上游存在可观测的新事件，不能单独证明本节点完成索引；落后距离未知时显示未知而不是零。重建索引与增量同步分开标记。

新鲜度判断应同时观察上游最新序列、本地已处理游标、最近成功同步时间和最近错误。链下信任索引器的 `latest_sequence` 继续增长，只能证明上游仍有可观察事件；还必须核对本地游标是否推进、是否存在序列或摘要缺口，以及发现服务节点是否完成对应索引。没有上游序列、接口失败或节点尚未产生数据时，状态应为 `unknown` 或 `unavailable`，不应伪装成落后距离为零。

20.6 治理与授权指标

治理指标记录事件来源、链上位置、事件类型、索引游标、节点状态变化和最后同步时间。授权资格、资源控制权与发布结果分别观测；治理事件被索引不等于所有资源状态已更新，节点正常也不等于每条授权材料都有效。验证失败保留证据和原因。治理观测还应保留原始事件摘要，并分别记录链下信任索引器的接收、解码、投影和对外提供时间；资源控制权由资源自身的签名和 DID 文档证明，节点授权由治理路径证明，二者应在指标和故障分析中分开。

20.7 网络级与节点级统计

Network 页面可同时呈现网络级聚合和节点级明细，但聚合来源、更新时间和缺失节点必须可追溯。所有周期统计采用同一时区、边界和去重规则；使用旧快照时携带快照时间和降级标识。聚合值不简单相加不同口径的累计值。

窗口	起点	终点	适用值
TODAY	当日零点	下一日零点	当日周期值
THIS WEEK	约定周起点	下一周期起点	周期值
THIS MONTH	当月一日	下月一日	月度周期值
ALL TIME	统计起始点	当前查询时刻	累计值

所有窗口使用同一时区和半开区间 `[start, end)`；数据缺失、请求失败和真实零值必须分别编码，不能用零填充缺失值。

20.7.1 网络级指标

网络级指标描述统计范围内的节点和资源，包括节点数量、注册、发布、发现、治理同步和网络状态。每项指标应说明纳入条件、去重方式、来源接口和更新时间；节点缺少数据时，聚合结果标明覆盖范围和缺失情况，不能把未上报节点按零计入。

20.7.2 节点级指标

节点级指标按节点 DID 或稳定节点标识记录健康状态、授权状态、资源处理量、同步游标、索引新鲜度和最近错误。节点离线、未授权、尚未同步以及统计接口失败使用不同状态，避免以零值或空字符串掩盖真实情况。

20.7.3 注册、发布和发现指标

注册、发布和发现指标分别表示注册服务节点受理、根平台可信发布和发现服务节点索引可见，不能合并为一个“成功数”。同一资源版本通过 DID、版本和发布序列关联，重试、重复索引和旧快照不应造成虚假增长。

20.7.4 指标时间窗口

时间窗口采用统一时区和明确边界；周期值使用固定起止时刻，累计值从定义的统计起点计算。接口、官网 Network 页面和后台报表必须采用同一口径，避免服务器时区、请求时间或缓存生成时间差异造成周期值异常。

20.7.5 数据缺失和降级展示

指标缺失、接口失败、尚未产生数据和旧快照展示分别标记。页面可以显示带时间的旧数据，但必须体现降级状态；后台仍记录真实失败，禁止把缺失转成零，或把快照当作实时健康结论。

20.8 链上治理事件可见性

治理事件展示应区分链上事件时间、信任索引器接收和索引时间、官网接口生成时间及页面显示时间。事件序列或链上位置用于排序和追溯，双语描述是展示层映射，不能替代原始事件。链上或索引服务暂不可用时，应显示不可用或延迟状态。

20.9 日志、追踪与关联标识符

跨服务记录至少关联请求标识、资源 DID、版本、发布序列、事件游标和节点标识，并记录时间、级别、结果和错误摘要。日志不得写入私钥或敏感请求内容；公开接口的错误信息应足够定位问题但不泄露内部配置。

接口层的关联字段应与协议类型保持对应：节点间请求使用 `requestId`、`requestNonce`、`bodyHash` 和 `upstreamAuth`；资源传播使用 `publicationCursor`、批序列和资源包版本；发现查询使用请求标识、发现节点 DID、候选数量、检索后端和耗时；链下信任索引器还应记录最后处理 `checkpoint`、治理序列和同步时间。建议把这些字段组织成结构化日志，而不是只拼接在一条自由文本消息中。

```
{
  "requestId": "req-example",
  "resourceDid": "did:oan:SKDM:example-resource",
  "packageVersion": "1.0.0",
  "publicationCursor": 42,
  "nodeDid": "did:oan:INDS:example-node",
  "operation": "discovery_query",
  "result": "completed",
  "elapsedMs": 18
}
```

上例只展示可用于关联处理过程的公开或低敏字段；生产日志仍须按部署策略过滤资源描述、端点凭证、签名材料和内部错误堆栈。`requestId` 便于跨服务追踪，但不能作为授权凭证，也不能替代资源控制证明或治理事件序列。

20.10 快照、缓存与降级模式行为

状态	前端行为	后台含义
live	展示最新响应	接口和数据正常
stale	展示快照并标识时间	后台刷新失败或数据过期
unavailable	展示不可用原因	接口不可达或返回错误
unknown	不作可信判断	缺少足够证据

快照覆盖必须满足“完整响应、结构校验、来源更新、时间更新”四项条件；部分响应不能覆盖完整旧数据。旧快照是用户体验机制，不是健康证明。前端可先显示浏览器侧快照，再后台刷新；成功后以完整新数据替换并更新来源和时间；失败时保留快照但显示降级状态，长期失败仍触发后台告警。无缓存首次访问才显示加载状态，不能让加载占位永久遮住已有数据。

快照记录应至少包含数据来源、生成时间、结构版本和数据状态，例如：

```
{
  "source": "official-discovery-summary",
  "capturedAt": "2026-09-10T10:00:00Z",
  "dataStatus": "stale",
  "schemaVersion": 1,
  "coverage": "official-nodes",
  "payload": { "resourceCount": 12 }
}
```

其中 `resourceCount: 12` 只是某次快照中的业务值，不代表当前实时数量。前端可以在刷新失败时继续展示它，但后台仍需记录刷新失败及其持续时间；当接口恢复时，只有通过结构校验的完整响应才能覆盖该快照。

20.10.1 旧快照加载

页面打开时先读取浏览器侧可用快照，并标明生成时间和来源。快照解析失败、结构版本不兼容或内容完整性校验失败时，应丢弃该快照并进入首次加载流程，不得把损坏数据渲染为正常统计。

20.10.2 后台数据刷新

旧快照显示后，前端在后台请求最新接口，不阻塞用户查看已有内容。请求应有超时和失败处理，刷新过程不改变用户正在查看的资源详情或查询结果，除非新响应通过完整结构校验。

20.10.3 新数据覆盖

只有完整且结构有效的新响应才覆盖旧快照，并同步更新缓存时间、来源和数据状态。新响应部分缺失时保留可用旧数据并标记降级，不能用空响应或零值覆盖有效内容。

20.10.4 长期失败与告警

连续失败、长期无数据和同步游标长期不增长应进入后台告警，记录接口、首个失败时间、最近成功时间和错误证据。页面继续展示快照不得抑制告警；恢复须由连续成功和业务检查确认。

20.10.5 无缓存时的首次加载

没有快照时才显示首次加载状态，并在成功、失败或超时后明确结束。失败状态应说明数据暂不可用，不得无限显示加载文字，也不得伪造零值或空的健康结论。

20.11 备份、恢复与灾难恢复

备份对象	恢复后核对	未通过时处理
节点身份和授权	DID、公钥、授权状态	停止节点对外服务
数据库	关键记录、迁移和索引	不启动写入任务
发布队列	未完成任务和幂等键	暂停重复投递
同步游标	游标连续性和上游序列	从可信位置补同步
网站产物	提交、哈希和配置对应	回退或重新构建

备份覆盖节点身份和授权材料、数据库、资源包、发布队列、同步游标、配置版本及网站部署产物；私钥和凭据单独加密并限制访问。恢复后先校验完整性、序列连续性和密钥对应关系，再启动写入任务，并通过健康检查、注册/发现烟测和索引追赶确认恢复。恢复顺序应先恢复配置和节点身份，再恢复数据库及其迁移、资源包和发布队列，随后校验同步游标、治理授权状态和网站构建产物，最后恢复对外写入。恢复过程要防止旧队列重复发布，也要防止游标倒退造成重复投影；可先以只读或暂停写入方式启动，核对 `resourceDid`、版本、发布序列和授权状态后再开放写入。`/health` 通过只说明进程恢复，不能单独证明历史索引、治理投影和未完成发布任务已经恢复。

20.12 部署烟测与功能验证

烟测最小闭环：

1. 记录部署提交、产物摘要和环境配置。
2. 检查各服务健康、数据库连接和索引游标。
3. 使用隔离身份完成注册及预期失败请求。
4. 检查根平台发布、分发和发现精确/语义查询。
5. 保存响应、日志、时间线和测试资源清理结果。烟测使用隔离测试身份和测试资源，保存提交时间、关联标识、响应、日志和清理结果。注册成功不能替代发现验证；还须检查根平台发布、内容分发、索引收敛和数据库状态。任一关键步骤失败、响应不确定或产生未清理的生产数据，都不得宣告部署成功。烟测结果应形成可复核的时间线，而不是只输出一个“通过”字符串。每一步记录目标地址或服务角色、提交号或产物摘要、请求标识、响应状态、耗时、关键响应字段和副作用清理结果；注册接口的受理成功、根平台发布成功和发现查询命中分别判定。对生产环境不适合写入真实资源时，应使用受控失败请求验证校验逻辑，并使用已有隔离测试资源验证发现路径。任何关键步骤结果不确定，都应保持部署状态为待核查。

20.12.1 注册功能烟测

使用隔离身份和受控资源验证请求校验、控制签名、注册服务节点受理、重复提交和错误响应。若接口返回注册 VC，检查其结构以及与资源 DID、版本或摘要的关联；测试结束后清理测试数据并保存证据。

20.12.2 发现功能烟测

使用已发布的隔离测试资源执行 DID 精确查询和语义查询，核对资源 DID、版本、状态、来源、发布证据和索引时间。结果为空、返回旧版本或接口失败时，结合发布记录和同步游标判断传播延迟还是功能故障。

20.12.3 节点健康检查

逐一检查根平台、注册服务节点、发现服务节点、内容分发平台、链下信任索引器和官网后端的存活、就绪及依赖状态，记录响应码、耗时、版本和检查时间，不能只检查进程是否存在。

20.12.4 数据库和索引器检查

检查数据库连接、迁移状态、关键记录和索引器游标，核对上游最新序列、本地游标和 `latest_sequence` 的关系及其连续变化，区分没有新事件与同步停止。

20.12.5 失败后的部署处理

关键烟测失败时停止发布确认，保留日志、提交号、产物哈希和测试资源信息，按修复或回滚流程处理。只有重试成功、数据副作用已清理且证据完整，才能重新出具验收结果。

20.13 数据保留、删除与隐私操作

公开指标只返回必要的聚合数据；访问统计、请求关联标识和运维日志应由后台权限控制。浏览器侧本地身份私钥、备份内容和用户提交的敏感材料不得写入访问统计。删除操作要说明在线数据、备份副本和不可变审计记录的不同保留规则。

20.14 运维告警与长期失败检测

告警记录应至少包括服务、接口、首次失败时间、最近成功时间、连续失败次数、错误摘要、影响范围和当前状态。

状态	触发依据	恢复条件
间歇失败	窗口内少量请求失败	后续请求连续成功
连续失败	连续请求超过阈值失败	健康和业务检查连续成功
长期停滞	指标或游标超过允许时长未更新	数据恢复且序列重新推进
依赖故障	上游或数据库不可用	依赖恢复并完成回归烟测

同一故障使用稳定指纹去重；恢复告警必须同时有连续健康检查 and 对应业务烟测，不能只依据进程重新启动。长期失败检测覆盖 `summary`、治理同步、发布队列和索引游标停滞。告警记录服务、接口、连续失败次数、首个失败时间、最近成功时间、错误摘要和影响范围，并对同一故障去重；恢复需有连续成功和业务验证。网站继续显示旧快照时，后台告警不得被抑制。`summary` 接口等周期性检查应保存“最近一次成功”和“最近一次失败”两条事实，而不是只保留当前状态。告警事件至少关联服务角色、接口、时间窗口、连续失败次数、最近成功序列或时间、错误摘要和影响范围；同一接口的短暂网络抖动可合并为一个故障指纹，持续失败则升级处理。恢复时应先确认接口连续成功，再确认数据时间或游标重新推进，最后执行受影响的注册/发现业务烟测。浏览器继续显示旧快照不能暂停或清除这条告警。

20.15 公共服务状态与维护通信

公共状态信息说明受影响服务、开始时间、当前状态、数据延迟和下一次更新，不暴露密钥、内部地址或用户数据。维护期间可以保留只读查询或旧快照，但应标明时间点和不可用功能；恢复公告以实际检查和业务烟测为依据。

20.16 参考来源

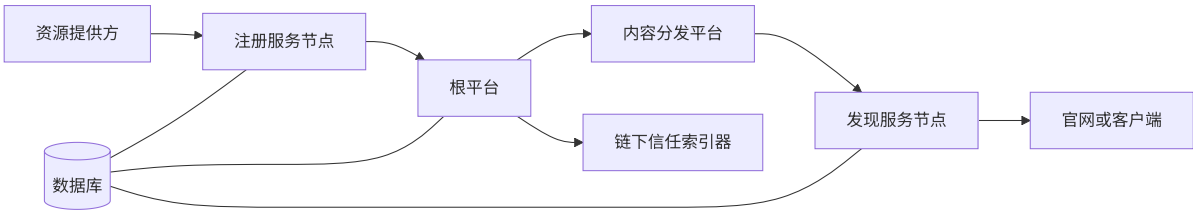
来源	类型	链接
oan-root-services	代码仓：根平台状态、发布和分发指标	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务状态、登记统计和健康接口	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：同步、索引、查询和新鲜度指标	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：同步游标、投影状态和治理指标	https://github.com/OpenAgenet/oan-trust-indexer
OAN Yellow Paper	arXiv 黄皮书：运行状态和信任证据边界	https://arxiv.org/abs/2606.03163

第二十一章 部署与符合性要求

本章说明部署符合性如何证明“部署的是什么、如何构建、依赖是否就绪、业务是否可用以及失败如何回退”。OpenAgenet (OAN) 的节点服务、网站后端和前端静态资源应分别记录源码提交、构建产物、配置版本和验收证据；官方部署值用于说明一个可运行实例，协议要求则适用于所有声称符合规范的实现。

21.1 参考部署拓扑

参考拓扑由根平台、注册服务节点、发现服务节点、内容分发平台、链下信任索引器、数据库和官网组成。对外只暴露经过 TLS 和反向代理保护的必要入口，节点间使用配置的服务地址和授权材料通信；第三方节点可以独立部署，但必须通过节点身份、授权、接口和同步符合性检查。



部署拓扑应把公网入口、内部服务、数据库和治理数据源分开标识；公开地址、端口和节点数量属于环境配置，不应被图示固化为协议必需项。

部署时可以按“公网入口、业务服务、状态存储、治理与同步”四个边界核对：官网和反向代理属于访问入口，节点服务承载注册、发布、分发或发现流程，数据库保存各自的运行状态，链下信任索引器读取治理事件并形成读模型。边界的具体主机和端口可以因环境变化，但每条跨边界连接都应能说明发起方、目标方、用途、认证材料和失败后的影响。

21.2 配置与秘密管理

配置类别	来源	部署检查	暴露边界
节点身份	受控密钥文件或密钥服务	DID、公钥和权限匹配	不进入公开接口
数据库	环境配置	连接、迁移和权限有效	不经网站代理暴露
上游端点	环境 profile	地址可达且协议匹配	仅必要服务访问
治理配置	授权配置和版本记录	包、对象、游标一致	不返回秘密

配置按环境注入，代码包不携带生产私钥、数据库凭据或外部服务令牌。节点 DID 私钥、根平台授权材料、数据库连接信息、链上包或对象配置、发现和分发端点应限制文件权限并单独备份；启动前校验必填项，轮换时保留旧新配置的审计记录。启动前失败项应阻止服务进入就绪状态；配置轮换应记录旧配置失效时间、新配置生效时间和验证结果。

配置检查可用以下最小记录作为发布前证据；值只表示存在和是否通过校验，不把秘密写入报告：

```
profile=production
service=oan-homepage-backend
OAN_HOMEPAGE_LISTEN_ADDR=present:valid
OAN_OFFICIAL_ROOT_ENDPOINT=present:https
OAN_OFFICIAL_REGISTRAR_ENDPOINT=present:https
OAN_OFFICIAL_DISCOVERY_ENDPOINT=present:https
database=present:connected
node_identity=present:did-public-key-match
secrets=not-printed
```

若必填端点、数据库、节点身份或治理配置无法校验，应停留在配置阶段，不应继续通过启动成功或页面可访问来掩盖缺项。

21.3 构建与发布产物

发布记录至少包含源代码版本、锁文件、Rust 和前端构建环境、构建步骤、产物哈希、配置版本和发布时间。部署可采用版本化源码包或等效方式同步已确认版本，在目标构建环境中生成 Rust 后端和前端静态资源；源码包、二进制、静态资源和配置应能相互对应。发布前应形成“源代码版本 → 构建 → 产物 → 部署”的可追溯链；目标环境中的临时源码修改或未记录的构建目录不能作为正式来源。

产物	来源	核对信息
源码归档	明确 Git 提交	提交、文件清单和归档摘要
Rust 二进制	锁定依赖和构建环境	产物哈希、版本和目标平台
前端静态资源	官网构建脚本	构建提交、资源清单和摘要
配置	环境独立材料	配置版本或指纹，不公开秘密

发布记录可以用一个不含秘密的摘要描述产物之间的对应关系：

```
{
  "sourceCommit": "<git-commit>",
  "lockfile": "<lockfile-sha256>",
  "backendArtifact": "<backend-sha256>",
  "frontendArtifact": "<frontend-sha256>",
  "configFingerprint": "<config-fingerprint>",
  "builtAt": "<utc-time>",
  "environment": "<profile>"
}
```

尖括号中的内容是记录模板，不是 OAN 官方固定值。实际发布时应替换为真实摘要，并把源码归档、构建日志、产物哈希和服务器部署记录放在同一验收记录中。

21.4 数据库与存储要求

部署前确认数据库版本、连接、迁移和索引状态，迁移必须幂等、可记录并先备份。存储应覆盖资源登记、版本、凭证关联、发布任务、同步游标、治理事件和统计所需数据；恢复后检查关键记

录、序列连续性、队列状态和索引一致性，不能只检查进程启动。

数据库验收应将“能连接”“迁移完成”“数据可用”和“恢复后仍一致”分别记录。重点核对资源 DID 与版本的唯一性、发布任务的幂等键、发现索引所需记录、治理事件与同步游标的连续性；迁移完成但关键索引缺失时，服务不应进入可写就绪状态。数据库检查不应通过官网公开接口返回内部表结构，报告中只保留必要的版本、计数、摘要和校验结果。

21.5 TLS、域名、反向代理与网络暴露

域名和 TLS 终止点应与实际服务入口一致，反向代理只转发必要路径并保留正确的 Host、协议和客户端关联信息。公开端口、跨域来源、防火墙和节点间访问规则应按环境配置；数据库、管理接口和内部队列不得因网站代理配置而暴露给普通互联网用户。验收时应分别从公网和内部网络检查允许路径、证书主机名、HTTP 到 HTTPS 跳转、CORS 范围和非公开路径的拒绝结果。

21.6 部署顺序与依赖检查

推荐顺序是记录基线、备份和同步代码，构建产物，准备数据库，启动根平台及依赖服务，再启动注册、分发、发现和索引器，最后部署官网并执行业务烟测。任一必需依赖、迁移、健康检查或关键烟测失败，都应停在当前步骤，保留证据并进入修复或回滚。每个步骤的通过条件应由下一步骤实际需要的依赖来定义，不能只以服务进程已启动作为放行条件。

21.6.1 代码同步与构建

以明确的源代码版本生成归档包或等效发布包，传送到目标构建环境后校验文件和版本信息。按项目锁定的 Rust 依赖编译节点服务，按前端构建脚本生成静态资源和后端产物；不从工作区临时文件或未记录的本地构建目录发布。

21.6.2 数据库准备

确认备份可恢复、数据库可连接、迁移版本和必需索引正确，再执行幂等迁移或启动校验。迁移失败不得启动会写入新数据的服务；需要人工处理时保存失败日志、数据库版本和恢复点。

21.6.3 节点启动顺序

先启动根平台及其数据库和发布依赖，再启动注册服务节点、内容分发平台、发现服务节点和链下信任索引器。实际顺序可按部署拓扑调整，但下游启动前必须确认上游地址、身份、授权和接口已就绪，避免以进程存在代替依赖可用。

21.6.4 服务健康检查

逐一检查进程存活、就绪接口、关键依赖和配置加载结果，并记录响应码、耗时、版本和时间。检查通过不代表业务路径通过，尤其要分别验证根平台发布、注册受理、发现查询、分发和索引同步。

21.6.5 部署后功能验证

部署后功能验证至少覆盖：

- 1. 健康和就绪接口。
- 2. 注册服务节点的合法请求与预期失败请求。
- 3. 根平台发布、内容分发和发现服务节点同步。
- 4. DID 精确查询和语义查询。
- 5. 数据库迁移状态、游标、日志和告警。

任一关键项失败时保留提交、产物、配置和响应证据，并停止发布完成确认。

接口验证应同时记录请求方法、实际 URL、HTTP 状态、响应类型、关键业务字段和关联标识。对于动态 JSON 接口，不能只检查响应为 2xx，还要检查注册响应是否包含资源 DID 和登记状态、根平台响应是否为 `resource-verified-and-queued`、内容分发响应是否包含发布游标、发现响应是否可反序列化为 `ResourceDiscoveryResponse`，以及索引器状态是否能说明当前同步位置。浏览器经官网代理访问时，还应额外验证代理路径与节点原始路径的映射没有改变请求体或错误语义。

21.7 注册节点功能烟测

使用隔离测试身份和受控资源验证表单或请求校验、控制签名、注册服务节点受理、重复提交和错误响应。若接口返回注册 VC，检查 VC 与资源 DID、版本或摘要的关联；测试资源的后续发布和清理不得污染正常资源统计。注册成功只证明注册服务节点完成受理，不自动证明根平台已经发布或发现服务节点已经索引。

验收步骤	结果	失败处置
合法注册	受理结果和关联标识可验证	停止确认并保留请求证据
重复注册	幂等，不新增错误登记事实	检查去重和数据库变化
错误签名	明确拒绝且原状态不变	检查无副作用
VC 结果	若返回则与 DID、版本或摘要一致	标记缺失或无效，不伪造成功
清理	测试资源可识别并清理	进入人工复核

21.8 发现节点功能烟测

对已发布的隔离测试资源执行 DID 精确查询和语义查询，核对 DID、资源类型、版本、状态、来源、根平台可信发布证明状态和索引时间。查询结果为空、返回旧版本或接口失败时，应结合游标和发布记录判断是传播延迟还是功能故障。语义查询的通过条件是返回可解释的候选和来源，不是要求某个固定排序或未经定义的相关性分数。

21.9 根平台、分发与索引器验证

分别检查根平台的发布状态、内容分发平台的资源或游标记录、链下信任索引器的同步状态和发现节点的索引结果。每一项检查都要给出来源、时间和正常判据；只看到根平台成功不能证明下游已经可发现。应使用同一个隔离资源版本贯穿核对，并记录根平台发布序列、分发摘要、索引版本和查询结果，避免以各接口的总数量相等替代内容一致性检查。

21.10 治理同步验证

选取可确认的治理事件，核对链上位置、事件类型、信任索引器游标、节点状态读模型和更新时间。连续两次或多次检查中游标和 `latest_sequence` 的关系应符合部署环境的同步状态；无新事件与同步停止必须区分。治理事件已被读取但投影未更新时，应判为同步链路未完成，而不是仅凭索引器接口响应成功放行。

21.11 升级、回滚与兼容性检查

升级前后检查数据库迁移、协议版本、资源模式、历史版本、节点互操作和回滚路径。升级只在兼容性和备份条件满足时继续；若新产物无法通过健康或业务验证，应按记录的版本和配置回退，并重新执行关键验收。升级验证应使用代表性的资源 DID、资源包、注册请求和发现查询，而不只验证服务能启动。

21.11.1 升级前检查

记录当前软件版本、二进制哈希、配置、数据库和游标状态，确认备份可读、目标版本可构建、迁移可回退或可恢复，并确认旧版本仍可取得。

21.11.2 数据和配置兼容

核对新增字段、资源包和凭证结构、协议路由、环境变量、节点授权材料及前端接口期望。兼容转换应可重复执行；不能兼容的历史数据必须在发布前停止，而不能在运行中静默丢弃。

21.11.3 回滚条件

出现数据库迁移失败、关键健康检查失败、注册或发现测试失败、发布队列异常、索引持续停滞或安全配置错误时触发回滚评估。涉及不可逆数据迁移时先暂停写入并按恢复方案处理，不能只替换二进制。

21.11.4 回滚后验证

回滚后重新检查服务版本、配置、数据库记录、节点身份、发布序列和同步游标，再执行健康、注册、发现、根平台、分发和索引验证。只有旧版本恢复到可证明状态，才能结束回滚。

21.12 第三方节点接入与符合性测试

第三方节点应提交节点身份、授权材料、公开端点、支持的接口和环境信息，并通过签名验证、授权检查、协议响应、资源发布、同步、查询、安全边界和故障恢复测试。通过测试只授予明确范围内的接入资格，不替代后续运行监测和治理管理。准入报告应明确节点角色、授权域、接口版本、测试环境和有效期；未测试的角色、域或接口不能由其它通过项推定为合格。

第三方节点的最低验收证据可以按以下顺序组织：

```
node identity -> authorization -> protocol response
               -> controlled registration/publication
               -> synchronization/discovery
```

-> failure recovery and evidence review

测试应优先在独立数据库和隔离端点中完成。节点能够调用接口不等于获得授权，能够同步数据也不等于可以发布资源；每一项结论都应绑定实际测试范围和证据。

21.13 构建可复现性与发布来源证明

构建记录应能从提交和锁文件追溯到产物，至少保存构建环境、命令、依赖版本、产物哈希和操作者。服务器上的运行二进制、前端静态资源和源码归档应能相互核验；配置和秘密不纳入公开产物哈希，但应记录其版本或指纹。对于 Rust 后端和官网前端，应分别保存构建结果，再在部署记录中说明二者如何组合为同一发布批次。

21.14 环境特定配置文件

开发、测试、预发布和生产环境使用独立 profile、端点、数据库和密钥，禁止把测试授权材料或生产私钥交叉复用。配置变更应有提交、审批或操作记录；同一代码在不同环境的差异应集中在配置，而不应通过未记录的手工改源码实现。验收时应核对当前 profile 与端点、数据库、节点身份和治理对象相互匹配，避免将测试环境的“通过”误写为生产环境通过。

21.15 部署后功能与安全验证

验收记录汇总版本、环境、配置、健康接口、业务烟测、数据库、治理同步、权限、TLS、公开端口和日志检查结果。还应确认私钥未进入日志或公开接口、管理数据未暴露给普通用户、错误响应未泄露秘密，并保留可复核的命令输出或接口响应摘要。最终结论应明确覆盖范围、证据时间、未覆盖项目和是否允许对外提供服务；任何关键项为 FAIL 或结果不确定时，不应写成整体通过。

21.16 参考来源

来源	类型	链接
oan-root-services	代码仓：根平台服务和部署配置	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务节点和部署配置	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现服务节点和部署配置	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：信任索引器和链端点配置	https://github.com/OpenAgenet/oan-trust-indexer
oan-protocol-common	代码仓：跨服务协议、配置相关类型和错误契约	https://github.com/wolfbrother/oan-protocol-common
OAN Yellow Paper	arXiv 黄皮书：部署、信任和验证边界	https://arxiv.org/abs/2606.03163

第二十二章 外部协议与生态映射

本章把外部协议接入理解作为一种可追溯的转换过程：先识别外部对象和版本，再把可核对的字段映射到 OpenAgenet (OAN) 资源元数据或资源包，随后分别验证资源控制权、注册凭证、根平台可信发布证明和治理状态，最后向调用方解释哪些信息可用于发现、哪些信息还需要在调用前复核。适配器应保留外部原文、来源、摘要和转换版本，不能把转换后的摘要直接当作 OAN 的可信发布事实。

外部材料	OAN 承载位置	可用于发现的内容	必须独立验证的内容
DID Core / DID 文档	资源 DID 文档及验证方法	id、验证方法、服务端点	控制关系、签名、状态和发布证据
A2A Agent Card	资源元数据、协议绑定或关联描述	能力、技能、用例、输入输出、交互入口	Card 发布者与 OAN 控制权、注册凭证和根平台可信发布证明
MCP 服务元数据	MCP 服务资源包及其协议绑定	服务器能力、工具清单、传输和认证声明	服务端点、版本、资源包哈希和生命周期
OpenAPI 或工具 API 描述	工具 API 资源包及其协议绑定	操作、参数、schema、错误和端点	文档摘要、控制签名、业务授权和实际可用性

映射结果回答的是“外部系统如何描述和连接这项资源”，不直接回答“该资源是否已经获得 OAN 网络授权”。后一个判断必须回到 OAN 的 DID、签名、注册凭证、根平台可信发布证明和治理状态。

22.1 W3C DID Core 与 did:oan

适配器应将 DID Core 通用字段和 did:oan 方法字段分层保存。通用字段用于让 DID 工具理解文档，OAN 字段用于表达资源类型、能力、版本、资源包和发布关系。下面的对象只是映射记录示例，不是新的 DID 文档：

```
{
  "source": {"type": "did-document", "version": "did-core-compatible"},
  "oan": {"method": "did:oan", "resourceDid": "did:oan:SKDM:<identifier>"},
  "mapping": {
    "preserved": ["id", "verificationMethod", "service"],
    "mapped": ["resourceType", "capabilityTags", "protocolBindings"]
  }
}
```

该记录只说明转换过程。did:oan 使用 DID Core 的通用身份和服务表达方式，并通过 OAN 的资源元数据、资源包及关联证明补充资源发现和发布所需的信息。

22.1.1 DID Core 数据结构对应

DID 文档中的 `id`、`verificationMethod`、控制关系和 `service` 对应 DID Core 的通用数据模型。OAN 的资源类型、能力标签、用例、版本、资源包引用和发布证明关联属于 OAN 资源模型或方法扩展。适配器应保留原字段和原始路径，不应把 OAN 扩展字段伪装成 DID Core 的标准字段。

DID Core 字段	OAN 对应内容	核验重点
<code>id</code>	资源或节点 DID	不因名称、URL 或展示格式改写
<code>verificationMethod</code>	控制方或节点验证方法	公钥、控制关系和签名对象一致
<code>authentication / assertionMethod</code>	认证与断言用途	不将任意公钥加入授权关系
<code>service</code>	资源端点和协议入口	可读取不等于可调用

22.1.2 did:oan 方法扩展

`did:oan` 方法扩展用于表达资源主体、资源类型、能力集合、端点、协议绑定、版本引用及发布证据关联。方法扩展必须明确字段类型、必填条件、规范化方式和签名覆盖范围，扩展字段变化应进入 DID 文档或资源包版本管理。

方法扩展的检查顺序为：

- 1. 解析方法名和方法特定标识符。
- 2. 根据主体代码核对资源类型或节点角色。
- 3. 核对元数据、服务、版本和资源包引用。
- 4. 独立验证控制证明、哈希、注册 VC 和根平台可信发布证明。

影响签名覆盖范围、资源类型、版本选择或治理判断的扩展，应按破坏性变更进入第二十五章的变更记录。

22.1.3 解析行为

解析器接收完整 `did:oan` 后，先校验方法名和方法特定标识符，再读取对应 DID 文档并验证结构、控制关系、状态和版本。未找到、方法不支持、文档无效、证明缺失和状态未知应分别返回，不能以空文档或最近缓存掩盖错误。

解析状态	含义	后续处理
<code>resolved</code>	文档可读取且结构合法	继续验证控制、状态和发布证据
<code>not-found</code>	没有找到对应文档	不以同名文件替代
<code>method-unsupported</code>	解析器不支持该方法	停止高风险调用
<code>invalid-document</code>	结构或绑定不合法	拒绝并留存错误证据
<code>status-unknown</code>	状态来源过期或不可用	降级或人工复核

22.1.4 互操作边界

DID Core 互操作保证通用身份结构可以被理解，不保证外部解析器理解 OAN 的资源语义、根平台可信发布证明或治理状态。外部系统可读取 DID 文档并获得入口，但在调用前仍需按 OAN 规则验证资源版本、发布证据和端点。

互操作报告应分别记录格式互操作和 OAN 语义互操作：前者证明外部解析器能够读取通用字段，后者证明调用方能够理解资源类型、能力、版本、资源包和发布证据。两者不能合并为单一的“兼容”结论。

22.2 A2A Agent Card 映射

OAN 与 A2A Agent Card 的关系可按“网络级资源发现”和“服务级调用描述”区分：OAN 通过 DID 文档、资源元数据和资源包提供稳定身份、来源、版本及治理关联；A2A Agent Card 以服务提供方、支持的交互入口、能力、技能、输入输出模式和安全声明描述智能体服务。二者可以互相引用，但映射不会改变任一方的身份、签名和授权语义。

映射方向	保留内容	不能推导的结论
OAN → A2A	资源 DID、名称、描述、能力标签、用例、服务端点、协议绑定和版本	不能由映射自动取得 A2A 会话授权或业务权限
A2A → OAN	服务提供方、能力、技能、输入输出模式、安全声明和原文摘要	不能由 Card 自动取得 OAN 注册资格或根平台可信发布证明

OAN 资源档案中的能力描述、用例、端点和协议绑定可以为 A2A Agent Card 提供可信的发现入口；反向接入时，Agent Card 只能作为注册草稿或补充描述的来源，仍需由资源控制方按 OAN 流程生成和签署资源材料。详细调用流程由 A2A、MCP、工具 API 或资源自身协议决定。

22.2.1 能力描述映射

OAN 的 `capabilityTags`、`capabilityDescription` 和用例信息可映射到 A2A Agent Card 的 `skills`、技能名称、描述和示例任务。映射时保留标签树路径、原始文本、资源包版本和来源；OAN 发现节点的语义匹配结果可以帮助定位候选资源，但不能证明资源已获授权或一定具备所声明能力。

建议同时保留规范化标签和原始描述：

```
{
  "tag": "code.repository",
  "sourcePath": "skills[0].name",
  "sourceVersion": "1.0",
  "resourcePackageVersion": "1.0.0"
}
```

规范化标签服务于筛选和语义治理，原始文本服务于解释和复核。

22.2.2 用例和技能映射

OAN 的用例字段和技能包信息可用于生成 Agent Card 的技能摘要、示例任务和输入输出说明。生成结果应保留资源包版本和字段来源；若外部 Card 只提供摘要，反向登记时只能形成待核对的资源描述，不能补齐 OAN 中没有声明的控制、版本、端点或治理信息。

用例字段	应表达的事实	缺失处理
任务	用户希望完成的工作	标记描述不完整
输入	类型、来源和前提	不凭空补写
输出	结果形态或交付物	不把标签当承诺

用例字段	应表达的事实	缺失处理
限制	权限、成本、时效和数据边界	调用前复核

22.2.3 服务端点映射

OAN DID 文档或资源包中的服务端点、协议类型、传输方式和认证要求可映射到 Agent Card 的 `supportedInterfaces`、协议绑定和安全声明。端点映射只提供连接准备信息，不代表端点可用；调用方仍需执行 TLS、认证、版本、状态和业务授权检查。端点映射可按“入口、协议、传输、认证、版本、状态”逐项记录；其中任何一项来自外部描述时，都应保留原始路径或摘要，便于在调用前重新核对。

端点信息	发现阶段用途	调用前复核
入口 URL	定位服务	TLS、域名和可达性
协议/传输	选择客户端	版本和协商结果
认证要求	准备凭据	用户授权和权限范围
版本	选择兼容实现	是否仍受支持
状态	提示可用性	最新状态和治理证据

22.2.4 身份、签名和可信证据映射

Agent Card 的 `provider`、签名或安全声明可以作为外部输入保存，但 OAN 的资源身份以 `did: oan` 和其控制关系为准。Agent Card 整体签名证明的是 Card 内容及其发布者声明；注册 VC、根平台可信发布证明和治理状态证明的是 OAN 流程中的不同事实，必须分别保留，不能互相替代。身份和证据映射应分层保存：

- 资源控制关系回答“谁能控制该 DID”。
- Agent Card 或外部声明回答“外部系统如何描述该资源”。
- 注册 VC 回答“注册服务节点声明了什么登记事实”。
- 根平台可信发布证明回答“根平台验证并发布了什么内容”。

22.3 MCP 资源与服务器元数据映射

OAN 将 MCP 服务作为可发现资源。映射时按“服务器、工具、调用”三层记录：服务器层描述身份、协议版本、传输和端点；工具层描述工具或资源清单、名称、说明和输入 `schema`；调用层描述会话、认证和错误约束。资源包应保存清单原文或稳定引用、版本和哈希，发现索引可以只保存检索摘要。一次 MCP 初始化成功只说明服务能够完成某次协议交互，不等于根平台已经验证并发布该资源。

22.3.1 MCP Server 映射

MCP Server 的名称、说明、协议版本、传输方式、端点和服务器能力映射到 OAN MCP 服务资源的元数据。服务器 DID、控制证明、资源版本、资源包哈希和根平台可信发布证明独立保存，以区分“服务器自我描述”和“网络已验证发布”。每条工具记录还应保存所属服务器和资源包版本，避免仅凭工具名称跨服务去重。

22.3.2 工具定义映射

MCP 工具名称、说明、输入 schema 和可用能力可映射到 OAN 的工具条目、能力描述和标签。工具条目应带有所属服务器、资源包版本、内容摘要和原始清单位置；工具名称相同不等于属于同一控制主体。发现索引可以使用摘要字段，但调用方应能回到完整清单核验工具边界。

22.3.3 输入输出约束映射

MCP 输入 schema、结果结构和错误约束映射到 OAN 的输入输出描述，并保留 schema 原文、媒体类型、版本和哈希。映射失败或字段不完整时，资源可以登记为描述不完整，但不能向调用方宣称约束已经验证。输入输出映射可用 schema 版本和摘要建立稳定引用；字段类型、必填关系、枚举值或错误结构发生变化时，应把它视为新的描述版本，而不是只更新展示文本。

22.3.4 调用和认证信息映射

MCP 传输、会话建立、认证要求和调用前条件可作为 OAN 服务信息的一部分，用于发现后的连接准备。OAN 的发布证明只证明声明材料及其来源，不能替 MCP 服务器授予用户业务权限或保证会话成功。调用和认证映射应区分“发现时可见的接入条件”和“运行时由服务端授予的权限”。OAN 只登记和验证声明及其来源，不代替 MCP 服务或工具 API 的业务鉴权。

22.4 工具与 API 描述映射

OpenAPI 映射建议以路径和操作为粒度建立稳定条目：

```
operationId: searchRepository
source: openapi.yaml#/paths/~1repositories~1search/get
oanResourceDid: did:oan:TLDM:<identifier>
schemaHash: sha256:<value>
```

规范化摘要可用于发现和去重，原始文档用于调用前复核。

工具 API 应以路径和操作为最小审计单元，至少保留 operationId、原始文档路径、服务器端点、schema 版本、认证要求和摘要。规范化摘要用于发现和去重，原始文档用于调用前复核，文档可读不等于接口可调用。

22.5 VC 与 VP 生态互操作

VC/VP 处理应采用“解析 → 密码学验证 → 状态检查 → 信任策略判断”的顺序。任何一步失败都要产生明确的中间状态，不能将“JSON 可解析”直接显示为“可信”。

阶段	输出	是否足以接受
解析	主体、签发者、类型和声明	否
签名验证	签名与公钥匹配	否，还需检查关系
状态检查	有效期、暂停/撤销状态	否，还需检查授权范围
OAN 信任策略	结合来源和治理作出决定	是/否/需复核

适配器可以解析外部 VC/VP 的主体、签发者、凭证类型、有效期、状态和证明材料，并将原始对象及验证结果保留。能够解析只表示格式可读；是否接受还要看签发者授权、控制关系、治理

状态和 OAN 的信任策略。对于无法取得状态信息、签名验证失败或签发者关系不明的输入，应保留失败原因并进入拒绝或人工复核路径。

22.6 外部组织与平台身份映射

外部组织、平台账号、域名或其 DID 可以作为资源提供方、运营方、委托方或端点托管方的关联信息保存，但应记录关系类型、证明来源、责任范围和有效期。域名、企业账号、平台认证和资源控制 DID 不是同一字段，外部账号名称不能替代资源 DID 控制权，域名或平台认证也不能直接产生根平台发布资格。

22.7 外部凭证与信任证据映射

外部证据记录建议包含 `issuer`、`subject`、`credentialType`、`issuedAt`、`expiresAt`、`statusSource`、`sourceHash`、`verifiedAt` 和 `verifier`。缺少状态端点时应记录“状态未检查”，而不是默认有效。OAN 将其作为补充证据，但节点授权、资源控制和根平台发布仍按各自的 OAN 证据链判定。

22.8 字段来源、版本与哈希要求

字段来源可用如下审计记录表示。示例中的 `agent-card.json`、摘要和时间均为占位值：

```
{
  "field": "capabilityTags[0]",
  "source": "agent-card.json#/skills/0",
  "sourceHash": "sha256:<value>",
  "mappingVersion": "oan-a2a-v1",
  "verifiedAt": "<timestamp>"
}
```

每个映射字段应记录来源对象、来源路径、原始版本、规范化版本、签名主体和内容摘要。规范化操作应可重放；排序、大小写、别名或字段裁剪改变摘要时，必须更新摘要和映射版本，并保留旧结果供审计。

22.9 兼容性配置文件与迁移路径

互操作 profile 应明确支持的 DID、A2A Agent Card、MCP、OpenAPI 版本、必选和可选字段、签名算法、传输、错误策略及迁移方式。迁移时先保留原始证据，双读旧、新 profile，核对字段摘要和签名边界，再切换单写；失败时回到旧 profile，不覆盖唯一来源。profile 本身应有名称和版本，便于调用方知道采用的是哪一组映射规则。

22.10 外部协议能力协商

协商结果可分为 `compatible`、`partially-compatible` 和 `incompatible`，并同时给出共同协议、版本、传输和安全能力，以及未满足的能力：

```
{
  "result": "partially-compatible",
  "commonProtocols": ["https"],
  "missing": ["required-auth-scheme"]
}
```

客户端或适配器声明支持的协议、版本、传输和安全能力，依据双方交集选择可用 profile；没有共同能力时返回明确的不兼容结果。协商成功只表示格式和交互能力可用，仍须单独验证资源身份、发布证据、治理状态和业务权限。

22.11 外部签名与证据保留

适配器输出应将原文、规范化摘要、外部签名和 OAN 签名分开存储，并为每份材料设置来源标识。重新打包或转发时，不能只保留转换后的对象；至少应能回答“原始对象是什么、谁签署、签署了什么、转换是否改变内容、OAN 哪一方重新签署”。适配过程中保留外部签名、凭证引用、原始对象、版本、摘要、验证时间、验证器和失败原因。重新编码或生成 OAN 资源包时，不能丢失外部材料与 OAN 控制签名之间的边界；两类签名分别说明覆盖的事实。

适配器输出应分开保存原始对象、规范化对象、外部签名、资源控制签名、注册 VC 和根平台可信发布证明。审计时应能回答：原始对象是什么、谁签署、签署了什么、转换是否改变内容、哪一方重新签署以及验证发生在何时。

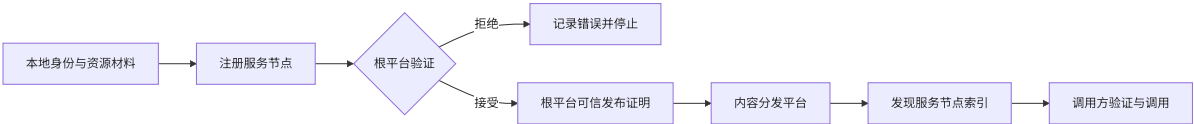
22.12 参考来源

来源	类型	链接
oan-protocol-common	代码仓：资源元数据、协议类型和外部描述绑定	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓：客户端资源材料和协议映射辅助	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓：社区接入脚本和外部资源注册流程	https://github.com/OpenAgenet/oan-community-skill
OAN White Paper	arXiv 白皮书：生态互操作和开放基础设施定位	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv 黄皮书：资源、信任和验证机制	https://arxiv.org/abs/2606.03163
Agentic Overlay Network Architecture	IETF 草案：智能体网络架构映射	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF 草案：发现元数据和查询 profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

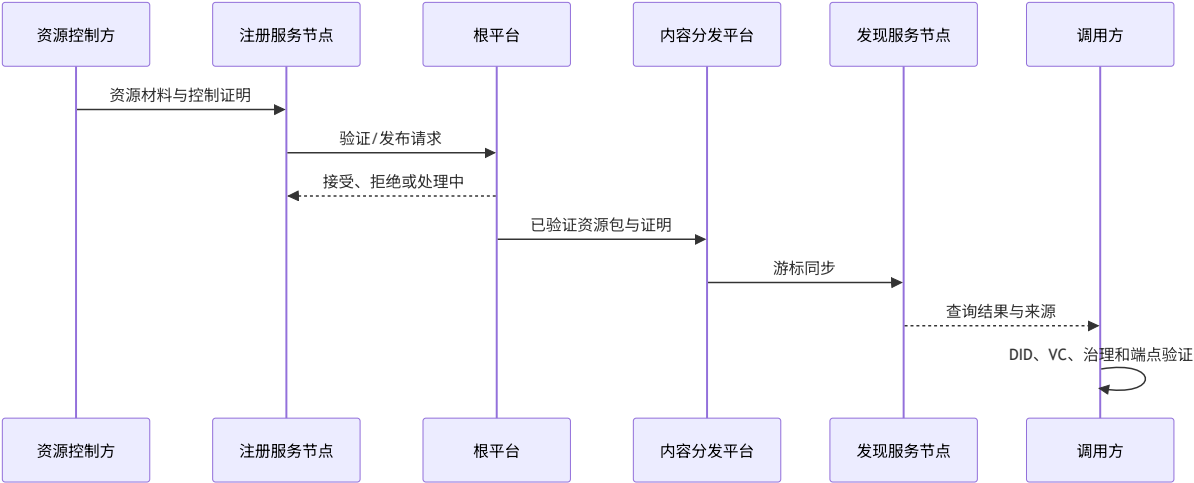
第二十三章 端到端参考流程

本章的流程统一采用“准备、提交、验证、传播、发现、调用、更新或恢复”的状态链。参考实现中的接口名称、网站按钮和部署地址属于示例环境；规范不要求所有实现采用同一界面，但要求关键事实可以被独立验证。端到端记录至少关联资源 DID、资源类型、版本、请求标识、来源节点、观察时间和证据摘要。流程中的“完成”应对应事实和证据为准：注册服务节点返回受理结果、根平台形成发布事实、内容分发平台完成材料传输、发现服务节点建立索引，分别是不同阶段，不能合并为一个成功状态。

阶段	主要事实	合格输出
准备	本地身份、DID 文档、资源元数据和资源包	可验证的控制材料
注册	注册服务节点受理和上游提交	登记结果、VC（如有）和关联 ID
发布	根平台验证并形成发布事实	根平台可信发布证明
传播	内容分发平台传输并记录游标	可校验的资源包和同步结果
发现	发现服务节点验证并建立索引	带来源和新鲜度的候选结果
调用	调用方重新验证并执行本地策略	允许或拒绝调用的决定



本章以“本地身份和测试资源已准备、节点端点可访问、测试环境与生产环境隔离”为前提，给出从资源创建、注册、可信发布、发现、验证到更新和故障处理的参考路径。示例展示可复现的交互关系，不把某个官网按钮、当前部署地址或一次成功响应扩大为所有实现的强制接口。执行时应将测试资源、测试主体和请求标识与生产资源隔离，并保存请求材料、响应摘要、DID 文档、注册凭证、根平台可信发布证明、同步游标和查询结果之间的关联。



每一步都应记录输入来源、事实产生者、验证者、状态变化和证据；页面提示或单个 HTTP 成功响应不能代表端到端完成。

23.1 创建本地身份

客户端在本地生成主体密钥和主体 DID，建立本地身份记录，并导出带时间标识的身份备份文件。注册智能体服务、技能包、MCP 服务或工具 API 时，再由该主体在本地生成或关联资源身份和资源控制证明。私钥只留在发布者控制范围内，不随普通注册请求发送；恢复时重新校验文件结构、公钥与私钥对应关系、主体 DID 一致性以及可否重新签名。验收证据包括 DID、公钥、备份摘要和恢复后的等价性，而不是把私钥上传成功作为结果。

阶段	本地动作	验收证据
生成	生成主体 DID、公钥和私钥	公钥与主体 DID 文档一致
建档	保存主体及受其控制的资源身份记录	记录结构和版本合法，控制关系清晰
备份	浏览器或客户端导出身份包	文件可解析，私钥未发送到节点
恢复	导入并校验密钥关系	恢复前后 DID、公钥和签名能力等价

23.2 注册智能体服务

以一个声明“代码仓库检索和项目结构总结”的智能体服务为例，发布者先准备资源 DID、能力标签、能力描述、用例、端点、协议声明、认证条件和版本，再使用本地控制密钥签署 DID 文档或资源包材料。客户端将公开注册材料和控制证明提交给注册服务节点；注册服务节点校验请求并向根平台提交，成功时返回登记结果和可能存在的注册 VC。随后还要等待根平台发布、内容分发和发现服务节点索引，分别取得相应状态和证据。

```
{
  "resourceType": "agent_service",
  "capabilityTags": ["code.repository", "analysis.summary"],
  "version": "1.0.0",
  "authorizedDomains": ["technology.software_engineering"]
}
```

注册材料、请求摘要、注册 VC（如有）、根平台状态、资源包摘要和发现结果应分别留存，不能把“提交成功”写成“已被全网发现”。同一资源再次提交时，应先判断资源 DID、版本和材料是否变化；相同版本的重复请求应按注册服务节点实际返回判断是否幂等，不能据页面提示推断产生了新版本。

23.3 注册技能包

技能包示例应给出名称、能力、输入输出、用例、依赖、版本、安装或调用入口、标签、主体 DID 和资源 DID，并计算资源包内容摘要。发布者先在本地完成材料规范化和签名，再向注册服务节点提交；提交后核对受理结果、资源类型、版本、摘要和注册凭证（如有），随后按 22.6 的顺序核对根平台发布和发现服务节点索引。技能包能够安装或在本地运行，不等于已经获得根平台发布或发现服务节点索引。

技能包验收同时检查两条线：**产品线**核对入口、依赖、输入输出和用例，**网络线**核对 DID、资源包版本、摘要、控制签名、登记、发布和索引状态。两条线的结果分开记录；产品线失败说明资源本身不可按声明使用，网络线失败说明资源尚未形成可验证的网络发布链路，不能用一条线的通过结果替代另一条线。

23.4 注册 MCP 服务

MCP 服务示例应准备服务器身份、工具或资源清单、传输方式、端点、认证方式、协议版本和兼容性声明。发布者将清单摘要、服务端点和资源版本纳入资源材料并在本地签署，注册服务节点验证字段和控制证明；调用方在发现后仍需按 MCP 流程建立会话，并按 OpenAgent (OAN) 规则核对 DID、资源包、发布证据和治理状态。

检查结果	表示的事实	不表示的事实
字段校验通过	注册服务节点能理解材料	根平台已发布
注册 VC 返回	登记事实有凭证	MCP 会话一定成功
根平台发布	特定资源版本形成网络发布事实	业务权限自动授予
MCP 会话成功	一次连接交互成功	资源持续可用或治理状态有效

23.5 注册工具 API

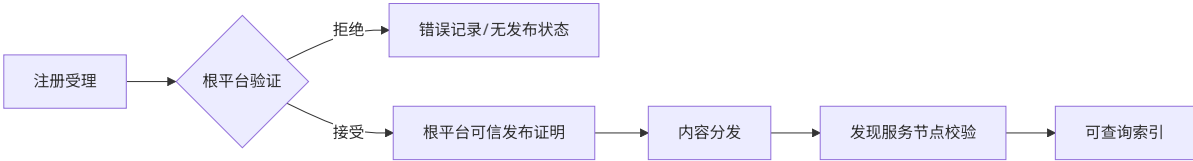
工具 API 以 OpenAPI 或等价描述作为输入，明确操作、参数、请求体、响应、错误、认证、服务器端点和版本。发布者先确定描述文件来源和版本，再将规范化描述摘要绑定到资源包并签名；注册服务节点验证材料，调用方可通过摘要确认取得的 API 契约未被替换。摘要验证不能证明接口当前可用，实际业务权限仍由 API 服务端决定。

```
operationId: searchRepository
source: openapi.yaml#/paths/~1repositories~1search/get
version: 1.0.0
schemaHash: sha256:<value>
```

原始描述、规范化摘要和 OAN 控制签名应分别保存；文档可读不等于接口可调用。

23.6 发布与分发资源

资源经注册服务节点受理后，根平台验证控制证明、节点授权、资源类型、版本、内容摘要和发布条件，形成根平台可信发布证明；内容分发平台按发布清单或游标保存、传输已验证材料，发现服务节点按游标接收、验签、校验并建立结构化和语义索引。每一步独立记录结果，不能以注册成功代替可发现，也不能以材料已分发代替索引已完成。



23.6.1 注册结果

注册结果至少包含资源 DID、资源类型、版本或摘要、受理状态、注册 VC（如接口返回）和关联标识。VC 为空时只能如实记录为空，不能自行伪造凭证；注册服务节点受理失败应保留错误原因，客户端可依据请求标识或协议允许的幂等方式重试。客户端还应记录注册结果由哪个节点返回以及观察时间，以免把后续异步发布状态混入登记结果。

```
{
  "resourceDid": "did:oon:AGDM:<identifier>",
  "status": "submitted",
  "registrationCredential": null,
  "observedAt": "<timestamp>"
}
```

`submitted` 仅表示登记阶段结果，不能解释为根平台发布或发现可见。

23.6.2 根平台发布

根平台核对注册服务节点授权、资源控制签名、资源包完整性、版本关系和治理条件，接受后记录发布序列并形成根平台可信发布证明。拒绝、重复或异步处理中均应有明确状态；根平台没有接受时，不得把内容分发或本地缓存标记为可信发布。发布证据至少要与资源 DID、版本和各项摘要绑定，重复处理不能产生互相冲突的当前版本。

根平台发布记录至少关联资源 DID、版本、DID 文档哈希、元数据哈希、包哈希、注册服务节点 DID、发布序列和处理状态；重复处理应保持幂等。

23.6.3 内容分发

内容分发平台按根平台发布清单或游标提供 DID 文档、资源包、摘要和证明材料，接收方校验完整性后记录接收游标。断点续传和重复投递应幂等；分发成功只说明材料可取得，不说明发现服务节点已经建立可查询索引。接收方发现摘要、版本或证明不一致时，应拒绝写入可见索引并保留失败材料的诊断信息。

分发校验顺序为“清单、内容、摘要、证明、游标”。发送端记录批次和游标，接收端先验证每项版本和摘要，再提交本地状态；传输成功不等于索引完成。

23.6.4 发现节点索引

发现服务节点接收材料后验证根平台可信发布证明、摘要、版本、治理状态和授权域，再写入结构化及语义索引。索引完成应有资源 DID、版本、来源、索引时间和游标证据；语义索引命中不能覆盖证明无效或资源已撤销。查询方应根据索引时间和同步游标判断结果是否新鲜，并在需要最新状态时回到可验证来源复核。

索引记录至少包含 `resourceDid`、版本、来源、授权域、生命周期、`indexedAt` 和同步游标。只有验证通过的材料进入可见索引，语义命中只提供检索入口。

23.7 按 DID 发现

调用方将完整资源 DID 作为精确查询条件提交发现服务节点，例如请求 `/discovery/resources/query` 时将 DID 放入查询字段。发现服务节点返回当前索引中的资源版本、摘要、来源、生命周期

状态、索引时间和根平台可信发布证明状态。得到结果后仍要执行调用前验证；未找到、节点未同步、索引过期和资源已撤销应区分处理，不能把空结果直接解释为资源不存在。

```
{
  "resourceDid": "did:oon:SKDM:<identifier>",
  "limit": 1
}
```

验收应区分精确命中、未找到、尚未同步、索引过期、暂停/撤销和查询错误，并保留查询节点、请求标识、观察时间和响应摘要。

23.8 按结构化条件发现

调用方按资源类型、主体类型、标签、协议、版本、状态和授权域组合条件查询，并明确字段编码、交集/并集关系、过滤顺序、分页和排序。发现服务节点先应用授权域和生命周期等硬条件，再进行标签、协议或文本相关性处理；不同接口的实际过滤顺序以其协议说明为准。结果中的标签匹配只说明检索条件满足，调用方仍需检查 DID、签名、凭证、发布证据、版本和端点。

条件	作用	示例
resourceType	限定资源类型	skill
protocol	限定协议	mcp
capabilityTags	能力匹配	code.repository
authorizedDomains	查询可见范围	technology. software_engineering
limit	数量边界	实现允许范围

23.9 按自然语言任务描述发现

调用方提交任务描述，例如 “I need a tool that can search code repositories and summarize the project structure.”。发现服务节点或其语义组件从描述中提取能力、用例、资源类型、协议、标签和状态条件，再执行硬条件过滤与语义排序；当前实现可能使用词法、结构化或向量/混合路径，具体取决于部署配置。返回结果时应保留命中资源 DID、资源版本、来源和解释字段（接口提供时），语义相关性不是控制权或安全授权。

```
任务描述 -> 能力/用例提取 -> 标签与结构化条件 -> 发现查询 -> 命中理由
```

解释结果应区分文本、标签、结构化过滤和语义相关性来源。

23.10 调用前验证已发现资源

调用方从发现结果取得 DID 文档、资源包、根平台可信发布证明、注册 VC 和状态信息，依次验证 DID 解析、控制关系、凭证、发布证据、治理授权、版本和端点，再根据本地策略决定是否调用。发现结果是候选资源和证据入口，不是调用授权；验证失败、状态未知、证据过期或端点认证条件不满足时，应停止高风险调用，并记录停止原因。

□ DID 和 DID 文档验证通过。

- ☐ 资源包、版本和摘要一致。
- ☐ 注册 VC 与根平台可信发布证明可验证。
- ☐ 治理状态和新鲜度满足策略。
- ☐ 端点、认证和业务权限满足调用条件。

调用方不应把发现接口返回的候选对象直接作为最终调用对象。候选通常是 `ResourceDiscoveryCandidate` 的索引表示，完整 DID 文档和资源包需要通过相应读取接口取得；内容分发平台的资源包读取返回 `ResourcePackage`，DID 文档读取返回 `DidDocument`，元数据读取返回 `ResourceMetadata`。读取成功后，再将资源 DID、版本、三类哈希、注册 VC、根平台可信发布证明和治理状态放在同一验证上下文中核对。

23.10.1 DID 解析

解析完整资源 DID，确认方法为 `did:oan`，读取 DID 文档并校验 `id`、验证方法、控制关系、服务端点和资源元数据一致性。解析未找到、方法不支持、文档结构无效和状态异常分别处理，不能以任意同名 JSON 替代。

解析结果至少应带出以下字段，供后续步骤继续核验：

字段	作用	缺失时处理
<code>did / id</code>	确认请求标识与文档标识一致	停止
<code>verificationMethod</code>	找到控制关系和公钥材料	停止控制权验证
<code>service</code>	找到资源访问或调用入口	标记端点不完整
<code>resourceMetadata</code>	对照资源类型、版本和能力	标记描述不完整
解析状态	区分成功、未找到和方法错误	不能默认为成功

23.10.2 控制权验证

依据 DID 文档中的验证方法和控制关系验证资源签名，确认签名覆盖的规范化内容、资源 DID、版本和前序关系。公钥与签名数学上一致不代表发布者有权控制另一个 DID，验证还必须绑定目标 DID 和既有控制关系。验证输入应来自当前取得的 DID 文档和资源包，而不是只使用发现结果中经过截断的展示字段。

控制权验证应按以下顺序完成：

- 从 DID 文档取得验证方法和公钥引用。
- 确认公钥属于目标 DID 的控制关系。
- 按声明的规范化规则重建签名输入。
- 验证签名覆盖的 DID、版本、资源包摘要和前序关系。
- 将验证结果与注册服务节点、根平台和发现服务节点返回的来源信息关联。

23.10.3 注册 VC 验证

验证注册 VC 的签发者、主体 DID、资源类型、版本或摘要、签名、有效期及吊销/暂停状态。VC 缺失或接口不返回 VC 时，按协议定义的未知或不完整处理，不能把页面提交成功文字当作凭证；注册 VC 只证明注册服务节点签发的登记事实，不自动证明根平台发布和发现索引已经完成。

检查对象	需要核对的事实	失败含义
签发者	是否为允许的注册服务节点或协议规定的签发者	凭证来源不可信
主体	是否指向当前资源 DID	凭证与资源不匹配
声明	类型、版本、摘要和登记事实	内容不能支撑当前版本
证明	签名、公钥和签名覆盖范围	凭证不可验证
状态	有效期、暂停或撤销信息	只能标记未知或拒绝

23.10.4 根平台可信发布证明验证

验证根平台可信发布证明的签发者、目标资源 DID、资源包版本、包摘要、元数据摘要、发布序列和签名，并确认根平台自身授权状态有效。证明只覆盖其声明事实，不保证端点业务质量。

根平台可信发布证明验证结果应与注册 VC 分开记录。下面是验证结果记录示例，状态值用于说明验证阶段，不是固定接口响应：

```
{
  "resourceDid": "did:oon:<method-specific-id>",
  "registrationCredential": "verified|missing|invalid",
  "rootPublicationProof": "verified|missing|invalid",
  "governanceStatus": "active|suspended|revoked|unknown"
}
```

只有证明之间的目标 DID、版本和摘要相互一致时，调用方才可以把“已登记”和“已由根平台发布”作为连续证据使用；其中任一项缺失都不能由另一项代替。发布证明验证通过后，还要根据发现服务节点的索引时间和游标判断是否已经可查询。

23.10.5 治理状态检查

从可信治理状态来源检查节点授权、授权域、暂停或撤销事件及其新鲜度，记录链上位置、索引游标和检查时间。治理数据过期或来源不可用时，对敏感调用采取拒绝或人工复核，而不是默认通过。

治理检查至少输出以下判断，而不是只返回一个布尔值：

判断	含义
授权资格	当前节点是否具有对应角色和授权范围
资源状态	目标资源是否暂停、撤销或仍有效
数据新鲜度	事件序列、索引游标和最近成功时间是否可接受
证据位置	状态对应的事件、交易或索引记录
调用决策	允许、拒绝、降级展示或人工复核

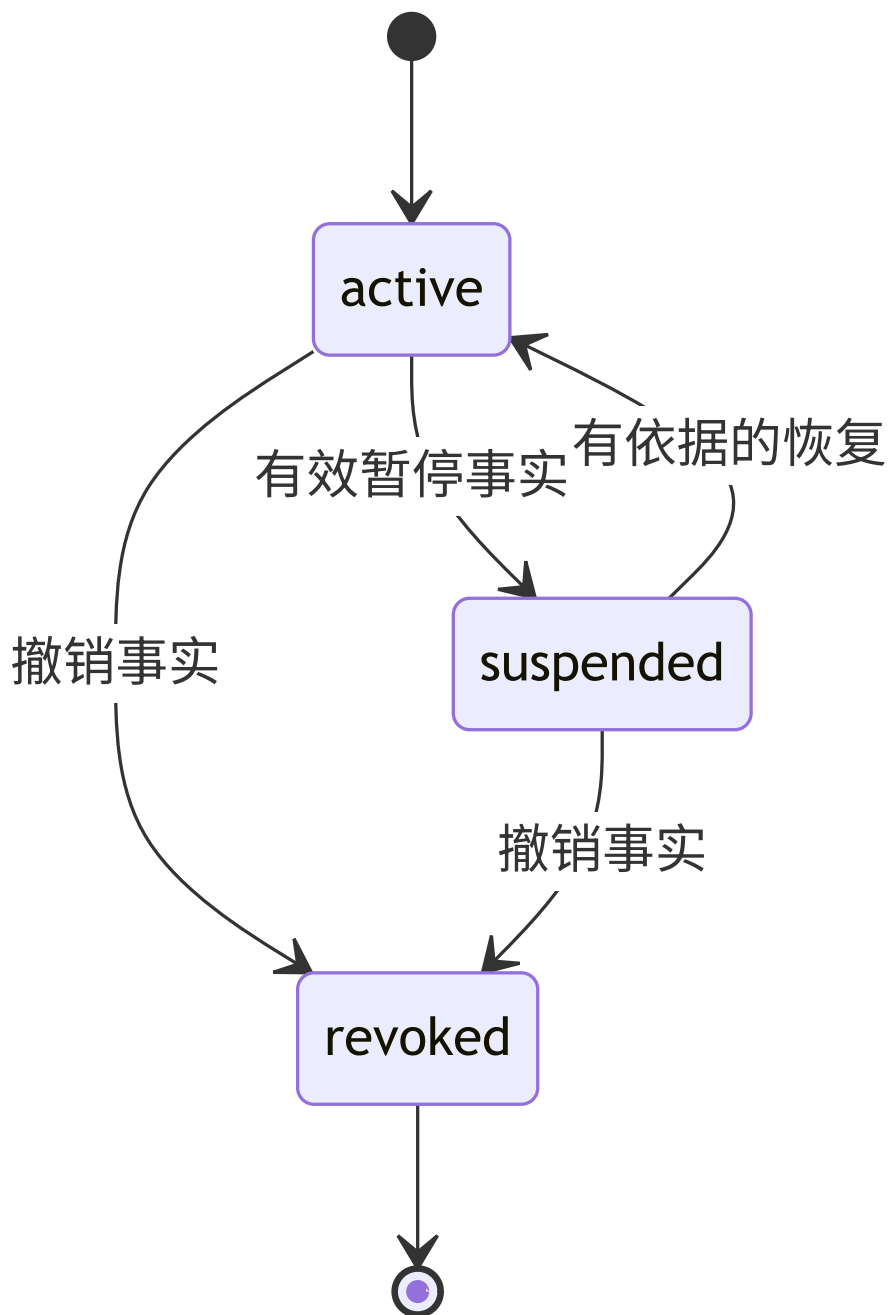
23.11 在保留 DID 的情况下更新资源

原控制方生成同 DID 的新 DID 文档或资源包版本，携带前序版本和内容摘要并使用既有控制关系签名。注册服务节点和根平台重新验证后产生新登记或发布证据，分发和发现节点按发布序列收敛；相同内容重复提交应幂等，未授权的新文档不得覆盖旧版本。更新完成的判定应同时看新版本发布证明、下游同步状态和查询结果，而不是只看注册接口再次返回成功。

检查项	更新前后要求
resourceDid	保持相同
资源版本	按规则递增，不回退
控制证明	覆盖新的规范化材料
发布证明	绑定新版本和摘要
发现结果	当前版本优先，历史可审计

23.12 暂停、撤销与恢复资源

资源暂停由控制者、治理或有效证据状态触发，撤销针对资源或版本，恢复需重新检查控制证明、凭证、发布证据和端点。各节点应按自身职责记录触发事实、处理时间和传播状态；发现结果中的状态是节点已同步到的状态，不一定等于刚发生的操作时间。网络超时或节点暂时离线不等于资源撤销；发现节点传播未完成时，应显示状态和新鲜度，保留历史证据。



23.13 接入第三方注册节点

第三方提交节点 DID 文档、授权申请、授权域、公开端点和运行配置，完成身份、签名、接口、资源校验、安全边界和上游提交测试。通过后只获得明确范围的注册资格；节点运行中仍需检查授权状态、版本和失败重试，不能用一次验收永久替代治理状态。接入测试应使用隔离资源，并保存测试请求、响应、错误分支、授权范围和有效期，避免将测试资源误认为正式网络资源。

- ☐ 节点 DID 文档、角色、端点和协议版本一致。
- ☐ 授权域与治理授权材料一致。
- ☐ 正向、负向、重试和恢复用例均有证据。

□ 私钥和治理秘密未进入公开材料。

23.14 接入第三方发现节点

第三方发现节点获取节点身份、授权材料和发布源配置，先完成全量同步建立基线，再进行增量同步、摘要校验、版本选择、DID/结构化/语义查询、去重、新鲜度和降级测试。对外提供结果时保留来源、版本、索引时间和授权域，不能把本地排序改写为根平台发布事实；查询接口可用也不代表同步数据已经覆盖最新发布序列。

接入顺序为“全量建立基线 → 增量同步 → 注入重复/乱序/缺失 → 查询与去重 → 新鲜度和恢复验证”。本地排序只影响展示顺序。

23.15 通过网络观察治理事件

治理事件在链上产生后，由链下信任索引器按序读取并形成节点授权状态读模型，根平台和发现服务节点再依据同步状态执行策略。观察者应记录事件类型、链上位置、索引游标、投影时间和页面显示时间，区分链上事实与网站展示。`latest_sequence` 继续增长只能证明索引器在读取新事件；具体节点授权状态是否已经变化，还要核对对应事件是否进入读模型。

链上事件 -> 索引器游标 -> 授权状态读模型 -> 根平台/发现服务节点判断 -> 页面观察

23.16 解析并验证资源 DID

客户端先解析资源 DID 和 DID 文档，再验证控制关系、资源元数据、服务端点、版本和相关发布证据。解析结果可以被 SDK 或本地验证器封装，但调用方仍应能取得原始文档、摘要和错误状态，以便审计和复核。解析成功只证明文档能够读取和通过结构检查，不能跳过资源包、凭证、根平台发布证明和治理状态检查。

建议输出 `resolved`、`invalid`、`not-found` 和 `status-unknown` 等可区分状态，避免所有失败都被压缩为空对象。

23.17 获取资源历史版本

调用方按 DID、版本或发布序列请求历史资源包及其 DID 文档、摘要和证明，并明确其已替代、已撤销或仅供审计的状态。历史内容不得因可读取而自动恢复为当前可发现或可调用版本；当历史版本缺少对应发布证明或状态证据时，应标记为不可完整验证，而不是补用当前版本证明。

历史记录至少关联 DID、版本/序列、摘要、发布证据、当前状态和查询时间；“可读取”和“当前可调用”必须是两个字段。

23.18 处理不可用或数据过期的发现节点

调用方根据响应错误、节点健康、索引时间、游标和声明的新鲜度判断发现结果是否可用。可以先展示已有旧快照以改善体验，但必须标注时间和降级状态；对需要最新授权或版本的动作，应重新从可信来源验证，不能只依赖旧缓存。

情况	只读展示	授权/调用动作
节点超时	可展示标记为过期的快照	停止或切换后重新验证
索引过期	显示过期时间	重新获取当前证据
来源不明	不显示可信标识	拒绝
恢复完成	更新快照	重做完整验证

23.18.1 发现节点不可用

发现请求超时、连接失败或返回明确服务错误时，记录节点和关联标识，按策略重试或切换其它授权发现服务节点。不可用表示基础设施状态，不自动表示资源撤销，也不能把空结果当作网络中不存在资源。

重试记录至少包含端点、请求标识、错误类型、次数、退避时间和最后已知索引时间；连接失败与合法空结果不可混淆。

23.18.2 返回旧快照

旧快照可用于有限只读展示，必须携带生成时间、来源和降级标识。它可以展示资源名称、类型、历史命中结果和上次索引时间，但不可单独用于判断当前授权、撤销状态、端点可用性或调用资格。若快照中的资源涉及调用、授权或敏感决策，应在动作前重新获取并验证当前 DID 文档、凭证、发布证明和治理状态；重新取得在线证据后，应以新结果覆盖快照，并保留新旧来源的时间关系。

```
{
  "source": "discovery-node",
  "snapshotAt": "<timestamp>",
  "stale": true,
  "usableFor": "read-only-display"
}
```

23.18.3 数据新鲜度判断

以索引时间、事件游标、上游最新序列和允许延迟判断新鲜度；latest_sequence 不增长可能是没有新事件，也可能是同步停止，需结合最近成功时间和错误证据判断。未知新鲜度不能默认为最新。

新鲜度报告应同时给出 indexedAt、lastSuccessfulSyncAt、latestSequence、cursor 和错误状态，不能只展示一个序列号。

23.18.4 切换其它发现节点

切换前确认候选节点的授权状态、端点和支持的查询 profile，并在结果中保留来源节点。来自不同节点的结果需要按 DID、版本、摘要和发布证据去重和交叉验证，不能简单拼接成同等可信结果。

切换检查单：

- ☐ 候选节点 DID、角色和授权状态有效。
- ☐ 查询 profile 和授权域满足调用方需求。

- 结果按 DID、版本、摘要去重。
- 冲突结果回到根平台发布证据核验。

23.18.5 恢复后的重新同步

节点恢复后从持久化游标或受信发布序列补齐缺口，重新验证受影响版本和撤销状态，再更新索引投影。恢复完成以游标追上允许范围、关键资源查询一致和错误告警恢复为判据，而不是进程重新启动。

恢复验证应至少抽查一条新增资源、一条更新资源、一条撤销或暂停资源和一条在故障期间未变化的资源，确认恢复过程没有只补齐数量而遗漏生命周期状态。

恢复完成判据可写成：

游标补齐 + 关键资源一致 + 生命周期重新过滤 + 错误告警恢复

23.19 参考来源

来源	类型	链接
oan-protocol-common	代码仓：端到端流程共享对象和协议类型	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓：根平台验证、发布和治理接口	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务节点流程	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现同步、索引和查询流程	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理状态同步和查询流程	https://github.com/OpenAgenet/oan-trust-indexer
oan-sdk-ts	代码仓：客户端端到端调用辅助	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓：社区端到端操作示例	https://github.com/OpenAgenet/oan-community-skill
OGAN White Paper	arXiv 白皮书：整体愿景和生态流程	https://arxiv.org/abs/2606.03161
OGAN Yellow Paper	arXiv 黄皮书：协议流程和验证机制	https://arxiv.org/abs/2606.03163
GRAIL Semantic Discovery Paper	arXiv 论文：语义发现流程	https://arxiv.org/abs/2605.02489
did:ogan DID Method Specification	标准/方法规范：端到端流程中的资源身份	https://github.com/OpenAgenet/ogan-public-docs/blob/main/did-ogan-specs/doc/OGAN%20DID%20Method%20Specification.md
OGAN Resource Identity and Discovery	IETF 草案：身份、注册和发现流程	https://datatracker.ietf.org/doc/draft-xu-ogan-resource-identity-discovery/

来源	类型	链接
Agentic Overlay Network Architecture	IETF 草案：节点协作流程	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF 草案：发现查询流程	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第二十四章 符合性配置文件与验收标准

本章说明符合性配置文件如何把每项要求落实为“测试对象、输入、步骤、判据、证据和结论”，不把未经测试的能力宣称标为合格。当前可直接复用的离线材料是 `oan-third-party-node-admission-test-kit/fixtures/v1-alpha`，其中 `manifest`、`case matrix`、`expected fixtures` 和 `requirement checks` 形成可审计基线；真实根平台、内容分发平台、链上治理和生产恢复应单独标明。

结论	含义	后续动作
PASS	在声明范围内满足全部判据	归档证据
FAIL	至少一项强制判据不满足	阻止发布或接入
NOT TESTED	没有可复现证据	不得宣称符合
CONDITIONAL	仅在明确限制下成立	记录限制和期限

符合性审查以隔离测试环境、固定测试向量、明确规范版本和可留证结果为前提。每个 `profile` 的“通过”只表示对应角色完成规定能力和拒绝行为测试，不自动证明整个 OpenAgetnet (OAN) 网络或资源业务质量通过。测试记录应能复现输入、步骤、响应、日志、版本和判定。

`profile` 的适用范围应在报告开头明确写出，例如“资源提供方—资源包生成”“注册服务节点—注册接口”“发现服务节点—查询和索引”。同一测试资源可以贯穿多个 `profile`，但每个角色只对自己产生或验证的事实负责；测试资源在注册、发布、分发和发现阶段的状态应通过 DID、版本、摘要和请求标识关联，而不能只凭名称关联。

24.1 资源提供方符合性配置文件

资源提供方应能生成或导入本地身份，创建资源 DID，形成合法 DID 文档、资源元数据和资源包，并用本地控制密钥签名。验收测试覆盖四类资源的必填字段、能力标签、用例、端点、版本和摘要；合格证据是可验证的签名材料、结构化输入和错误输入的拒绝结果，私钥不得出现在上传请求或日志中。

验收对象	测试输入	通过判据	必留证据
本地身份	新建身份、导入有效备份、导入损坏备份	能生成或恢复主体控制关系，损坏文件被拒绝	DID、公钥、导入结果；不留存私钥
资源 DID	四类资源各一份	DID 与 DID 文档中的控制关系一致	DID 文档、规范化摘要、签名验证结果
资源元数据	必填、缺失、类型错误字段	合法数据可生成资源包，非法数据明确拒绝	输入样例、错误响应、校验日志
资源包	内容、版本、哈希和端点	摘要可重算，篡改后验证失败	资源包摘要、重算结果、版本信息

验收顺序应固定为“身份可用 → DID 文档生成 → 元数据校验 → 资源包摘要和签名 → 提交材料检查”。任一步失败都不得继续把材料标记为可发布；测试记录应关联资源 DID、测试向量和规范版本。

资源提供方的通过条件不是“网页能够下载一个文件”，而是能够在本地重建同一份材料并验证控制关系。测试人员至少应保存一份合法材料、一份缺少必填字段的材料和一份签名或摘要被修改的材料，分别核对生成结果、明确错误和无副作用结果。

24.2 注册节点符合性配置文件

以隔离测试身份提交合法、缺字段、错误签名、越权节点、重复和过期材料，检查注册节点的状态码、错误结构、登记记录、幂等结果、VC（如有）和上游提交。合格条件是只受理授权范围内的有效请求，拒绝请求不改变有效资源状态，并能用日志和关联标识复核处理路径。

注册节点验收至少覆盖以下请求序列：

1. 提交合法材料，确认返回结构、登记状态和关联请求标识。
2. 重复提交同一 DID、版本和摘要，确认结果幂等，不产生第二个有效登记事实。
3. 替换控制签名、节点授权或资源摘要，确认请求被拒绝且原登记不变。
4. 提交过期或超出授权范围的材料，确认拒绝原因可区分。
5. 在上游暂时不可用时检查失败状态、重试边界和待处理记录。

每个请求都应保存发送端的规范化请求摘要和注册节点返回的关联标识；对于异步上游调用，还应把“注册服务节点已受理”和“根平台已处理”作为两条独立记录。成功响应中没有注册 VC 时，应记录字段缺失或未返回，而不能在验收材料中补造凭证。

检查点	观察内容	合格结果
请求校验	必填字段、版本、摘要和签名	非法请求在产生登记前被拒绝
控制权	资源 DID 与控制签名关系	不能用其他主体签名覆盖资源
授权范围	注册服务节点身份和允许资源范围	越权提交不改变有效状态
VC 结果	签发者、主体、类型和状态	VC（如返回）可验证且与登记事实对应
上游提交	根平台或后续发布任务	失败有明确状态，不冒充全网可见

24.3 根平台符合性配置文件

使用合法和篡改后的资源包、控制证明、注册节点授权、版本序列和摘要进行验证。根平台只有在控制关系、节点授权、内容完整性和治理条件均满足时才形成根平台可信发布证明；重复发布应幂等，拒绝和异步失败不得产生可被发现节点当作有效版本的发布事实。

根平台验收采用“验证输入、判断资格、形成发布事实、再次读取结果”四步检查：

- ☐ 控制签名能够绑定资源 DID、版本和待发布内容。
- ☐ 注册服务节点授权有效，且提交范围与授权范围一致。
- ☐ 资源包、DID 文档和摘要能够相互校验。

- ☐ 版本序列没有倒退、跳跃或无法解释的分叉。
- ☐ 根平台可信发布证明的签发者、主体、覆盖内容和状态可验证。
- ☐ 拒绝、重复和异步失败不会生成有效发布记录。

验收证据应同时保存首次处理结果和随后查询到的发布状态，用于区分“请求已接收”“验证已通过”“发布事实已形成”和“可被发现节点索引”这几个不同阶段。

根平台 profile 的关键判据是“拒绝输入不会形成可传播的有效发布事实”。因此，测试人员除检查接口响应外，还应重新读取发布记录、内容分发清单或发布序列，确认错误摘要、错误控制签名和越权注册服务节点没有进入后续队列；这类检查应使用独立的测试资源和请求标识。

24.4 内容分发服务符合性配置文件

向内容分发平台投递完整包、缺包、错误摘要、重复批次、断点游标和旧版本，检查清单、响应、接收游标、完整性和审计记录。合格条件是只交付根平台允许的材料，重复和断点不丢失或重复改变事实，接收方可以用摘要和证明核验内容。

测试步骤	构造条件	通过判据
完整投递	清单、内容、摘要和发布证明齐全	平台接收并返回可追踪批次
完整性失败	删除内容或修改摘要	拒收，不更新可供发现的版本
重复投递	相同批次和游标再次提交	幂等，不重复计入发布事实
断点恢复	从中间游标继续	只补齐缺口，最终摘要一致
旧版本投递	低于当前有效版本	不覆盖较新的有效内容

测试结束时应用清单和摘要重新核对接收端内容，并记录投递批次、游标、响应时间、重试次数和最终状态。分发成功只表示内容可取得并可校验，不替代根平台发布证明。

断点和重复测试应在接收端重启或网络中断后再次执行，检查持久化游标是否仍指向最后一个已确认批次。若接收端已经保存内容但没有保存对应证明或游标，应将其视为未完成的分发状态，不能直接交给发现服务节点建立可信索引。

24.5 发现节点符合性配置文件

先执行全量和增量同步，再用已发布测试资源执行 DID、结构化和语义查询，并加入重复材料、旧版本、撤销状态、序列缺口和过期数据。合格条件是验证后索引、同 DID 默认选择最新有效版本、结果不重复、返回来源和新鲜度，相关性不覆盖控制或治理检查。

发现服务节点测试顺序如下：

1. 检查全量同步后的资源数量、版本和摘要。
2. 注入一个增量发布，确认仅新增或更新对应索引记录。
3. 分别执行 DID 精确查询、结构化条件查询和语义查询。
4. 加入重复材料、旧版本、撤销资源和过期快照，检查过滤与去重。
5. 核对结果中的来源节点、版本、新鲜度和发布证据引用。

查询类型	必须验证	不能替代的检查
DID 精确查询	DID、当前版本和来源	控制权与治理状态
结构化查询	类型、标签、协议和状态条件	资源实际可用性
语义查询	输入描述到标签/文本条件的转换	授权和签名验证

同一 DID 的多条记录应按版本和发布状态进行确定性选择；无法确认新鲜度、治理状态或发布证据时，应标注不确定并交给调用方复核。

发现节点 profile 的查询验收应使用同一组已发布资源覆盖“精确命中、结构化命中、语义命中、重复记录、旧版本和不可见资源”六类结果。报告中应同时记录查询输入、过滤条件、返回的资源 DID、版本、来源、索引时间和重复数量，避免只记录返回总数。

24.6 信任索引器符合性配置文件

提供按序治理事件、断点、重复事件、缺失事件和暂时不可用的上游数据，检查授权状态读模型、事件游标、延迟和错误状态。合格条件是顺序和幂等性正确，重启后可从游标恢复，不能把未读事件或人工展示当作链上事实；证据包括事件位置、游标和投影记录。

输入场景	预期投影	证据字段
连续事件	授权状态按序更新	事件位置、游标、投影时间
重复事件	不重复改变状态	事件唯一标识、幂等结果
序列缺口	暂停或标记不完整	缺口范围、错误状态、告警
重启恢复	从持久化位置继续	重启前后游标、补齐记录
上游不可用	保留最后可信状态并标明过期	最近成功时间、失败原因

链下信任索引器的“状态已更新”必须能够回指治理事件或明确的上游证明；数据库中存在一条投影记录，不能单独作为治理事实。

当测试事件连续到达时，应分别检查事件顺序、重复事件和序列缺口；当 latest_sequence 暂时不增长时，应结合最近成功读取时间和错误状态判断是“没有新事件”还是“同步停滞”。只有游标、事件位置和授权状态读模型相互对应，才能把该次测试记为通过。

24.7 解析器与客户端符合性配置文件

使用合法 DID、未知 DID、方法不支持、结构无效、控制签名错误、状态异常、历史版本和证明缺失的输入，检查结果类型和停止行为。合格条件是错误可区分、无效材料不被静默接受、历史版本不冒充当前版本，且原始文档、摘要和错误信息可供复核。

输入状态	客户端应展示或返回	后续动作
合法且已解析	当前 DID 文档和解析状态	继续控制、发布和治理验证
未找到	明确的 not-found	不用同名缓存替代
方法不支持	明确的 method-unsupported	停止高风险调用
文档或签名无效	invalid-document 或验证失败	拒绝并保留原始证据
状态未知或过期	status-unknown	降级展示或请求重新验证
历史版本	明确版本和时间	不当作当前版本使用

验收还要检查错误路径不会把部分解析结果误标为可信资源，也不会因自动重试而绕过控制权、凭证状态或治理状态检查。

24.8 SDK 符合性配置文件

SDK 应覆盖本地身份、资源 DID、元数据、控制签名、注册、DID 精确发现、语义发现、VC 和发布证明验证。使用 `oan-sdk-ts` 的测试输入检查序列化、错误映射和跨环境端点配置；合格条件是 SDK 不上传私钥、不丢失版本和证据字段，并能让调用方区分登记成功与可发现。

SDK 验收清单：

- ☐ 新建和导入本地身份的结果可复核，私钥不进入网络请求。
- ☐ 生成资源 DID、DID 文档和签名时，字段与 Rust 协议类型一致。
- ☐ 注册成功、注册失败和“尚未可发现”能够映射为不同结果。
- ☐ DID 精确查询和语义查询保留来源、版本和新鲜度信息。
- ☐ VC、根平台可信发布证明和治理状态的验证结果不被合并为单一布尔值。
- ☐ 网络错误、服务端错误和格式错误能够被调用方区分。

跨环境测试应使用固定节点端点配置和测试向量，不能以某个线上地址或某次成功请求作为 SDK 永久兼容性的证明。测试报告还应把 SDK 的单元测试、节点联调和官网实际请求分开记录，避免把本地对象测试结果误写成端到端互操作结果。验收结论至少应关联 SDK 提交、协议公共类型版本、测试向量、目标端点和响应摘要。

24.9 安全与密钥保管验收标准

测试主体、资源和节点密钥的生成、保存、备份、恢复、轮换及权限边界，检查签名、VC 和根平台可信发布证明的验签。合格条件是私钥只在控制方边界内使用，备份可恢复且受保护，错误密钥和越权签名被拒绝；证据应脱敏，不能提交私钥原文。安全验收任务清单：

- ☐ 证明主体私钥、资源控制私钥和节点运行凭据的使用边界。
- ☐ 检查备份文件的生成、导入、权限和恢复结果，不把私钥写入日志或请求体。
- ☐ 使用错误公钥、错误签名和其他主体签名执行拒绝测试。
- ☐ 验证 VC 和根平台可信发布证明的签发者、主体、签名覆盖范围和状态。
- ☐ 检查异常、调试日志和证据包中的敏感字段是否已脱敏。

测试报告应明确“控制方能够签名”与“服务节点能够验证”是两个不同事实；后者不意味着节点取得前者的私钥。安全验收应把客户端、注册服务节点、根平台、发现服务节点和证据存储分别作为观察边界，逐项记录私钥是否出现在请求体、日志、临时文件、错误堆栈或测试归档中。控制签名、注册 VC 和根平台可信发布证明应分别验证其签发者、主体、签名覆盖范围和状态。

24.10 互操作与兼容性验收标准

使用 Rust 协议类型、TypeScript SDK、官网 API、注册服务节点、发现服务节点及 A2A/MCP 映射的同一组向量，比较 DID 文档、资源包、摘要、签名、错误和版本结果。合格条件是跨语言规范化和字段语义一致，映射保留来源，不能因适配而削弱 OAN 的控制和治理验证。

对照维度	Rust 实现	TypeScript/HTTP 实现	合格条件
序列化	协议公共类型	SDK 和官网请求体	字段、类型和缺省规则一致
摘要	规范化后的内容摘要	前端或 SDK 生成/读取摘要	同一输入得到同一摘要
签名	控制签名和证明验证	SDK/节点返回验证结果	签名覆盖范围一致
错误	节点错误结构	SDK 错误映射	不丢失失败原因
映射	A2A Agent Card、MCP 元数据	适配或展示结果	保留原文和来源

跨语言对照必须使用同一版本、同一输入和同一规范化规则；只比较最终页面文本，不能证明协议互操作。互操作结论还应区分“同一输入产生同一对象”“不同实现能够解析对象”和“跨节点端到端流程成功”三个层次，分别记录对应证据，不能只凭最终页面显示相同就判定互操作通过。

跨语言验收可以采用“同一夹具、三次核对”的方式：先比较 Rust 协议类型和 TypeScript 对象的规范化结果，再比较 SDK 或官网发出的 HTTP 请求与节点实际解析结果，最后比较注册、发布和发现链路中的资源 DID、版本和摘要。任一层出现差异，都应记录差异字段和适用版本，不能只把失败归因于网络问题。

24.11 性能与可用性验收标准

在明确并发量、资源规模、网络条件和统计窗口的测试环境中测量注册、发布、同步、查询和治理读取的成功率、延迟、积压、恢复时间和资源消耗。阈值应在 profile 或部署验收记录中事先定义；没有测量数据时只能标为未测，不得把一次演示表现写成通用性能承诺。

链路	建议记录字段	解释边界
注册	请求数、成功率、P50/P95 延迟、错误分类	不等同于全网可发现时间
发布	发布批次、排队时长、失败重试	不等同于分发完成
同步	游标、积压量、单批耗时、恢复时间	不等同于查询质量
查询	查询类型、命中率、P50/P95、重复率	语义相关性需另行说明
治理读取	最新序列、延迟、异常时长	不把停滞序列解释为无事件

报告至少区分冷启动、稳定运行、故障恢复三个窗口，并同时给出环境、数据规模、并发量和统计方法。

性能结果应注明是否包含网络往返、数据库写入、语义检索和异步传播等待；注册接口延迟不能直接当作资源全网可见延迟，查询命中率也不能直接当作语义质量。出现超时、重试或部分失败时，应同时报告请求数量、失败分类和恢复后的最终状态。证据包的最小目录可以固定为：

```
evidence/  
  manifest.json
```

```
spec-and-profile/  
fixtures/  
requests-and-responses/  
logs-and-metrics/  
deployment/  
review-and-result/
```

`manifest.json` 应列出每份材料的摘要、来源、生成时间、测试环境和对应条款；目录结构只规定证据组织方式，不要求公开生产配置或敏感数据。

证据清单可以采用如下不含秘密的记录形式：

```
{  
  "profile": "discovery-node",  
  "profileVersion": "v1-alpha",  
  "implementationCommit": "<git-commit>",  
  "fixtureId": "valid-resource-v1",  
  "environment": "isolated-test",  
  "result": "PASS",  
  "artifacts": [  
    {"path": "requests-and-responses/query.json", "sha256": "<digest>"},  
    {"path": "logs-and-metrics/discovery.log", "sha256": "<digest>"}  
  ],  
  "reviewedAt": "<timestamp>"  
}
```

尖括号内容是证据清单模板中的占位值；正式报告应替换为真实提交、摘要和时间，但不能写入私钥、数据库凭据或未脱敏的内部端点。

24.12 符合性实现所需证据

审查材料至少包括规范和协议版本、代码提交、配置摘要、测试向量、接口请求响应、日志、指标、部署记录和失败处置。敏感内容使用摘要、脱敏片段或受控阅览方式提交；证据必须标明测试时间、环境、操作者、工具版本和覆盖条款。

24.12.1 协议和数据结构证据

提供 DID、DID 文档、资源元数据、资源包、VC、发布证明和 API 响应样例，标明 schema、规范化、版本和内容摘要，并说明合法、缺失和非法样例的预期结果。每个数据样例至少标注 `fixtureId`、对象类型、schema/profile 版本、规范化方式、内容摘要、预期结论和覆盖条款；合法样例与负向样例应成对保存。

协议证据还应覆盖 HTTP 方法和路径、请求 JSON 的反序列化类型、响应类型或动态字段、错误状态和关键校验结果。对注册、根平台验证发布、内容分发和发现查询，至少分别保存一份合法请求、一份关键失败请求及其响应；共享类型应能追溯到 `oan-protocol-common`，节点本地类型和动态 JSON 应能追溯到对应 handler。仅保存接口地址或一份成功截图，不能证明输入输出契约完整。

24.12.2 密码学和身份证据

提供公钥、签名、验证方法、控制关系、凭证签发者和验证结果，私钥只提供受控测试环境中的存在性或摘要证据。材料应能证明签名覆盖对象和目标 DID，而非只展示签名字符串。密码学证据应能够复算签名覆盖的规范化对象和用于验证的公钥来源，并分别记录控制签名、注册 VC、根平台可信发布证明的验证结果。

24.12.3 节点行为证据

提供注册、根平台发布、分发、发现、治理同步和错误处理的请求响应、事件、游标、日志及时间线，关联资源 DID、版本和请求标识。对异步流程还要记录“已接收、已验证、已发布、已索引”等阶段，不能只保留最后一次成功响应。

24.12.4 部署和烟测证据

提供源码提交、构建产物、配置摘要、服务健康、数据库状态、注册和发现烟测结果及失败处理记录。公开材料脱敏，生产密钥和内部管理端点不纳入证据。部署证据至少能把源码提交、构建产物、运行配置和烟测结果串联起来；烟测应包含一个成功路径和一个预期失败路径。

24.12.5 版本和来源证据

提供代码提交、锁文件、schema/协议版本、资源包和证明摘要、产物哈希及发布记录，使审查者能够确认测试对象与部署对象是同一来源。来源核对至少比较代码提交、依赖锁定文件、构建产物摘要和运行时版本标识；对象不一致时不得认定为同一测试结论。

24.13 测试向量与互操作夹具

测试夹具应固定合法 DID、DID 文档、资源元数据、资源包、哈希、控制签名、注册 VC、根平台可信发布证明、治理事件和错误输入，并提供 Rust、TypeScript 或 JSON 可读取形式。每个向量标注预期成功/失败、版本、摘要和覆盖条款，避免依赖实时公网数据。

推荐的固定夹具记录形态如下：

```
{
  "fixtureId": "valid-resource-v1",
  "input": "resource-package.json",
  "expected": "PASS",
  "schemaVersion": "<version>",
  "contentHash": "sha256:<value>",
  "covers": ["23.1", "23.3", "23.10"]
}
```

同一 fixtureId 在 Rust、TypeScript 和 HTTP 测试中应指向相同的原始输入；若适配层必须转换，应同时保存转换前后的摘要和转换规则。

24.14 负向测试与拒绝行为

负向测试覆盖非法 DID、字段缺失、错误签名、越权节点、过期或吊销凭证、旧版本、重复请求、错误摘要、恶意端点和序列缺口。验收不仅看返回错误，还要确认没有写入无效索引、生成错误发布证明、推进游标或泄露秘密等副作用。

负向场景	必须观察的拒绝结果	不得发生的副作用
非法 DID 或结构缺失	可区分的格式/字段错误	不写入登记或索引
错误或越权签名	控制权/授权失败	不覆盖有效资源
过期或吊销凭证	状态无效	不生成有效发布证明
错误摘要或恶意端点	完整性/策略失败	不推进发布或同步游标
重复或旧版本	幂等或版本冲突	不产生错误新版本

每项负向测试都要同时检查响应、数据库或索引变化、事件游标、日志敏感信息和后续查询结果。

24.15 符合性证据留存与审查

证据按规范版本、代码提交、环境、时间、操作者和测试夹具归档，变更影响协议、schema、签名、治理、节点行为或部署路径时必须重测相关 profile。审查结论应注明通过、失败、未测和条件通过，不用单一总分掩盖关键失败。

结论	复核要求	重新测试条件
PASS	所有强制判据和拒绝行为均有证据	规范、代码或部署对象变化时重测
FAIL	明确失败输入、影响范围和阻断动作	修复后重跑原向量及回归向量
NOT TESTED	列出缺失环境或证据	补齐前提后不得沿用旧结论
CONDITIONAL	写明限制、期限和责任人	限制解除或到期前复核

审查结论还应区分“节点 profile 合格”“端到端链路合格”和“生产部署合格”，避免一个局部测试结果被扩大解释为整个网络符合性。

24.16 参考来源

来源	类型	链接
oan-protocol-common	代码仓：协议类型、schema 和错误契约	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓：根平台符合性和发布测试对象	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册流程和服务端校验	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现查询、索引和语义测试对象	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理状态和同步测试对象	https://github.com/OpenAgenet/oan-trust-indexer

来源	类型	链接
oan-sdk-ts	代码仓: 客户端符合性和跨语言调用验证	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓: 社区 Skill 烟测和操作验证	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv 黄皮书: 协议安全性质和符合性边界	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF 草案: 资源身份和发现测试方向	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF 草案: 发现 profile 测试方向	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

第二十五章 版本、兼容性与变更管理

本章说明 OpenAgenet (OAN) 的版本对象不是单一数字，而是多层坐标：

层级	例子	作用
规范	统一技术规范主/次/修订版本	定义语义和要求
协议	节点 API、错误和事件版本	定义交互契约
数据模式	DID 文档、资源包、VC、治理事件 schema/profile	定义可解析结构
方法	did:oan 方法规则	定义标识符和解析语义
实现	Rust crate、npm 包、官网构建提交	提供具体代码
部署	配置、二进制、服务提交	定义运行环境

发布记录必须把上述层级关联起来，不能用 Cargo 或 npm 版本号替代协议、模式或部署版本。一个实现版本可以同时支持多个协议或模式版本，一个资源 DID 也可以对应多个历史 DID 文档和资源包版本；因此发布、兼容、迁移和回滚都必须记录实际映射关系。版本记录还要区分“代码已提交”“产物已构建”“服务已部署”和“变更已生效”，避免把仓库状态误认为线上状态。

版本层级之间的关系可按以下方式核对：

变更对象	直接影响	不应直接推断
Rust crate 或 npm 包	某个实现或客户端能力	协议、DID 方法或线上部署已升级
协议/API	节点间请求、响应和错误处理	历史资源包已经迁移
资源模式/profile	DID 文档、资源包、VC 或事件的解析	控制权或资源 DID 已改变
did:oan 方法	标识符解析和控制语义	普通资源版本必须更换 DID
部署产物和配置	某一环境的运行行为	所有节点或第三方实现都已升级

25.1 规范版本管理

统一技术规范采用主版本、次版本和修订版本记录结构、语义和勘误变化。主版本变化表示既有实现可能无法直接互操作，次版本用于兼容性扩展，修订版本用于不改变规范语义的澄清或错误修正；每次发布保留受影响章节、发布日期、状态和实现映射。

级别	适用变化	必须保留
主版本	破坏既有互操作或语义	影响范围、迁移和批准
次版本	向后兼容扩展	新字段、兼容测试和实现映射
修订版本	澄清或勘误	条款、原意和发布日期

25.2 协议版本管理

节点协议和公共 API 应明确版本标识、支持范围、请求/响应模式、错误语义和协商方式。旧客户端在过渡期内只能使用服务端仍声明支持的 profile；服务端不得因为兼容旧请求而静默改变签名、版本、发布序列或治理判断，无法兼容时返回可识别的不兼容错误。

```
client-supported profiles  server-supported profiles
empty intersection -> explicit incompatible response
```

因此，协议兼容不仅要检查请求能否被反序列化，还要检查同一输入在签名验证、发布序列、治理判断、错误码和重试行为上的结果是否仍然一致。服务端不能因为兼容旧请求而静默改变这些语义；若旧版本仍可读取但不能安全写入，应分别声明读、写和查询能力。

能力	兼容性问题	记录方式
读取	旧客户端是否能解析响应和未知字段	支持的响应 profile 与样例
写入	请求是否会生成不同签名、版本或发布事件	支持的写入版本和拒绝条件
查询	分页游标、排序、语义过滤是否保持含义	查询版本、错误和边界样例
重试	重试是否造成重复注册或重复发布	幂等键、序列和重复请求测试

25.3 资源模式版本管理

资源元数据、资源包、VC、治理事件和发现响应分别记录 schema 或 profile 版本。它们并非一个版本：资源包中的 packageVersion 描述资源内容版本，DID 文档的模式版本描述文档结构，VC 的模式版本描述凭证声明和证明结构，治理事件版本描述事件载荷和事件语义。读取方应先识别各自版本，再选择解析器和验证规则，不能用资源包版本替代凭证或事件模式版本。

字段变化的处理边界如下：

变化	默认处理	额外要求
新增可选字段	兼容候选	旧读取方应忽略或保留，新增方提供回退行为
新增必填字段	可能破坏	提供默认语义、升级 profile 或迁移旧数据
字段改名、改类型或改变单位	破坏性	新版本、转换规则和负向样例
改变摘要算法或签名覆盖范围	破坏性	新验证规则、测试向量和重新发布
新增未知状态或治理事件	需显式处理	旧实现不得将未知状态当作有效或已完成

```
{
  "schemaVersion": "<resource-package-schema>",
  "packageVersion": "<resource-version>",
  "resourceDid": "did:oa:<method-specific-id>",
  "metadataHash": "<hash>",
  "verificationProfile": "<verification-profile>"
}
```

上例仅为字段关系示例，尖括号内容不是 OAN 的固定取值。对于历史材料，读取方应保留原始版本、摘要和验证结果；无法理解的必填字段、摘要算法、签名结构或状态值必须拒绝或进入明确的隔离状态，不能静默当作当前有效数据。

读取方应先识别 schema/profile，再校验必填字段、摘要算法、签名结构和状态值。删除、改名、改类型或改变签名覆盖范围时，应生成新 profile、迁移说明和负向样例。

25.4 DID 方法兼容性

did:oaan 方法变化必须同时评估 DID Core 数据结构、方法特定标识符、解析器、DID 文档、历史资源和发布证据。正常资源版本升级通常保持资源 DID 不变，通过新的 DID 文档、资源包、内容摘要、控制证明或发布证据表达版本变化；这与解析器实现版本或节点软件版本不是同一层面的变化。若方法解析规则、标识符规范化或控制语义发生破坏性变化，应定义旧 DID 的继续解析、迁移或停用策略，不能只更新实现版本。

对象	必须检查
标识符	语法、大小写、语义代码和规范化
DID 文档	id、验证方法、服务和扩展字段
控制语义	公钥、签名覆盖和更新授权
历史资源	旧 DID、版本、VC 和发布证明
解析器	成功、错误、未知方法和迁移行为

资源 DID 不变并不意味着文档内容永远不变。更新方仍须证明其拥有该 DID 对应的控制权，并使新文档、资源包版本、VC 和发布记录能够相互校验；发现服务节点应按其支持的版本和新鲜度规则选择当前可用版本，保留历史版本的可追溯关系。反过来，单纯更换服务器、SDK 或解析器，也不能自动产生新的资源版本。

25.5 向后兼容变更与破坏性变更

兼容性判断以旧客户端、旧节点、历史数据和签名验证行为的可验证结果为准，而不只看字段是否仍能反序列化。评估至少覆盖注册提交、根平台发布、内容分发、发现索引、治理校验、SDK/Skill 调用和失败恢复；任一环节把同一输入解释为不同资源、版本、状态或信任结果，都不能仅按“可解析”认定为兼容。

变更类型	默认判定	必要动作
新增可选字段	兼容候选	旧客户端回归测试
错误澄清	需评估	检查拒绝行为是否变化
删除必填字段	破坏性	提高版本并迁移
改变签名或治理语义	破坏性	治理审批、重发布和重测

25.5.1 兼容性变更

兼容变更保持既有必填字段、签名语义、状态含义和接口成功/失败规则，新增内容应可被旧实现安全忽略或通过协商识别。即使代码可以编译，也要用旧客户端、旧节点和历史数据验证结果一致。

- ☐ 旧必填字段和错误语义仍有效。
- ☐ 旧签名、状态和版本选择结果不变。
- ☐ 旧客户端能读取或安全忽略新增内容。
- ☐ 历史资源、DID 文档和凭证仍可验证。

25.5.2 破坏性变更

破坏性变更包括删除或重命名必需字段、改变 DID 或签名语义、改变资源状态、协议路径、版本选择或治理授权判断。此类变更必须提高版本、公布影响、安排迁移窗口，并阻止未完成迁移的节点误把新旧材料混用。破坏性变更发布前应完成以下检查：

- ☐ 列出受影响的节点、客户端、SDK/Skill、DID 文档、资源包、VC、发布证明和索引。
- ☐ 明确旧版本在过渡期内的读取、写入和发现行为。
- ☐ 提供迁移前后样例、负向样例、回滚数据和重测记录。
- ☐ 阻止未迁移组件把新旧材料混合解释为同一有效版本。

发布前应冻结影响范围，明确受影响节点、客户端、DID 文档、资源包、VC、发布证明、治理状态和数据库；未迁移对象不得混用新旧材料。对于无法一次迁移的历史对象，应明确它们是继续可读、仅可验证、禁止更新，还是进入隔离区，并在发现结果中避免把过期或未验证对象显示为当前有效版本。

25.5.3 版本标识

版本标识应说明所属层级、格式、来源和比较规则，例如规范版本、协议版本、资源包版本、schema 版本、Rust crate 版本、npm 包版本和部署提交不能互相替代。资源包版本与资源 DID 稳定性分开记录，证据中同时保存版本和内容摘要。

```
{
  "specVersion": "<spec>",
  "protocolVersion": "<protocol>",
  "schemaVersion": "<schema>",
  "resourceVersion": "<resource>",
  "implementationCommit": "<commit>",
  "deploymentId": "<deployment>"
}
```

25.5.4 迁移和过渡期

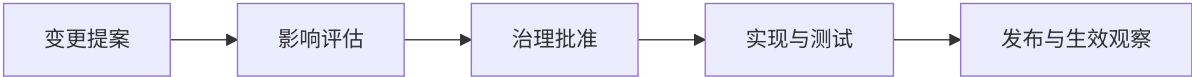
过渡期应规定旧新版本并行支持范围、开始和结束时间、数据双读/单写策略、节点升级顺序、失败回滚和停止旧版本条件。过渡期内不得让不同版本对同一 DID 产生无法解释的有效版本分叉；结束时重测注册、根平台发布、内容分发、发现查询和治理链路。若旧版本只能读取而不能写入，服务端应在协议能力声明和错误响应中明确表达，不能让调用方通过试错判断迁移状态。迁移计划至少应记录：

阶段	允许操作	必须观察
双读	读取旧、新格式	解析和摘要一致性
单写切换	只生成新格式	旧客户端错误是否可识别
收敛	补齐历史和索引	DID、版本和发布证据一致
结束	关闭旧路径	回滚入口和残留数据

推荐先扩展读取能力，再切换写入方，最后关闭旧读取路径；全程保存数据库迁移、回滚和发现/发布重测证据。

25.6 治理控制的规范变更

涉及节点授权、治理事件、资源状态、根平台可信发布证明、信任判断或节点协议的变更，应经过正式提案、影响评估、治理批准、发布和生效记录。提案、审批、代码实现、构建产物、部署状态和线上生效是不同证据，必须分别记录；签发者、验证者和受影响节点必须明确，不能以代码已经合并或网站已更新替代治理生效。



25.6.1 变更提案

提案说明动机、范围、受影响章节和对象、兼容性等级、迁移方案、测试证据、风险、回滚方式及拟生效时间。涉及签名、授权或 DID 解析的提案还应列出历史数据和第三方节点的影响。

项目	应填写内容
动机与范围	原因、角色和对象
兼容性	主/次/修订级别及依据
迁移	数据、节点、客户端和证据
验证	测试向量、集成测试和回滚演练
生效	批准主体、条件和时间

25.6.2 治理审批

审批应形成可验证的治理记录，说明决策主体、批准内容、版本、范围、条件和生效依据。普通实现提交、运营口头确认或单个节点本地配置不能替代需要治理授权的事实。治理审批记录应至少回答“谁批准、批准了什么、何时生效、由谁验证、哪些对象受影响”：

```
approver: <governance-authority>
approvedScope: [protocol, schema, node-policy]
effectiveCondition: <event-or-time>
verificationEvidence: <record>
```

25.6.3 发布和生效

发布记录关联规范版本、协议/schema 版本、代码提交、产物哈希和配置。生效应按声明的时间、治理事件或节点升级条件执行，并在生效前完成备份、兼容检查和符合性重测；页面更新不等于协议已经生效。

生效后还应观察跨节点状态、客户端错误、发布游标、索引延迟、版本分布和回滚可行性，确认实现状态与规范声明一致。若部署完成但发现服务节点仍使用旧协议、旧模式或旧治理配置，应将其记录为“已部署但未完全生效”，不能直接标记为完成。

25.6.4 回滚和紧急变更

紧急变更仅限修复安全、数据完整性或重大可用性风险，并记录原因、授权、影响范围和临时期限。回滚必须考虑数据库、发布序列、治理状态和历史证据，不能只把服务二进制换回旧版本；事后补齐正式审查记录。紧急变更的回滚检查应按对象执行：

对象	回滚要求
二进制和配置	恢复可验证的构建产物和配置摘要
数据库	保留迁移前备份和恢复校验
同步游标	防止重复处理或跳过事件
发布状态	不把失败发布留作有效事实
缓存和索引	清理与重新同步范围明确

回滚检查还应覆盖同步游标、缓存和未完成的发布任务；紧急变更结束后必须补齐正式审批和重测。

25.7 弃用与迁移策略

弃用对象可以是接口路径、字段、摘要算法、协议版本、节点二进制、SDK/Skill 包或配置项。通知应说明对象、替代方案、支持期限、影响版本、迁移工具、验证方法和停止服务条件，并分别说明旧对象在过渡期内是可读、可写、可发现还是可调用。旧接口或字段在过渡期可返回明确的弃用信息，但不得静默改变结果；节点和客户端完成迁移后，按记录的条件停止旧版本并保留审计证据。通知至少应给出迁移前后样例、验证命令、支持截止时间和失败回滚入口。

25.8 变更记录与发布说明

每次变更记录受影响章节、数据结构、接口、代码仓、数据库迁移、测试、部署、回滚、操作者和已知影响，并明确规范条款与实现状态不一致时的责任人和期限。发布说明应让维护者知道“改了什么、为什么改、谁需要动作以及如何验证”，并能沿着变更编号追溯到批准、提交、产物、部署和生效检查。

```
changeId: CHANGE-<id>
scope: [spec, protocol, schema, implementation, deployment]
affectedChapters: []
repositories: []
compatibility: compatible|breaking|deprecated
tests: []
deployment: <commit-or-artifact>
rollback: <procedure>
owners: []
knownImpact: <summary>
```


上述 YAML 是发布记录的示例，不是 OAN 固定配置。状态核对可按以下顺序进行：代码提交后记录提交标识，构建完成后记录产物摘要，部署完成后记录目标环境和服务状态，烟测通过后记录测试结果，满足协议、数据和治理生效条件后再记录 **effective**。任一阶段失败，都应保留失败原因和回滚动作，而不是覆盖为成功。

25.8.1 规范变更记录

记录章节、条款、版本、变更类型、原文与新意图、兼容性结论、批准记录和生效时间。勘误、澄清和规范语义变化分开标识，避免把实质变更包装成文字修订。规范变更记录可用如下字段核对：

字段	内容
affectedChapters	受影响章节和条款
changeType	勘误、澄清、兼容扩展或语义变更
approval	批准记录
effectiveAt	生效条件或时间
implementationMap	对应代码、模式和部署版本

25.8.2 协议变更记录

记录路由、请求/响应、状态码、错误、签名、游标、版本协商和节点互操作变化，并关联协议测试和旧版本支持期限。至少应说明旧请求是否仍能注册、发布、查询和重试，以及不兼容时返回什么稳定的错误语义。协议变更记录应把路由、请求/响应、状态码、错误、签名、游标和协商行为分别列出，并至少附一组旧版成功样例、一组新版成功样例和一组不兼容样例。

25.8.3 实现变更记录

记录受影响仓库、提交、依赖、Cargo/package 版本、数据库迁移、构建产物和部署环境。实现修复若尚未同步规范，应明确临时状态和后续规范更新责任；Rust crate、npm 包、官网构建和节点二进制分别记录，不能把其中一个版本号当作整套 OAN 实现的版本号。实现变更记录应区分“代码已合并”“构建已完成”“已部署”“烟测通过”四个状态，避免提交号存在就被解释为线上已生效。

25.8.4 迁移提示和已知影响

列出需要升级的节点、客户端、SDK/Skill、配置和数据库，说明旧资源、历史 DID、VC、发布证明、索引和缓存的影响，以及迁移失败和回滚后的已知限制。迁移提示应按角色列出动作和影响，并把“需要升级”与“可以继续读取”区分开：

角色	需要关注
资源提供方	资源包、DID 文档和签名生成方式
节点运营方	配置、数据库、游标和服务升级顺序
客户端和 SDK	版本协商、错误处理和缓存失效
审查方	旧资源、历史证据和重测范围

25.9 参考来源

来源	类型	链接
oan-protocol-common	代码仓: 协议、schema、错误码和版本相关结构	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓: 根平台发布和兼容性变更	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓: 注册协议和版本迁移	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓: 索引、查询和同步兼容性	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓: 治理事件和投影版本迁移	https://github.com/OpenAgenet/oan-trust-indexer
oan-sdk-ts	代码仓: 客户端版本和兼容性	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓: 脚本、示例和发布兼容性	https://github.com/OpenAgenet/oan-community-skill
OAN White Paper	arXiv 白皮书: 生态演进和开放基础设施定位	https://arxiv.org/abs/2606.03161
OAN Yellow Paper	arXiv 黄皮书: 协议版本、信任和变更边界	https://arxiv.org/abs/2606.03163
did:oan DID Method Specification	标准/方法规范: DID 方法兼容性	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF 草案: 资源身份和发现版本方向	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Agentic Overlay Network Architecture	IETF 草案: 网络架构演进方向	https://datatracker.ietf.org/doc/draft-xu-agentic-overlay-network-architecture/
Efficient Agent Discovery Profile	IETF 草案: 发现 profile 演进方向	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

附录 A 规范性数据模式

本附录中的模式是协议实现、测试向量和接口校验的共同依据。字段名称以实际 JSON 序列化结果为准，Rust 类型、TypeScript 类型和 HTTP 示例不得各自发明同义字段；如果某字段只在参考实现或特定资源类型中存在，应明确标为可选或 profile 扩展。模式版本、内容摘要、签名覆盖范围和未知字段处理规则应与正文第五至第八章、第十六章和第二十五章一致。

A.1 DID Document 模式

did:oa DID 文档的公共 Rust 类型包含 `@context`、`id`、`verificationMethod`、`authentication`、`assertionMethod`、`service` 和可选的 `oanMetadata`。其中 `id` 必须与被解析的 DID 完全一致；验证方法的 `id`、`controller`、公钥材料和引用关系必须能够支持签名验证。`oanMetadata` 中的资源类型、能力标签、用例、协议绑定、实现链接和资源包信息属于 OpenAget (OAN) 扩展，具体字段以 `oan-core` 的序列化定义为准。示例中的公钥、摘要和 DID 均为占位值；缺少 `id`、验证方法引用无法解析、服务端点不是有效 URI，或资源类型与 DID 语义码不匹配时，注册和索引校验应失败。当前公共类型未将 `controller` 作为 DID 文档顶层字段；若某个 DID profile 使用该字段，应单独说明其兼容范围，不能与 `verificationMethod.controller` 混同。

```
{
  "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oa/v1"],
  "id": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "verificationMethod": [{
    "id": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
    "type": "Ed25519VerificationKey2020",
    "controller": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "publicKeyMultibase": "zExamplePublicKey"
  }],
  "authentication": ["did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
  "assertionMethod": ["did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
  "service": [],
  "oanMetadata": {
    "subjectType": "skill",
    "resourceType": "skill",
    "resourceDescription": {
      "name": "Example repository structure skill",
      "description": "Searches a repository and summarizes its project structure.",
      "capabilityTags": ["code-repository", "project-structure"],
      "useCaseExamples": ["Summarize the layout of a source repository."]
    },
    "capabilityTags": ["code-repository", "project-structure"],
  }
}
```

```
"authorizedDomains": ["software-development"]
}
}
```

校验项	合法条件	失败处理
id	与请求目标 DID 完全一致并符合 did:oan 语法	拒绝注册或索引
验证方法	公钥编码、controller 和引用关系可验签	返回密钥或签名结构错误
oanMetadata	subjectType 与 resourceType 使用公共资源类型枚举	返回资源类型不匹配
未知扩展	不改变既有字段语义	按模式版本决定忽略或拒绝

A.2 资源元数据模式

四类资源共用资源 DID、资源类型、名称、描述、版本、能力标签、用例、端点、协议绑定和实现链接等信息，但当前公共类型在不同层次承载这些信息：ResourceMetadata 使用 resourceDid、resourceType、subjectType、name、description、capabilityTags、authorizedDomains、protocolBindings、services、lifecycleState、packageVersion、哈希和 updatedAt；DID 文档则通过 oanMetadata.resourceDescription、capabilityTags、protocolBindings 和 packageInfo 表达对应信息。四类资源的枚举值是 agent_service、skill、mcp_server 和 tool_api，不能把展示名称 agent 当作协议值。字符串、数组和 URI 的长度、重复项及格式限制，应由具体 profile 或节点校验规则补充；未知资源类型、空 DID、摘要缺失或端点无法解析时，不能把数据当作完整元数据。兼容新增可选字段，破坏性变化应通过版本管理处理。

字段	类型	必填性	约束
resourceDid	string	是	必须是资源自身 DID
resourceType	enum	是	agent_service、skill、mcp_server、tool_api 等公共枚举
subjectType	enum	是	与资源主体类型一致
name	string	是	ResourceMetadata 的直接字段；多语言表达由 profile 承载
description	string	否	不替代结构化能力字段
capabilityTags	array[string]	否	用于语义治理和发现，元素应可比较
authorizedDomains	array[string]	否	用于授权域判断，不等同于能力标签
services	array[ServiceEndpoint]	否	每项包含 id、type、serviceEndpoint
packageVersion	string	是	与资源包版本保持一致
packageHash	string	是	按 hashAlgorithm 表示摘要引用
/metadataHash			

```
{
  "resourceDid": "did:oan:SKDM:ExampleResourceDid",

```

```

"resourceType": "skill",
"subjectType": "skill",
"name": "Repository analysis",
"description": "Searches a repository and summarizes its project structure.",
"capabilityTags": ["software-development", "code-analysis"],
"authorizedDomains": ["software-development"],
"packageVersion": "1.2.0",
"packageHash": "sha256:example-package-hash",
"metadataHash": "sha256:example-metadata-hash",
"hashAlgorithm": "sha256",
"lifecycleState": "active",
"updatedAt": "2026-09-07T01:02:08Z"
}

```

A.3 ResourcePackage 模式

资源包是当前公共 oan-package 类型定义的可验证、可分发版本对象，字段包括 packageVersion、resourceDid、resourceType、完整 didDocument、didDocumentHash、metadataHash、packageHash、hashAlgorithm、metadata、rootProof 和 createdAt。资源包内部没有通用的 manifest、contentDigest 或 resourceMetadata 顶层字段；如具体资源需要文件清单，应放入约定的扩展字段或外部资源材料，并明确 profile。包内摘要按当前实现分别覆盖 DID 文档、资源元数据和包声明，资源包版本不能代替 DID 文档或资源版本。缺少资源 DID、版本、摘要、元数据或根平台可信发布证明，摘要不一致，或资源类型不一致时拒绝验证。

```

{
  "packageVersion": "1.2.0",
  "resourceDid": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
  "resourceType": "skill",
  "didDocument": {
    "@context": ["https://www.w3.org/ns/did/v1", "https://w3id.org/oan/v1"],
    "id": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "verificationMethod": [{
      "id": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1",
      "type": "Ed25519VerificationKey2020",
      "controller": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "publicKeyMultibase": "zExamplePublicKey"
    }],
    "authentication": ["did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
    "assertionMethod": ["did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu#key-1"],
    "service": []
  },
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "hashAlgorithm": "sha256",
  "metadata": {
    "resourceDid": "did:oan:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
    "resourceType": "skill",

```

```
    "subjectType": "skill",
    "name": "Repository analysis",
    "description": "Searches a repository and summarizes its project structure.",
    "capabilityTags": ["software-development", "code-analysis"],
    "authorizedDomains": ["software-development"],
    "lifecycleState": "active",
    "packageVersion": "1.2.0",
    "packageHash": "sha256:example-package-hash",
    "metadataHash": "sha256:example-metadata-hash",
    "hashAlgorithm": "sha256",
    "updatedAt": "2026-09-07T01:02:08Z"
  },
  "rootProof": {
    "rootDid": "did:oa:INRT:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
    "packageClaims": {
      "resourceDid": "did:oa:SKLG:5HkPq7Vm3RdT9Ya2WcX8Ns4Bf6GjLeZu",
      "resourceType": "skill",
      "version": "1.2.0",
      "didDocumentHash": "sha256:example-did-document-hash",
      "metadataHash": "sha256:example-metadata-hash",
      "packageHash": "sha256:example-package-hash",
      "hashAlgorithm": "sha256",
      "lifecycleState": "active",
      "authorizedDomains": ["software-development"]
    }
  },
  "createdAt": "2026-09-07T01:02:08Z"
}
```

发布校验按以下顺序执行：

1. 解析包版本、资源类型和必填结构。
2. 校验资源 DID、DID 文档、元数据和包声明的一致性。
3. 按规范化规则重新计算内容摘要。
4. 验证根平台可信发布证明后，才进入分发或发现流程。

A.4 根平台可信发布证明模式

根平台可信发布证明在当前公共 `RootProof` 类型中至少包含 `rootDid`，并可以包含 `bulletinEventHash`、`signature`、`packageClaims`、`proof`、`cryptoSuite` 和 `hashAlgorithm`。资源 DID、资源类型、版本、DID 文档摘要、元数据摘要、包摘要、生命周期状态和授权域主要位于 `packageClaims`，而不是强制平铺在证明对象顶层。签发者必须是当前有权执行根平台发布的根平台身份；验证方应检查根平台签名、声明范围、哈希绑定、版本关系、公告事件引用和治理状态，而不能只检查 JSON 字段存在。证明失效、摘要不匹配、签发者未授权或目标版本已撤销时，发现服务节点不得把对应包作为当前有效资源。

字段	证明的事实	验证重点
rootDid	哪个根平台身份形成证明	根平台 DID 文档和授权状态
resourceDid	对应的资源	与包和 DID 文档一致
packageClaims.packageHash	具体发布内容	与资源包摘要相等
packageClaims.version	资源包版本	与 packageVersion 一致
bulletinEventHash	对应的根平台公告事件	公告链和事件顺序可验证
proof/signature	声明完整性	套件、签名输入和签发者公钥

A.5 注册 VC 模式

注册 VC 的公开响应采用 W3C 风格的 `type`、`issuer`、`issuanceDate`、`credentialSubject`、`credentialStatus` 和 `proof` 表达注册服务节点签发的登记事实；公共 Rust 凭证模型还提供 `subject`、`status`、`issuedAt`、`expiresAt` 和 `claims` 形式的内部结构，二者不能在同一 profile 中混写。当前官方注册节点返回的资源注册 VC 使用 `credentialSubject.resourceDid`、`resourceType`、`didDocumentHash`、`metadataHash`、`packageHash`、`packageVersion`、`hashAlgorithm`、`authorizedDomains` 和 `lifecycleState` 等声明。它证明注册服务节点对登记请求的受理，不自动证明根平台已发布或资源端点可用。缺少签发者、目标 DID、声明绑定或 `proof` 时验证失败；VC 为空、过期、吊销或状态未知时，调用方应区分不完整、无效和暂不可判定。

```
{
  "@context": ["https://www.w3.org/2018/credentials/v1", "https://openagenet.org/credentials/v1"],
  "type": ["VerifiableCredential", "OANResourceRegistrationCredential"],
  "issuer": "did: oan: INRG: RegistrarExample",
  "issuanceDate": "2026-09-07T00:00:00Z",
  "credentialSubject": {
    "id": "did: oan: SKDM: ExampleResourceDid",
    "resourceDid": "did: oan: SKDM: ExampleResourceDid",
    "resourceType": "skill",
    "didDocumentHash": "sha256: example-did-document-hash",
    "metadataHash": "sha256: example-metadata-hash",
    "packageHash": "sha256: example-package-hash",
    "packageVersion": "1.2.0",
    "hashAlgorithm": "sha256",
    "authorizedDomains": ["software-development"],
    "lifecycleState": "active"
  },
  "credentialStatus": {
    "type": "OANResourceRegistrationStatus",
    "status": "active"
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

注册 VC 证明注册服务节点已受理并签发登记凭证；只有根平台发布证明和发现索引状态均满足要求时，资源才可作为全网可发现资源使用。

A.6 节点授权 VC 模式

节点授权 VC 在公共凭证模型中以 `credential_type`、`issuer`、`subject`、`role`、`status`、`issuedAt`、可选的 `expiresAt`、`claims` 和 `proof` 表达节点资格；通过 JSON 序列化后字段名分别为 `type`、`issuer`、`subject`、`role`、`status`、`issuedAt`、`expiresAt`、`claims` 和 `proof`。授权域和能力范围通常位于 `claims`，而不是固定平铺字段。角色值应与节点角色枚举和部署 `profile` 一致；授权域和能力范围应使用可比较的结构化值。验证方须同时检查凭证签名、治理状态和当前有效期，不能因历史 VC 存在就忽略暂停或撤销事件。下面的 `credentialSubject`、`validFrom` 等字段仅用于展示层兼容示例，采用公共凭证模型时应使用上述字段。

```
{
  "type": "NodeAuthorizationCredential",
  "issuer": "did:oa:INRT:RootPlatformExample",
  "subject": "did:oa:INDS:DiscoveryNodeExample",
  "role": "discovery",
  "status": "active",
  "issuedAt": "2026-09-01T00:00:00Z",
  "expiresAt": "2027-09-01T00:00:00Z",
  "claims": {
    "authorizedDomains": ["software-development"],
    "capabilities": ["query", "index", "serve_public_results"]
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

A.7 治理事件模式

治理事件的公共公告模型由 `Bulletin`、`BulletinEvent` 和 `BulletinEventCore` 组成。事件核心字段为 `sequence`、`previousHash`、`eventType`、`subjectDid`、`actorDid`、`payload` 和 `createdAt`；事件本体再增加 `eventHash`、`signature`，以及可选的 `proof`、`cryptoSuite` 和 `hashAlgorithm`。当前事件类型包括根平台初始化、内容分发平台信息更新、注册服务节点授权/撤销、发现服务节点授权/域更新/撤销、节点撤销和能力标签树更新。链下投影可以另外记录接收时间、投影时间、游标和错误，但这些不是 `BulletinEvent` 的公共核心字段。链上或公告链原始事件是事实来源，页面展示文本和投影状态是派生信息；事件缺少序列、前序哈希、签名或目标主体，或投影顺序不连续时不得静默更新授权状态，应保留异常并等待补齐。

```
{
  "sequence": 1842,
  "previousHash": "sha256:example-previous-event-hash",
  "eventType": "DISCOVERY_NODE_AUTHORIZED",
  "subjectDid": "did:oa:INDS:DiscoveryNodeExample",
  "actorDid": "did:oa:INRT:RootPlatformExample",
  "payload": { "authorizedDomains": ["software-development"] },
  "createdAt": "2026-09-07T01:02:03Z",
  "eventHash": "sha256:example-event-hash",
  "signature": "zExampleSignature",
  "cryptoSuite": "ed25519-sha256",
}
```



```
"hashAlgorithm": "sha256"
}
```

A.8 注册请求与响应模式

当前官方注册服务节点的浏览器/客户端入口为 POST /resources/register，请求反序列化为 ResourceRegistrationSubmission，包含 resourceDid、resourceType、didDocument、didDocumentHash、metadata、packageVersion、packageHash、metadataHash、hashAlgorithm、可空的 registrationCredential 和 subjectControlProof。节点内部向根平台转发时，才会包入 registrarDid 和 upstreamAuth，使用 POST /root/resources/verify-and-publish。因此，不能把内部转发包直接当作公网注册请求。响应通常包含 status、resourceDid、resourceType、registrationCredential 和 rootResponse；不同部署可以追加状态或关联标识。相同 DID、版本和摘要的重试是否完全幂等，应以节点存储和根平台实现为准；缺失控制证明、字段不完整或前序版本不匹配时不得写入有效登记。

```
POST /resources/register
Content-Type: application/json

{
  "resourceDid": "did:oan:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "didDocument": {},
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadata": {
    "resourceType": "skill"
  },
  "packageVersion": "1.2.0",
  "packageHash": "sha256:example-package-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "hashAlgorithm": "sha256",
  "registrationCredential": null,
  "subjectControlProof": {
    "challenge": {},
    "proof": {}
  }
}
```

情形	响应状态	处理
校验通过并完成上游处理	2xx	返回登记结果、注册 VC 和根平台响应
请求体无法反序列化或形状校验失败	400	修正 DID、摘要、控制证明或必填字段
上游根平台拒绝	依节点映射	根据返回错误码处理，不把失败当作已登记
网络失败或上游暂不可用	5xx 或连接错误	按请求 DID、版本和摘要查询后再决定是否重试

A.9 发现请求与响应模式

公共 `ResourceDiscoveryQuery` 的字段是可选的 `query`、`resourceType`、`capabilityTags`、`protocol`、`version`，以及带默认值的 `versionMode` 和 `limit`。完整 DID 精确查询、结构化条件查询、标签查询和自然语言查询是节点对这些字段的使用方式，不是另设的 `queryType` 核心字段；分页和新鲜度字段可以出现在节点读模型或扩展 `profile` 中。公共 `ResourceDiscoveryResponse` 包含发现服务节点 DID、候选数组、创建时间和可选响应 `proof`；候选包含资源 DID、资源类型、分数、版本、生命周期、能力标签、授权域、服务、协议绑定、资源包信息和根平台可信发布证明引用。索引时间、来源说明和查询解释属于节点或官网派生展示信息。语义分数不能替代可信证据；未知条件、非法 DID、过期游标或查询服务不可用应返回可识别错误，不以空结果掩盖失败。

```
{
  "query": "I need a tool that can search code repositories and summarize the project structure
    .",
  "resourceType": "tool_api",
  "capabilityTags": ["code-analysis", "repository-search"],
  "protocol": "https",
  "versionMode": "latest",
  "limit": 20
}
```

```
{
  "discoveryDid": "did:oon:INDS:DiscoveryNodeExample",
  "candidates": [{
    "resourceDid": "did:oon:SKDM:ExampleResourceDid",
    "resourceType": "skill",
    "score": 0.94,
    "version": "1.2.0",
    "lifecycleState": "active",
    "capabilityTags": ["code-analysis", "repository-search"],
    "authorizedDomains": ["software-development"],
    "services": [],
    "protocolBindings": [],
    "packageInfo": { "packageHash": "sha256:example-package-hash" },
    "rootProof": { "rootDid": "did:oon:INRT:ExampleRootDid" }
  ]},
  "createdAt": "2026-09-07T01:02:08Z",
  "proof": null
}
```

A.10 错误响应模式

当前节点公开错误结构以 `{ "error": "machine_readable_error_code" }` 为基础；可信调用拒绝时还可能包含 `reason`。未来或特定网关可以扩展为对象，加入消息、关联请求标识和重试提示，但这不是所有节点当前都保证提供的字段。错误码应区分输入错误、签名/凭证错误、未授权、未找到、重复、版本冲突、依赖不可用、同步滞后和内部错误。面向公众的消息不得泄露私钥、数据库或内部配置；未知错误码的客户端应按失败处理而不是默认成功。

```
{
  "error": "sync_stale",
  "reason": "discovery_index_behind_publication"
}
```

若网关采用扩展错误对象，客户端应先判断 `error` 是字符串还是对象，再读取可选字段；不能假定对象形状，也不能把 HTTP 状态码单独当作业务错误码。注册服务节点当前会将部分上游错误映射为自身的 HTTP 错误响应，调用方应同时保留 HTTP 状态和响应体。

A.11 节点授权与治理状态模式

节点授权状态和治理状态通常由链下信任索引器或节点状态接口形成读侧投影，不等同于 `NodeAuthorizationCredential` 或公告事件本体。投影可包含节点 DID、节点角色、授权域、当前状态、状态原因、有效期、最近治理事件、事件序列、同步游标、数据时间和新鲜度；具体字段以对应接口为准。状态值至少应能区分有效、暂停、撤销、过期、未知和同步中；缺失治理来源或索引器落后时，不得伪造有效状态。敏感写入动作在状态未知或超过部署允许的新鲜度时采用拒绝或人工确认策略，展示接口可以单独标注延迟。

状态	允许的只读用途	对写入或授权操作的影响
active	可展示并进入常规验证	按能力范围允许
suspended	可展示历史状态	拒绝新的敏感操作
revoked	仅保留审计和历史引用	拒绝所有需要当前授权的操作
expired	可提示授权已到期	不得作为当前授权依据
unknown/syncing	只能显示不确定状态	等待确认，不得放行敏感操作

A.12 分发清单与同步游标模式

分发侧当前公共结构分为两类：内容分发平台索引响应使用 `items`、`count`、`afterCursor`、`nextCursor` 和 `hasMore`，每个条目包含 `cursor` 与完整资源包；根平台向发现服务节点发送的批量通知使用批次标识、根平台 DID、目标发现服务节点 DID、授权域、序列范围、条目、内容分发平台清单地址、更新地址和 `proof`。条目再携带资源 DID、操作、资源包版本、DID 文档摘要、元数据摘要、包摘要、哈希算法、生命周期状态、公告序列和公告事件哈希。清单版本、生成时间、错误或确认字段可由具体接口补充，不能把一个接口的字段直接套到另一接口。游标必须单调推进，重复批次应可安全重放，序列缺口或摘要不匹配时暂停该条目处理。游标是传输进度，不等于资源已经在发现服务节点建立索引。

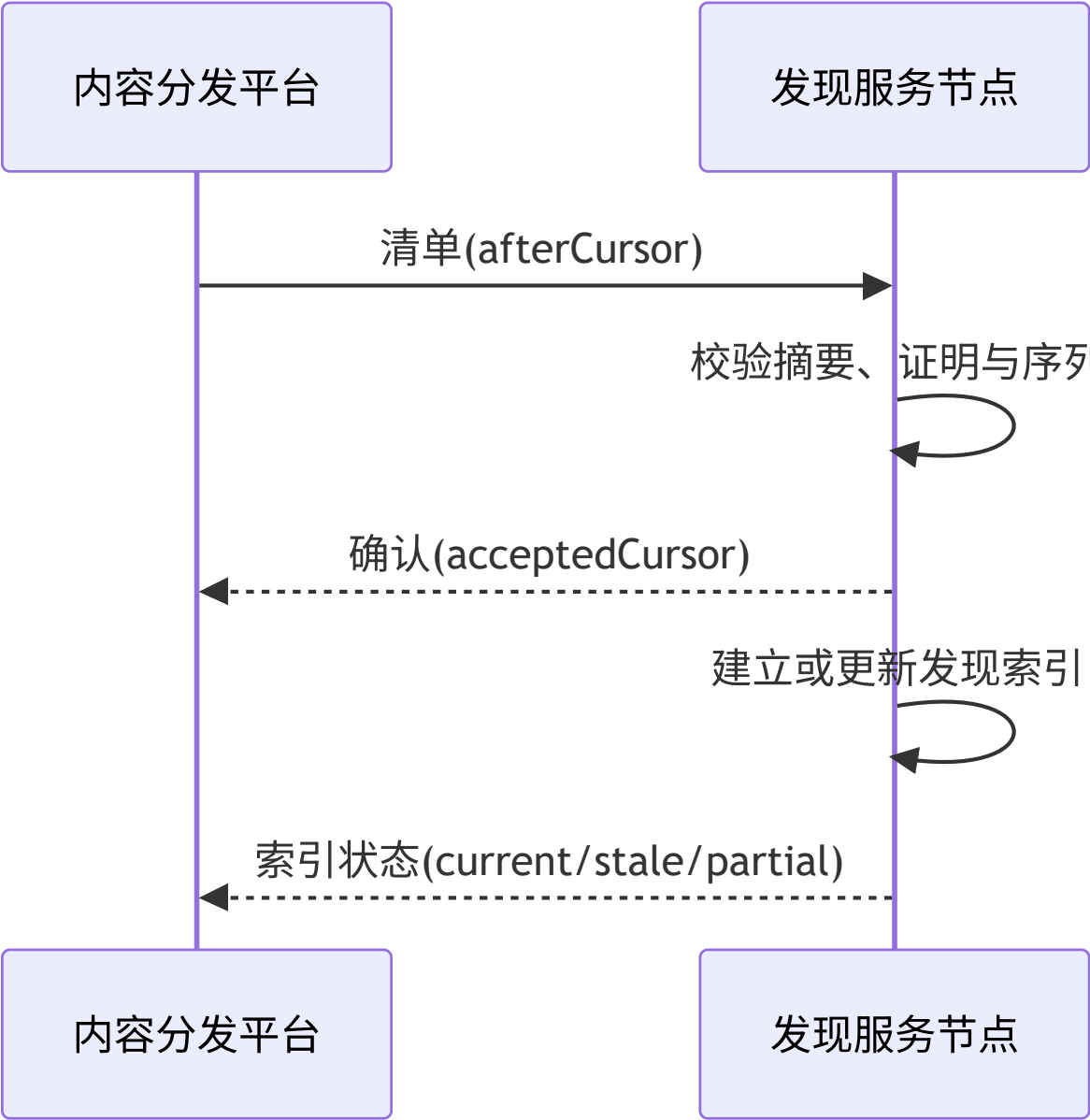
```
{
  "items": [{
    "cursor": 1842,
    "package": {
      "packageVersion": "1.2.0",
      "resourceDid": "did:oa:SKDM:ExampleResourceDid",
      "resourceType": "skill",
      "didDocument": { "id": "did:oa:SKDM:ExampleResourceDid" },
      "didDocumentHash": "sha256:example-did-document-hash",
    }
  ]
}
```

```
"metadataHash": "sha256:example-metadata-hash",
"packageHash": "sha256:example-package-hash",
"hashAlgorithm": "sha256",
"metadata": {
  "resourceDid": "did:oon:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "subjectType": "skill",
  "name": "Repository analysis",
  "lifecycleState": "active",
  "packageVersion": "1.2.0",
  "packageHash": "sha256:example-package-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "hashAlgorithm": "sha256",
  "updatedAt": "2026-09-07T01:02:08Z"
},
"rootProof": { "rootDid": "did:oon:INRT:ExampleRootDid" },
"createdAt": "2026-09-07T01:02:08Z"
}
}],
"count": 1,
"afterCursor": 1800,
"nextCursor": 1842,
"hasMore": false
}
```

根平台通知条目的最小形状示例为:

```
{
  "resourceDid": "did:oon:SKDM:ExampleResourceDid",
  "resourceType": "skill",
  "operation": "upsert",
  "packageVersion": "1.2.0",
  "didDocumentHash": "sha256:example-did-document-hash",
  "metadataHash": "sha256:example-metadata-hash",
  "packageHash": "sha256:example-package-hash",
  "hashAlgorithm": "sha256",
  "lifecycleState": "active",
  "bulletinSequence": 1842,
  "bulletinEventHash": "sha256:example-event-hash",
  "capabilityTags": ["software-development"],
  "authorizedDomains": ["software-development"]
}
```

清单/通知 -> 校验摘要、证明、序列和授权域 -> 确认传输进度 -> 建立或更新发现索引



A.13 搜索解释与证据模式

搜索解释应说明查询类型、解析后的条件、命中的能力标签或用例、结构化过滤、相关性分数、结果来源、版本、资源状态、索引时间、根平台可信发布证明和新鲜度。解释是帮助调用方理解结果的派生信息，不能作为签名或授权证据。字段缺失、语义模型版本不兼容或来源不可验证时，应明确标注未知或不完整，而不是补写推测性理由。

解释层	可表达内容	不能替代的证据
查询解析	查询类型、标准化条件、标签和用例命中	资源控制权
相关性	分数、匹配字段、排序原因	根平台发布证明
来源	发现节点、索引时间、索引版本	节点当前授权状态
可信状态	VC、根平台证明、治理状态和新鲜度	实际调用结果保证

```
{
  "queryExplanation": {
    "queryType": "semantic",
    "normalizedIntent": "repository structure summarization",
    "matchedTags": ["code-analysis", "repository-search"],
    "filtersApplied": { "resourceTypes": ["skill", "tool_api"] },
    "modelVersion": "semantic-index-v1"
  },
  "evidence": {
    "resourceDid": "did: oan: SKDM: ExampleResourceDid",
    "rootPublication": "verified",
    "governance": "active",
    "indexedAt": "2026-09-07T01:02:08Z",
    "freshness": "current"
  }
}
```

A.14 参考来源

来源	类型	链接
oan-protocol-common	代码仓: 规范性数据结构和 schema	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓: TypeScript 数据类型和客户端校验	https://github.com/OpenAgenet/oan-sdk-ts
did: oan DID Method Specification	标准/方法规范: DID 文档和标识数据结构	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Resource Identity and Discovery	IETF 草案: 资源身份数据结构	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

附录 B 协议与 API 参考

本附录提供接口阅读和实现对照规则。路径、字段和状态以各节点服务入口、`oan-protocol-common/schemas/agent-contract/openapi.yaml`、官网前后端 API 封装及实际部署配置共同核对；当前只在通用契约中声明的能力不能被写成已由所有节点暴露。接口变更必须关联协议版本、错误模式、认证方式、幂等规则和第二十五章变更记录。

B.1 端点目录

端点目录按服务角色列出健康、DID、资源注册、根平台发布、内容分发、发现、治理状态和统计接口，并区分公网入口、节点间入口、仅限内部入口和部署前缀。当前官网通过同源后端转发部分注册和发现请求；节点本身还暴露独立服务端点，因此官网路径不能自动推导为所有节点的协议路径。示例基础 URL 使用占位符，不把官网当前域名或某台服务器地址固化为协议要求；每行至少记录调用方向、HTTP 方法、认证、请求/响应模式、失败和版本。下表只列出当前代码中可以核对的代表性入口，未列出的管理、统计和调试接口仍以对应节点路由为准。

服务角色	调用方向、方法与路径	认证边界	主要返回
注册服务节点	客户端 -> 注册服务节点： <code>POST /resources/register</code> ；客户端 -> 注册服务节点： <code>GET /registrar/status</code> 、 <code>GET /registrar/did</code>	注册入口接收资源控制证明；向根平台转发时使用节点间签名	注册 VC、受理状态、根平台响应
根平台	注册服务节点 -> 根平台： <code>POST /root/resources/verify-and-publish</code> ；客户端或节点 -> 根平台： <code>GET /root/resources/{did}</code> 、 <code>GET /root/status</code>	发布写入仅限授权节点或管理调用；公开读取取决于部署	根平台可信发布证明、资源版本、状态和游标
内容分发平台	根平台或发布者 -> 内容分发平台： <code>POST /cdn/resources</code> 、 <code>POST /cdn/resources/batch</code> ；客户端或节点 -> 内容分发平台： <code>GET /cdn/resources/{did}</code> 、 <code>GET /cdn/resources/index</code>	发布使用节点间认证；资源读取和索引读取取决于部署	资源包、索引条目、同步状态

服务角色	调用方向、方法与路径	认证边界	主要返回
发现服务节点	客户端 -> 发现服务节点: POST /discovery/ resources/query; 节点或 运维方 -> 发现服务节点: GET /discovery/index/ resources/{did}、 GET /discovery/status	查询可公开, 管理和同步接 口受限	资源结果、来源和证明引用
运维接口	运维方 -> 节点: GET /health 及节点状态、 指标和审计接口	运维认证和网络边界	状态摘要, 不含秘密

核心载荷类型与端点的对应关系如下。共享类型来自 `oan-protocol` 或 `oan-package` 时, 字段名称以 `serde` 的 JSON 映射为准; `Value` 表示当前实现的动态响应, 不能据此推导不存在的固定 schema。

端点	请求 JSON 类型	响应类型或主要字段	备注
POST /resources/ register	ResourceRegistrationSubmitRequest	动态 JSON: 登记状态、资源 DID、注册 VC、根平台响应	/resources/submit 绑定同一处理函数
POST /root/resources/ verify-and-publish	ResourceVerifyAndPublishRequest	动态 JSON: resource-verified-and-queued、版本、三类哈希和 CDN 队列状态	仅限授权注册服务节点的节点间调用
POST /cdn/resources	ResourceCdnPublishRequest	动态 JSON: published、版本和发布游标	资源包必须通过根平台来源和哈希校验
POST /discovery/ resources/query	ResourceDiscoveryQuery	ResourceDiscoveryResponse	limit 默认 10, versionMode 默认 latest
POST /discovery/query/ explain	ResourceDiscoveryQuery	动态解释 JSON	不等同于普通发现响应
GET /cdn/resources/{did} }	路径参数 did	ResourcePackage	直接读取资源包
POST /cdn/resources/ batch-get	本地 ResourceBatchGetRequest	requestedCount、 foundCount、items	请求体包含 resourceDids 数组

B.2 请求头字段与签名

请求头和签名材料按接口 profile 定义。当前节点间 SignedRequestEnvelope 使用 requestId、protocolVersion、purpose、method、path、aud、requestTimestamp、requestNonce、bodyHash 和 proof 等 JSON 字段承载签名语义; 官网面向浏览器的公开注册和发现入口不应凭空增加一组未在服务入口定义的 X-OAN-* 必需头。若接口使用协议信封承载这些信息, 应以 JSON 字段为准; 若部署层额外使用 HTTP 头, 只能作为部署 profile 说明。服务端需校验签名覆盖的规范化请求、目标 DID、时间窗和 nonce; 缺失、过期、重放或目标不符时拒绝。

节点间协议信封字段的作用可按下表对照。method 和 path 绑定被保护的 HTTP 请求, aud 绑定接收方, bodyHash 绑定请求体; 这些字段共同避免把一个节点发出的有效证明挪用于另一条请求。浏览器提交资源时, 资源控制证明位于请求体的 subjectControlProof, 不应把它误写成节点间 HTTP 头。

字段	作用	校验重点
requestId	标识一次节点间请求	用于日志关联，不能替代防重放机制
protocolVersion、purpose	标识协议版本和调用目的	与接收接口允许的 profile 匹配
method、path、aud	绑定 HTTP 方法、路径和接收方	不能与实际请求或目标节点不一致
requestTimestamp、requestNonce	限制时间范围并阻止重复使用	检查时钟窗口和 nonce 重放缓存
bodyHash、proof	绑定请求体并提供完整性证明	重新计算请求体摘要并验证证明

```
POST /resources/register
Host: registrar.example
Content-Type: application/json
```

节点间请求的 `upstreamAuth.proof` 应覆盖方法、规范化路径、目标、时间/nonce、内容摘要和实际请求体；服务器完成时间窗、重放缓存、签名者授权和资源控制权检查后才执行写入。浏览器注册请求的资源控制证明位于请求体的 `subjectControlProof`，它与节点间的 `upstreamAuth` 不是同一种凭证。

B.3 响应头字段与缓存指令

响应可以在适用时提供来源、协议/模式版本、数据时间、新鲜度、ETag、缓存控制、关联标识和必要安全头；这些字段不是当前所有节点接口都统一保证的公共响应头，下面的 `x-*` 项仅可作为部署层或接口 profile 的扩展约定。ETag 或缓存命中只说明内容版本未变，不证明资源状态或治理授权仍有效；返回旧快照时，响应或正文应有可识别的时间和降级标识，不能用缓存掩盖上游失败。

字段或指令（适用时）	作用	使用注意
ETag	标识响应内容版本	不等同于治理状态有效
Cache-Control	指定缓存和重新验证策略	旧快照须同时标识新鲜度
X-Request-Id	关联客户端、节点和日志	不得包含私密信息
X-OAN-Data-Time	表示数据生成或采集时间	与服务器响应时间区分
X-OAN-Freshness	表示 <code>current</code> 、 <code>stale</code> 等状态	不能掩盖上游失败

B.4 状态码与错误码

HTTP 状态码表达传输层和通用处理结果，OpenAgenet (OAN) 错误码表达字段错误、签名无效、未授权、未找到、重复、版本冲突、依赖失败、同步滞后和内部错误等具体语义。当前公共错误码词汇以 `oan-protocol-common/docs/error-codes.md` 为准，例如 `invalid_did_document_structure`、`invalid_registration_credential`、`invalid_request_signature`、`unauthorized_domains`、`invalid_root_proof` 和 `package_decode_failed`；该文件定义错误语义，但没有把每个错误码固定绑定到唯一 HTTP 状态，不同节点可能通过 HTTP 状态或响应体映射这些错误。客户端按实际错误结构决定重试；不能把所有 2xx 都当作发布成功，也不能把 404、空结果和同步未完成混为一谈。

HTTP 状态	OAN 错误码示例	客户端动作
400	<code>invalid_did_document_structure</code> 、 <code>invalid_authorized_domains</code> 、 <code>package_decode_failed</code>	修正输入或资源包后重试；具体状态以接口实现为准

HTTP 状态	OAN 错误码示例	客户端动作
401/403	invalid_request_signature、 unauthorized_domains	重新认证、改用授权节点或停止请求
404	节点返回的资源未找到错误	区分未找到和索引未同步
409	节点返回的重复或版本冲突错误	获取当前版本并按更新规则处理
429	部署层限流错误	遵循重试提示并退避
5xx	依赖不可用或内部错误	只有在接口标明可重试时才退避重试

B.5 分页与游标规则

列表接口使用明确的 limit、游标和排序规则，游标只能由服务端生成和解释，客户端不得假设其可读或可修改。当前 ResourceDiscoveryQuery 公共类型固定提供 limit，默认值为 10，但 POST /discovery/resources/query 的请求模型没有游标字段；发现服务会按 limit 截取候选结果。实际提供 afterCursor、nextCursor 和 hasMore 的是内容分发平台 GET /cdn/resources/index，其游标与发现查询的 limit 不是同一类型。页大小应有部署上限，排序键在分页期间保持稳定；过期、非法或与过滤条件不匹配的游标应返回错误。发布游标、同步游标、事件游标和分页游标不能交叉使用。

```
GET /cdn/resources/index?limit=20&afterCursor=1800
Accept: application/json

{
  "items": [],
  "count": 0,
  "afterCursor": 1800,
  "nextCursor": 1800,
  "hasMore": false
}
```

分页游标只表示内容分发平台列表的读取位置；发现查询的 limit 只控制本次候选结果数量，不产生可续接的发现分页游标。发布序列、分发同步游标和治理事件游标必须使用各自接口定义的字段和校验规则。

B.6 版本与兼容性头字段

协议版本、资源模式版本、能力声明和兼容策略通过已定义的协议信封字段、请求体字段或部署 profile 表达，并在请求和响应中保持可追溯。当前公共协议常量包括 oan-resource-2026，而 ResourceDiscoveryQuery.versionMode 默认值为 latest；这类协议字段不能直接改写成未经代码支持的统一 HTTP 头。服务端遇到不支持的版本应返回明确不兼容结果；兼容扩展不得改变 DID、签名、状态或发布序列语义，版本选择规则参见第二十五章。

```
{
  "query": "a tool that can search code repositories and summarize project structure",
  "capabilityTags": ["code-search", "project-structure-summary"],
  "versionMode": "latest",
  "limit": 10
}
```

上例是 POST /discovery/resources/query 的发现查询请求示例, 字段均对应当前 ResourceDiscoveryQuery ; protocolVersion 则属于节点间 SignedRequestEnvelope, 不应混入该查询对象。当前查询模型没有统一的 capabilities 字段; 能力条件应通过 capabilityTags 或目标接口明确支持的扩展表达。服务端不支持请求版本时, 应返回明确的版本不兼容错误及可接受版本范围; 客户端不得仅因收到 HTTP 2xx 就忽略响应模式版本。

B.7 健康、就绪与存活响应格式

当前各节点公开 /health 响应的公共最小结构是 HealthResponse: status、nodeType 和可选的节点 DID; 节点状态接口可以另外提供数据库、上游、同步游标或错误摘要。运维系统可以据此区分进程存活、依赖就绪、业务可用和数据同步状态, 但不能把所有字段都当作每个节点都存在的统一 schema。liveness 通过不等于 readiness, 通过 readiness 也不等于注册或发现烟测通过; 敏感配置和令牌不得出现在响应。

```
{
  "status": "ok",
  "nodeType": "discovery",
  "did": "did:oan:INDS:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz"
}
```

若部署提供扩展状态接口, 可以在上述最小响应之外增加检查时间、依赖状态和同步游标; 扩展字段应标明来源和版本, 不能把该扩展响应反向当作所有节点 /health 的强制返回结构。

检查类型	证明内容	不代表的内容
liveness	进程仍能响应	依赖和业务可用
readiness	服务具备接收请求的条件	注册或发现烟测成功
dependency	数据库、上游或索引器状态	所有业务路径正常
smoke test	关键业务流程可完成	长期性能和全量正确性

B.8 公开、认证与仅限运维 API 矩阵

接口矩阵至少区分公开只读接口、发布/注册认证接口、节点间接口和仅限运维接口。公开发现、DID、状态和必要统计可以供互联网访问; 写入、管理、数据库、完整日志、访问统计和密钥相关接口必须受认证、网络边界和审计保护。官网后端的中转接口不能扩大下游服务权限, 普通用户不可通过公开接口取得后台统计或秘密。

访问级别	示例	允许的数据	保护要求
公开只读	DID、发现、必要状态	已发布资源和公开证明	限流、输入校验、审计
已认证写入	注册、更新、节点间同步	授权范围内对象	签名、nonce、幂等和授权检查
仅运维	指标、完整日志、管理操作	内部运行数据	强认证、网络隔离和最小权限
禁止公开	数据库、私钥、访问统计明细	不对互联网用户开放	不得通过官网中转泄露

B.9 参考来源

来源	类型	链接
oan-protocol-common	代码仓: 协议类型、OpenAPI 契约和错误码	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓: 根平台 API	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓: 注册 API	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓: 发现 API	https://github.com/OpenAgenet/oan-discovery-node
oan-sdk-ts	代码仓: 客户端 API 封装	https://github.com/OpenAgenet/oan-sdk-ts
OAN Resource Identity and Discovery	IETF 草案: 资源身份和发现 API	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/
Efficient Agent Discovery Profile	IETF 草案: 发现 API profile	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

附录 C 密码学与编码

本附录固定 OpenAget (OAN) 在身份、凭证、资源发布和跨节点校验中使用的密码学与编码边界。当前共享 Rust 实现以 oan-crypto、oan-credentials、oan-did-oan 为主要依据，浏览器侧 TypeScript SDK 使用 Web Cryptography API 生成 Ed25519 本地身份；任何跨语言实现都应以相同规范化输入复算相同摘要，并对算法、编码和错误状态显式处理。

C.1 支持的算法

当前 CryptoSuite 包含 Ed25519Sha256Legacy、Ed25519Sha256 和 Sm2Sm3：前两者使用 Ed25519 签名与 SHA-256 哈希，后者使用 SM2 签名与 SM3 哈希。对应验证方法类型为 Ed25519VerificationKey2020 或 SM2VerificationKey2020，证明类型为 Ed25519Signature2020 或 SM2Signature2020。算法套件必须与公钥、proof 和哈希算法一致；未知套件拒绝，旧套件只能按兼容策略保留，不能悄然降级。

算法套件	签名算法	哈希算法	典型用途
Ed25519Sha256Legacy	Ed25519	SHA-256	兼容既有凭证或摘要流程
Ed25519Sha256	Ed25519	SHA-256	当前 Ed25519 证明流程
Sm2Sm3	SM2	SM3	支持 SM2 的身份和证明流程

算法套件、验证方法、公钥编码和 proof 类型必须成组匹配；未知或被弃用的套件不能静默切换到另一套算法。

在实现选择上，Ed25519Sha256Legacy 与 Ed25519Sha256 使用同一种 Ed25519 密钥和 SHA-256 哈希，但签名输入不同：前者先对规范化 JSON 求 SHA-256，再对得到的 64 个小写十六进制字符的 UTF-8 字节签名；后者直接对规范化 JSON 的 UTF-8 字节签名。Sm2Sm3 直接对规范化 JSON 的 UTF-8 字节使用 SM2 签名，摘要函数使用 SM3。跨语言实现不能只根据签名算法名称判断输入，必须同时读取 cryptoSuite 和 hashAlgorithm。

选择项	当前实现中的判断	失败处理
cryptoSuite	决定签名算法、哈希算法和签名输入形式	未识别时拒绝
proof.type	与套件对应的 Ed25519Signature2020 或 SM2Signature2020	类型不匹配时拒绝
验证方法类型	从 Ed25519VerificationKey2020 或 SM2VerificationKey2020 推导套件	无法推导时拒绝

选择项	当前实现中的判断	失败处理
公钥载体	二选一使用 <code>publicKeyMultibase</code> 或 <code>publicKeyJwk</code>	两者均缺失或无法解析时拒绝

C.2 规范化 JSON 与哈希

共享实现先将可序列化对象转换为 JSON，再按对象键的字典序递归排列，数组保持原顺序，字符串使用 JSON 转义，数字使用序列化后的表示，省略空字段的规则由 Rust/TypeScript 数据模型决定。`Ed25519Sha256Legacy` 对规范化 JSON 的 SHA-256 十六进制结果签名，`Ed25519Sha256` 和 `Sm2Sm3` 对规范化 JSON UTF-8 字节签名。跨语言实现必须固定 UTF-8、键排序、数字和空字段规则；任何差异都会导致摘要或签名验证失败。

```
{
  "b": 2,
  "a": {
    "y": 1,
    "x": "OAN"
  },
  "items": ["one", "two"]
}
```

规范化时键递归按字典序排列、数组保持原顺序，输入使用 UTF-8。对应的规范化结果为：

```
{
  "a": {
    "x": "OAN",
    "y": 1
  },
  "b": 2,
  "items": ["one", "two"]
}
```

跨语言测试应同时固定空值、数字表示、字符串转义和对象字段省略规则，不能仅依赖某一语言运行时的默认序列化行为。

例如，上述规范化字符串的 SHA-256 结果为 `03332fb018e2f0376e263dabf4b2231e3bd70767e19fb7e263060efb68476e0`。这个值只对应示例对象、示例键顺序和示例 UTF-8 编码；它不是协议固定常量。Rust 侧可使用 `oan-crypto::canonical_json` 与 `hash_json_with_suite` 产生基准，TypeScript 侧应先按相同规则递归排序，再使用 `crypto.subtle.digest("SHA-256", ...)` 对 UTF-8 字节求摘要。

实现时要特别区分三种变化：对象成员顺序变化在规范化后不应改变结果，数组元素顺序变化会改变结果，数字从 1 改为 1.0 是否被视为相同则取决于输入在进入规范化函数前的 JSON 数值表示。生产代码不得先把 JSON 转成带缩进的文本再直接签名，也不得依赖数据库返回字段的顺序。

C.3 JWS 与分离式签名

OAN 当前共享实现使用 `DataIntegrityProof` 表达分离式证明，包含 `type`、`creator`、`created`、`proofPurpose`、`proofValue`、`cryptoSuite`、`hashAlgorithm` 和 `verificationMethod`。`proofValue` 是 Base64url

无填充的签名值；签名输入由待签名对象的规范化结果按算法套件确定。验证方必须先确定套件、重建签名输入、验证公钥与签名，再检查 creator、verificationMethod、目标 DID、有效期和状态；不能只比较 proofValue 字符串。

```
{
  "type": "DataIntegrityProof",
  "creator": "did:oan:SKDM:ExampleResourceDid#key-1",
  "created": "2026-09-07T01:02:03Z",
  "proofPurpose": "assertionMethod",
  "verificationMethod": "did:oan:SKDM:ExampleResourceDid#key-1",
  "cryptoSuite": "Ed25519Sha256",
  "hashAlgorithm": "SHA-256",
  "proofValue": "base64url-without-padding"
}
```

验签失败应区分规范化输入不一致、公钥不匹配、签名无效、证明用途不符和状态不可接受，便于调用方决定是修正请求、重新获取数据还是停止处理。

一个最小的验签顺序如下：

- 1. 从 proof.cryptoSuite 或兼容的 proof.type 确定套件，并确认它与验证方法、公钥格式一致。
- 2. 从 DID 文档中按 verificationMethod 定位公钥，检查方法标识、控制者和目标 DID 的关系。
- 3. 移除待验证对象中的 proof，按套件重建规范化签名输入；遗留套件还要先计算 SHA-256 十六进制摘要。
- 4. 对 proofValue 做 Base64url 无填充解码并执行签名验证，再检查 created、用途、签发者授权和凭证状态。

```
payload without proof -> canonical JSON -> suite-specific signature input
                                -> Base64url proofValue -> verify with DID public key
```

proofValue 能够被解码只说明编码形式可读，不能说明签名正确；签名正确也不自动说明签发者获得了根平台授权或资源仍处于可用状态。

C.4 DID 与密钥编码

资源 DID 的现行语法为 did:oan:<四位大写语义码>:<32位 Base58 后缀>，后缀使用 Bitcoin Base58 字符集并排除 0、0、I、l。did:oan 解析器还校验资源类型与语义码主体部分的对应关系。公钥可按算法使用 z 前缀的 Base58 multibase，或使用 JWK：Ed25519 使用 kty=OKP、crv=Ed25519 和 Base64url 无填充的 x，SM2 使用 kty=EC、crv=SM2 以及 x、y。私钥不得进入 DID 文档、普通协议请求或服务日志。

```
did:oan:<四位大写语义码>:<32位 Base58 后缀>
```

材料	推荐表示	不得出现
资源 DID	did:oan 字符串	错误语义码、非法 Base58 字符
Ed25519 公钥	multibase 或 Ed25519 JWK	私钥、未声明算法的裸字符串

材料	推荐表示	不得出现
SM2 公钥	SM2 JWK 或实现支持的公钥表示	不完整的 x/y 坐标
私钥	客户端受保护存储	DID 文档、普通请求、服务日志

`publicKeyMultibase` 当前由 `z` 前缀加 Base58 编码的公钥字节组成；Ed25519 使用 32 字节公钥，SM2 使用未压缩 SEC1 公钥字节。JWK 使用 Base64url 无填充编码的字段，Ed25519 的 `x` 解码后应为 32 字节，SM2 的 `x` 和 `y` 解码后各为 32 字节。`oan-crypto` 会根据验证方法或显式 `cryptoSuite` 解析这些载体，解析成功后再进行签名验证。

示例 DID 的语义码只负责表达主体或资源类型与应用域，不由后缀直接承载公钥。随机生成的 DID 与密钥对是两个独立生成步骤；如果业务需要可重复派生，应明确使用 `DidOan::derive` 的控制材料和 `nonce`，并将派生规则视为特定实现能力，不能把随机生成的 DID 当作可从公钥反推的标识。

C.5 VC 与 VP 编码

注册 VC 和节点授权 VC 使用 JSON 对象表达 `issuer`、`subject`、状态、签发/到期时间、`claims` 和 `proof`；共享凭证实现以 `AgentRegistrationCredential`、`NodeAuthorizationCredential` 及 `DataIntegrityProof` 为基础。签发时对无 `proof` 对象生成签名，验证时移除 `proof` 重建原始载荷并按 `proof` 指定套件验签。VP 若由外部系统呈现，应保留原始结构、凭证引用和验证结果；可解析不等于 OAN 信任策略接受，过期、吊销、主体不符或签发者未授权必须拒绝或标为不可接受。

```
{
  "type": ["VerifiableCredential", "AgentRegistrationCredential"],
  "issuer": "did:oan:REGS:RegistrarExample",
  "credentialSubject": {
    "id": "did:oan:SKDM:ExampleResourceDid"
  },
  "proof": { "type": "DataIntegrityProof", "proofValue": "..." }
}
```

VC 解析成功只表示结构可读；调用方仍必须验证签名、签发者授权、主体、有效期和凭证状态。VP 作为外部呈现容器时，应保留凭证来源和验证结果，不把呈现动作本身当作授权。

注册 VC 的验证对象是去除 `proof` 后的 `AgentRegistrationCredential`，节点授权 VC 的验证对象是去除 `proof` 后的 `NodeAuthorizationCredential`。验证链可用下表作为实现检查顺序：

检查	关注对象	不通过时的含义
结构	<code>type</code> 、 <code>issuer</code> 、 <code>subject</code> 、时间、 <code>claims</code> 、 <code>proof</code>	凭证不能解析或不能进入后续验证
签名	<code>proof</code> 、签发者 DID 文档中的公钥	凭证内容未被对应私钥证明
绑定	<code>issuer</code> 、 <code>subject</code> 、资源 DID、DID 文档哈希和资源包哈希	凭证与当前资源不对应
授权	签发者身份、节点角色、授权域和治理状态	签发者不能在当前边界内作出该声明
状态	<code>status</code> 、 <code>issuedAt</code> 、 <code>expiresAt</code> 及 可用的吊销事实	凭证已失效或不应继续使用

验证方应保存足以复核的凭证原文、DID 文档、验证时间和错误码；不应只保存一个“验证成功”的布尔值。

C.6 密钥备份与恢复格式

本地身份备份是客户端敏感文件，可包含版本、更新时间、默认主体/智能体标识、主体/智能体/节点记录，以及 DID 文档、公私钥 JWK 和资源 profile。TypeScript SDK 当前使用版本 1 的身份存储快照；官网注册流程下载的身份备份文件名带有时间标识，资源提交成功时还可将注册 VC、DID 文档和本地身份文件打包保存。备份文件应由用户私下保管，导入时校验 JSON 结构、版本、公钥私钥对应关系和 DID 一致性；不支持的版本、缺失密钥或解析失败不得部分加载。

```
{
  "version": 1,
  "updatedAt": "2026-09-07T01:02:03Z",
  "defaultSubjectDid": "did:oon:SUBJ:ExampleSubjectDid",
  "subjects": [],
  "agents": [],
  "importedNodes": []
}
```

导入流程应先完成完整性和版本检查，再一次性加载主体、智能体和节点记录；发现公钥与私钥不匹配、DID 不一致或版本不支持时，应拒绝整个备份，不能生成部分可用身份。

备份文件中的私钥 JWK 是高敏感材料。浏览器下载、导入和再次导出都应由用户主动触发，页面、注册服务节点、根平台和发现服务节点不需要接收该私钥。导入后，客户端可以用私钥对一段测试消息签名，再用备份中的公钥验证，以确认密钥对一致；该测试不得把私钥或测试签名写入日志。备份文件名中的时间标识只用于区分下载文件，不参与身份或签名计算。

```
{
  "version": 1,
  "subjects": [
    {
      "did": "did:oon:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
      "verificationMethodId": "did:oon:DVFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz#key-1",
      "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "example-public-x" },
      "privateKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "example-public-x", "d": "example-private-d" }
    }
  ],
  "agents": [],
  "nodes": []
}
```

上例中的密钥值是不可用的占位值，只用于说明字段位置；真实备份不得采用该示例内容，也不得在公开文档或测试输出中放入生产私钥。

C.7 DID 与资源标识符测试向量

测试向量应覆盖合法的四类资源 DID、基础设施节点 DID、组织和开发者 DID，以及语义码大小写错误、部分缺失、错误分隔符、后缀长度错误、Base58 禁用字符、额外段和资源类型不匹配。每个向量固定输入字符串、解析结果或错误类型；生成型 DID 只验证格式和类型关系，随机后缀不能要求跨次运行相同。跨语言测试应使用相同 JSON 编码和算法套件，并保存预期摘要或签名验证结果。

向量类别	输入	合格判据
合法 DID	四类资源和基础设施 DID	解析成功且类型关系正确
格式错误	错误大小写、分隔符或长度	稳定返回解析错误
禁用字符	0、0、I、I 等	拒绝，不静默修正
类型不匹配	语义码与资源类型不一致	拒绝或返回明确错误

测试向量应同时记录输入、预期错误和副作用判据。例如，`did:oan:SKFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz` 应能通过语法解析并通过 `skill` 类型检查；把同一 DID 作为 `mcp_server` 资源提交时，应得到类型不匹配错误。把后缀中的 `0`、`0`、`I` 或 `I` 替换进去时，应在解析阶段失败，客户端不应自动改写后再提交。

测试字段	示例值	预期结果
合法语义码	SKFI	解析为 SK 主体码和 FI 应用域码
合法后缀	7 YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz	长度为 32 且仅含允许的 Base58 字符
资源类型匹配	SKFI + skill	类型校验通过
资源类型冲突	SKFI + mcp_server	返回 <code>ResourceTypeMismatch</code> 或对应错误码

C.8 哈希与签名验证夹具

夹具至少包含规范化原文、规范化 JSON、算法套件、哈希算法、摘要、签名输入、公钥、proof 和预期验签结果，并分别覆盖正确载荷、字段顺序变化、数值/字符串变化、公钥变化、签名截断和 proof 套件不一致。Rust 实现可由 `oan-crypto` 测试产生基准，TypeScript 实现应读取同一夹具复算；夹具不得包含生产私钥，生成夹具时使用专用测试密钥并公开公钥和签名结果。

夹具建议采用一个可被 Rust 和 TypeScript 同时读取的 JSON 文件，并将输入对象与预期结果分开保存。示例结构如下：

```
{
  "suite": "Ed25519Sha256",
  "payload": { "a": 1, "b": "OAN" },
  "canonicalJson": "{ \"a\":1, \"b\": \"OAN\" }",
  "signatureInputEncoding": "utf-8-canonical-json",
  "publicKeyJwk": { "kty": "OKP", "crv": "Ed25519", "x": "test-public-key" },
  "proofValue": "test-signature-base64url-no-padding",
  "expected": { "canonicalJson": true, "signatureValid": true }
}
```

夹具中的公钥、签名和摘要必须由同一组专用测试密钥真实生成；上例的 `x` 和 `proofValue` 仍是结构占位值，不能直接作为通过测试的输入。至少应运行以下负向变体：只改变 JSON 成员顺序、改变数组顺序、改变一个字符串、替换公钥、截断签名，以及将 `cryptoSuite` 与 `proof.type` 交叉替换。正确实现应只接受原始载荷和匹配公钥的组合，并在其他变体上给出稳定失败结果。

C.9 参考来源

来源	类型	链接
oan-protocol-common	代码仓: 密码套件、密钥、签名和哈希实现	https://github.com/wolfbrother/oan-protocol-common
oan-sdk-ts	代码仓: 客户端密码学材料和 DID 操作	https://github.com/OpenAgenet/oan-sdk-ts
did:oan DID Method Specification	标准/方法规范: DID 标识和验证方法	https://github.com/OpenAgenet/oan-public-docs/blob/main/did-oan-specs/doc/OAN%20DID%20Method%20Specification.md
OAN Yellow Paper	arXiv 黄皮书: 密码学证明和信任模型	https://arxiv.org/abs/2606.03163

附录 D 符合性测试矩阵

本附录把“符合性”限定为可复现的输入、规则执行和证据判断，而不是把某次线上服务成功访问视为完整证明。测试执行器应记录测试套件版本、fixture 版本、被测实现版本、配置摘要、运行时间、环境类型和脱敏后的输出；每个用例必须产生稳定的用例编号、PASS 或 FAIL 结论、实际错误原因和可定位的证据文件。对于拒绝类用例，合格条件不仅是请求失败，还包括拒绝发生在正确的边界、没有写入不应产生的发布或索引状态，并且错误码或错误原因符合约定。当前可直接复现的离线测评以 `oan-third-party-node-admission-test-kit/fixtures/v1-alpha` 为主；其中 `case-matrix.md` 和 `requirements-traceability-matrix.md` 明确了固定 fixture、预期输出和 49 项要求检查。该套件验证的是测评包和协议判断材料，不自动等同于对官方线上 Root、链、内容分发平台或链下信任索引器的端到端证明。

统一的测试记录至少包含以下字段：

字段	记录要求
用例标识	使用 manifest 中稳定的 case ID，不以测试文件的临时行号作为唯一标识。
前置条件	记录 fixture、协议版本、授权域集合、治理快照、模拟响应和必要的服务状态。
操作与输入	记录被调用的接口、HTTP 方法、请求体摘要、查询条件、签名或哈希输入；私钥、令牌和数据库连接串必须脱敏。
实际结果	记录状态码、规范化结果、错误码、生命周期、版本、游标和必要的响应摘要。
合格判据	将实际结果与 expected fixture 的 <code>outcome</code> 、 <code>reasonCode</code> 及规范条款逐项比对。
证据产物	保存 JSON 报告、Markdown 报告、输入与期望 fixture 的引用、版本和脱敏规则。

测试失败的处理分为三类：阻止发布的协议或安全失败、阻止节点接入的身份或授权失败、以及不阻止发布但必须告警和留痕的运维或新鲜度失败。测试套件不应为了得到“通过”而静默重试、自动修正输入或忽略未知字段；如果实现具有明确的兼容模式，应在报告中单独标出。对于并未被当前 V1-alpha fixture 覆盖的链上事件、真实内容分发和跨节点网络时序，应标注为未覆盖，而不是从离线通过结果推导出已覆盖。

用例执行结果应把“测试结果为通过”和“被测对象接受输入”区分开来。对一个预期拒绝的输入，服务返回拒绝是被测对象的正确行为，因此测试用例本身应记录为 PASS，而不是把业务拒绝直接统计为失败。报告中的最小记录可以采用如下结构：

```
{
  "caseId": "fixture.registrar.rejects-did-mismatch",
```

```
"expectedOutcome": "invalid",
"actualOutcome": "invalid",
"verdict": "PASS",
"reasonCode": "DID_ID_MISMATCH",
"sideEffect": "no_resource_record_written",
"evidence": ["evidence/fixture.registrar.rejects-did-mismatch.json"]
}
```

expectedOutcome 描述夹具预期，actualOutcome 描述实现实际判断，verdict 描述测试是否通过；三者不能合并成一个含义不清的 status 字段。写入型测试还应在请求前后比较资源记录、发布队列和索引条目数量，证明拒绝没有产生后续副作用。

不同测试层级使用不同的结论范围：离线夹具检查规则和报告生成，节点集成测试检查 HTTP 路由、持久化和节点间调用，端到端测试检查发布、分发、发现和治理状态的传播。报告应记录层级，而不能把离线规则检查、服务健康检查和完整业务验收合并成一个“系统通过”结论。

D.1 资源注册测试

注册测试以 agent_service、skill、mcp_server 和 tool_api 四类资源作为首要覆盖对象。正向样例应同时满足 resourceDid、DID 文档 id、DID 文档元数据、资源包元数据、VC 主体和哈希字段的一致性，并具有可验证的主体控制证明、资源类型和授权域。Registrar 接口层测试应覆盖 /resources/register 及其兼容入口 /resources/submit，验证形状校验、授权域校验、向根平台提交以及返回注册凭证和根平台响应的边界。负向样例应逐项替换一个事实，例如 DID 不一致、VC 主体不一致、包哈希不一致、缺少 packageProof、缺少凭证证明、控制挑战不一致、元数据不完整或资源处于暂停状态，避免一个 fixture 同时含有多个无法区分的错误。

测试项	前置数据与步骤	合格与拒绝判据	证据及影响
最小正向注册	加载 registrar-resource.valid.json，向测试 Registrar 提交固定资源包。	接受，资源类型、DID、VC、授权域和哈希全部绑定；返回结果不得声称超出实际发布阶段。	请求摘要、响应摘要、资源记录和 expected fixture；通过后允许进入后续发布测试。
字段与类型一致性	分别使用 did-mismatch、credential-subject-mismatch、metadata-incomplete。	拒绝并返回稳定原因；不得写入可被发现节点读取的有效资源记录。	错误响应和存储前后对照；失败阻止注册发布。
证明与哈希绑定	使用 tampered-proof、credential-hash-mismatch、package-hash-mismatch、缺少证明 fixture。	拒绝；验证必须覆盖证明主体、签发者、DID 文档哈希、元数据哈希和包哈希。	验证路径和脱敏证明摘要；失败阻止发布。
重复提交与更新	以同一 DID 和版本重复提交，再以控制方签名递增版本提交。	重复提交不得产生两个冲突的当前版本；更新必须通过控制权和版本规则；历史与当前版本的含义可区分。	两次响应、资源记录、版本列表和重试日志；异常重复写入阻止发布。

当前固定 fixture 直接覆盖正向注册和多种拒绝条件；真实 Registrar 的数据库幂等性、并发写入和同 DID 更新传播还需在运行服务场景中补充执行，并在报告中标记为 live integration coverage，而不能用离线 fixture 代替。

四类资源应采用同一组绑定检查、分别使用各自合法的 `resourceType` 和 DID 语义码。测试数据可以按下表组织，便于复用同一断言而不把某一类资源的字段误套到另一类资源：

资源类型	DID 主体码示例	最小业务材料	重点断言
智能体服务	AG	DID 文档、服务端点、能力 标签	服务端点和资源类型与 DID 文档一致
技能包	SK	DID 文档、包信息、下载或 清单地址	包哈希、版本和资源元数据 一致
MCP 服务	MC	DID 文档、MCP 服务端 点、协议绑定	协议绑定和端点引用一致
工具 API	TL	DID 文档、API 端点、输 入输出描述	工具/API 类型和接口描述 一致

同一个负向用例只改变一个被测事实，例如只替换 DID 文档 `id`，而保持 VC、哈希、授权域和生命周期不变。这样可以把失败原因准确回指到对应条款，也能防止服务因为另一个更早的校验失败而掩盖目标缺陷。

D.2 根平台发布测试

根平台发布测试应把 Registrar 的受理与根平台的接受、验证和发布分开。输入包含注册服务节点身份材料、资源 DID 文档、资源包、注册凭证、主体控制证明、版本和哈希；根平台先验证节点授权和请求签名，再验证资源绑定，只有全部通过后才形成根平台可信发布证明和可供分发的发布事实。测试应检查失败时不存在“先发布后校验”的中间状态，也不存在只凭 Registrar 返回成功就可被发现节点接受的路径。对于重复投递，重处理应保持幂等：相同发布事实不生成冲突版本，内容或证明发生变化时必须按版本和控制权规则重新判断。

当前第三方接入测试套件没有把 Root、链和真实发布队列纳入 V1-alpha 评估范围，因此本节的离线部分只能验证发布输入材料和 `expected` 规则，不能宣称完成真实根平台发布覆盖。运行服务时应补充记录根平台接口响应、发布作业标识、资源版本、根平台可信发布证明摘要、拒绝原因和重处理结果；任何包含私有治理材料的日志均不得进入公开报告。

根平台发布测试的关键断言是“先验证、后产生可传播发布事实”。可按以下顺序执行：

1. 准备已授权注册服务节点的身份材料，以及一个资源注册提交。
2. 分别篡改节点间签名、资源控制证明、DID 文档哈希、资源包哈希和授权域，逐个调用 `POST /root/resources/verify-and-publish`。
3. 检查每次拒绝的 HTTP 结果、机器可读错误、资源版本、发布队列和根平台可信发布证明。
4. 对原始有效请求重复发送，确认重复发布不会生成冲突的当前版本。

其中第 3 步的存储和队列对照是必要证据：只保存 HTTP 响应不能证明根平台没有在失败请求中留下部分发布状态。

D.3 分发与同步测试

分发测试围绕“发布事实可验证地到达授权节点”展开。应先用全量同步建立基线，再以游标执行增量同步，并分别注入重复事件、乱序事件、缺失事件、断点恢复和内容哈希错误。接收节点不

能仅按到达时间覆盖数据，而应按资源 DID、版本、发布证明和游标规则判断是否接受；缺失或哈希错误的材料应保持待处理或失败状态，不能进入可见索引。恢复测试需记录最后成功游标、重试次数、退避结果和最终收敛状态，区分“传输完成”“本地验证完成”和“发现可见”。内容分发平台的缓存命中或 HTTP 成功只能证明传输层结果，不能替代接收方对根平台可信发布证明的验证。

本节与当前 V1-alpha fixture 的边界应明确写入报告：现有测试套件验证授权域、资源可见性、来源和固定证据规则，但其 manifest 明确不包含真实 CDN、Root 队列和链上同步。实现接入完整分发测试后，应使用可重复的资源版本和人为构造的游标，保存发送端、接收端、验证端和索引端四类证据；任何一次失败若可能导致旧版本重新可见、撤销状态丢失或未经验证的资源进入索引，应作为发布阻断问题。

分发测试至少要验证游标推进与资源接受是两个相关但不同的结果：接收节点可以成功读取一页数据，但其中某个资源因证明或哈希错误被拒绝；此时游标是否推进、失败条目是否进入重试队列，必须按接口和实现的既定规则记录。重复发送相同发布事实时，结果应可安全重放；发送不同内容但复用同一资源 DID 和版本时，应进入冲突处理路径，不能静默覆盖已接受内容。

```
{
  "afterCursor": 1800,
  "nextCursor": 1812,
  "hasMore": true,
  "accepted": 11,
  "rejected": 1,
  "rejectedReasons": ["invalid_root_proof"]
}
```

上例是测试报告中的结果摘要示意，字段应以实际内容分发接口返回结构为准；nextCursor 前进不等于所有条目都已验证接受，报告必须同时保存被拒条目的 DID 和原因。

D.4 发现与语义搜索测试

发现测试应覆盖 /discovery/resources/query 的精确 DID 查询、资源类型、协议、能力标签、授权域和生命周期过滤，以及自然语言查询和查询解释接口（若该实现启用）。测试输入先经过语法、范围和授权域检查，再转换为结构化条件、标签条件或语义检索条件；结果必须保留资源 DID、版本、来源、治理状态和必要的证明摘要。相关性分数、关键词命中、标签匹配和语义相似度只能用于排序或解释，不能绕过授权域、暂停/撤销过滤、DID 文档校验和根平台可信发布证明检查。

查询类别	代表输入	必须验证的行为
DID 精确查询	固定资源 DID 或不存在的 DID。	命中唯一资源或返回稳定空结果；不得因文本相似返回其它资源。
结构化组合查询	resourceType、protocol、capabilityTags、limit 和授权域组合。	字段按协议规则解析，非法枚举、负数或超限被拒绝，分页不重复、不遗漏已接受结果。
语义查询	例如 Find an MCP server for security audit。	查询文本转为可解释的候选条件；返回结果需标明语义/标签/文本等来源，不能把相似度当作信任证明。
可见性边界	域内、域外、暂停、撤销、协议不匹配和类型不匹配资源。	仅返回授权域内且状态可用的资源；空结果是合法结果，不得用推荐项填充。

查询类别	代表输入	必须验证的行为
去重与解释	同一 DID 的多版本、重复索引记录 和 explain 请求。	依据 DID 与版本规则去重，能说明 候选来源和过滤原因。

`fixtures/v1-alpha/discovery` 已提供可见性、空结果、任意标签匹配、非法过滤、非法标签、非法资源类型、非法协议、`limit` 边界和过短查询等固定用例；这些用例应作为回归基线。语义模型、向量库和自然语言排序的具体质量不应由固定规则 `fixture` 推断，需另行记录模型版本、索引构建时间和解释结果。

发现结果测试应把候选生成、过滤、排序和证明检查分别记录。一个资源即使在文本或能力标签上高度匹配，只要授权域不覆盖、生命周期不可用、DID 文档与资源包不一致或根平台可信发布证明无效，就不能出现在可调用结果中。反过来，空结果不表示发现服务异常，只有在存在满足全部过滤条件的有效资源而响应仍为空时，才构成召回或索引缺陷。

测试阶段	观察值	合格条件
输入解析	查询文本、DID、资源类型、标签、 协议和 <code>limit</code>	合法输入进入对应检索路径，非法输入受控失败
候选生成	候选 DID、匹配来源和原始分数	候选来自本地已索引数据，不凭空生成资源
治理过滤	授权域、生命周期和发布证明状态	不满足边界的候选被排除
结果输出	DID、版本、来源、状态和证明摘要	排序不改变信任结论，结果可复核

D.5 治理授权测试

治理测试必须把节点授权、资源控制权和资源发布结果作为三个独立判断。节点 DID 文档、授权凭证、授权域、协议版本和生命周期状态一致时，节点才可执行其角色允许的动作；资源拥有者的控制证明只说明资源操作权，不能替代节点授权。测试应分别使用 `active`、`suspended`、`revoked`、未知和非规范大小写状态，验证敏感操作在状态不能确认或状态已失效时采取拒绝或 `fail-closed` 行为。对于授权域，应覆盖空集合、通配符、父子域、兄弟域、重复、大小写和未知域，确认标签树或资源描述不会扩大节点授权范围。

`governance`、`node`、`domains` 和 `lifecycle` `fixture` 已为离线判断提供稳定输入；治理快照中的来源、事件序列或投影时间必须与测试报告一同保存。链下信任索引器的投影延迟、链上最终性和真实撤销传播属于运行集成范围，不能仅凭快照测试宣称完成。

治理授权测试宜采用“授权事实 -> 节点动作 -> 资源可见性”的闭环断言。对于一个被撤销的发现服务节点，测试不只验证授权快照状态为 `revoked`，还要验证其同步或查询动作被拒绝、既有授权范围不会继续扩大，并且已经索引的资源不会因旧缓存重新变成有效结果。对于授权域变更，应同时检查新域和旧域，避免只验证新增域而遗漏旧域权限是否被正确收回。

D.6 版本更新与生命周期测试

生命周期测试以状态转换和版本关系为中心，而非以时间戳先后作为唯一依据。测试应建立一个 `active` 资源，使用原控制方的有效证明发布新版本，再验证当前版本、历史版本和同 DID 查询的返回关系；随后分别施加暂停、撤销和恢复事实，检查 Registrar、Root、内容分发平台、发现服务节点和调用方看到的状态是否按允许的传播边界变化。未经控制方授权的更新、版本回退、同版本

内容冲突和恢复无依据都应拒绝。发现查询默认只返回可用当前版本，若接口支持历史查询，则必须明确历史记录的状态，不能让旧版本绕过当前治理状态重新成为可调用资源。

现有 lifecycle fixture 可以验证状态值的规范性和 active/inactive 语义；同 DID 更新、真实历史持久化和跨节点收敛则应使用服务集成测试，并保留每次提交的 DID 文档哈希、资源包哈希、版本、控制证明、发布证明和索引时间。

同 DID 更新的测试记录至少包含以下关系：

```
same resource DID
-> controller proof for the update
-> strictly selected package version
-> new DID document/package hashes
-> Root publication evidence
-> Discovery current-version result
```

若只更新描述或端点，也必须重新计算受影响的 DID 文档哈希、元数据哈希或资源包哈希，并验证相应证明；不能用版本号递增替代内容绑定。历史查询测试还应确认历史版本可追溯但不会绕过当前暂停、撤销或授权状态。

D.7 安全与滥用防护测试

安全测试应验证“拒绝且无副作用”。固定签名样例用于确认有效 Ed25519 签名可验证、篡改后的同一载荷不能通过；在服务级测试中还应替换请求 nonce、request ID、受众、路径、协议版本和签名者，检查重放、跨接口转用和越权调用。请求大小、查询 limit、文本长度、非法 JSON、错误 Content-Type、恶意端点和异常授权域应在进入昂贵的语义检索、网络请求或持久化前被限制。限流和超时测试应记录响应状态、等待时间、资源占用和恢复情况，不能用无限重试掩盖拒绝。

敏感证据测试应确保私钥、访问令牌、数据库连接串、内部文件路径和未脱敏的控制材料不会进入报告、网站公开接口或普通节点响应。security/sensitive-evidence.json 与报告脱敏检查是当前可复现的基线；真实服务还应抽样查看应用日志、反向代理日志和错误响应，确认异常分支没有泄露内部堆栈或秘密。

安全测试的副作用检查应覆盖三个位置：请求处理前的输入缓冲或临时文件、服务端持久化记录、下游发布或索引队列。对超大请求、恶意端点和无效签名，合格结果应包括请求被限制、无资源记录写入、无节点间发布调用和无公开错误堆栈。对限流测试，应区分“服务拒绝请求”和“服务进程不可用”：前者是策略结果，后者才是稳定性缺陷。

D.8 第三方节点接入测试

第三方节点测试从节点身份一致性开始，要求配置、DID 文档、状态响应、授权凭证、角色、端点、协议版本、生命周期和授权域之间没有矛盾。接入后分别验证注册服务节点能否按授权域受理并向根平台提交，发现服务节点能否只同步和查询授权范围内的根平台发布内容；还要验证无效节点、域外资源、协议版本不一致和撤销状态不会因配置缓存或历史凭证继续获得权限。日志和故障恢复测试应检查请求 ID、游标、拒绝原因和重试记录足以重建事件，但不包含私钥和内部秘密。

当前套件的 node-identity.*、授权域、治理快照和发现可见性 fixture 适合作为第三方接入的离线准入门槛。真实节点接入还必须用隔离端点和独立数据库进行，不得把测试节点直接接入官方生产数据面；报告应注明被测节点、授权域、配置版本和撤销/恢复时间。

第三方节点的接入结论应拆成身份、协议、授权、业务和运维五个观察面。只有身份材料一致，说明“节点是谁”；只有协议调用成功，说明“接口能否通信”；只有授权和业务闭环通过，才说明“节点能否在授权范围内完成注册、同步或发现”；日志、限流和恢复证据则用于判断节点是否具备可持续运行条件。任何一个观察面失败，都应在报告中保留具体失败原因，而不能用其它面通过进行抵消。

D.9 DID 解析与方法符合性测试

解析测试应验证 `did:oa` 的方法前缀、语义代码、主体类型映射、标识符字符集和规范化规则；不能把大小写修正、字符串可解析或服务端返回 JSON 当作控制权证明。DID 文档测试需检查 `id`、控制者、公钥、认证/断言引用、服务端点、资源类型和资源元数据的一致性，并对错误方法、错误段数、非法字符、未知主体代码和错误服务引用给出确定性失败。若实现提供历史版本解析，应同时记录解析所依据的版本、哈希和状态，区分当前解析结果与历史记录。

测试证据应包含规范化前后的标识符、解析字段、验证方法和错误原因；私钥仅用于测试运行时签名，不得写入 `fixture` 报告。

方法符合性测试还应区分语法层、结构层和控制权层：语法层确认字符串符合 `did:oa` 形式，结构层确认 DID 文档中的 `id`、验证方法和服务引用相互一致，控制权层则需要使用对应私钥生成证明并由公钥验证。仅通过前两层，不能得出资源由某个主体控制的结论。测试向量中使用的固定后缀可以复现解析结果，但不应被解释为生产环境的密钥派生结果。

D.10 API 兼容性与错误契约测试

API 测试应同时检查成功契约和失败契约。每个接口至少记录方法、路径、`Content-Type`、必填字段、未知字段处理、状态码、错误体结构、错误原因、分页字段和重试建议；客户端不得把任意 `2xx` 都当作业务成功，也不得把网络超时自动解释成资源被拒绝。对兼容版本，测试应区分可忽略的扩展字段、必须拒绝的协议版本和不能改变语义的字段顺序或 JSON 表示差异。重试仅适用于明确可重试的传输或临时依赖失败，带有签名和写入副作用的请求必须依靠 `request ID`、`nonce` 或服务端幂等规则防止重复提交。

`oa-protocol-common/docs/error-codes.md`、协议类型和测试套件中的 `requests fixture` 是错误契约的主要核对来源。当前 `fixture` 已覆盖空请求体、非法方法、格式错误 JSON、缺失字段、过大请求、错误 `Content-Type` 和错误查询类型；各服务新增错误码时应同步更新公共错误文档、客户端映射和 `expected fixture`。

兼容性测试应把请求发送结果和业务语义结果分别断言。例如，未知可忽略字段不应改变既有字段的含义；未知协议版本应产生明确的不兼容结果；缺失必填字段应在业务写入前失败；网络超时则只能记录为传输结果未知，不能直接判定资源没有注册成功。对具有写入副作用的接口，重试测试必须使用固定 `requestId`、`nonce` 或等价幂等条件，并检查服务端是否生成重复 VC、重复版本或重复发布任务。

场景	预期测试结论	需要保存的证据
合法请求与已知版本	业务处理成功	状态码、响应摘要和资源状态
增加未知扩展字段	按接口兼容规则处理	原始请求与字段处理结果
缺少必填字段	受控拒绝且无写入副作用	错误码和存储前后计数
网络超时后重试	不产生重复业务事实	两次请求标识、响应和最终记录

D.11 新鲜度、收敛与恢复测试

新鲜度测试应把观测时间、事实产生时间、同步游标、索引更新时间和允许的最大延迟分开记录。测试流程可先发布一个带唯一版本和哈希的资源，确认根平台接受，再观察内容分发平台、发现服务节点和链下信任索引器的状态，最后比较各层是否收敛到相同的 DID、版本、状态和证明。人为中断网络、重启服务、删除临时传输状态或注入重复事件后，恢复过程必须从可验证的游标或全量重建继续，不能凭“最新一条响应”猜测缺失事实。旧快照可以用于非权威展示或有限降级，但在授权、撤销和新发布判断中必须遵守明确的新鲜度边界。

报告至少应列出各层的 `observedAt`、`source`、`cursor`、`version`、`lifecycleState`、验证结果和延迟；对于超出阈值的结果，标注是阻止敏感操作、降级为只读，还是仅产生运维告警。当前 V1-alpha 套件并不提供完整的真实网络收敛测评，因此这部分应与 Root、CDN、发现服务节点和链下信任索引器的集成报告分开发布。

收敛测试应使用一个可追踪的资源版本作为关联键，而不是只比较各接口返回的资源数量。建议在报告中保留如下最小证据链：

```
{
  "resourceDid": "did:oa:SKFI:7YpQm9Kx2VnRb6Ts3WfHa4Cd5Ej8LgNz",
  "version": "1.0.1",
  "rootPublishedAt": "2026-09-07T01:02:03Z",
  "cdnObservedAt": "2026-09-07T01:02:05Z",
  "discoveryObservedAt": "2026-09-07T01:02:07Z",
  "trustIndexObservedAt": "2026-09-07T01:02:08Z",
  "didDocumentHash": "sha256:example",
  "packageHash": "sha256:example",
  "converged": true
}
```

示例中的时间、DID 和摘要仅用于说明报告字段。实际验收应依据测试环境设定的阈值判断 `converged`，同时保留各层原始响应、日志关联标识和游标，才能区分传播延迟、索引失败、治理状态滞后和版本绑定错误。

D.12 参考来源

来源	类型	链接
oan-protocol-common	代码仓：测试数据结构、schema 和错误契约	https://github.com/wolfbrother/oan-protocol-common
oan-root-services	代码仓：根平台验证和发布测试对象	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册协议和校验测试对象	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现、索引和语义查询测试对象	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：治理状态和同步测试对象	https://github.com/OpenAgenet/oan-trust-indexer

来源	类型	链接
OAN Yellow Paper	arXiv 黄皮书：安全性质和验证要求	https://arxiv.org/abs/2606.03163
Efficient Agent Discovery Profile	IETF 草案：发现 profile 的测试方向	https://datatracker.ietf.org/doc/draft-xu-efficient-agent-discovery-profile/

附录 E 参考部署与示例

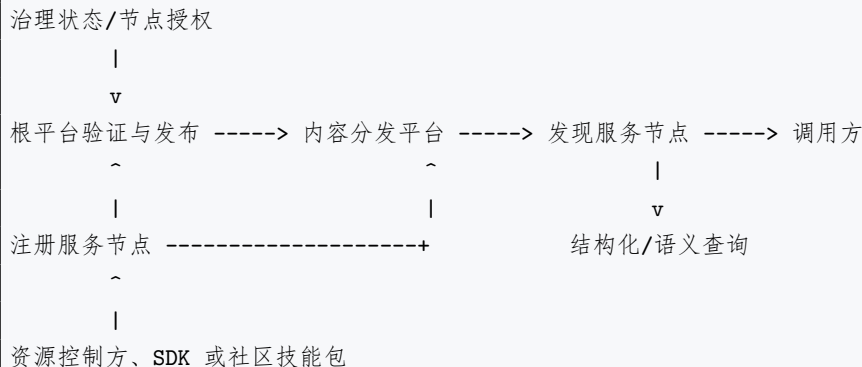
本附录给出可用于开发、验证和运维交接的参考路径。规范要求关注角色边界、证据绑定、状态传播和接口语义；参考实现使用 Rust 节点、TypeScript SDK、社区技能包、PostgreSQL/NATS 及网站网关；具体域名、端口、密钥、数据库 URL、模型目录和系统服务名均属于环境配置。部署人员应先确认目标网络的信任根、授权域、节点 DID 和协议版本，再写入配置，不应复制示例中的私钥、令牌或线上地址。服务端编译产物、日志、缓存和数据库数据与源码归档分离，秘密只通过受保护的运行时配置注入。

采用版本化源码归档同步时，归档内容应来自已经确认的提交或发布版本，而不是工作区中未审查的文件。以下命令只表示一种通用的源码归档方式：

```
git archive --format=tar --prefix=release/ <COMMIT_OR_TAG> | gzip -c > <RELEASE_ARCHIVE>.tar.gz
```

该归档只代表源码交付物；构建目录、依赖缓存、数据库文件、运行时密钥和日志不应混入源码包。源码、构建产物、配置、运行数据和网站静态文件应分别存放在由部署环境定义的受控目录中，并通过发布记录关联其版本和摘要。目录名称、文件系统布局和传输方式属于环境实现，不是协议要求。

推荐的通用启动链路如下：



启动顺序应先满足数据库、消息系统和链下信任索引器等依赖，再启动根平台、注册服务节点、内容分发组件和发现服务节点，最后启动网站后端、静态前端和反向代理。关闭或升级时应先停止新的写入，再等待发布和分发队列达到可接受状态，保留数据库、游标、版本记录和脱敏日志后再替换程序。若某个依赖暂时不可用，服务应按照自身配置进行有限重试和退避，并对外报告 `unknown`、不可用或待同步，而不生成虚假的发布或授权事实。

E.1 官方网络部署

官方网络的参考分层是：注册服务节点承接资源接入，根平台执行网络级验证、发布和协调，内容分发平台传输根平台已验证的资源包和证明，发现服务节点建立授权域内的结构化与语义索引，链

下信任索引器把治理事件投影为可查询状态，网站后端通过公共网关为前端提供展示和用户操作入口。网站不是信任权威；网站异常不应改变资源或节点的协议状态。

生产部署至少应核验以下链路：

1. 数据库、消息系统和各节点监听地址已就绪，且外网只暴露经过反向代理保护的公共 HTTPS 入口。
2. 根平台、注册服务节点、发现服务节点、内容分发平台和链下信任索引器分别通过 `/health` 或其角色状态接口返回可解释状态。
3. 注册服务节点能够完成受控的注册请求校验和根平台提交；注册烟测应使用专用测试资源或受控失败输入，不能污染生产资源目录。
4. 根平台发布结果能被内容分发平台消费，发现服务节点能校验并索引，随后通过精确查询和语义查询观察可见性。
5. 网站前端的 `VITE_*` 端点指向当前公共网关，构建产物不得残留 `localhost` 或 `127.0.0.1`；网站部署后执行注册、发现、Network、路由和浏览器控制台烟测。

部署记录应保存构建版本、配置摘要、服务切换时间、健康检查结果、注册和发现烟测结果、索引游标及失败重试信息。公共域名、节点地址和证书属于目标环境配置，迁移到其它环境时必须使用该环境经过授权的端点和证书。

各服务的发布应采用可回退的替换顺序：先验证新二进制和静态产物，再通过目标环境的服务管理器切换对应服务，最后通过公共入口检查路由。网站静态文件可先上传到受控的临时发布目录，完成文件数量、入口文件和端点引用检查后再切换当前版本；节点服务则应先保留旧二进制、配置和运行状态，再替换新版本。回退时只恢复已验证的旧版本，不重置数据库、发布游标或治理状态。

层次	典型观测	失败时的处理边界
进程层	服务管理器状态、监听端口、启动日志	修复配置或二进制后重启；不以进程存活代替业务可用
接口层	<code>/health</code> 、节点状态和公共路由状态码	区分反向代理、上游服务和请求格式问题
业务层	注册受理、根平台发布、发现查询	使用受控测试数据；失败时阻止发布或回退
数据层	数据库连接、游标、索引统计和版本记录	先保留现场和备份，不直接删除或重建业务数据

E.2 私有网络与联盟网络部署

私有或联盟网络可以把根平台、注册服务节点、发现服务节点和内容分发平台部署在组织内网或隔离网络中，但仍应保留相同的协议责任：根平台是网络级验证、信任授权、数据分发和语义治理枢纽，注册服务节点负责资源接入，发现服务节点只对已发布材料建立索引。网络隔离改变可达性，不改变 DID、控制证明、根平台可信发布证明、授权域和生命周期状态的判断规则。

典型配置边界如下：

配置项	私有/联盟网络处理方式	不应改变的事实
信任根	使用组织或联盟约定的根 DID、根密钥和治理流程，并通过安全配置注入。	节点授权必须可验证，不能只依赖网段或 IP 白名单。
资源可见性	通过授权域、发现节点分区和反向代理控制可见范围。	可见性过滤不能把域外资源伪装成域内资源。
数据分发	可使用内网 CDN、对象存储或节点间同步。	接收方仍需校验哈希、版本和根平台可信发布证明。
运维边界	将数据库、链下信任索引器和管理接口置于受控网络。	普通调用方不能因网络可达而获得私钥或治理写权限。

联盟成员接入时，应先提交节点身份、角色、端点、授权域和协议版本材料，再由治理权威授权；撤销或暂停应能够传播到根平台和发现服务节点。联调阶段使用独立数据库和测试域，避免将私有网络的测试资源误同步到官方公共网络。

联盟网络的边界还应体现在运维账号和数据备份上：节点运维人员可以管理进程、配置和日志，但不应从应用配置中读取资源控制方的私钥；资源发布者只提交公开 DID 文档、证明和资源包，不因网络位于内网而获得治理写权限。跨成员联调时，应为每个成员记录节点 DID、授权域、配置版本和撤销状态，测试结束后清理临时凭据和测试资源索引，但保留必要的审计摘要。

E.3 最小注册节点部署

最小注册服务节点只需要自身 DID 文档和运行密钥、资源记录存储、根平台地址、根平台 DID 或受众、监听地址、CORS/网络策略和必要的数据库配置。参考 Rust 实现提供 `/health`、`/registrar`、`/did`、`/resources/register`、兼容提交入口、注册状态、资源记录、能力标签树、标签建议和授权查询等接口；具体可用接口应以目标版本源码和配置为准。注册流程是：接收资源包，检查 DID 与资源类型、元数据、授权域、控制证明和哈希，生成面向根平台的签名请求，等待根平台验证和发布，再保存登记记录并返回注册凭证和根平台响应。

最小验证应包括：

```
curl -fsS "${REGISTRAR_BASE_URL}/health"
curl -fsS "${REGISTRAR_BASE_URL}/registrar/did"
curl -X POST "${REGISTRAR_BASE_URL}/resources/register" \
  -H 'content-type: application/json' \
  --data-binary @registration-request.json
```

`${REGISTRAR_BASE_URL}` 是由部署环境注入的示例变量，不是固定地址或端口要求。注册服务节点不得接收资源控制私钥；私钥在资源控制方的 SDK、浏览器或本地 Skill 中使用，节点只接收公钥、签名和公开证明。根平台不可达时，节点必须返回可区分的上游失败，不得把本地保存或受理显示为根平台已发布。

E.4 最小发现节点部署

最小发现服务节点需要自身身份和授权材料、根平台或内容分发平台同步端点、授权域、数据库、索引配置和监听地址。它先获取或接收根平台发布材料，验证根平台可信发布证明、资源包哈希、DID 文档、版本和生命周期，再写入本地结构化索引；语义检索可在此基础上使用标签树、文本字段和

向量索引，但不能跳过发布和授权验证。参考实现提供 `/discovery/did`、`/discovery/resources/query`、查询解释、查询建议、能力标签树、同步历史和索引统计等角色接口。

初始同步完成的判据不是“HTTP 请求成功”，而是：同步游标已记录，资源包的来源和哈希校验通过，授权域内资源进入索引，域外、暂停、撤销或协议不匹配资源被过滤，索引统计与同步记录一致。建议用固定资源集执行如下查询：

```
{
  "query": "Find an MCP server for security audit",
  "resourceType": "mcp_server",
  "limit": 10
}
```

返回结果应能区分文本、标签、结构化条件和语义相关性等匹配来源，并保留资源 DID、版本、状态、来源和新鲜度字段。若根平台或内容分发平台暂时不可用，可以展示已知旧快照，但不得据此产生新的可信发布或授权结论。

发现服务节点的同步状态宜按下表区分，供 Network 页面、运维接口和告警规则共同使用：

状态	含义	可执行动作
ready	游标推进、索引可查询且最近一次验证成功	允许正常查询
syncing	正在全量或增量同步	可查询已确认数据，标明同步状态
stale	仍有旧索引，但超过环境设定的新鲜度边界	降级展示或阻止敏感调用
unknown	无法确认上游、游标或治理状态	不得据此确认资源可信或可调用

这里的状态是运维和展示层的观察结果，不是对资源生命周期状态的替代。具体新鲜度阈值应由部署环境和运维策略给出，不能从一次查询响应推导出协议固定时延。最小注册服务节点的配置可抽象为以下环境配置示意；其中值均为占位值，不能直接用于生产：

```
REGISTRAR_LISTEN_ADDR=<LISTEN_ADDRESS>
ROOT_ENDPOINT=<AUTHORIZED_ROOT_ENDPOINT>
DATABASE_URL=<DATABASE_CONNECTION_REFERENCE>
NODE_DID=<NODE_DID>
AUTHORIZED_DOMAINS=<AUTHORIZED_DOMAIN_SET>
```

启动后至少要分别记录“节点可达”“请求被本地校验”“请求已提交根平台”三个结果。后两个结果不能由同一个 HTTP 200 状态码含混表示；如果根平台返回超时或未知，注册服务节点应保留可重试的请求关联信息，并把结果标为未知或待处理。

E.5 SDK 注册示例

SDK 示例应体现私钥只在调用方本地使用、DID 文档与资源包绑定、注册请求由指定注册服务节点受理以及注册结果需要继续观察发布和发现状态。以下代码展示接口关系，端点、域名和资源内容须按环境替换：

```
import { createAgentIdentity, createRegistrationSubmissionFromIdentity } from "@openagenet/oan-sdk-ts";
import { OanClient } from "@openagenet/oan-sdk-ts/client";
```



```
const identity = await createAgentIdentity(
  "Repository structure summarizer",
  "skill",
  undefined,
  {
    domainCode: "DM",
    authorizedDomains: ["technology.software_engineering"],
    manifestUrl: "https://example.invalid/skill/manifest.json",
    description: "Searches code repositories and summarizes project structure.",
    capabilityTags: ["code.repository", "analysis.summary"],
  },
);

const submission = createRegistrationSubmissionFromIdentity(identity, {
  endpoint: "https://example.invalid/skill/manifest.json",
  packageVersion: "1.0.0",
  packageHash: "sha256:replace-with-real-hash",
  metadataHash: "sha256:replace-with-real-hash",
});

const client = new OanClient({
  registrarEndpoint: process.env.OAN_REGISTRAR_ENDPOINT!,
});

const result = await client.registerResource(submission);
console.log(result.status, result.resourceDid, result.registrationCredential);
```

这是 SDK 流程示意，真实发布前必须使用实际包哈希、完整资源描述、可访问端点和 Registrar 接受的授权域；不能把 `example.invalid` 或占位哈希作为可发布材料。SDK 返回的 VC、根平台响应和资源 DID 应由调用方保存并按状态继续验证，SDK 不会替调用方保管私钥或自动授予业务调用权限。

注册结果处理时，应把网络异常、注册服务拒绝、注册服务已受理但根平台结果未知、以及根平台已发布分别记录。示意代码只展示结果分支，不把 `result` 的某个字段当作所有版本都存在的固定字段：

```
try {
  const result = await client.registerResource(submission);
  console.log("registrar response received", result);
  // 由调用方继续读取并验证 VC、根平台发布状态和发现可见性。
} catch (error) {
  console.error("registration transport or client error", error);
  // 不要在无法确认结果时自动生成新的 DID 或无条件重复提交。
}
```

真实接入应把请求关联标识、资源 DID 和本地身份备份安全保存；错误日志只能保存脱敏摘要，不能输出签名私钥、本地身份备份全文或数据库连接串。

E.6 DID 与语义发现示例

DID 精确查询用于确认一个已知资源是否在指定发现服务节点的授权视图中可见；语义查询用于从任务描述出发寻找候选资源。两者都应返回候选资源的 DID、资源类型、版本、状态、能力标签、授权域、端点或端点引用、来源和验证摘要。调用方不能因为语义匹配分数高、标签命中或网站显示推荐，就跳过 DID 文档、根平台可信发布证明、VC、生命周期和端点验证。

```
const exact = await client.discoverResources({
  resourceDid: "did:oon:SKDM:replace-with-a-real-identifier",
  limit: 1,
});

const semantic = await client.suggestDiscoveryQuery({
  query: "Find an MCP server for security audit",
});

const candidates = await client.discoverResources({
  query: "Find an MCP server for security audit",
  resourceType: "mcp_server",
  capabilityTags: semantic.capabilityTags,
  limit: 10,
});
```

若实现支持查询解释，应进一步读取 explain 结果，说明查询如何转为结构化条件、标签或语义条件，以及候选为何被保留或过滤。没有命中时返回稳定空结果；查询过短、域外或非法筛选条件应返回受控错误。分页和去重应依据 DID 与版本规则处理，不能把同一资源的多个索引副本展示为多个独立资源。

精确查询和语义查询可以使用同一发现入口，但验收断言不同：精确查询重点检查唯一性、DID 完整匹配和版本选择；语义查询重点检查候选来源、过滤原因和排序解释。一个最小的人工验收记录可以写成：

```
queryType=semantic
query=Find an MCP server for security audit
resourceDid=did:oon:MC:example-identifier
matchedBy=capabilityTags,text
governanceState=active
source=authorized-discovery-index
verification=root-proof-and-package-hash
```

其中 example-identifier 只是占位值。若结果只有文本相似度而没有可验证的来源、治理状态或证明摘要，应视为候选推荐，不应直接作为可信调用对象。

E.7 资源验证示例

调用方收到发现结果后，应按“身份、绑定、来源、治理、端点”顺序验证：首先解析 did:oon 并检查 DID 文档中的公钥、控制者和服务引用；然后检查资源包的资源 DID、类型、版本、DID 文档哈希、元数据哈希和包哈希；再验证根平台可信发布证明及其生命周期状态；随后检查注册 VC 的签发者、主体、授权域、哈希绑定和证明；最后根据业务策略验证端点、协议、证书和调用权限。任何一项失败都应阻止高风险调用，并保存脱敏的失败原因。

```
import { verifyResourcePackageShape, assertUsableLifecycle } from "@openagenet/oan-sdk-ts";

verifyResourcePackageShape(resourcePackage);
assertUsableLifecycle(resourcePackage);
// 继续使用调用方自己的 VC、节点授权和端点验证策略。
```

SDK 的结构校验函数不能替代治理状态读取、网络新鲜度判断或业务授权。验证结果应记录所依据的版本、来源 URL、哈希、签发者、状态时间和验证组件版本；不要记录私钥、令牌或完整的敏感请求。

E.8 运维烟测示例

部署后烟测应按依赖顺序执行，并把“服务可达”“协议动作成功”“状态已收敛”分开记录。推荐检查序列为：健康接口和端口；数据库连接与迁移状态；根平台状态和治理/授权读取；注册服务节点健康与受控注册请求；内容分发队列和发布状态；发现服务节点同步游标、索引统计、精确查询和语义查询；链下信任索引器最新游标；网站后端健康、前端路由、注册/发现传输和浏览器控制台。部署流程应在公共端点、注册传输、发现查询、Network 摘要和浏览器路由检查通过后才视为完成。

一个烟测记录可采用以下结构：

```
{
  "deploymentId": "2026-09-07T12:00:00Z-<commit>",
  "environment": "<environment>",
  "checks": [
    { "name": "root-health", "status": "pass", "observedAt": "..."},
    { "name": "registration-transport", "status": "pass", "resourceDid": "..."},
    { "name": "discovery-query", "status": "pass", "candidateCount": 1},
    { "name": "trust-indexer-freshness", "status": "unknown", "reason": "..."}
  ]
}
```

示例中的 DID、时间和状态为记录格式示意。烟测失败时，应先判断是网络传输、编译产物、配置、数据库、治理状态、同步延迟还是功能错误，再决定重试、回滚或阻止对外发布；不能通过删除日志、重置游标或跳过注册/发现检查来获得表面成功。

E.9 同 DID 更新示例

同 DID 更新必须由原资源控制方或明确获授权的控制方产生新的控制证明，DID 本身保持不变，资源包或 DID 文档中的可更新内容形成新的版本和哈希。流程是：读取本地身份和当前版本，修改元数据或资源包，递增符合版本方案的版本号，重新计算元数据和包哈希，签署更新材料，提交注册服务节点，由根平台重新验证并形成新的根平台可信发布证明，再经内容分发平台和发现服务节点同步。发现服务节点默认以最新可用版本响应，但必须保留可审计的版本和状态信息。

更新测试应覆盖无控制证明、控制证明对应旧材料、版本回退、同版本内容冲突、更新后哈希不一致和旧版本重新发布等情况。失败更新不得覆盖当前有效版本，也不得因为注册服务节点返回 VC 就被发现节点当作新版本；注册凭证、根平台发布证明和发现状态应分别记录。

更新完成后的对照记录至少应包含：旧版本和新版本的资源 DID、DID 文档哈希、资源包哈希、控制证明摘要、根平台发布证明摘要、发现索引版本和查询时间。若内容未变化，客户端可以复用已有预览或幂等结果；若任一影响绑定关系的字段变化，则必须重新计算哈希并重新验证，不能只修改展示用的版本字符串。

E.10 资源暂停与恢复示例

暂停、撤销和恢复是生命周期事实，不等同于删除 DID 或抹除历史。治理权威或资源控制方按允许的治理流程产生状态变更，根平台和链下信任索引器读取并投影该事实，发现服务节点同步后过滤不可用资源，调用方在验证时重新检查状态。暂停通常表示暂时不可用，撤销表示当前凭证或资源状态不再可用；恢复必须有明确的恢复依据，不能由发现节点自行把资源改回 active。

状态变更事实 -> 治理事件/授权材料 -> 根平台与索引器读取
-> 发现服务节点更新过滤状态 -> 调用方重新验证后决定是否调用

审计记录应包括资源 DID、原状态、新状态、事实来源、事件或请求标识、观察时间、传播时间和执行结果。治理状态未同步或超过允许新鲜度时，敏感调用应 fail-closed；网站或旧快照可以展示“待同步”，但不能显示为当前可调用。

E.11 跨协议 A2A 与 MCP 映射示例

跨协议映射的目标是降低现有 Agent 平台、MCP 客户端和工具调用方接入 OpenAgetnet (OAN) 的成本，不是把不同协议的对象强行视为同一个对象。映射时可将 OAN DID 文档和资源描述中的名称、能力标签、用途、输入、输出、服务端点、协议绑定和版本映射到 A2A Agent Card、MCP 服务元数据或工具 API 描述；反向转换时必须保留原始 DID、原始字段、版本、来源和哈希。A2A Agent Card 或 MCP 元数据说明能力和连接方式，但不替代 OAN 的控制证明、VC、根平台可信发布证明、治理状态和授权域。

OAN DID文档/资源包
- id, resourceType, capabilityTags, useCases
- protocolBindings, service endpoint, version, hashes
|
+--> A2A Agent Card: identity reference, skills, interfaces
+--> MCP metadata: server capabilities, tools, transport
+--> Tool/API description: operation, schema, endpoint

映射器应明确字段来源和转换方向：能力名称可用于展示和检索，DID 和哈希用于身份与完整性追溯，签名和 VC 作为独立验证材料保留，端点映射只代表连接入口而不会自动授权调用。若目标协议没有等价字段，应使用扩展字段或附加引用，并在结果中标明丢失信息；不得把语义相似、协议兼容或元数据转换成功解释为资源可信或业务可用。

下面的映射关系只说明信息如何被携带，不规定 A2A Agent Card、MCP 元数据或工具 API 描述必须采用 OAN 的字段名：

OAN 信息	外部描述中的可能承载位置	保留要求
资源 DID、DID 文档来源	身份引用或扩展字段	保留原始 DID 和来源，不改写成普通名称

OAN 信息	外部描述中的可能承载位置	保留要求
能力标签、用例和描述	Agent Card skills、MCP capabilities 或工具说明	可用于展示和检索，不能替代证明
协议绑定、端点和输入输出	接口、传输和 schema 描述	保留版本和来源，端点可用性需另行验证
VC、哈希和根平台可信发布证明	外部对象的验证附件或引用	不因转换成功而删除或覆盖原始证明

映射后的对象如果重新进入 OAN 注册或发现流程，应重新执行 DID、哈希、控制证明、授权域和生命周期检查；一个外部协议对象可作为接入材料，但不能单独成为 OAN 的发布事实。

E.12 参考来源

来源	类型	链接
oan-root-services	代码仓：根平台部署和发布示例	https://github.com/OpenAgenet/oan-root-services
oan-registrar-node	代码仓：注册服务部署和注册示例	https://github.com/OpenAgenet/oan-registrar-node
oan-discovery-node	代码仓：发现服务部署、同步和查询示例	https://github.com/OpenAgenet/oan-discovery-node
oan-trust-indexer	代码仓：信任索引器部署和治理同步示例	https://github.com/OpenAgenet/oan-trust-indexer
oan-sdk-ts	代码仓：客户端集成示例	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓：社区注册发现操作示例	https://github.com/OpenAgenet/oan-community-skill
OAN Yellow Paper	arXiv 黄皮书：参考部署和端到端机制	https://arxiv.org/abs/2606.03163
OAN Resource Identity and Discovery	IETF 草案：资源身份和发现示例边界	https://datatracker.ietf.org/doc/draft-xu-oan-resource-identity-discovery/

附录 F 仓库与实现映射

本附录采用“规范条款 -> 数据结构/接口 -> 实现文件 -> 测试证据 -> 部署配置 -> 维护责任”的链路描述实现，而不把仓库名称本身当作功能证明。当前核心运行实现由共享协议库、根平台、注册服务节点、发现服务节点、内容分发组件和链下信任索引器组成；SDK、社区技能包和官网是客户端与公共入口；第三方节点准入测试套件是独立测评支持，不是生产运行节点。`oan-onboard-tests` 和根目录 `genesis` 在当前工作区中不是可直接核对的独立目录，因此涉及它们的历史引用只能视为资料来源约定，不能作为当前实现证据。

映射状态使用以下含义：

状态	含义
已实现	在当前代码、测试或配置中能定位到对应逻辑，并可通过指定方式复现或检查。
部分实现	已有数据结构、接口或局部流程，但仍缺少端到端、生产级或完整失败场景覆盖。
测评支持	由固定 <code>fixture</code> 、报告生成器或验证脚本提供，不代表运行节点已经实现该能力。
参考/待核对	规范要求或部署设计已提出，但当前工作区没有足够代码或测试证据确认。

每项实现事实应同时记录来源版本和验证时间。对于输入，说明它如何转换成校验条件、授权域过滤条件或查询条件；对于输出，说明它是原始事实、验证结果、缓存投影、搜索排序还是展示摘要；对于根平台可信发布证明、VC、签名和治理状态，必须保留签发者、验证者、绑定内容和状态来源。能力标签、语义相关性、A2A Agent Card 或 MCP 元数据只描述或辅助发现能力，不能替代控制权和治理验证。映射记录至少应区分三种证据：源码证据说明逻辑和字段在哪里定义，测试证据说明该逻辑在固定输入下得到什么结果，运行证据说明指定环境中的服务、配置和依赖实际能够完成什么。三者分别回答“写了什么”“测过什么”和“运行了什么”，不能用 README、一次编译成功或一个健康接口相互替代。对于动态数据，还应记录证据生成时的提交、配置摘要、数据库/索引状态和请求关联标识。

F.1 oan-protocol-common

该仓是 Rust workspace 级共享基础，主要映射第五至九章、第十六章和附录 A 至 C。`crates/oan-core` 承载 DID 文档、资源类型和核心领域对象；`oan-crypto` 负责密码套件、密钥和哈希相关逻辑；`oan-credentials` 负责凭证构造或验证；`oan-package` 负责资源包及其绑定；`oan-protocol` 负责跨节点请求、响应和协议常量；`oan-publication-events` 负责发布事件；`oan-storage` 提供 JSON、SQLite 或 PostgreSQL 存储抽象；`oan-service-security` 提供请求信封、nonce、request ID 和签名相关公共逻辑；`oan-semantic-recommender` 及其 `data` 目录提供能力标签树、授权域分类和推荐/规范化辅助。

映射对象	主要位置	对应规范事实	当前验证方式与边界
DID 文档与资源类型	<code>crates/oan-core/src/lib.rs</code>	<code>did:oan</code> 标识、主体类型和核心字段	Rust 单元测试与各节点调用；需继续以规范模式和方法测试确认完整覆盖。
哈希、签名与密码套件	<code>crates/oan-crypto/src/lib.rs</code>	控制证明、请求签名、包/元数据哈希	<code>oan-third-party-node-admission-test-kit</code> 的有效/篡改签名 fixture；不同于密钥托管或生产 HSM 保证。
凭证与资源包	<code>crates/oan-credentials</code> 、 <code>crates/oan-package</code>	注册 VC、资源包、根平台可信发布证明的结构绑定	注册/准入 fixture 的主体、哈希和证明一致性检查；真实链路需服务集成验证。
协议类型与错误	<code>crates/oan-protocol</code> 、 <code>docs/error-codes.md</code> 、 <code>schemas/agent-contract</code>	HTTP 载荷、协议版本、错误契约和外部描述	节点编译、客户端测试和固定请求 fixture；版本兼容需按发布版本复测。
存储与事件	<code>crates/oan-storage</code> 、 <code>crates/oan-publication-events</code>	资源记录、游标、发布事件和数据库边界	各服务测试及本地集成；不能由共享库单独证明事务、备份和恢复策略。
语义与授权域辅助	<code>crates/oan-semantic-recommender/data/*</code>	能力标签树、语义治理词汇和授权域分类	推荐器测试和发现/注册消费方测试；推荐输出不是授权决定。

共享库的维护责任是保持跨节点数据结构、序列化、错误码和密码学绑定的一致性；任何字段变更都应同步检查根平台、注册服务节点、发现服务节点、SDK、社区技能包和固定 fixture。共享库没有权限代替上层服务决定节点授权、资源发布或业务调用。

从 HTTP 接口向下追踪时，可以按实现链定位契约：注册服务节点的 `services/registrant-node/src/main.rs` 将 `/resources/register` 和 `/resources/submit` 绑定到 `register_resource`，根平台的 `services/root-node/src/main.rs` 将 `/root/resources/verify-and-publish` 绑定到 `verify_resource_and_publish`，发现服务节点的同名服务目录将 `/discovery/resources/query` 绑定到 `resource_query`，内容分发平台的 `services/cdn-node/src/main.rs` 负责 `/cdn/resources`、批量发布和资源读取，链下信任索引器的 `src/api.rs` 负责 `/v1/status`、`/v1/events` 等读接口。handler 签名是确认输入与输出类型的直接证据，路由表是确认 HTTP 方法和路径的直接证据。

维护接口时应把“共享 DTO 变更”和“动态响应字段变更”分开记录：前者需要检查 `oan-protocol`、`oan-package`、所有反序列化调用方和跨语言 SDK；后者需要检查生成 JSON 的 handler、错误码测试、公开代理和附录 B 的端点表。Rust 编译通过只能证明类型层可编译，不能证明 JSON 字段、HTTP 路径、认证边界和异步状态语义仍与对外文档一致。维护人员定位一个共享字段时，应按“领域结构 -> 序列化实现 -> 调用方 -> fixture”顺序核对。例如，资源包字段先在 `oan-package` 的结构和校验方法中确认，再检查 `oan-protocol` 的请求/响应类型、注册服务节点和发现服务节点的反序列化路径，最后用资源 fixture 验证缺失、篡改和版本冲突时的行为。只修改某个节点能够编译，并不能说明跨仓协议已经兼容；至少还要检查对应 SDK 导出和社区技能包的输入输出。当共享库中的字段发生变化时，维护人员应先判断变化属于序列化兼容、校验语义变化还是纯内部实现变化。序列化字段、枚举值、错误码和哈希计算方式属于跨仓影响面；即使 Rust workspace 内部编译通过，也要继续核对节点 HTTP 载荷、TypeScript 类型、社区技能包脚本和准入测试夹具。若只改变内部辅助函数且外部载荷与错误行为不变，则应保留相应单元测试作为边界证据，不必扩大协议版本范围。

F.2 oan-root-services

该仓对应第四章、第十至十一章、第十四至十六章及相关生命周期、分发和运维章节。根平台主服务位于 `services/root-node/src/main.rs`，其路由包括根 DID 文档、状态、注册服务节点和发现服务节点目录、资源详情与版本、能力标签树、公告/事件、授权、撤销以及面向内容分发和发现服务节点的内部接口。`services/cdn-publisher` 负责消费发布任务、从根平台准备资源包、向 CDN 批量发布并回写发布结果；`services/cdn-node` 负责内容分发平台的数据面服务。配置样例位于各服务目录的 `config.example.toml`。

根平台实现的责任边界可按以下方式核对：

根平台动作	输入	输出或持久化事实	证据与偏差
节点授权与状态读取	节点 DID 文档、授权材料、治理/状态数据	授权节点目录、状态和授权域	根服务路由与授权处理逻辑；链上最终性取决于链下信任索引器和运行配置。
资源验证与发布	注册服务节点提交、资源 DID 文档、资源包、控制证明和哈希	接受/拒绝结果、资源版本、根平台可信发布证明或发布任务	根服务测试和共享类型；应以服务集成测试确认失败不产生错误发布状态。
内容分发协调	已验证资源版本、分发任务和游标	CDN 发布队列、批量响应、已发布标记	<code>cdn-publisher/src/main.rs</code> 的队列、批处理和回写逻辑；不等于 CDN 外部可达性。
语义治理	能力标签树、别名、授权域分类	注册推荐和发现服务节点可同步的语义材料	<code>oan-semantic-recommender/data</code> 及根平台接口；词汇演进需版本化。

根平台维护者还需关注幂等、版本冲突、发布重试、队列积压、数据库备份和治理状态失联等运行边界。根平台签发根平台可信发布证明，只证明其验证和发布范围内的事实；不证明资源端点业务质量，也不持有资源控制私钥。根平台代码排查应把公开查询路由和内部写入路由分开记录。`/root/resources/{did}`、版本查询和能力标签查询属于读路径；注册服务节点提交、节点授权、撤销、内容分发队列和发现服务节点通知属于受保护的写或协调路径。维护人员在修改 `services/root-node/src/main.rs` 时，应同时核对 `services/cdn-publisher/src/main.rs` 对发布任务的消费、`services/cdn-node/src/main.rs` 对资源包的读取，以及根平台配置中的队列、数据库和上游端点；不能仅凭根平台 HTTP 响应判断内容已经完成分发。根平台变更的核对重点是发布事实何时产生。维护人员应分别记录注册服务节点提交时间、根平台验证结果、资源版本与哈希、根平台可信发布证明、内容分发任务状态和发现服务节点通知状态；只有通过根平台验证并形成发布事实，后续分发和发现链路才有可引用的根来源。根平台的查询接口、内容分发平台的缓存和发现服务节点的索引条目应作为不同证据保存。

F.3 oan-registrar-node

该仓主要对应第六至九章、第十六章和资源生命周期部分。实现入口为 `services/registrar-node/src/main.rs`，当前路由包含 `/health`、`/registrar/did`、`/resources/register`、兼容的 `/resources/submit`、资源列表与详情、状态、统计、根平台授权、能力标签树、标签建议、标签规范化、注册域目录和注册建议。配置样例位于 `services/registrar-node/config.example.toml`，数据目录、密钥目

录和可选 SQLite/PostgreSQL 数据库由配置解析后确定。

注册服务节点处理链路是：

资源控制方提交公开材料和控制证明

- > 形状、DID、资源类型、元数据和授权域校验
- > 构造面向根平台的签名请求
- > 接收根平台验证/发布响应
- > 生成并保存注册 VC 与登记记录
- > 向调用方返回登记结果和根平台响应

这里的注册 VC 由注册服务节点针对登记事实签发，不能代替根平台可信发布证明；根平台拒绝时，Registrar 不应把本地受理作为全网发布。注册服务节点可以看到资源公开材料、公钥、签名和凭证，但正常流程不应取得资源控制私钥。oan-third-party-node-admission-test-kit/fixtures/v1-alpha/resources 提供 DID、资源类型、授权域、VC 主体、哈希、控制挑战、证明完整性和生命周期等固定验证样例；SDK 与官网测试覆盖客户端调用形状，但不取代 Registrar 的服务端校验。

维护重点是保持 /resources/register 与协议类型的字段一致、明确错误码和 HTTP 状态、确保上游失败不造成假成功、避免重复提交产生冲突记录，并同步更新配置样例、公共错误文档、SDK 和社区技能包。注册问题的定位应沿请求关联关系展开：先在 register_resource 的输入校验和错误转换处确认请求是否被本地拒绝，再检查面向根平台的 ResourceVerifyAndPublishRequest 是否正确构造，最后对照注册记录、VC 和根平台响应判断是否只是受理成功。一个可复用的排查记录至少保留资源 DID、请求标识、版本、错误码、上游状态和数据库写入结果；不要用“页面显示成功”替代节点之间的实际结果。注册服务节点的实现映射还应标明每个结果属于哪一层：本地请求校验、注册记录写入、向根平台提交、注册 VC 返回和根平台发布结果不能合并为一个“注册成功”。对网络超时或响应不完整为重试，应使用原请求的关联信息核对服务端是否已经产生登记事实；只有确认幂等条件和最终状态后，客户端才能决定是否再次提交。

F.4 oan-discovery-node

该仓对应第四章、第七章、第十一至十三章、第十五至十六章及新鲜度和生命周期要求。services/discovery-node/src/main.rs 提供节点 DID、状态、授权域、同步历史、索引统计、查询统计、资源索引、资源查询、查询建议、查询解释、拒绝包和能力标签树等接口；semantic_aliases.v1.json 是发现侧语义别名材料。services/embedding-service 提供可选的文本向量服务，配置、测试和真实模型 smoke 文件位于该服务目录，不能把启用语义模型写成所有部署的强制依赖。

发现节点的数据路径应按四层区分：

1. 同步层接收根平台或内容分发平台材料，记录来源、游标和同步结果。
2. 验证层检查根平台可信发布证明、资源包哈希、DID 文档、版本、授权域和生命周期。
3. 索引层把已验证材料写入结构化索引，并按能力标签、文本字段或可选向量建立检索索引。
4. 查询层把 DID、结构化条件和自然语言输入转换为条件，返回候选及来源、版本、状态和解释信息。

实现点	代码/材料	需要保持的边界
同步与拒绝	<code>src/main.rs</code> 、配置和同步相关测试	未验证、域外、暂停、撤销或哈希不一致的材料不得进入可见索引。
精确与结构化查询	<code>/discovery/resources/query</code> 及查询模型	DID 精确查询、资源类型、协议、标签、域和 <code>limit</code> 应有稳定校验与错误契约。
语义检索	<code>services/embedding-service</code> 、语义别名和推荐数据	相关性只影响排序/解释，不能替代控制权、根平台发布或治理状态。
去重与版本	资源索引和版本字段	同一 DID 的同版本副本不能被展示为多个独立资源；当前版本和历史版本须可区分。
新鲜度与降级	同步状态、索引统计和查询响应	旧快照可用于有限展示，但不能创造新的发布、授权或调用许可。

当前固定 `fixture` 已覆盖域内可见、域外不可见、空结果、任意标签匹配和非法查询；真实同步、向量模型质量、并发索引和跨节点收敛仍需服务集成或部署测试确认。发现节点维护者负责索引一致性、查询错误契约、来源保留、治理状态过滤和语义配置版本化。发现查询的维护定位可以从入口向后追踪：`/discovery/resources/query` 负责请求解析和查询调度，索引表负责已验证资源的读模型，语义别名和能力标签材料负责规范化与候选辅助，`services/embedding-service/src/server.ts` 和 `src/embedding.ts` 只在启用向量模式时参与文本向量生成。排查“结果重复”时，应分别检查同一 DID 的版本去重、索引副本去重、分页游标和前端展示映射，不能只调整排序或前端数组。发现服务节点的代码映射还要保留“原始材料 -> 验证结果 -> 索引记录 -> 查询候选”的转换关系。原始资源包或根平台发布通知不应直接等同于可查询结果；只有完成来源、哈希、版本、授权域和生命周期检查后，才可进入索引。查询响应中的相关性分数、标签命中和文本片段属于排序或解释证据，资源 DID、版本、来源和治理状态才是调用方继续验证的事实入口。

F.5 oan-trust-indexer

该仓对应第十四章、第二十至二十一章以及授权状态和恢复要求。`src/event_source.rs` 负责事件来源，`src/sync.rs` 负责游标推进和同步，`src/store.rs` 负责投影存储，`src/domain.rs` 负责领域状态，`src/api.rs` 和 `src/grpc.rs` 提供查询接口，`src/config.rs` 负责运行配置。`configs/mainnet.example.toml`、`mainnet.current-events.toml` 和单检查点 `smoke` 配置用于区分持续同步、当前事件观察和诊断性运行方式。

索引器的可信链路是“读取链上事件 -> 按顺序或游标处理 -> 校验事件结构和状态转换 -> 写入链下读模型 -> 对外提供状态查询”。读模型必须保存事件来源、游标、最后成功处理时间、投影时间、错误和必要的版本信息。链下投影是治理事实的可查询缓存，不创造链上授权；根平台、注册服务节点和发现服务节点在使用它时应考虑延迟、过期和 `fail-closed` 边界。`trust/v1/status` 或等价接口返回的最新序列只能作为索引进度观察，不能单独证明某项链上交易已经最终确认。

现有准入 `fixture` 可验证 `active`、`suspended`、`revoked` 状态和治理快照的一致性，但不等同于真实链 RPC、GraphQL 端点、最终性、断点恢复和持续增长的证明。维护责任包括事件源兼容、游标不回退、重复事件幂等、异常重试、数据库备份和状态新鲜度监控；更换链端点时不得只修改地址而忽略 `package`、对象、事件类型和解析逻辑的联动核对。链下信任索引器的接口返回只能说明其读模型状态。维护人员应把事件来源、最后处理游标、投影时间、错误和链端点配置放在同一份运行记录中，并将“链上事件已读取”“链下状态已投影”“根平台已采用该状态”分别记录；三者不

能因一个健康接口返回成功而合并为单一结论。链下信任索引器的版本升级应至少做一次小范围回放或检查点恢复验证：使用已知事件序列检查解析结果、游标、重复事件处理和状态投影，再比较 API 返回的主体、节点、凭证签发者和事件序列。若更换 package、对象或事件类型，旧读模型是否可以继续使用必须单独判断；不能因为进程重新启动且 /health 返回成功，就认定历史治理状态已经正确恢复。

F.6 oan-sdk-ts

该仓对应第五至九章、第十二章、第十六至十七章和附录 A 至 C。packages/sdk-ts/src/index.ts 集中提供资源 DID 文档草稿、资源包形状验证、生命周期检查、注册、发现、查询解释、能力标签辅助和治理读接口；identity.ts 负责本地主体/资源身份记录、密钥对生成、身份包导入导出和从身份生成注册材料；did-oan.ts 负责 did:oan 解析、规范化、主体代码到资源类型映射和语义冲突检查。packages/protocol-types/src/index.ts 提供跨包协议类型。测试位于 tests/sdk-core-tests.ts、identity-store-node-tests.ts 和 client-workflow-tests.ts。

SDK 的定位是调用方工具，而不是 Registrar、Root 或 Discovery 的权威实现。它可以在本地生成密钥、构造 DID 文档和注册材料、调用公共接口并对资源包进行结构验证，但不能替调用方确认链上最终状态，也不应把推荐结果当作授权结论。测试中的 endpoint stub、固定 DID 和占位哈希用于接口行为验证，发布示例必须替换为真实端点、包哈希、授权域和资源描述。私钥和本地身份包由调用方保存，不发送给 OpenAgenet (OAN) 节点。

SDK 能力	对应实现	验证证据	维护边界
DID 与身份	identity.ts、did-oan.ts	SDK 核心和身份存储测试	保持 DID、主体类型、公钥和本地私钥绑定，不实现网络治理。
注册材料	index.ts、身份记录转换函数	客户端工作流测试	生成材料后仍由 Registrar 和根平台重新校验。
发现与解释	index.ts 的 Discovery client	stub 工作流和查询辅助测试	结果排序/解释不能替代调用方最终验证。
资源验证	资源包形状和生命周期函数	SDK 核心测试	结构验证不等同于真实 VC、治理和端点可用性验证。

SDK 发布维护还需检查构建输出、导出入口、Node/浏览器运行边界和协议版本兼容，避免 README、npm 包内容和源码 API 不一致。SDK 的公开入口以 package.json 的 exports 为准：根入口、./client、./governance、./protocol-types 和 ./identity-store-node 分别对应构建输出中的模块。变更 packages/*/src 后，应检查 npm run build 是否生成相应 JavaScript 和声明文件，再运行 tests/sdk-core-tests.ts、tests/client-workflow-tests.ts 和 tests/identity-store-node-tests.ts；如果只是内部实现变化但导出路径、请求字段或错误类型变化，仍应视为客户端兼容性变更。SDK 映射应同时记录“本地生成”和“远端确认”的边界：身份、DID 文档草稿、资源包形状和签名材料可以在调用方本地生成或验证，注册 VC、根平台可信发布证明、发现索引状态和治理状态则来自节点或调用方的后续查询。SDK 方法名存在并不代表远端业务已经完成；示例和测试应明确哪些响应由 stub 提供，哪些需要真实注册服务节点或发现服务节点。

F.7 oan-community-skill

该仓库对应第十七章以及第 4 至第 8、11、15 和 18 章面向社区调用方的操作路径。其 `SKILL.md` 明确技能包用于资源描述整理、本地身份管理、注册材料生成、注册服务节点提交、发现查询、语义查询解释、生命周期观察和只读治理辅助；它不负责启动或维护根平台、注册服务节点、发现服务节点、内容分发平台、NATS、PostgreSQL 或链下信任索引器，也不执行链上治理写操作。实现说明要求继续核对 `src/index.ts`、`validation.ts`、`registration.ts`、`resource-description.ts`、`discovery.ts`、`lifecycle.ts`、`capability-assist.ts` 和 `tests/oan-community-skill-tests.ts`。

社区技能包的典型输入是资源 README、技能包说明或用户提供的产品描述；它先生成候选登记材料，再报告缺失字段和质量问题，要求用户补充名称、描述、公开访问地址、授权域、能力标签、用例、输入输出、许可证和维护者等事实，最后调用 SDK 的正常注册路径。它不能凭空发明端点、维护者、许可证或授权域，也不能把能力标签当作授权范围。对批量资源，应逐项报告成功、跳过、失败、缺失输入和下一步，不因某一项失败而静默丢弃其它结果。

Skill 维护责任是保持用户操作语义、SDK 调用、公共端点默认值、错误分类和隐私边界一致；发布包必须包含运行所需脚本、文档和资源，而不只包含 TypeScript 编译产物。它可以调用公开或用户明确提供的 Registrar、Discovery 和可选的信任查询端点，但不得暴露节点私有治理材料或读取根平台私有索引器。社区技能包的实现核对应从 `src/index.ts` 的公开调用入口开始，分别追踪 `registration.ts`、`resource-description.ts`、`validation.ts`、`discovery.ts` 和 `lifecycle.ts` 的输入输出，再对照 `tests/oan-community-skill-tests.ts` 和发布包文件清单。技能包负责整理和调用，不负责替用户生成未经确认的事实；因此新增自动补全、标签建议或重试逻辑时，必须确认它不会把推测内容直接写入注册材料，也不会在结果未知时无条件重复提交。社区技能包与 SDK 的变更也应区分“帮助生成材料”和“写入网络事实”。技能包可以整理描述、提示缺失字段和调用公开接口，但不能把推测出的端点、标签、授权域或许可证直接当作用户确认的注册事实；当请求结果未知时，也不能通过无条件重试制造重复 VC、重复版本或重复发布任务。

F.8 官网与公开开发者入口

官网是公共展示和操作入口，对应第 15、16、19、20、22 和 24 章的部分实现。前端使用公开的注册、发现、根平台、内容分发平台和链下信任索引器端点；后端通过 `/api/*` 和 `/health` 提供网站所需的聚合、统计、访问分析或代理能力。生产部署说明、构建与烟测脚本以及 Nginx 和 systemd 配置描述构建、配置、反向代理和验证边界。官网的 Home、Register、Discover、Network 和 Docs 页面可以调用或展示协议数据，但页面文案、地图、统计、缓存快照和推荐排序都属于展示层。

官网注册页可以在浏览器侧生成或导入本地身份，构造 DID 文档和注册材料，再向指定注册服务节点提交；私钥不应发送到网站后端。发现页可以提交 DID 或自然语言任务描述，展示发现服务节点返回的候选资源和 DID 文档预览；用户仍需根据资源 DID、VC、根平台可信发布证明、状态和端点执行最终验证。Network 页面展示节点和指标时，应区分当前实时数据、旧快照、未知状态和统计聚合，不能把 UI 状态作为治理权威。

官网部署维护的硬性核对项包括：生产前端构建变量不得残留本机地址；后端应只通过反向代理暴露；静态资源和后端版本一致；公共 HTTPS、语言切换、注册传输、发现查询、Network 摘要和浏览器路由均通过烟测。SEO、访问统计和页面缓存只能是松耦合增强，不能改变注册、发现、治理或节点服务的协议行为。官网问题定位应先区分浏览器层、官网后端聚合层和节点服务层：前端页面及 `frontend/src/api.ts` 负责请求发起和展示，后端 `backend/src/http/router.rs` 与相应 handler 负责公共聚合接口，注册服务节点和发现服务节点才是注册/发现业务的直接处理者。访问统计写入、

SEO 元数据、浏览器缓存和旧快照读取失败时，页面可以降级或不记录，但不得阻塞注册提交、发现查询或节点状态判断；任何跨层修改都要分别执行页面路由、公共接口和节点业务烟测。网站组件与协议实现的变更边界可以用接口方向核对：浏览器到官网后端的 `/api/*` 是公共聚合或代理路径，官网后端到节点的请求是服务间调用，节点自身的 `/resources/register` 和 `/discovery/resources/query` 才是协议业务入口。修改官网缓存、统计、SEO 或页面文案时，应验证这些功能失败不会改变节点请求体、响应语义、注册/发现结果和治理状态；修改节点接口时，则不能只做网页回归而跳过协议和客户端测试。

F.9 规范章节与实现模块映射

下表提供面向维护和发布的主索引。章节编号是规范责任的入口，具体实现仍需结合前述仓库小节、当前提交和目标部署配置核对。

规范范围	主要实现仓/模块	主要接口或数据	主要测试/证据	实现状态与维护责任
资源模型、DID 文档、资源包	<code>oan-protocol-common</code> 、 <code>oan-sdk-ts</code>	<code>DidDocument</code> 、资源类型、资源包、哈希字段	Rust/TS 单元测试、资源 fixture	已实现基础结构；协议类型变更由共享库维护者牵头。
注册与控制证明	<code>oan-registrar-node</code> 、 <code>oan-sdk-ts</code> 、 <code>oan-community-skill</code>	<code>/resources/register</code> 、注册材料、主体控制证明、注册 VC	<code>registrar-resource</code> 、*.、客户端工作流程测试	部分实现；真实幂等、并发和上游失败需持续集成验证。
根平台验证与可信发布	<code>oan-root-services</code> 、 <code>oan-protocol-common</code>	根资源验证/发布接口、根平台可信发布证明、版本	根平台单测、Root/CDN harness、发布日志	部分实现；链上治理和生产最终性不能从离线测试推断。
内容分发与同步	<code>oan-root-services/services/cdn-publisher</code> 、 <code>cdn-node</code> 、发现节点同步逻辑	发布队列、资源包、游标、哈希	Root/CDN 流程测试、同步日志	部分实现；断点、乱序和真实故障恢复需部署级验证。
发现与语义治理	<code>oan-discovery-node</code> 、 <code>oan-protocol-common/oan-semantic-recommender</code>	查询请求、能力标签树、语义别名、索引记录	discovery fixture、查询解释和语义服务测试	已有结构化/语义路径；模型质量和跨节点收敛需单独记录。
节点授权与治理状态	<code>oan-trust-indexer</code> 、根平台、准入测试套件	治理事件、授权域、生命周期、游标	<code>governance/node/domain</code> 、 <code>fixture</code>	部分实现，索引延迟、链端点和最终性属于运行验证。
公开 API 与错误契约	<code>oan-protocol-common</code> 、各节点 <code>main.rs</code> 、 <code>oan-sdk-ts</code>	HTTP 路径、状态码、错误码、分页	<code>docs/error-codes</code> 、 <code>md</code> 、 <code>requests</code> fixture、SDK 测试	已实现主要接口；新增字段/错误码须同步所有客户端。
社区开发者体验	<code>oan-community-skill</code> 、 <code>oan-sdk-ts</code>	本地身份、资源描述、注册/发现助手	Skill 测试、SDK tests、npm 构建产物	已实现用户路径；发布包完整性和版本兼容由包维护者负责。
官网和运维入口	网站前端、网站后端与部署流程	<code>/api/*</code> 、静态前端、健康和烟测	生产部署说明、构建/烟测脚本、浏览器检查	展示/聚合层；不承担协议权威，部署由运维维护者负责。

规范范围	主要实现仓/模块	主要接口或数据	主要测试/证据	实现状态与维护责任
第三方节点符合性	<code>oan-third-party-node-admission-test-kit</code>	固定 manifest、 fixture、expected、 报告	49 项要求检查、case matrix、traceability matrix	测评支持；当前 V1-alpha 不覆盖真实 Root、链、CDN、索 引器。

交叉索引维护遵循四条规则：第一，新增或修改共享字段时，先更新规范与 `oan-protocol-common`，再检查所有生产服务和客户端；第二，新增接口时同时更新错误契约、SDK、社区技能包和固定请求样例；第三，治理状态、根平台可信发布证明和语义匹配必须保留各自事实来源，不得在聚合层合并成一个无法解释的“可信”布尔值；第四，部署脚本和烟测通过只证明目标环境达到该次检查条件，不把环境变量、域名、模型、节点数量或数据库选型写成协议固定要求。

当实现与规范不一致时，维护者应在变更记录中说明受影响章节、输入输出差异、兼容策略、测试证据、是否产生数据迁移以及回滚方式。只有在代码、测试、配置和运行验证共同支持时，才能把“参考/待核对”提升为“已实现”；任何缺少事实来源、验证主体或状态边界的聚合结果，都只能作为观察信息，不能作为跨节点信任证明。

一次跨仓变更的最小追踪记录可以按以下顺序建立：

章节或字段变化

- > `oan-protocol-common` 类型/校验/错误
- > Root、Registrar、Discovery 或索引器调用路径
- > SDK 与社区 Skill 输入输出
- > fixture、单元测试和集成测试
- > 配置、systemd/反向代理和部署烟测
- > 兼容结论、发布提交和回滚依据

如果变更只涉及官网文案、SEO 或访问统计，应明确记录为官网展示层变更，不得把它列入协议版本变更；如果变更涉及协议字段、DID 文档、资源包、授权域、错误码或生命周期状态，则必须回到共享类型和所有消费方逐项核对。该追踪链用于维护和审查，不改变各仓库原有的职责边界。

交叉索引中的“已实现”应以最小证据集为准：对应代码入口可定位、至少有一项针对性测试或可重复检查、运行配置能够解析所需依赖，并且没有与当前协议字段或错误契约冲突。只有目录中存在文件而没有调用路径、测试结果或运行配置时，应保留为“参考/待核对”；只有测试夹具通过而没有服务集成时，应标为“测评支持”或“部分实现”，不能直接提升为生产能力。

F.10 参考来源

来源	类型	链接
<code>oan-protocol-common</code>	代码仓：共享协议基础	https://github.com/wolfbrother/oan-protocol-common
<code>oan-root-services</code>	代码仓：根平台和内容分发协调	https://github.com/OpenAgenet/oan-root-services
<code>oan-registrar-node</code>	代码仓：注册服务节点	https://github.com/OpenAgenet/oan-registrar-node
<code>oan-discovery-node</code>	代码仓：发现服务节点	https://github.com/OpenAgenet/oan-discovery-node

来源	类型	链接
oan-trust-indexer	代码仓: 链下信任索引器	https://github.com/OpenAgenet/oan-trust-indexer
oan-sdk-ts	代码仓: TypeScript SDK	https://github.com/OpenAgenet/oan-sdk-ts
oan-community-skill	代码仓: 社区 Skill	https://github.com/OpenAgenet/oan-community-skill